শূন্য থেকে React

Job Application Tracker প্রজেক্ট দিয়ে শেখা

React · TypeScript · React Router · Redux Toolkit · Tailwind CSS

ইন্টারভিউ প্রস্তুতির জন্য সম্পূর্ণ গাইড

সূচিপত্র

শূন্য থেকে React — Job Application Tracker প্রজেক্ট দিয়ে শেখা

অধ্যায় ১: ওয়েব কীভাবে কাজ করে

অধ্যায় ২: React-এর জন্য দরকারি JavaScript

অধ্যায় ৩: Asynchronous JavaScript

অধ্যায় ৪: TypeScript

অধ্যায় ৫: Tools — যেগুলো ছাড়া project চলে না

অধ্যায় ৬: React-এর মূল ধারণা

অধ্যায় ৭: State আর Event

অধ্যায় ৮: Side Effect আর useEffect

অধ্যায় ৯: Routing — React Router

অধ্যায় ১০: Global State — Redux Toolkit

অধ্যায় ১১: Server-এর data আর Redux — createAsyncThunk

অধ্যায় ১২: Authentication — login থেকে logout

অধ্যায় ১৩: পুরো feature একসাথে — Applications

অধ্যায় ১৪: Project structure আর নতুন feature যোগ করার নিয়ম

অধ্যায় ১৫: Interview প্রশ্নব্যাংক

শূন্য থেকে React — Job Application Tracker প্রজেক্ট দিয়ে শেখা

এই বই কার জন্য

এই বই তোমার জন্য, যদি:

- তুমি আগে কখনো programming করোনি, অথবা React আর JavaScript একদম নতুন।
- তুমি একটা **আসল project-এর code** পড়ে পড়ে শিখতে চাও।
- তুমি **job interview**-এর জন্য প্রস্তুতি নিচ্ছ।

বইটা ধরে নেয় না যে তুমি কিছু জানো। প্রতিটা নতুন শব্দ প্রথমবার আসার সময় ব্যাখ্যা করা হয়েছে।

উদাহরণগুলো **যেখানে সম্ভব project-এর আসল code** থেকে নেওয়া, `frontend/src` folder থেকে। তবে শুরুর দিকের অধ্যায়ে কিছু জায়গায় TypeScript-এর type সরিয়ে code সহজ করা হয়েছে (তখন লেখায় বলে দেওয়া থাকবে "type বাদ দিয়ে দেখানো হলো" বা "সংক্ষেপে")। আর কিছু ছোট খেলনা উদাহরণও আছে, শুধু একটা ধারণা বোঝানোর জন্য।

একটা কথা: বইয়ের code block-গুলোতে কোনো comment নেই। প্রতিটা code কী করে আর তার result কী, সেটা code-এর ঠিক আগে বা পরে সাধারণ লেখায় বলা আছে।

কীভাবে পড়বে

1. **ক্রম অনুযায়ী পড়ো।** প্রতিটা অধ্যায় আগের অধ্যায়ের উপর দাঁড়িয়ে আছে।
2. **Code পাশে খুলে রাখো।** বইয়ে যে file-এর কথা বলা হচ্ছে, VS Code-এ সেটা খুলে মিলিয়ে দেখো।
3. **প্রতিটা অধ্যায়ের শেষে "ইন্টারভিউ প্রশ্ন"** অংশ আছে। উত্তর দেখার আগে নিজে মুখে উত্তর দেওয়ার চেষ্টা করো।
4. **শেষ অধ্যায়ে** পুরো বইয়ের সব গুরুত্বপূর্ণ প্রশ্ন একসাথে আছে, interview-এর আগের রাতে revision-এর জন্য।

কোন অধ্যায়ে কী

পুরো সূচিপত্র বইয়ের শুরুতে আছে। এখানে শুধু এক নজরে দেখে নাও কোন অধ্যায়ে মূলত কী শিখবে:

অধ্যায়	বিষয়
১–৩	Web কীভাবে কাজ করে, React-এর জন্য দরকারি JavaScript, asynchronous JavaScript

অধ্যায়	বিষয়
৪–৫	TypeScript আর tools (Node.js, npm, Vite, Tailwind CSS)
৬–৯	React-এর মূল জিনিস: component, state, effect, routing
১০–১২	Redux Toolkit দিয়ে global state, server থেকে data আনা, login/authentication
১৩–১৪	পুরো একটা feature শুরু থেকে শেষ, আর project-এর folder সাজানো
১৫	সব অধ্যায়ের interview প্রশ্ন একসাথে

Project-টা কী করে

Job Application Tracker দিয়ে তুমি চাকরির আবেদনগুলোর হিসাব রাখতে পারো:

- Account খোলা আর login করা যায়।
- নতুন চাকরির post যোগ করা যায়: post-এর নাম, link, deadline, আবেদনের মাধ্যম (link বা mail), বেতন, CV link।
- আবেদন করেছ কিনা চিহ্ন দেওয়া যায়।
- List-এ খোঁজা (search), filter করা, সাজানো (sort) আর page বদলানো যায়।
- Edit আর delete করা যায়।

Project-এর দুটো অংশ

```
JobApplicationTracker/
├── backend/jat/
└── frontend/
```

- `frontend/` : এটা **React** দিয়ে বানানো, browser-এ চলে, user যা দেখে আর click করে সেটা। **এই বই মূলত এটা নিয়ে।**
- `backend/jat/` : এটা server-এর code। এটা বানানো **NestJS** দিয়ে। NestJS হলো একটা **backend framework**, মানে server বানানোর জন্য তৈরি করা একগাদা ready-made code আর নিয়মকানুন। Backend data রাখে **PostgreSQL**-এ।
- **Database** হলো এমন একটা program যেটা data **স্থায়ীভাবে** রাখে, যাতে computer বন্ধ করে আবার চালালেও তোমার আবেদনগুলো হারিয়ে না যায়। **PostgreSQL** হলো একটা জনপ্রিয় database।

দুটো অংশই **TypeScript**-এ লেখা। TypeScript হলো JavaScript-এর সাথে কিছু বাড়তি নিয়ম যোগ করা একটা ভাষা (অধ্যায় ৪)। আগে JavaScript শিখব (অধ্যায় ২–৩), তারপর TypeScript।

Project চালানোর আগে যা লাগবে

যে program-গুলো install করতে হবে

Program	কী জিনিস	কেন লাগবে
Node.js	Browser-এর বাইরে, তোমার computer-এ JavaScript চালানোর program। এর সাথে **npm** আসে, যেটা দিয়ে project-এর দরকারি package নামানো আর command চালানো হয় (অধ্যায় ৫)	Frontend আর backend দুটোই চালাতে
Docker Desktop	অন্য program-কে একটা আলাদা ছোট বাক্সে (container) চালানোর program	আমাদের PostgreSQL database এটার ভেতরে চলে, তাই PostgreSQL আলাদা করে install করতে হয় না
VS Code	Code লেখা আর পড়ার জন্য একটা free editor	File খুলে বইয়ের সাথে মিলিয়ে পড়তে

Node.js নামাও nodejs.org থেকে (LTS version), Docker Desktop নামাও docker.com থেকে, আর VS Code নামাও code.visualstudio.com থেকে। Install করার পর Docker Desktop একবার খুলে রাখো, ওটা চালু না থাকলে database চলবে না।

Terminal কী

Terminal হলো একটা window, যেখানে mouse দিয়ে click না করে লিখে লিখে computer-কে command দেওয়া হয়। VS Code-এর ভেতরেই terminal আছে: উপরের menu থেকে **Terminal** → **New Terminal** দিলে নিচে একটা terminal খুলবে।

Terminal সবসময় কোনো একটা folder-এর "ভেতরে" থাকে। `cd` (change directory) command দিয়ে folder বদলানো হয়। যেমন `cd frontend` লিখে Enter চাপলে terminal `frontend` folder-এ ঢোকে। এরপর যে command দেবে, সেটা ওই folder-এর ভেতরে চলবে।

একটা জিনিস জেনে রাখো: localhost আর port

- **localhost** মানে "**নিজের computer**"। Development-এর সময় frontend আর backend দুটোই তোমার নিজের computer-এ চলে, তাই address-এ `localhost` থাকে।
- একই computer-এ অনেক program একসাথে network-এ কথা বলতে পারে। কে কোনটা, সেটা আলাদা করা হয় **port** number দিয়ে। এটাকে একটা building-এর flat number-এর মতো ভাবতে পারো। আমাদের project-এ frontend চলে port **5173**-এ, backend চলে port **3000**-এ, আর database চলে port **5432**-এ।

Project চালানো, ধাপে ধাপে

প্রথমে VS Code দিয়ে `JobApplicationTracker` folder-টা খোলো (**File → Open Folder**)। তাহলে terminal শুরুতে এই folder-এই থাকবে। দুটো আলাদা terminal লাগবে, কারণ backend আর frontend দুটোই একসাথে চালু থাকতে হবে।

Terminal ১: backend

ধাপ ১. Backend folder-এ ঢোকো, আর setting-এর file বানাও। Folder-এ `.env.example` নামে একটা file আছে। এটা copy করে `.env` নামে নতুন file বানাও:

```
cd backend/jat
cp .env.example .env
```

(Windows-এর Command Prompt ব্যবহার করলে `cp`-এর বদলে লেখো `copy .env.example .env`। VS Code-এর ভেতরের terminal-এ `cp` চলে।)

এরপর VS Code-এ `.env` খুলে খালি জায়গাগুলো পূরণ করো:

- `POSTGRES_USER` আর `POSTGRES_PASSWORD`: database-এর জন্য যেকোনো একটা নাম আর password দাও, যেমন `jat` আর `jat123`।
- `DATABASE_URL`: এখানে `USER` আর `PASSWORD`-এর জায়গায় ঠিক ওই একই নাম আর password বসাও। যেমন `postgresql://jat:jat123@localhost:5432/jat`।
- `JWT_SECRET`: যেকোনো লম্বা secret string, যেমন `my-very-long-secret-text-123`। এটা login token বানাতে লাগে (অধ্যায় ১২)।

এই ধাপটা **আগে** করতে হবে। পরের ধাপের `npm install` ভেতরে ভেতরে `.env`-এর `DATABASE_URL` পড়ে, তাই `.env` না থাকলে সেটা error দেয়।

ধাপ ২. দরকারি package নামাও। **Package** মানে অন্যদের লেখা ready-made code, যেটা আমাদের project ব্যবহার করে (অধ্যায় ৫-এ বিস্তারিত):

```
npm install
```

`npm install` প্রথমবার একটু সময় নেবে। শেষ হলে `node_modules` নামে একটা folder তৈরি হবে, আর সব package থাকে ওখানেই।

ধাপ ৩. Docker-এর ভেতরে PostgreSQL database চালু করো (Docker Desktop খোলা থাকতে হবে):

```
npm run db:up
```

ধাপ ৪. Database-এ দরকারি table-গুলো তৈরি করো। **Table** হলো database-এর ভেতরে data সাজিয়ে রাখার জায়গা, অনেকটা Excel sheet-এর মতো: প্রতিটা row একটা জিনিস (যেমন একটা application), আর প্রতিটা column একটা তথ্য (যেমন deadline)। Table বানানো বা বদলানোর নির্দেশগুলো যে file-এ লেখা থাকে, তাকে বলে **migration**:

```
npm run db:migrate
```

প্রথমবার `npm run db:up`-এর পরে database পুরোপুরি চালু হতে কয়েক সেকেন্ড লাগে। তাই একটু অপেক্ষা করে migrate চালাও। যদি "Can't reach database server" ধরনের error আসে, কয়েক সেকেন্ড পর একই command আবার চালাও।

ধাপ ৫. Backend server চালু করো:

```
npm run start:dev
```

কিছুক্ষণ পর terminal-এ দেখবে server চালু হয়েছে। এই terminal খোলা রাখো, বন্ধ করলে backend থেমে যাবে।

Terminal ২: frontend

Terminal-এর পাশের **+** চিহ্নে click করে নতুন terminal খোলো। নতুন terminal আবার `JobApplicationTracker` folder থেকে শুরু হবে।

```
cd frontend
npm install
npm run dev
```

`npm run dev` চালালে terminal-এ `http://localhost:5173` লেখা একটা address দেখাবে।

Browser-এ খোলো

Browser-এ **http://localhost:5173** খোলো। Login page আসবে। "Register" দিয়ে নতুন account খুলে ব্যবহার শুরু করো।

কী চালাচ্ছ	কোন folder-এ	Command
`.env` বানানো (প্রথমবার, backend-এ সবার আগে)	`backend/jat`	`cp .env.example .env`

কী চালাচ্ছ	কোন folder-এ	Command
Package নামানো (প্রথমবার)	`backend/jat` আর `frontend` দুটোতেই	`npm install`
Database চালু	`backend/jat`	`npm run db:up`
Table তৈরি (প্রথমবার)	`backend/jat`	`npm run db:migrate`
Backend চালু	`backend/jat`	`npm run start:dev`
Frontend চালু	`frontend`	`npm run dev`

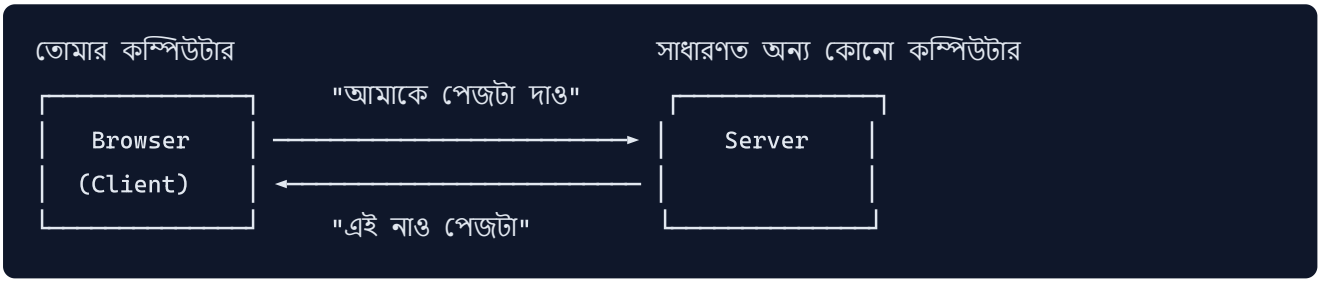
পরের দিন আবার চালাতে চাইলে শুধু Docker Desktop খুলে `npm run db:up`, `npm run start:dev` (backend) আর `npm run dev` (frontend) দিলেই হবে। `npm install` আর `db:migrate` প্রথমবারের কাজ।

অধ্যায় ১: ওয়েব কীভাবে কাজ করে

React শেখার আগে বুঝতে হবে React কোন সমস্যার সমাধান করে। তার জন্য আগে বুঝতে হবে একটা website আসলে কীভাবে কাজ করে। এই অধ্যায়ে কোনো code লিখতে হবে না, শুধু ধারণাগুলো পরিষ্কার করব।

১.১ Browser আর Server

যখন তুমি browser-এ (Chrome, Firefox) একটা address লেখো, তখন দুটো program কথা বলে:



- **Client** হলো যে চায়। এখানে তোমার browser।
- **Server** হলো যে দেয়। Server কোনো আলাদা জাদুর যন্ত্র না, **server আসলে একটা program**। সে চুপচাপ বসে থাকে আর কেউ কিছু চাইলে উত্তর দেয়। সাধারণত server অন্য কোনো computer-এ চলে (যেমন Google-এর computer-এ)।

Client যখন কিছু চায়, সেই চাওয়াটাকে বলে **request**। Server যা ফেরত দেয়, সেটাকে বলে **response**। এখন থেকে এই দুটো শব্দ English-এই বলব।

আমাদের project-এ:

- **Frontend** (React) চলে browser-এ।
- **Backend** (NestJS) হলো server program। Development-এর সময় এটা **তোমার নিজের computer-এই** চলে, তাই address-এ `localhost` থাকে (ভূমিকা অধ্যায়ে দেখেছ)। আসল website-এ এটা অন্য কোনো computer-এ চলত।
- Backend data রাখে **database**-এ। Database হলো এমন program যেটা data স্থায়ীভাবে জমা রাখে, যাতে server বন্ধ হলেও data হারিয়ে না যায়।

১.২ একটা web page-এর তিনটা ভাষা

প্রতিটা web page তিনটা জিনিস দিয়ে তৈরি:

ভাষা	কাজ	তুলনা
HTML	কাঠামো: কোথায় কী আছে	বাড়ির দেয়াল, দরজা, জানালা
CSS	styling: color, size, spacing	রং, পর্দা, ফার্নিচার
JavaScript	আচরণ: click করলে কী হবে	বিদ্যুৎ: সুইচ টিপলে বাতি জ্বলে

HTML

HTML লেখা হয় **tag** দিয়ে। প্রতিটা tag একটা জিনিস বোঝায়:

```
<h1>Job Application Tracker</h1>
<p>তোমার সব আবেদন এক জায়গায়।</p>
<button>Log in</button>
<input type="email" />
```

এই চারটা লাইন browser-এ দেখাবে: একটা বড় শিরোনাম, তার নিচে একটা লাইন লেখা, একটা "Log in" button, আর একটা খালি ঘর যেখানে email লেখা যায়।

- `<h1>` হলো বড় শিরোনাম, `<p>` একটা paragraph, `<button>` একটা button, আর `<input>` হলো একটা **input field**, মানে user যেখানে কিছু type করতে পারে।
- বেশিরভাগ tag জোড়ায় আসে: `<p>` দিয়ে শুরু, `</p>` দিয়ে শেষ। মাঝের লেখাটা হলো ভেতরের content।
- কিছু tag একা আসে, যেমন `<input />`।
- Tag-এ **attribute** থাকে, মানে বাড়তি তথ্য। যেমন `type="email"` বলে দেয় এই input field-এ email লেখা হবে।

CSS

CSS দিয়ে বলা হয় জিনিসটা দেখতে কেমন হবে:

```
button {
    background-color: indigo;
    color: white;
    padding: 8px 16px;
}
```

এর মানে: page-এর সব button-এর পেছনের রং হবে নীলচে-বেগুনি (indigo), লেখা হবে সাদা, আর **padding** হবে উপর-নিচে 8px, ডানে-বামে 16px। **Padding** হলো জিনিসটার লেখা আর তার বর্ডারের মাঝের ফাঁকা জায়গা।

আমাদের project-এ আমরা CSS সরাসরি খুব কম লিখি। তার বদলে **Tailwind CSS** ব্যবহার করি (অধ্যায় ৫-এ বিস্তারিত)। HTML tag-এ `class` নামে একটা attribute দেওয়া যায়, যেটা দিয়ে tag-টাকে এক বা একাধিক নাম দেওয়া হয়। Tailwind আগে থেকেই অনেক ছোট ছোট class নাম বানিয়ে রেখেছে, আর প্রতিটা নাম একটা করে style বোঝায়:

```
<button class="bg-indigo-600 text-white px-4 py-2">Log in</button>
```

Class	মানে
`bg-indigo-600`	background রং indigo (600 হলো রঙের গাঢ়ত্ব)
`text-white`	লেখার রং সাদা
`px-4`	ডানে-বামে padding (x মানে আড়াআড়ি)
`py-2`	উপরে-নিচে padding (y মানে খাড়াখাড়ি)

ফলে উপরের CSS-এর মতোই একটা রঙিন button দেখাবে, কিন্তু আলাদা CSS file লিখতে হলো না। একটা কথা মনে রাখো: React-এ `class` না লিখে `className` লেখা হয় (অধ্যায় ৬-এ কারণ দেখবে)।

JavaScript

JavaScript হলো আসল programming ভাষা। এটা দিয়ে:

- Button-এ click করলে কিছু ঘটানো যায়
- Server থেকে data আনা যায়
- Page-এর লেখা বদলানো যায়, page reload না করেই

১.৩ DOM: browser-এর memory-তে page

Browser HTML পড়ে memory-তে page-এর একটা **tree-আকারের representation** বানায়। একটা গাছের যেমন গোড়া থেকে ডাল, ডাল থেকে আরো ডাল বের হয়, এটাও তেমন। এই ধরনের সাজানোকে বলে **tree structure**। Browser-এর বানানো এই tree-এর নাম **DOM** (Document Object Model)।

এই HTML থেকে:

```
<main>
    <h1>Applications</h1>
    <button>New</button>
</main>
```

DOM হয় এমন:

```
main
├── h1  → "Applications"
└── button → "New"
```

অর্থাৎ `main`-এর ভেতরে দুটো জিনিস আছে: একটা `h1`, যার লেখা "Applications", আর একটা `button`, যার লেখা "New"।

DOM-এর প্রতিটা অংশ JavaScript-এর কাছে একটা **object**। Object হলো অনেকগুলো তথ্য একসাথে বাঁধা একটা জিনিস, যেমন একটা `h1`-এর object-এর ভেতরে তার লেখা, তার attribute ইত্যাদি থাকে (object নিয়ে বিস্তারিত অধ্যায় ২-এ)।

JavaScript এই DOM বদলাতে পারে, আর DOM বদলালে screen-ও সাথে সাথে বদলায়:

```
document.querySelector('h1').textContent = 'আমার আবেদনগুলো';
```

এটাকে বাম থেকে ডানে পড়ো:

1. `document` হলো পুরো page।
2. `.querySelector('h1')` page-এ **প্রথম যে** `h1` পায়, সেটা খুঁজে বের করে।
3. `.textContent = 'আমার আবেদনগুলো'` সেই `h1`-এর লেখা বদলে দেয়।

Result: screen-এ "Applications" লেখাটা সাথে সাথে বদলে "আমার আবেদনগুলো" হয়ে যাবে, page reload ছাড়াই।

সমস্যাটা হলো, বড় app-এ এভাবে হাতে হাতে DOM বদলানো খুব কঠিন হয়ে যায়: কোন জিনিস কখন বদলাতে হবে, কোনটা বদলাতে ভুলে গেলাম, ইত্যাদি। **React ঠিক এই সমস্যার সমাধান করে।** তুমি শুধু বলো "data এরকম হলে screen দেখতে এরকম হবে", আর DOM বদলানোর কাজটা React নিজেই করে (অধ্যায় ৬)।

১.৪ HTTP: client আর server-এর ভাষা

Browser আর server যে নিয়মে কথা বলে, তার নাম **HTTP**। প্রতিটা কথোপকথনে থাকে একটা request আর একটা response।

URL কী

URL হলো একটা address, যেমন `http://localhost:5173/api/applications/42`। এর অংশগুলো:

অংশ	মানে
`http://`	কোন নিয়মে কথা হবে (HTTP)
`localhost:5173`	কোন computer (localhost মানে নিজের computer) আর কোন port
`/api/applications/42`	ওই server-এর ভেতরে কোন জিনিসটা চাই

আমাদের frontend code-এ প্রায় সবসময় শুধু শেষ অংশটা লেখা হয়, যেমন `/api/applications/42`। একে বলে **relative URL**। Browser নিজে থেকেই সামনে বর্তমান page-এর address (`http://localhost:5173`) বসিয়ে নেয়। পুরোটা লেখা থাকলে সেটাকে বলে full URL।

আমাদের project-এ একটা নিয়ম আছে: যে URL `/api` দিয়ে শুরু, সেটা **backend-এর** জন্য। বাকি URL, যেমন `/login` বা `/applications`, হলো frontend-এর page। `/api` দিয়ে শুরু হওয়া request কীভাবে port 3000-এর backend-এ পৌঁছায়, সেটা অধ্যায় ৩ আর ৫-এ দেখবে।

Request-এর অংশ

একটা request-এ চারটা অংশ থাকতে পারে:

অংশ	মানে	উদাহরণ
Method	কী করতে চাই	`GET`, `POST`, `PATCH`, `DELETE`
URL	কোথায়	`/api/applications/42`
Headers	বাড়তি তথ্য	`Authorization: Bearer abc123`
Body	পাঠানো data (সবসময় থাকে না)	`{ "email": "a@b.com" }`

Header-এর উদাহরণে `abc123` হলো একটা **token**। Login করার পর server তোমাকে একটা লম্বা গোপন লেখা দেয়, সেটাই token। পরের প্রতিটা request-এ সেটা দেখিয়ে তুমি প্রমাণ করো যে তুমি login করা user। `Bearer` শব্দটা শুধু বলে দেয় "এর পরে যেটা আছে সেটা একটা token" (বিস্তারিত অধ্যায় ১২-এ)।

Body-র উদাহরণটা **JSON** format-এ লেখা, যেটা ১.৫-এ দেখবে।

চারটা প্রধান method

Method	কাজ	আমাদের project-এ
`GET`	Data পড়া	আবেদনের list আনা
`POST`	নতুন কিছু তৈরি	নতুন আবেদন যোগ করা, login

Method	কাজ	আমাদের project-এ
`PATCH`	কিছু অংশ বদলানো	আবেদন edit করা
`DELETE`	মুছে ফেলা	আবেদন delete করা

Response-এর status code

Server প্রতিটা response-এর সাথে একটা **status code** পাঠায়। এটা তিন অঙ্কের একটা number, যেটা দেখে বোঝা যায় কাজটা সফল হলো কিনা। মোটা দাগে: **2xx** মানে সফল, **4xx** মানে client (মানে আমরা) কিছু ভুল করেছি, **5xx** মানে server-এ সমস্যা।

Code	মানে
`200` OK	সফল
`201` Created	নতুন কিছু তৈরি হয়েছে
`204` No Content	সফল, তবে ফেরত দেওয়ার কিছু নেই (যেমন delete)
`400` Bad Request	তুমি ভুল data পাঠিয়েছ (যেমন email-এর জায়গায় উল্টাপাল্টা লেখা)
`401` Unauthorized	Server জানে না তুমি কে: login করোনি, অথবা token ভুল বা পুরনো
`403` Forbidden	Server জানে তুমি কে, কিন্তু এই কাজের অনুমতি তোমার নেই
`404` Not Found	জিনিসটা নেই
`409` Conflict	আগে থেকেই আছে (যেমন একই email দিয়ে আবার register)
`500` Server Error	Server-এ সমস্যা

১.৫ API আর JSON

API

API (Application Programming Interface) হলো server-এর কিছু নির্দিষ্ট URL, যেখানে request পাঠিয়ে data পাওয়া বা বদলানো যায়। এমন প্রতিটা URL-কে বলে একটা **endpoint**। আমাদের backend-এর API:

Method + URL	কাজ
`POST /api/auth/register`	Account খোলা

Method + URL	কাজ
`POST /api/auth/login`	Login
`GET /api/auth/me`	আমি কে? (login করা user-এর তথ্য)
`GET /api/applications`	আবেদনের list
`POST /api/applications`	নতুন আবেদন
`GET /api/applications/:id`	একটা আবেদন
`PATCH /api/applications/:id`	আবেদন বদলানো
`DELETE /api/applications/:id`	আবেদন মোছা

`:id` মানে এখানে আসল id বসবে, যেমন `/api/applications/42` মানে "42 নম্বর id-র আবেদনটা"।

খেয়াল করো, একই URL-এ method আলাদা হলে কাজ আলাদা। `GET /api/applications` list আনে, আর `POST /api/applications` নতুন আবেদন বানায়।

JSON

JSON (JavaScript Object Notation) হলো data লিখে পাঠানোর একটা format। Server আর browser দুজনেই এটা বোঝে। দেখতে এমন:

```
{
    "success": true,
    "data": {
        "id": "42",
        "jobPost": "Backend Engineer",
        "isApplied": false,
        "salaryMin": 50000
    }
}
```

পড়ার নিয়ম:

- `{ }`-এর ভেতরে থাকে কিছু **key-value pair**। বাম পাশে নাম (key), কোলন `:`, তারপর তার value। যেমন `"jobPost": "Backend Engineer"` মানে jobPost-এর value হলো "Backend Engineer"।
- Key সবসময় double quote `" "`-এর ভেতরে থাকে।
- Value হতে পারে লেখা (`"42"`), number (`50000`), `true`/`false`, `null` (কিছু নেই), অথবা আরেকটা `{ }`। উপরে `data`-র value নিজেই আরেকটা `{ }`।

- `[ ]` দিয়ে একটা **list** বোঝায়, যেমন `["a", "b", "c"]`। Programming-এ list-কে বলে **array** (অধ্যায় ২)।

আমাদের backend **সবসময় একই আকারে** response দেয়। এটা মনে রাখো, অধ্যায় ৩-এ কাজে লাগবে:

- **সফল হলে:** `{ "success": true, "data": ... }`
- **ব্যর্থ হলে:** `{ "success": false, "error": { "statusCode": 400, "message": "...", "details": [...] } }`

এখানে `...` কোনো code না। এর মানে শুধু "এখানে কিছু একটা থাকবে"। যেমন `data`-তে থাকবে আসল data (একটা আবেদন, বা আবেদনের list), `message`-এ থাকবে error-এর লেখা, আর `details`-এ থাকবে error-এর খুঁটিনাটি লেখার একটা list (`details` সবসময় থাকে না, শুধু validation error-এ)।

১.৬ Single Page Application (SPA)

পুরনো website-এ প্রতিটা click-এ server থেকে **পুরো নতুন page** আসত, আর screen সাদা হয়ে আবার load হতো।

আমাদের React app একটা **SPA (Single Page Application)**:

1. প্রথমবার browser একটাই HTML page (`index.html`) আর JavaScript file load করে।
2. এরপর page বদলানো, data দেখানো সব কাজ browser-এই JavaScript দিয়ে হয়।
3. Server থেকে শুধু **JSON data** আসে, পুরো page আর আসে না।

ফলে app-টা খুব দ্রুত আর মসৃণ লাগে, mobile app-এর মতো।

আমাদের `frontend/index.html`-এর `<body>`-তে শুধু এটুকু আছে (পুরো file-এর বাকি অংশ বাদ দিয়ে দেখানো হলো):

```
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
```

- `<div id="root"></div>` একটা খালি বাক্স। `id="root"` দিয়ে বাক্সটার নাম দেওয়া হয়েছে "root"।
- `<script>` tag দিয়ে browser-কে বলা হয় "এই JavaScript file-টা load করে চালাও"। `src` attribute-এ file-এর path থাকে।
- File-টার নাম `main.tsx`। `.tsx` মানে file-টা **TypeScript + JSX**-এ লেখা। TypeScript হলো type-যুক্ত JavaScript (অধ্যায় ৪), আর JSX হলো JavaScript-এর ভেতরে HTML-এর মতো লেখা (অধ্যায় ৬)। Browser এগুলো সরাসরি বোঝে না। Development-এ **Vite** নামের একটা tool এগুলোকে browser-এর বোঝার মতো সাধারণ JavaScript-এ convert করে দেয় (অধ্যায় ৫)।

Result: page খুললে প্রথমে কিছুই দেখা যায় না, তারপর `main.tsx` চলে আর React পুরো app-টা ওই খালি `<div id="root">`-এর ভেতরে বানিয়ে দেয় (অধ্যায় ৬)।

SPA আর SEO

SEO (Search Engine Optimization) মানে এমনভাবে website বানানো যাতে Google-এর মতো search engine page-এর লেখা পড়তে পারে আর search result-এ দেখায়। Search engine একটা program পাঠিয়ে page-এর HTML পড়ে। সাধারণ website-এ server থেকে আসা HTML-এই সব লেখা থাকে। কিন্তু SPA-তে server থেকে আসা HTML-এ শুধু খালি `<div id="root">` থাকে, আসল লেখা পরে JavaScript দিয়ে তৈরি হয়। ফলে search engine অনেক সময় page-টাকে প্রায় খালি দেখে।

আমাদের app-এর জন্য এটা সমস্যা না, কারণ এটা login করে ব্যবহার করার app, Google-এ দেখানোর দরকার নেই। কিন্তু blog বা online shop-এর মতো site-এ SEO খুব জরুরি। তখন **server-side rendering** নামের কৌশল ব্যবহার করা হয়, যেখানে server নিজেই লেখাসহ পুরো HTML বানিয়ে পাঠায়।

সারসংক্ষেপ

- Browser (client) server-এর কাছে **HTTP request** পাঠায়, আর server **response** দেয়। Server আসলে একটা program, development-এ আমাদের backend নিজের computer-এ (localhost) চলে।
- **HTML** কাঠামো দেয়, **CSS** সাজায়, **JavaScript** আচরণ ঠিক করে।
- **DOM** হলো memory-তে page-এর tree-আকারের representation। React DOM বদলানোর কাজটা আমাদের হয়ে করে।
- Request-এ থাকে method, URL, headers আর body। Response-এ থাকে **status code** (2xx সফল, 4xx client-এর ভুল, 5xx server-এর সমস্যা)।
- **API** হলো server-এর endpoint-গুলো, আর **JSON** হলো data-র format। আমাদের backend সবসময় `{ success, data }` বা `{ success, error }` আকারে response দেয়।
- আমাদের app একটা **SPA**: একবার load হয়, তারপর শুধু JSON data আসা-যাওয়া করে।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: GET আর POST-এর পার্থক্য কী?
উত্তর: GET data পড়ার জন্য। এর body থাকে না, আর একই GET request বারবার পাঠালে server-এর data বদলায় না। POST নতুন কিছু তৈরির জন্য। এর data body-তে যায়, আর প্রতিবার পাঠালে নতুন একটা জিনিস তৈরি হতে পারে। যেমন আমাদের project-এ `GET /api/applications` list আনে, আর `POST /api/applications` নতুন আবেদন বানায়।

প্রশ্ন: 401 আর 403-এর পার্থক্য কী?

উত্তর: 401 Unauthorized মানে server জানে না তুমি কে: login করা নেই, অথবা token ভুল বা পুরনো হয়ে গেছে। আবার login করলে ঠিক হয়ে যাবে। 403 Forbidden মানে server জানে তুমি কে, কিন্তু এই কাজের অনুমতি তোমার নেই। এখানে আবার login করে লাভ নেই।

প্রশ্ন: SPA কী? এর সুবিধা আর অসুবিধা কী?

উত্তর: যে app একবার একটা HTML page load করে, তারপর browser-এই JavaScript দিয়ে page বদলায় আর server থেকে শুধু data আনে, সেটাই SPA।

- সুবিধা: page বদলাতে পুরো reload হয় না, তাই দ্রুত আর মসৃণ লাগে। Server-কে শুধু data পাঠাতে হয়, পুরো page না।
- অসুবিধা: প্রথমবার load-এ অনেক JavaScript নামাতে হয়, তাই প্রথম load ভারী। আর server থেকে আসা HTML প্রায় খালি থাকে বলে search engine লেখা ঠিকমতো পড়তে পারে না, মানে SEO খারাপ হয়। এর সমাধান server-side rendering।

প্রশ্ন: DOM কী?

উত্তর: Browser HTML পড়ে memory-তে page-এর যে tree-আকারের representation বানায়, সেটাই DOM। এর প্রতিটা অংশ JavaScript-এর কাছে একটা object। JavaScript DOM বদলালে screen-ও বদলায়। React এই DOM বদলানোর কাজটা আমাদের হয়ে করে।

প্রশ্ন: Status code-এর 2xx, 4xx, 5xx মানে কী?

উত্তর: 2xx মানে request সফল, যেমন 200 OK বা 201 Created। 4xx মানে client-এর দিক থেকে ভুল, যেমন 400 ভুল data, 401 login নেই, 404 জিনিসটা নেই। 5xx মানে server-এর ভেতরে সমস্যা, যেমন 500।

অধ্যায় ২: React-এর জন্য দরকারি JavaScript

React আসলে JavaScript-ই। তাই React code পড়তে JavaScript-এর কিছু জিনিস খুব ভালো করে জানতে হবে। এই অধ্যায়ে শুধু সেগুলোই আছে যেগুলো আমাদের project-এ ব্যবহার হয়েছে।

> **পড়ার আগে একটা সতর্কবার্তা।** এই অধ্যায়ের কিছু উদাহরণ project-এর আসল code থেকে নেওয়া, তাই সেখানে এমন কিছু জিনিস চোখে পড়বে যেগুলো এখনো শেখোনি:
>
> - **JSX**: JavaScript-এর ভেতরে HTML-এর মতো লেখা, যেমন `<p>{error}</p>`। এটা অধ্যায় ৬-এ শিখব। এখন শুধু এটুকু জানো: JSX-এর ভেতরে `{ }` মানে "এখানে একটা JavaScript value বসাও"।
> - **TypeScript type**: যেমন `: number`, `: string`, `readonly`, `<T>`। এগুলো অধ্যায় ৪-এ শিখব। এখন এগুলো চোখ বুলিয়ে পার হয়ে যাও।
> - **React/Redux-এর নাম**: যেমন `useAppSelector`, `dispatch`, `state`, `useState`। এগুলো পরের অধ্যায়গুলোতে আসবে।
>
> এখন প্রতিটা উদাহরণের শুধু **JavaScript অংশটা** পড়ো, বাকিটা নিয়ে চিন্তা কোরো না।

২.১ Code কোথায় চালাবে: `console.log`

এই অধ্যায়ের ছোট উদাহরণগুলো তুমি নিজে চালিয়ে দেখতে পারো, কিছু install না করেই:

1. Chrome খোলো।
2. Keyboard-এ **F12** চাপো (Mac-এ Cmd + Option + J)। পাশে বা নিচে একটা panel খুলবে, এর নাম **DevTools**।
3. উপরের tab থেকে **Console** বেছে নাও।
4. সেখানে code লিখে Enter চাপো।

কোন উদাহরণ Console-এ চালানো যাবে, সেটা বোঝার সহজ নিয়ম: যে ছোট উদাহরণ নিজেই সম্পূর্ণ, মানে যেখানে ব্যবহার হওয়া সব variable ওই উদাহরণের ভেতরেই বানানো (যেমন `const nums = [1, 2, 3];` তারপর `nums.map((n) => n * 2)`), সেটা Console-এ চালিয়ে দেখো। আর যেখানে project-এর নাম আছে (`values`, `application`, `state`...), সেগুলো শুধু পড়ার জন্য: Console-এ চালালে error আসবে, কারণ ওই variable-গুলো সেখানে নেই। একইভাবে যেখানে TypeScript type (`: string`) বা `export` আছে, সেগুলোও Console-এ চলবে না, কারণ browser-এর Console type বা `export` বোঝে না।

`console.log(...)` দিয়ে কোনো কিছু Console-এ print করা হয়, মানে দেখানো হয়:

```
console.log('Hello');
console.log(2 + 3);
```

Console-এ প্রথমে দেখাবে `Hello`, তারপর `5`। কোনো কিছুর value কী হলো দেখতে চাইলে সেটাকে `console.log`-এর ভেতরে দিয়ে দাও।

২.২ Variable: `const` আর `let`

Variable হলো একটা নাম দেওয়া বাক্স, যেখানে একটা **value** রাখা হয়। Value মানে যেকোনো data, যেমন একটা লেখা বা একটা number।

```
const appName = 'Job Tracker';
let count = 0;
count = count + 1;
console.log(count);
```

- প্রথম লাইনে `appName` নামের variable-এ `'Job Tracker'` লেখাটা রাখা হলো।
- এখানে `=` মানে "সমান" না। `=` মানে **assign**: ডান পাশের value-টা বাম পাশের variable-এ রাখো।
- তৃতীয় লাইন `count = count + 1` পড়তে হয় ডান থেকে: "`count`-এর এখনকার value (`0`) নাও, তার সাথে 1 যোগ করো, result আবার `count`-এ রাখো"। এরপর `count`-এর value `1`।
- তাই Console-এ দেখাবে `1`।

Keyword	পরে নতুন value assign করা যায়?	কখন ব্যবহার করবে
`const`	না	**প্রায় সবসময়**
`let`	হ্যাঁ	যখন সত্যিই পরে value বদলাতে হবে
`var`	হ্যাঁ	**কখনো না** (পুরনো, এর scope-এর নিয়ম গোলমেলে; ২.৬-এ দেখবে)

আমাদের project-এ প্রায় সব জায়গায় `const` আছে। `let` আছে শুধু `frontend/src/shared/api.ts`-এ। নিচের snippet-এ `: Response` হলো TypeScript type (অধ্যায় ৪), আর `try`, `await`, `fetch` শিখব ২.১৩ আর অধ্যায় ৩-এ। এখন শুধু খেয়াল করো `response` variable-টা কীভাবে তৈরি হলো (সংক্ষেপে দেখানো হলো):

```
let response: Response;
try
{
    response = await fetch(API_BASE_URL + url, { method, headers, body });
}
```

(এখানে `catch` অংশ বাদ রাখা হয়েছে)

প্রথম লাইনে `response` শুধু **declare** করা হয়েছে, মানে বাক্সটা বানানো হয়েছে কিন্তু ভেতরে কিছু রাখা হয়নি। পরে `try`-এর ভেতরে তাতে value **assign** করা হয়েছে। Declare করে পরে value assign করতে হলে `let` লাগে, কারণ `const`-এ তৈরির সময়ই value দিতে হয়।

মনে রাখো: `const` object-এর **ভেতরের জিনিস** বদলাতে বাধা দেয় না (object কী, সেটা ২.৭-এ দেখবে; এখন শুধু জানো `{ }`-এর ভেতরে কিছু তথ্য রাখা থাকে)। `const` বাধা দেয় শুধু variable-টাতে নতুন value assign করতে:

```
const user = { name: 'Rahim' };
user.name = 'Karim';
user = {};
```

দ্বিতীয় লাইন ঠিকঠাক চলবে, আর `user.name` হয়ে যাবে `'Karim'`, কারণ object-এর ভেতরটা বদলানো যায়। তৃতীয় লাইনে error হবে (`TypeError: Assignment to constant variable.`), কারণ `const` variable-এ নতুন value assign করা যায় না।

২.৩ Data-র Type

Type	উদাহরণ	মানে
`string`	`'hello'`, `"hello"`	Text। Single বা double quote-এর ভেতরে লেখা হয়
`number`	`42`, `3.14`	সংখ্যা
`boolean`	`true`, `false`	হ্যাঁ/না
`null`	`null`	ইচ্ছা করে "কিছু নেই"
`undefined`	`undefined`	এখনো কোনো value assign করা হয়নি
`object`	`{ name: 'Rahim', age: 25 }`	অনেক তথ্য একসাথে (২.৭)
`array`	`[10, 20, 50]`	List (২.৮)

`''` (দুটো quote, মাঝে কিছু নেই) হলো **empty string**, মানে এমন string যার ভেতরে কোনো অক্ষর নেই।

কোনো value কোন ধরনের, সেটা জানতে `typeof` লেখা হয়। যেমন `typeof 'hello'` দেয় `'string'`, আর `typeof 42` দেয় `'number'`। একটা অদ্ভুত ব্যাপার আছে: `typeof [10, 20]` দেয় `'object'`, `'array'` না। কারণ JavaScript-এ array আসলে এক বিশেষ ধরনের object। আরেকটা পুরনো ভুল হলো `typeof null`-ও দেয় `'object'`।

২.৪ Operator আর `if` / `else`

Operator হলো এমন চিহ্ন যেটা এক বা একাধিক value নিয়ে নতুন একটা value বানায়। যেমন `+`।

Arithmetic আর comparison

Operator	কাজ	উদাহরণ	Result
`+` `-` `*` `/`	যোগ, বিয়োগ, গুণ, ভাগ	`10 / 2`	`5`
`===`	দুটো value কি একই?	`5 === 5`	`true`
`!==`	দুটো value কি আলাদা?	`5 !== 3`	`true`
`>` `<`	বড়, ছোট	`15 > 10`	`true`
`>=` `<=`	বড় বা সমান, ছোট বা সমান	`10 <= 10`	`true`

খেয়াল করো: একটা `=` মানে assign (২.২), আর তিনটা `===` মানে তুলনা। দুটো গুলিয়ে ফেলা খুব সাধারণ ভুল।

`==` বনাম `===`

JavaScript-এ দুই রকম তুলনা আছে:

- `==` তুলনার আগে দুই পাশের value-কে একই type-এ convert করে নেয়। তাই `'5' == 5` হয় `true`, যদিও একটা string আর একটা number।
- `===` type-ও মেলায়, কিছু convert করে না। তাই `'5' === 5` হয় `false`।

`==`-এর এই নিজে নিজে convert করা অনেক অপ্রত্যাশিত bug তৈরি করে। আমাদের project-এ সবসময় `===` আর `!==` ব্যবহার করা হয়েছে, তুমিও তাই করবে।

Logical operator: `!`, `&&`, `||`

Operator	মানে	উদাহরণ	Result
`!`	উল্টে দাও	`!true`	`false`
`&&`	দুটোই সত্য? (AND)	`true && false`	`false`
`||`	যেকোনো একটা সত্য? (OR)	`true || false`	`true`

`if` / `else`

`if` দিয়ে বলা হয় "condition সত্য হলে এই কাজটা করো, না হলে (`else`) ওই কাজটা করো":

```
const age = 20;
if (age >= 18)
{
    console.log('Adult');
}
else
{
    console.log('Child');
}
```

`age >= 18` সত্য, তাই Console-এ দেখাবে `Adult` । `age` -এর value `10` হলে দেখাত `Child` । `( )` -এর ভেতরে condition, আর `{ }` -এর ভেতরে কাজ।

`NaN`

`NaN` মানে "Not a Number"। কোনো হিসাব থেকে অর্থবহ number না এলে এটা আসে। যেমন `Number('hello')` দেয় `NaN` , কারণ 'hello' লেখাটাকে number-এ convert করা যায় না।

Truthy আর Falsy

`if` -এর `( )` -এ শুধু `true` / `false` না, যেকোনো value দেওয়া যায়। JavaScript তখন value-টাকে true বা false হিসেবে ধরে নেয়:

- **Falsy** (false-এর মতো আচরণ করে): `false` , `0` , `''` (empty string), `null` , `undefined` , `NaN`
- **বাকি সব truthy**, যেমন `'hello'` , `42` , `[]` , `{}` ।

Project থেকে একটা উদাহরণ, `frontend/src/features/applications/applications.utils.ts` -এর `validateForm` function-এর ভেতর থেকে:

```
if (!values.jobPost.trim())
{
    errors.jobPost = 'Job post is required';
}
```

`errors` হলো form-এর error message রাখার একটা object। `values.jobPost` হলো form-এ user-এর লেখা job post-এর নাম। `.trim()` লেখার আগে-পিছের ফাঁকা জায়গা কেটে দেয়, যেমন `' abc '.trim()` দেয় `'abc'`। User কিছু না লিখলে বা শুধু space দিলে `trim()` দেয় `''`, যেটা falsy। `!''` হয় `true`, তাই `if`-এর ভেতরের লাইন চলে আর error message সেট হয়। User `'Backend Engineer'` লিখলে `!'Backend Engineer'` হয় `false`, তাই কোনো error সেট হয় না।

২.৫ Function

Function হলো নাম দেওয়া একটা কাজ, যেটা বারবার ব্যবহার করা যায়। একটা সহজ উদাহরণ দিয়ে শুরু করি:

```
function add(a, b)
{
    return a + b;
}

console.log(add(2, 3));
console.log(add(10, 20));
```

Console-এ দেখাবে `5`, তারপর `30`।

- `function add(a, b)` দিয়ে `add` নামের function বানানো হলো।
- **Parameter** (`a`, `b`) হলো function-এর input। `add(2, 3)` লিখলে `a` হয় `2`, `b` হয় `3`। এভাবে function চালানোকে বলে function **call** করা।
- `return` দিয়ে function result ফেরত দেয়। `return`-এর পরে function-এর বাকি কাজ আর চলে না।

এবার project-এর একটা আসল function, `frontend/src/shared/utils.ts` থেকে (type বাদ দিয়ে দেখানো হলো)। এখানে দুটো নতুন জিনিস দেখবে: `new URL(...)` আর `try` / `catch`। `try` / `catch` ২.১৩-এ বিস্তারিত শিখব। এখন শুধু এটুকু জানো: `new URL(value)` লেখাটাকে একটা web address হিসেবে পড়ার চেষ্টা করে, আর লেখাটা address না হলে error দেয়। `try`-এর ভেতরে error হলে code সোজা `catch`-এ চলে যায়।

```
function isValidUrl(value)
{
    try
    {
        const url = new URL(value);
        return url.protocol === 'http:' || url.protocol === 'https:';
    }
    catch
    {
        return false;
    }
}

console.log(isValidUrl('https://google.com'));
console.log(isValidUrl('hello'));
console.log(isValidUrl('ftp://files.com'));
```

Result:

- `isValidUrl('https://google.com')` দেয় `true` । Address পড়া গেল, আর তার protocol (শুরুর অংশ) হলো `'https:'` ।
- `isValidUrl('hello')` দেয় `false` । 'hello' কোনো address না, তাই `new URL` error দিল আর `catch` থেকে `false` ফেরত এলো।
- `isValidUrl('ftp://files.com')` দেয় `false` । Address পড়া গেল, কিন্তু protocol `'ftp:'`, যেটা `'http:'` বা `'https:'` কোনোটাই না।

Arrow function

Function লেখার আরেকটা ছোট উপায় হলো **arrow function**, `=>` চিহ্ন দিয়ে। `frontend/src/shared/utils.ts`-এর `toDateTimeInputValue` function-এর ভেতরে এমন একটা আছে (type বাদ দিয়ে দেখানো হলো):

```
const pad = (part) => String(part).padStart(2, '0');
```

এটা হুবহু এর সমান:

```
function pad(part)
{
    return String(part).padStart(2, '0');
}
```

`String(part)` number-কে string-এ convert করে। `.padStart(2, '0')` string-টা অন্তত 2 অক্ষরের না হলে সামনে `'0'` বসিয়ে 2 অক্ষরের বানায়। তাই:

- `pad(5)` দেয় `'05'`
- `pad(12)` দেয় `'12'` (আগেই 2 অক্ষর, তাই কিছু বসে না)

তারিখ দেখানোর সময় মাস বা ঘণ্টা `05`-এর মতো দুই অঙ্কে দেখাতে এটা লাগে।

Expression হলো code-এর এমন একটা টুকরো যেটা হিসাব করলে একটা value পাওয়া যায়। যেমন `2 + 3`, `count > 10`, বা `String(part).padStart(2, '0')`।

- `=>`-এর পরে যদি **একটাই expression** থাকে, তাহলে `return` লিখতে হয় না, expression-এর value নিজে থেকেই ফেরত যায়।
- একাধিক লাইন লাগলে `{ }` দিতে হয়, আর তখন `return` লিখতে হয়।

React-এ arrow function সব জায়গায় আছে। যেমন `frontend/src/features/auth/LoginPage.tsx`-এ (type বাদ দিয়ে দেখানো হলো):

```
onChange={(event) => setEmail(event.target.value)}
```

এখানে একটা arrow function বানানো হয়েছে, যেটা `event` নামের একটা parameter নেয় আর `setEmail(...)` call করে। এর মানে কী, সেটা অধ্যায় ৭-এ দেখবে।

Function-কে value হিসেবে পাঠানো: callback

JavaScript-এ function-ও একটা value। তাই একটা function-কে অন্য function-এর **parameter হিসেবে দেওয়া** যায়। এভাবে দেওয়া function-কে বলে **callback**, কারণ পরে সেটাকে "ফিরে ডাকা" (call back) হয়।

```
setTimeout(() => console.log('৩ সেকেন্ড পর'), 3000);
```

`setTimeout` browser-এর নিজের একটা function। এর প্রথম parameter একটা function (এখানে arrow function), আর দ্বিতীয় parameter সময়, **millisecond**-এ (1000 millisecond = 1 সেকেন্ড)। Result: ৩ সেকেন্ড পরে Console-এ দেখাবে `৩ সেকেন্ড পর`।

২.৬ Scope আর closure

Scope

Scope মানে একটা variable code-এর কোন অংশ থেকে দেখা যায়। `const` আর `let` **block scoped**: যে `{ }`-এর ভেতরে তৈরি, শুধু তার ভেতরেই দেখা যায়।

```
if (true)
{
    const message = 'ভেতরে';
    console.log(message);
}
console.log(message);
```

প্রথম `console.log` দেখাবে `ভেতরে`। দ্বিতীয়টা error দেবে (`ReferenceError: message is not defined`), কারণ `message` তৈরি হয়েছিল `{ }`-এর ভেতরে, বাইরে থেকে সেটা দেখা যায় না।

পুরনো `var` এই নিয়ম মানে না। `var` **function scoped**: পুরো function-এর যেকোনো জায়গা থেকে দেখা যায়, `{ }`-এর বাইরে থেকেও। উপরের উদাহরণে `const`-এর জায়গায় `var` লিখলে দ্বিতীয় `console.log`-ও `ভেতরে` দেখাত। এই কারণেই `var` থেকে bug হয়।

Closure

একটা function-এর ভেতরে আরেকটা function বানালে, ভেতরের function বাইরের function-এর variable মনে রাখে, এমনকি বাইরের function শেষ হয়ে যাওয়ার পরেও। একে বলে **closure**।

```
function makeCounter()
{
    let count = 0;
    return () =>
    {
        count = count + 1;
        return count;
    };
}

const next = makeCounter();
console.log(next());
console.log(next());
```

Console-এ দেখাবে `1`, তারপর `2`।

- `makeCounter()` call করলে `count` তৈরি হয়, আর একটা arrow function ফেরত আসে। সেটা রাখা হলো `next`-এ।
- `makeCounter` শেষ হয়ে গেছে, তবুও `next` তার `count`-কে মনে রেখেছে। প্রতিবার `next()` call করলে সেই একই `count` এক করে বাড়ে।

React-এ closure সব জায়গায় আছে। যেমন একটা component-এর ভেতরে বানানো event handler function সেই component-এর variable ব্যবহার করতে পারে closure-এর কারণেই (অধ্যায় ৭ আর ৮-এ এর উদাহরণ দেখবে)।

২.৭ Object

Object হলো **key-value pair**-এর একটা collection। প্রতিটা pair-এ একটা নাম (**key**) আর তার **value** থাকে:

```
const application = {
    id: '42',
    jobPost: 'Backend Engineer',
    isApplied: false,
    salaryMin: 50000,
};

console.log(application.jobPost);
console.log(application['jobPost']);
console.log(application.salaryMin);
```

প্রথম দুটো `console.log` দুটোই দেখাবে `Backend Engineer`, আর তৃতীয়টা `50000`। Object-এর ভেতরের value পাওয়ার দুটো উপায়: dot দিয়ে (`application.jobPost`) অথবা `[ ]`-এর ভেতরে key-কে string হিসেবে দিয়ে (`application['jobPost']`)।

Object-এর প্রতিটা key-value pair-কে বলে একটা **property**। যেমন `jobPost` হলো `application`-এর একটা property।

কোনো property-র value যদি একটা function হয়, তাহলে তাকে বলে **method**। Method call করা হয় `( )` দিয়ে। আগে দেখা `' abc '.trim()`-এ `trim` হলো string-এর একটা method, আর `console.log`-এ `log` হলো `console` object-এর একটা method।

Shorthand

Variable-এর নাম আর key একই হলে একবার লিখলেই হয়:

```
const email = 'a@b.com';
const password = '12345678';

const credentials = { email, password };
```

`credentials` হবে `{ email: 'a@b.com', password: '12345678' }`। অর্থাৎ `{ email, password }` হলো `{ email: email, password: password }`-এর ছোট রূপ। Project-এ এটা আছে `frontend/src/features/auth/LoginPage.tsx`-এর `loginUser({ email, password })`-এ।

২.৮ Array আর তার method

Array হলো একটা list:

```
const pageSizes = [10, 20, 50];
console.log(pageSizes[0]);
console.log(pageSizes[2]);
console.log(pageSizes.length);
```

Result:

- `pageSizes[0]` দেয় `10`। List-এর প্রতিটা জিনিসের একটা নম্বর থাকে, যাকে বলে **index**, আর গোনা শুরু হয় **0** থেকে।
- `pageSizes[2]` দেয় `50`।
- `pageSizes.length` দেয় `3`। `length` হলো array-এর একটা property (তাই `( )` লাগে না), যেটা বলে list-এ কয়টা জিনিস আছে।

এর বিপরীতে নিচের `map`, `filter`, `find`, `forEach` হলো array-এর **method**, তাই `( )` দিয়ে call করতে হয়। প্রতিটাই একটা callback নেয় (২.৫), আর সেটা list-এর প্রতিটা জিনিসের উপর একবার করে চালায়।

React-এ সবচেয়ে বেশি লাগে `map`, `filter` আর `find`। তিনটাই **নতুন array বা value বানায়, আসল array বদলায় না।**

`map`: প্রতিটা জিনিসকে অন্য কিছুতে transform করা

```
const doubled = [1, 2, 3].map((n) => n * 2);
```

`doubled` হবে `[2, 4, 6]`। Callback প্রতিটা number (`n`) নিয়ে তার দ্বিগুণ ফেরত দিল, আর `map` সেগুলো দিয়ে নতুন array বানাল। আসল `[1, 2, 3]` যেমন ছিল তেমনই থাকল।

React-এ list দেখাতে `map` লাগে। `frontend/src/features/applications/ApplicationFilters.tsx` থেকে (এখানে JSX আছে, অধ্যায় ৬):

```
{PAGE_SIZES.map((size) => <option key={size} value={size}>{size}</option>)}
```

`PAGE_SIZES` হলো `[10, 20, 50]`। `map` প্রতিটা number থেকে একটা `<option>` বানায়। Result: dropdown-এ তিনটা option দেখায়: 10, 20, 50।

`filter`: condition মিললে রাখো, না মিললে বাদ দাও

```
const big = [5, 15, 25].filter((n) => n > 10);
```

`big` হবে `[15, 25]`। Callback যার জন্য `true` দিল, শুধু সেগুলো রাখা হলো। `5 > 10` হলো `false`, তাই `5` বাদ গেল।

Project-এ একটা আবেদন delete করার পর list থেকে সেটা বাদ দিতে `frontend/src/features/applications/applications.slice.ts`-এ এই লাইন আছে:

```
state.items = state.items.filter((item) => item.id !== action.payload);
```

`action.payload` হলো delete করা আবেদনের id। `item.id !== action.payload` মানে "এই item-এর id কি delete করা id থেকে আলাদা?" আলাদা হলে `true`, তাই item-টা থাকে। মিলে গেলে `false`, তাই সেটা বাদ যায়। Result: delete করা আবেদনটা ছাড়া বাকি সব নিয়ে নতুন array, আর সেটা `state.items`-এ assign করা হয়।

এখানে `state.items`-এ নতুন value assign করা দেখে মনে হতে পারে আসল data বদলানো হচ্ছে। Redux Toolkit-এর ভেতরে এভাবে লেখা নিরাপদ, কারণ সে পেছনে নিজেই নতুন copy বানিয়ে নেয়। এটা অধ্যায় ১০-এ বুঝবে। এখন শুধু `filter`-এর কাজটা খেয়াল করো।

`find`: condition মেলে এমন প্রথম জিনিস

```
const first = [5, 15, 25].find((n) => n > 10);
const none = [5, 15, 25].find((n) => n > 100);
```

`first` হবে `15` (condition মেলা প্রথম জিনিস; `25`-ও মেলে, কিন্তু `find` প্রথমটা পেয়েই থেমে যায়)। `none` হবে `undefined`, কারণ কোনোটাই 100-এর বেশি না।

`forEach` : প্রতিটার উপর শুধু একটা কাজ চালাও

```
[10, 20, 50].forEach((size) => console.log(size));
```

Console-এ পরপর দেখাবে `10` , `20` , `50` । `forEach` কোনো নতুন array ফেরত দেয় না (ফেরত দেয় `undefined`)। শুধু প্রতিটা জিনিসের উপর কাজ চালাতে চাইলে এটা ব্যবহার হয়। নতুন array দরকার হলে `map` ।

২.৯ Destructuring: object বা array থেকে value বের করা

Object destructuring

```
const user = { name: 'Rahim', email: 'r@x.com' };

const { name, email } = user;
```

এরপর `name` -এর value `'Rahim'` , আর `email` -এর value `'r@x.com'` । এটা এই দুই লাইনের সমান:

```
const name = user.name;
const email = user.email;
```

`{ }` -এর ভেতরে যে নাম লেখা, object থেকে ঠিক সেই key-র value বের করে একই নামের variable-এ রাখা হয়।

React-এ এটা সব জায়গায় আছে। `frontend/src/features/applications/ApplicationsPage.tsx` থেকে (`useAppSelector` অধ্যায় ১০-এ শিখব; এখন শুধু জানো এটা একটা object ফেরত দেয়):

```
const { filters, items, meta, listStatus, listError } = useAppSelector((state) =>
state.applications);
```

এক লাইনেই object থেকে পাঁচটা value বের করে পাঁচটা variable-এ রাখা হলো।

Array destructuring

Array থেকেও একইভাবে value বের করা যায়, তবে `[ ]` দিয়ে। এখানে নাম না, **অবস্থান** (index) দেখে value বসে:

```
const [first, second] = [10, 20];
```

`first` হবে `10`, `second` হবে `20`। প্রথম নাম পায় প্রথম value, দ্বিতীয় নাম পায় দ্বিতীয় value। নামগুলো তুমি যা খুশি দিতে পারো।

React-এর `useState` ঠিক এটাই ব্যবহার করে। `frontend/src/features/auth/LoginPage.tsx` থেকে:

```
const [email, setEmail] = useState('');
```

`useState` দুটো জিনিসের একটা array ফেরত দেয়, আর array destructuring দিয়ে সেগুলো `email` আর `setEmail` নামে রাখা হয়েছে (অধ্যায় ৭)।

Default value

```
const { id = '' } = useParams();
```

এটা `frontend/src/features/applications/ApplicationDetailsPage.tsx` থেকে (type বাদ দিয়ে দেখানো হলো)। `useParams()` URL থেকে value-গুলো একটা object-এ দেয় (অধ্যায় ৯)। Object-এ `id` থাকলে সেটাই আসে, যেমন `'42'`। না থাকলে (`undefined` হলে) `id`-এর value হবে `''`। `= ''` অংশটাকে বলে **default value**।

Function parameter-এ destructuring

`frontend/src/features/applications/StatusBadge.tsx` থেকে (type বাদ দিয়ে, আর ভেতরের JSX-এর বদলে সাধারণ string দিয়ে সহজ করে দেখানো হলো):

```
function StatusBadge({ isApplied })
{
    return isApplied ? 'Applied' : 'Not applied';
}

StatusBadge({ isApplied: true });
```

Function-টা একটা object পায়, আর parameter-এর জায়গাতেই সেটার ভেতর থেকে `isApplied` বের করে নেয়। শেষ লাইনের call `'Applied'` ফেরত দেবে (`? :` চিহ্নের মানে ২.১১-এ দেখবে)। আসল file-এ লেখার জায়গায় JSX দিয়ে রঙিন badge ফেরত দেওয়া হয়। React component-এ props এভাবেই নেওয়া হয় (অধ্যায় ৬)।

২.১০ Template literal: string-এর ভেতর value বসানো

Backtick (`` ` ``) দিয়ে string লিখলে তার ভেতরে `${ }` দিয়ে যেকোনো JavaScript value বসানো যায়। এমন string-কে বলে **template literal**:

```
const page = 3;
const text = `Page ${page}`;
const next = `Next: ${page + 1}`;
```

`text` হবে `'Page 3'`, আর `next` হবে `'Next: 4'`। `${ }`-এর ভেতরের expression হিসাব হয়ে তার value লেখায় বসে যায়।

Project-এ `frontend/src/shared/api.ts`-এ:

```
headers.Authorization = `Bearer ${token}`;
```

`token`-এর value `'abc123'` হলে `headers.Authorization` হবে `'Bearer abc123'` (অধ্যায় ১-এর request header মনে করো)।

২.১১ Condition লেখার ছোট উপায়

Ternary: `condition ? সত্য হলে : মিথ্যা হলে`

```
const isApplied = true;
const label = isApplied ? 'Applied' : 'Not applied';
```

`label` হবে `'Applied'`। `isApplied` `false` হলে হতো `'Not applied'`। এটা ছোট একটা `if`/`else`, তবে এটা একটা expression, তাই এর result সরাসরি variable-এ রাখা যায়।

`&&`: condition truthy হলেই পরেরটা

২.৪-এ `&&` দেখেছ `true`/`false`-এর সাথে। আসলে `&&` বাম পাশের value falsy হলে সেটাই দেয়, আর truthy হলে ডান পাশের value দেয়। যেমন `'x' && 'y'` দেয় `'y'`, আর `'' && 'y'` দেয় `''`।

React-এ এটা খুব বেশি ব্যবহার হয়। `frontend/src/shared/components/Field.tsx` থেকে (JSX):

```
{error && <p className="text-sm text-red-600">{error}</p>}
```

`error`-এ কোনো message থাকলে (truthy) লাল রঙের `<p>`-এ message-টা দেখায়। `error` না থাকলে কিছুই দেখায় না।

`?.`: optional chaining, null হলে থেমে যাও

```
const user = null;
const email = user?.email;
```

`email` হবে `undefined`, কোনো error হবে না। `?.` মানে "বাম পাশের জিনিস `null` বা `undefined` হলে আর ভেতরে ঢুকো না, শুধু `undefined` দাও"। `user` যদি `{ email: 'a@b.com' }` হতো, তাহলে `email` হতো `'a@b.com'`। `?.` ছাড়া `user.email` লিখলে error হতো (`TypeError: Cannot read properties of null`), আর তার পরের code আর চলত না।

`??`: nullish coalescing, null/undefined হলে fallback value

`frontend/src/features/applications/applications.utils.ts` থেকে (সংক্ষেপে):

```
application.applyLink ?? ''
```

`applyLink` যদি `null` বা `undefined` হয়, result `''`। না হলে আসল value, যেমন `'https://jobs.com/apply'`। ডান পাশের value-টাকে বলে **fallback value**, মানে আসলটা না থাকলে যেটা দিয়ে কাজ চালানো হবে।

`??` আর `||`-এর পার্থক্য: `||` বাম পাশের **যেকোনো falsy** value-তে (`0`, `''` সহ) ডান পাশেরটা নেয়, আর `??` শুধু `null` আর `undefined`-এ। যেমন `0 || 10` দেয় `10`, কিন্তু `0 ?? 10` দেয় `0`।

২.১২ Spread আর rest (`...`)

প্রথমে একটা নিয়ম

এই বইয়ে কোনো code block-এর মাঝে যদি শুধু `...` লেখা একটা লাইন দেখো, তার মানে সেখানে কিছু অংশ বাদ রাখা হয়েছে, যাতে code ছোট থাকে। আর নিচে যে spread `...` শিখব, সেটা JavaScript-এর আসল একটা চিহ্ন, যেটা সবসময় কোনো value বা নামের ঠিক আগে বসে (মাঝে কোনো space নেই), যেমন `...filters` বা `...[1, 2]`। কোনটা কোন অর্থে, সেটা context থেকে বোঝা যায়।

Spread: ছড়িয়ে দেওয়া

Spread (`...`) একটা object বা array-এর সব ভেতরের জিনিস "ছড়িয়ে" অন্য জায়গায় বসিয়ে দেয়।

```
const filters = { search: '', page: 3, limit: 10 };
const next = { ...filters, page: 1 };
```

`next` হবে `{ search: '', page: 1, limit: 10 }`, আর `filters` আগের মতোই `{ search: '', page: 3, limit: 10 }` থাকবে।

`...filters` মানে "filters-এর সব key-value এখানে বসাও"। তারপর `page: 1` আগের `page`-এর উপর লিখে দেয়। **একই key দুবার থাকলে পরে যেটা লেখা, সেটাই থাকে।**

Array-তেও একই কাজ করে: `[...[1, 2], 3]` দেয় `[1, 2, 3]`।

React আর Redux-এ এটা খুব গুরুত্বপূর্ণ, কারণ সেখানে আসল object সরাসরি বদলানো হয় না, বদলাতে হলে নতুন copy বানানো হয় (অধ্যায় ৭)।

Shallow copy: একটা সাবধানতা

Spread দিয়ে বানানো copy হলো **shallow copy**: শুধু উপরের স্তরের key-গুলো নতুন object-এ copy হয়। কোনো key-র value যদি নিজেই একটা array বা object হয়, তাহলে নতুন copy-তে সেই **একই** array-টা থাকে, তার copy না।

```
const a = { tags: ['x'] };
const b = { ...a };
b.tags.push('y');
console.log(a.tags);
```

Console-এ দেখাবে `['x', 'y']`! (`push` হলো array-এর শেষে নতুন জিনিস যোগ করার method।) আমরা `b.tags` বদলালাম, কিন্তু `a.tags`-ও বদলে গেল। কারণ `a.tags` আর `b.tags` দুটো আলাদা array না, দুটোই memory-র একই array-কে দেখাচ্ছে। একে বলে দুটো variable-এর **reference** একই। ভেতরের array-ও আলাদা করতে চাইলে সেটাকেও spread করতে হয়: `const b = { ...a, tags: [...a.tags] };`।

Project থেকে: computed key আর object ফেরত দেওয়া arrow function

`frontend/src/features/applications/ApplicationForm.tsx`-এ form-এর একটা field বদলানোর লাইনটা এমন:

```
setValues((previous) => ({ ...previous, [field]: value }));
```

`setValues` অধ্যায় ৭-এ শিখব। এখন শুধু ভেতরের arrow function-টা দেখো। এখানে দুটো নতুন জিনিস আছে।

১. Computed key `[field]`। Object-এ key-কে `[ ]`-এর ভেতরে লিখলে key-র নাম হিসেবে variable-টার **value** বসে, variable-এর নাম না।

```
const field = 'jobPost';
const value = 'Frontend Developer';
const result = { [field]: value };
```

`result` হবে `{ jobPost: 'Frontend Developer' }`। `[ ]` ছাড়া `{ field: value }` লিখলে key-র নাম হতো সত্যি সত্যি `field`। এই কৌশলে একটাই function দিয়ে form-এর যেকোনো field বদলানো যায়: `field`-এ যে নাম আসবে, সেই key-টাই বদলাবে।

২. `({ ... })`: arrow function থেকে object ফেরত দেওয়া। Arrow function-এ `=>`-এর পরে সরাসরি `{` লিখলে JavaScript সেটাকে function-এর body-র `{ }` মনে করে, object না। তাই object ফেরত দিতে চাইলে সেটাকে `( )`-এর ভেতরে রাখতে হয়:

```
const makeUser = (name) => ({ name, isActive: true });
```

`makeUser('Rahim')` দেয় `{ name: 'Rahim', isActive: true }`। `( )` না দিলে function কিছুই ফেরত দিত না (`undefined`)।

সব মিলিয়ে: `previous` হলো form-এর আগের সব value। নতুন object-এ আগের সব value ছড়িয়ে দেওয়া হলো, তারপর শুধু `field` নামের key-টা নতুন `value` দিয়ে বদলানো হলো। যেমন `previous` যদি `{ jobPost: '', cvLink: '' }` হয়, `field` হয় `'jobPost'` আর `value` হয় `'Frontend Developer'`, তাহলে result `{ jobPost: 'Frontend Developer', cvLink: '' }`।

Rest: বাকিগুলো জড়ো করা

একই `...` চিহ্ন অন্য জায়গায় উল্টো কাজ করে। একে বলে **rest**:

`...` কোথায় বসেছে	নাম	কাজ
Function parameter-এ, বা destructuring-এর বাম পাশে	**Rest**	বাকি সব জিনিস **জড়ো** করে একটা object বা array-এ রাখে
অন্য যেকোনো জায়গায় (নতুন object/array বানানোর সময়, JSX-এ)	**Spread**	ভেতরের জিনিসগুলো **ছড়িয়ে** দেয়

একটা ছোট উদাহরণ:

```
const { page, ...others } = { search: '', page: 3, limit: 10 };
```

`page` হবে `3`, আর `others` হবে `{ search: '', limit: 10 }`। `page` বের করে নেওয়ার পর বাকি সব key `others`-এ জড়ো হলো।

এবার project-এর আসল code, `frontend/src/shared/components/Field.tsx` থেকে (type বাদ দিয়ে দেখানো হলো; এখানে JSX আছে):

```
export function Input({ className = '', ...props })
{
    return <input {...props} className={`${CONTROL_CLASSES} ${className}`} />;
}
```

একই function-এ rest আর spread দুটোই আছে:

- **Parameter-এ `...props` (rest):** "যা আসবে তার থেকে `className` আলাদা করে নাও (না এলে `''`), আর বাকি সব কিছু `props` নামের একটা object-এ জড়ো করো"।
- **`<input {...props} />` (spread):** "`props`-এর সব key-value input-এর attribute হিসেবে বসিয়ে দাও"। যেমন `props` যদি `{ type: 'email', id: 'email' }` হয়, তাহলে input পাবে `type="email"` আর `id="email"`।
- **`className={...}`:** এখানে template literal (২.১০) দিয়ে দুটো string জোড়া হয়েছে। `CONTROL_CLASSES` হলো একই file-এ রাখা Tailwind class-এর একটা লম্বা string (`'w-full rounded-md border ...'`; string-এর ভেতরের `...` মানে বাকি class-গুলো বাদ রাখা হয়েছে), যেটা সব input-কে একই রকম দেখায়। তার পরে একটা space, তারপর বাইরে থেকে আসা `className`। যেমন কেউ `className="mt-2"` দিলে শেষে হবে `'w-full rounded-md border ... mt-2'`। মানে সাধারণ style-ও থাকল, বাড়তি style-ও যোগ হলো।

২.১৩ Error সামলানো: `try` / `catch` / `throw`

Code চলার সময় কোনো সমস্যা হলে তাকে বলে **error**। নিজে থেকেও error তৈরি করা যায়:

```
function divide(a, b)
{
    if (b === 0)
    {
        throw new Error('Cannot divide by zero');
    }
    return a / b;
}

try
{
    console.log(divide(10, 2));
    console.log(divide(10, 0));
    console.log('এই লাইন চলবে না');
}
catch (error)
{
    console.log('সমস্যা হয়েছে:', error.message);
}
```

Console-এ দেখাবে `5`, তারপর `সমস্যা হয়েছে: Cannot divide by zero`। তৃতীয় `console.log` কখনো চলবে না।

- `throw` দিয়ে error "ছুঁড়ে দেওয়া" হয়। `throw`-এর পরে function-এর বাকি code আর চলে না।
- `new Error('...')` একটা error object বানায়, যার `message` property-তে লেখাটা থাকে।
- `try`-এর ভেতরে কোথাও error হলে code সাথে সাথে `catch`-এ লাফ দেয়। `catch (error)`-এ `error` হলো ছুঁড়ে দেওয়া জিনিসটা।
- `error`-টা ব্যবহার না করলে শুধু `catch` লেখা যায়, `(error)` ছাড়া। ২.৫-এর `isValidUrl`-এ ঠিক এটাই আছে।
- কোনো `catch` না থাকলে error উপরের দিকে, মানে যে function call করেছিল তার দিকে যেতে থাকে। কোথাও ধরা না পড়লে browser সেটা Console-এ লাল রঙে দেখায়, আর ওই কাজের বাকি অংশ থেমে যায়।

২.১৪ Class

Class হলো একই রকম object বানানোর **blueprint** বা template। একবার blueprint বানিয়ে `new` দিয়ে তার থেকে যত খুশি object বানানো যায়। ২.১৩-এর `new Error(...)`-এ `Error` আসলে JavaScript-এর নিজের একটা class।

আমাদের project-এ নিজেদের বানানো class একটাই, `frontend/src/shared/api.ts`-এ (type বাদ দিয়ে দেখানো হলো; আসল file-এ `readonly status: number;` লাইনও আছে, অধ্যায় ৪):

```
export class ApiError extends Error
{
    constructor(status, message)
    {
        super(message);
        this.name = 'ApiError';
        this.status = status;
    }
}

const error = new ApiError(404, 'Application not found');
console.log(error.message);
console.log(error.status);
```

এই code চালালে Console-এ দেখাবে `Application not found`, তারপর `404`। তবে Console-এ চালাতে চাইলে শুরুর `export` শব্দটা বাদ দিয়ে paste করো, কারণ Console `export` বোঝে না। (`export` কী, সেটা ২.১৫-এ।)

- `extends Error` মানে `ApiError` হলো `Error`-এর একটা **subclass** (child class)। Child class parent class-এর সব কিছু পায়। তাই `ApiError`-এর `message` আছে, আর আমরা বাড়তি যোগ করেছি `status`।
- `constructor` হলো একটা বিশেষ function, যেটা `new ApiError(...)` লেখার সময় নিজে থেকে চলে। `new ApiError(404, '...')`-এর `404` যায় `status`-এ আর লেখাটা যায় `message`-এ।
- `this` মানে "যে object-টা এখন বানানো হচ্ছে"। `this.status = status` মানে "নতুন object-এর `status` property-তে parameter-এর value রাখো"।
- `super(message)` দিয়ে parent class `Error`-এর constructor call করা হয়, যাতে `message` ঠিকমতো সেট হয়।
- `this.name = 'ApiError';` error-এর নাম সেট করে। ফলে Console-এ error দেখালে `ApiError` লেখা আসে।

যেহেতু `ApiError` আসলে একটা `Error`, তাই এটা দিয়েও `throw` করা যায়: `throw new ApiError(404, 'Application not found')`। Server-এর error-কে status code সহ ছুঁড়ে দিতে project-এ ঠিক এটাই করা হয় (অধ্যায় ৩)।

২.১৫ Module: `import` আর `export`

বড় program অনেক file-এ ভাগ করা থাকে। প্রতিটা file-কে বলে একটা **module**। একটা file-এর জিনিস অন্য file-এ ব্যবহার করতে হলে প্রথম file সেটা **export** করে, আর দ্বিতীয় file **import** করে।

`frontend/src/shared/utils.ts` জিনিসটা **দেয়** (export)। নিচে function-এর ভেতরের অংশ আর type বাদ দিয়ে শুধু প্রথম লাইন দেখানো হলো:

```
export function formatDate(value)
```

`frontend/src/features/applications/ApplicationsTable.tsx` জিনিসটা **নেয়** (import):

```
import { buildPath, formatDate } from '@/shared/utils';
```

- `{ }`-এর ভেতরে লেখা হয় ঠিক কোন কোন নামের জিনিস নিতে চাও। এখানে দুটো: `buildPath` আর `formatDate`। এভাবে নাম ধরে export আর import করাকে বলে **named export/import**। এই `{ }` object না, শুধু নামের list।
- `from`-এর পরে file-এর path। খেয়াল করো, শেষে `.ts` লেখা নেই। Tool নিজেই `.ts` বা `.tsx` খুঁজে নেয়।
- `@/` মানে `frontend/src/` folder। তাই `'@/shared/utils'` মানে `frontend/src/shared/utils.ts`। এই shortcut কোথায় সেট করা, সেটা অধ্যায় ৫-এ দেখবে।

আরেক রকম import আছে, `frontend/src/features/auth/auth.thunks.ts` থেকে:

```
import * as authApi from '@/features/auth/auth.api';
```

এর মানে: "ওই file-এর **সব** export একটা `authApi` object-এ নাও"। তখন ব্যবহার করতে হয় `authApi.login(...)` লিখে, মানে object-এর method-এর মতো।

সারসংক্ষেপ

- `console.log` দিয়ে value দেখা যায়, Chrome-এর F12 → Console-এ ছোট code চালানো যায়।
- `const` ব্যবহার করো, `let` কেবল দরকার হলে, আর `var` কখনো না। `const` আর `let` block scoped।
- `=` হলো assign, `===` / `!==` হলো তুলনা। `==` ব্যবহার কোরো না। `if`-এ falsy value: `false`, `0`, `''`, `null`, `undefined`, `NaN`।

- Function input নেয় (parameter) আর `return` দিয়ে result দেয়। Arrow function (`=>`) ছোট রূপ। অন্য function-কে দেওয়া function হলো callback। ভেতরের function বাইরের variable মনে রাখে, এটাই closure।
- Object হলো key-value pair-এর collection। Property-র value function হলে সেটা method।
- `map`, `filter`, `find` নতুন array বা value দেয়, আসলটা বদলায় না। `forEach` কিছু ফেরত দেয় না।
- Destructuring: object থেকে `const { a } = obj`, array থেকে `const [x, y] = arr`।
- `...` parameter-এ বা destructuring-এর বাম পাশে হলে rest (জড়ো করা), অন্যখানে spread (ছড়ানো)। Spread copy shallow। Computed key `[field]` আর `({ })` দিয়ে object ফেরত দেওয়া মনে রাখো।
- `?.` null থেকে বাঁচায়, `??` null/undefined-এ fallback value দেয়। `try` / `catch` / `throw` দিয়ে error সামলানো হয়। Class হলো object-এর blueprint।
- `import` / `export` দিয়ে code অনেক file-এ ভাগ করা হয়।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: `let`, `const`, `var`-এর পার্থক্য কী?
উত্তর:

- `const`-এ একবার value দেওয়ার পর নতুন value assign করা যায় না। তবে value যদি object হয়, তার ভেতরের property বদলানো যায়।
- `let`-এ পরে নতুন value assign করা যায়।
- `const` আর `let` দুটোই **block scoped**, মানে যে `{ }`-এর ভেতরে তৈরি, তার বাইরে থেকে দেখা যায় না। যেমন `if (true) { const x = 1; }` লেখার পর বাইরে `x` ব্যবহার করলে error হয়।
- `var` **function scoped**: পুরো function-এর ভেতরে যেকোনো জায়গা থেকে দেখা যায়, `{ }`-এর বাইরে থেকেও। উপরের উদাহরণে `var x = 1` লিখলে বাইরে থেকেও `x` পাওয়া যেত।
- `var`-এর আরেকটা অদ্ভুত আচরণ হলো **hoisting**: JavaScript `var` variable-কে function-এর শুরুতে তুলে নিয়ে যায়, তাই declare করার আগেই ব্যবহার করলে error না দিয়ে `undefined` দেয়। যেমন `console.log(y); var y = 5;` চালালে `undefined` দেখায়। একই কাজ `let` দিয়ে করলে error দেয়, যেটা আসলে ভালো, কারণ ভুলটা সাথে সাথে ধরা পড়ে।
- এই কারণে আধুনিক code-এ `var` ব্যবহার করা হয় না।

প্রশ্ন: `==` আর `===`-এর পার্থক্য কী?
উত্তর: `==` তুলনার আগে দুই পাশের value-কে একই type-এ convert করে নেয়, তাই `'5' == 5` হয় `true`। `===` type-ও মেলায় আর কিছু convert করে না, তাই `'5' === 5` হয় `false`। `==`-এর নিজে নিজে convert করা থেকে অপ্রত্যাশিত bug হয়, তাই সবসময় `===` (আর আলাদা কিনা দেখতে `!==`) ব্যবহার করা উচিত।

প্রশ্ন: `map` আর `forEach`-এর পার্থক্য কী?

উত্তর: দুটোই array-এর প্রতিটা জিনিসের উপর একটা callback চালায়। `map` প্রতিটা callback-এর return value দিয়ে একটা **নতুন array ফেরত দেয়**, যেমন `[1, 2, 3].map((n) => n * 2)` দেয় `[2, 4, 6]`। `forEach` কিছু ফেরত দেয় না (`undefined`), শুধু কাজ চালায়, যেমন প্রতিটা জিনিস print করা। React-এ data-র list থেকে screen-এ দেখানোর জিনিসের list বানাতে হয়, তাই সেখানে `map` লাগে।

প্রশ্ন: Spread operator দিয়ে copy করলে কি deep copy হয়?

উত্তর: না, **shallow copy** হয়। শুধু উপরের স্তরের key-গুলো নতুন object-এ যায়। কোনো key-র value যদি array বা object হয়, তাহলে নতুন আর পুরনো object দুটোই একই array বা object-এর reference রাখে। যেমন `const a = { tags: ['x'] }; const b = { ...a }; b.tags.push('y');` করলে `a.tags`-ও `['x', 'y']` হয়ে যায়। ভেতরের অংশও আলাদা করতে চাইলে সেটাকেও আলাদা করে spread করতে হয়।

প্রশ্ন: `??` আর `||`-এর পার্থক্য কী?

উত্তর: `||` বাম পাশের যেকোনো falsy value-তে (`0`, `''`, `false`, `null`, `undefined`, `NaN`) ডান পাশের fallback value নেয়। `??` শুধু `null` আর `undefined`-এ fallback নেয়। যেমন `0 || 10` দেয় `10`, কিন্তু `0 ?? 10` দেয় `0`। তাই যেখানে `0` বা `''` একটা বৈধ value, সেখানে `??` নিরাপদ।

প্রশ্ন: Callback কী?

উত্তর: যে function অন্য একটা function-কে parameter হিসেবে দেওয়া হয়, আর পরে সেই function তাকে call করে, সেটাই callback। JavaScript-এ function-ও একটা value, তাই এটা সম্ভব। যেমন `setTimeout(callback, 1000)` এক সেকেন্ড পরে callback চালায়, `array.map(callback)` প্রতিটা জিনিসের জন্য callback চালায়, আর React-এ `onClick={callback}` button click হলে callback চালায়।

প্রশ্ন: Closure কী?

উত্তর: একটা function-এর ভেতরে বানানো function বাইরের function-এর variable মনে রাখে আর ব্যবহার করতে পারে, এমনকি বাইরের function শেষ হয়ে যাওয়ার পরেও। একেই closure বলে। যেমন `makeCounter` function-এর ভেতরে `let count = 0` রেখে একটা arrow function ফেরত দিলে, সেই arrow function যতবার call করা হয়, একই `count` বাড়াতে থাকে। React-এ component-এর ভেতরে বানানো event handler component-এর variable ব্যবহার করতে পারে closure-এর কারণেই।

প্রশ্ন: Rest আর spread-এর পার্থক্য কী?

উত্তর: দুটোই `...` দিয়ে লেখা হয়, কিন্তু কাজ উল্টো। Function parameter-এ বা destructuring-এর বাম পাশে `...` হলো rest: বাকি সব জিনিস একটা object বা array-এ জড়ো করে, যেমন `function Input({ className, ...props })`। নতুন object বা array বানানোর সময় `...` হলো spread: ভেতরের জিনিসগুলো ছড়িয়ে দেয়, যেমন `{ ...filters, page: 1 }`।

অধ্যায় ৩: Asynchronous JavaScript

Server থেকে data আনতে সময় লাগে। এই অধ্যায়ে শিখব JavaScript কীভাবে "অপেক্ষা" করে, অথচ app জমে যায় না। শেষে আমাদের project-এর আসল API code (`frontend/src/shared/api.ts`) লাইন ধরে ধরে পড়ব।

৩.১ Synchronous বনাম asynchronous

Synchronous মানে একটার পর একটা কাজ। আগেরটা শেষ না হলে পরেরটা শুরু হয় না:

```
console.log('১');
console.log('২');
console.log('৩');
```

Console-এ আসে ১, ২, ৩, এই ক্রমে।

ধরো server থেকে data আসতে ২ সেকেন্ড লাগে। যদি synchronous ভাবে অপেক্ষা করতাম, তাহলে ওই ২ সেকেন্ড **পুরো app জমে থাকত**: button চাপা যেত না, scroll হতো না।

Asynchronous মানে: "কাজটা শুরু করে দাও, শেষ হলে আমাকে জানিও। ততক্ষণ আমি অন্য কাজ করি।"

```
console.log('১');
setTimeout(() => console.log('২'), 1000);
console.log('৩');
```

`setTimeout` (অধ্যায় ২.৫) প্রথম parameter-এর callback-টা দ্বিতীয় parameter-এর সময় পরে চালায়। সময় লেখা হয় **millisecond**-এ, তাই `1000` মানে ১ সেকেন্ড। Console-এ আসে ১, ৩, তারপর ১ সেকেন্ড পরে ২। JavaScript `setTimeout`-এর জন্য বসে থাকেনি, সোজা পরের লাইনে চলে গেছে।

৩.২ Promise: "ভবিষ্যতে একটা value দেব"

Promise হলো একটা object, যেটা বলে: "আমি এখন value-টা দিতে পারছি না, কিন্তু পরে দেব (অথবা জানাব যে ব্যর্থ হয়েছি)।" যে কাজ শেষ হতে সময় লাগে, যেমন server থেকে data আনা, সেটা সাথে সাথে result না দিয়ে একটা Promise দেয়।

একটা Promise-এর তিনটা **state** থাকে:

State	মানে
pending	কাজ চলছে, অপেক্ষা করো
fulfilled	সফল, value পাওয়া গেছে
rejected	ব্যর্থ, error পাওয়া গেছে

Promise শুরু হয় pending state-এ, তারপর হয় fulfilled নয়তো rejected হয়। একবার fulfilled বা rejected হয়ে গেলে আর বদলায় না।

এই তিনটা নাম মনে রাখো। Redux Toolkit-এর `createAsyncThunk` হুবহু এই নামগুলোই ব্যবহার করে (অধ্যায় ১১)।

`.then()` আর `.catch()`

Promise-এর result নেওয়ার পুরনো উপায় হলো `.then()` আর `.catch()`। `.then(callback)` বলে "Promise fulfilled হলে এই callback-টা value দিয়ে চালাও"। `.catch(callback)` বলে "rejected হলে এই callback-টা error দিয়ে চালাও"।

নিচের উদাহরণে `fetch` হলো server-এ request পাঠানোর function (৩.৪-এ বিস্তারিত)। এখন শুধু জানো, `fetch` একটা Promise দেয়, যেটা server-এর response এলে fulfilled হয়। এটা চালাতে Chrome Console-টা `http://localhost:5173` tab-এ খুলতে হবে (backend চালু থাকতে হবে)।

```
fetch('/api/health')
    .then((response) => response.json())
    .then((data) => console.log(data))
    .catch((error) => console.log('ব্যর্থ', error));
```

এখানে দুটো `.then` কেন লাগল?

- **Response দুই ধাপে আসে।** প্রথমে server-এর status code আর headers আসে, তখনই `fetch`-এর Promise fulfilled হয়ে `response` object দেয়। কিন্তু response-এর **body** (আসল data) তখনো পুরোটা এসে না-ও পৌঁছাতে পারে। তাই `response.json()` সাথে সাথে data দেয় না, সেটাও **একটা Promise** দেয়। Body পুরো আসার পর সেটা parse করে (parse মানে string পড়ে বুঝে object বানানো) আর fulfilled হয়।
- **`.then` chain করা যায়।** একটা `.then`-এর callback যা return করে, সেটা পরের `.then`-এ যায়। Return করা জিনিস নিজেই Promise হলে পরের `.then` সেটা শেষ হওয়া পর্যন্ত অপেক্ষা করে। তাই প্রথম `.then` `response.json()`-এর Promise return করল, আর দ্বিতীয় `.then` সেটা শেষ হলে parse করা `data` পেল।
- **একটাই** `.catch` শেষে বসিয়ে দিলে chain-এর যেকোনো ধাপে error হলে সেটা ধরা পড়ে।

Result: backend চালু থাকলে Console-এ দেখায় `{ success: true, data: { status: 'ok', timestamp: '...' } }`, যেখানে `timestamp` হলো server-এর এখনকার সময়। `/api/health` এমন একটা endpoint যেটা login ছাড়াই চলে। আর backend বন্ধ থাকলে দেখায় `ব্যর্থ` আর একটা error। (তখন Vite proxy খালি body-সহ `502` দেয়, তাই error-টা আসে `response.json()` থেকে, কারণ খালি লেখাকে JSON হিসেবে parse করা যায় না।)

৩.৩ `async` / `await`: পড়তে সহজ উপায়

একই কাজ, কিন্তু দেখতে synchronous code-এর মতো:

```
async function loadApplications()
{
    try
    {
        const response = await fetch('/api/health');
        const data = await response.json();
        console.log(data);
    }
    catch (error)
    {
        console.log('ব্যর্থ', error);
    }
}

loadApplications();
```

Result আগের উদাহরণের মতোই: backend চালু থাকলে Console-এ `{ success: true, data: { status: 'ok', timestamp: '...' } }` দেখায়, আর backend বন্ধ থাকলে `ব্যর্থ` আর error-টা, ঠিক আগের মতো।

- `async` লেখা function **সবসময় একটা Promise ফেরত দেয়**। Function-এর ভেতরে যা return করা হয়, সেটা ওই Promise-এর value হয়।
- `await` মানে: "এই Promise শেষ না হওয়া পর্যন্ত **এই function-টা** এখানে থামো, তারপর Promise-এর value দাও।" বাকি app কিন্তু চলতে থাকে, button-ও চাপা যায়।
- এখানেও দুটো `await` লেগেছে, একই কারণে: `fetch` একটা Promise দেয়, আর `response.json()`-ও আরেকটা Promise দেয়।
- `await` শুধু `async` function-এর ভেতরে লেখা যায়। একটা ব্যতিক্রম আছে: Chrome Console আর module file-এর একদম বাইরের level-এ (top level) `async` ছাড়াও `await` চলে। পরের উদাহরণগুলো সেভাবে লেখা।

- Promise rejected হলে `await` সেই error **throw** করে, তাই সাধারণ `try` / `catch` দিয়ে ধরা যায় (অধ্যায় ২.১৩)।

৩.৪ `fetch`: server-এ request পাঠানো

`fetch` হলো browser-এর নিজের একটা function, যেটা HTTP request পাঠায় (অধ্যায় ১.৪)। প্রথম parameter URL, দ্বিতীয় parameter একটা object, যেখানে method, headers আর body বলে দেওয়া হয়:

```
const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Accept': 'application/json', 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: 'a@b.com', password: '12345678' }),
});
const json = await response.json();
```

এটা login-এর request পাঠায়। ঠিক email আর password হলে `json` হবে এমন কিছু: `{ success: true, data: { accessToken: '...' } }`।

- **`method`**: `'GET'`, `'POST'` ইত্যাদি। না দিলে `GET`।
- **`headers`**: request-এর বাড়তি তথ্য।
  - `Content-Type: application/json` মানে "আমি body-তে JSON পাঠাচ্ছি"।
  - `Accept: application/json` মানে "আমি response-এ JSON চাই"।
- **`body`**: পাঠানোর data। HTTP দিয়ে শুধু লেখা পাঠানো যায়, object না। তাই **`JSON.stringify`** object-কে JSON string-এ convert করে। উপরের উদাহরণে `JSON.stringify({ email: 'a@b.com', password: '12345678' })` দেয় এই string: `'{"email":"a@b.com","password":"12345678"}'`।
- **`response.json()`**: response body parse করে object বানায়। এটা `JSON.stringify`-এর উল্টো কাজ। Body-টা পুরো আসতে সময় লাগে বলে এটাও একটা Promise দেয়, তাই `await` লাগে।
- **`response.ok`**: status code 200 থেকে 299-এর মধ্যে হলে `true`।
- **`response.status`**: status code-টা, যেমন `200` বা `401`।

`/api/...` কীভাবে backend-এ পৌঁছায়

URL-টা relative (`/api/auth/login`), তাই browser এটা পাঠায় `http://localhost:5173/api/auth/login`-এ, মানে frontend-এর নিজের port-এ। কিন্তু backend তো চলে port 3000-এ! Development-এ এই কাজটা করে **Vite**। `frontend/vite.config.ts`-এ বলে দেওয়া আছে, `/api` দিয়ে শুরু হওয়া সব request `http://localhost:3000`-এ পাঠিয়ে দাও। এভাবে মাঝখানে থেকে request অন্য জায়গায় পাঠিয়ে দেওয়াকে বলে **proxy** (অধ্যায় ৫-এ বিস্তারিত)।

একটা গুরুত্বপূর্ণ বিষয়

`fetch` শুধু **network ব্যর্থ** হলে reject করে (মানে error throw করে), মানে server-এ একদমই পৌঁছানো না গেলে, যেমন internet নেই। Server 400 বা 500 status code ফেরত দিলেও `fetch` সফল ধরে নেয়! কারণ `fetch` -এর চোখে server একটা response তো দিয়েছে। তাই status code বা body নিজে চেক করতে হয়। যেমন login ছাড়া `fetch('/api/applications')` চালালে server 401 আর `{ success: false, error: {...} }` ফেরত দেয়, কিন্তু `.catch` চলে না।

৩.৫ আমাদের project-এর API client: `frontend/src/shared/api.ts`

প্রতিটা জায়গায় `fetch` লিখলে একই code বারবার লিখতে হতো: token যোগ করা, body-কে JSON string-এ convert করা, error চেক করা। তাই সব এক জায়গায় রাখা হয়েছে, এই file-এ।

আগে কয়েকটা ছোট জিনিস

`API_BASE_URL` : `frontend/src/shared/constants.ts` -এ লেখা আছে `export const API_BASE_URL = '/api';` । তাই code-এ শুধু `'/auth/me'` লিখলে `API_BASE_URL + url` হয় `'/api/auth/me'` ।

`getToken()` : একই `api.ts` file-এ আছে। এটা browser-এর **localStorage** থেকে login token পড়ে আনে। localStorage হলো browser-এর ভেতরে ছোট একটা জায়গা, যেখানে লেখা রাখলে page reload বা browser বন্ধ করলেও থেকে যায় (অধ্যায় ১২)। Token থাকলে সেটা string হিসেবে দেয়, না থাকলে `null` ।

`clearToken()` : localStorage থেকে token মুছে দেয়।

`parseJson` : response-এর লেখাকে object-এ convert করার একটা ছোট helper function। Type বাদ দিয়ে দেখানো হলো:

```
function parseJson(text)
{
    try
    {
        return text ? JSON.parse(text) : null;
    }
    catch
    {
        return null;
    }
}
```

`JSON.parse` হলো `JSON.stringify` -এর উল্টো: JSON string থেকে object বানায়। তাই:

- `parseJson('{"success":true}')` দেয় `{ success: true }`।
- `parseJson('')` দেয় `null`। Empty string falsy (অধ্যায় ২.৪), তাই ternary সোজা `null` দেয়।
- `parseJson('<html>...')` দেয় `null`। লেখাটা JSON না, তাই `JSON.parse` error throw করে, আর `catch` থেকে `null` আসে। (`catch`-এর পরে `(error)` নেই, কারণ error-টা ব্যবহার করা হচ্ছে না; অধ্যায় ২.১৩।)

মূল function: `request`

এবার আসল function। Code-এ TypeScript-এর অনেক কিছু আছে, যেমন `<T>`, `: string`, `: HttpMethod`, `body?`, `Promise<...>`, `Record<string, string>`, `: Response`, `parseJson<T>`। এগুলো অধ্যায় ৪-এ শিখব, এখন পার হয়ে যাও। (`HttpMethod` হলো এই file-এর একটা type, যেটা বলে method শুধু `'GET'`, `'POST'`, `'PATCH'` বা `'DELETE'` হতে পারে।) বাকি সব, যেমন `if`, `?.`, `??`, `||`, template literal, `try` / `catch`, অধ্যায় ২-এ শিখেছ। প্রথম লাইনটা সাধারণ ভাষায় পড়লে: "`request` একটা async function, যেটা তিনটা input নেয়: `method` (যেমন `'GET'`), `url` (যেমন `'/auth/me'`), আর ইচ্ছা হলে `body` (পাঠানোর data; `?` মানে এটা না দিলেও চলে)। এটা একটা Promise ফেরত দেয়, যেটা শেষে server-এর সফল response অথবা `null` দেয়।"

```
async function request<T>(method: HttpMethod, url: string, body?: unknown):
Promise<ApiSuccessResponse<T> | null>
{
    const token = getToken();
    const headers: Record<string, string> = { Accept: 'application/json' };
    if (token)
    {
        headers.Authorization = `Bearer ${token}`;
    }
    if (body !== undefined)
    {
        headers['Content-Type'] = 'application/json';
    }

    let response: Response;
    try
    {
        response = await fetch(API_BASE_URL + url, {
            method,
            headers,
            body: body === undefined ? null : JSON.stringify(body),
        });
    }
    catch
    {
        throw new ApiError(0, 'Cannot reach the server');
    }

    if (response.status === 401 && token)
    {
        clearToken();
        window.location.assign(AppRoutes.Login);
    }

    const json = parseJson<T>(await response.text());
    if (json?.success === true)
    {
        return json;
    }
    if (json?.success === false)
    {
        throw new ApiError(json.error.statusCode, json.error.details?.join(', ') ??
json.error.message);
```

```
    }
    if (response.ok)
    {
        return null;
    }
    throw new ApiError(response.status, response.statusText || 'Request failed');
}
```

ধাপে ধাপে কী হচ্ছে:

ধাপ	কী হয়
1	Token থাকলে (login করা থাকলে) `Authorization: Bearer <token>` header যোগ হয় (অধ্যায় ১২)। `Accept` header সবসময় থাকে
2	Body থাকলে `Content-Type: application/json` header যোগ হয়, আর body `JSON.stringify` দিয়ে JSON string-এ convert হয়। Body না থাকলে (যেমন GET) `body: null`
3	`fetch` চলে। Network ব্যর্থ হলে `fetch` throw করে, `catch` সেটা ধরে আর তার বদলে `ApiError(0, 'Cannot reach the server')` throw করে। `0` মানে কোনো status code পাওয়াই যায়নি। (Development-এ কিন্তু request Vite proxy হয়ে যায়। Backend বন্ধ থাকলে proxy নিজেই `502 Bad Gateway` দেয়, তাই তখন `fetch` throw করে না; শেষ ধাপে `ApiError(502, 'Bad Gateway')` আসে।)
4	Token পাঠানো সত্ত্বেও 401 এলে বুঝতে হবে token expire করেছে বা ভুল। তখন token মুছে login page-এ পাঠানো হয়। তারপরও function থামে না, পরের ধাপগুলো চলে আর শেষে `ApiError` throw হয়
5	Response body-কে আগে **text** হিসেবে পড়া হয়, তারপর `parseJson` দিয়ে object বানানো হয়
6	Response `{ success: true, ... }` হলে সেটাই ফেরত দেয়
7	Response `{ success: false, error: ... }` হলে server-এর error message দিয়ে `ApiError` throw করে
8	Body খালি বা JSON না, কিন্তু status সফল (যেমন DELETE-এর 204) হলে `null` ফেরত দেয়
9	বাকি সব ক্ষেত্রে status code আর status-এর লেখা দিয়ে `ApiError` throw করে

কয়েকটা লাইন একটু খুলে বলি:

- **`window.location.assign(AppRoutes.Login)`**: `AppRoutes.Login`-এর value হলো `'/login'` (`frontend/src/shared/constants.ts`)। `window.location.assign` browser-কে ওই address-এ পাঠায়, আর পুরো page নতুন করে load হয়। ফলে app-এর memory-তে থাকা সব পুরনো data মুছে যায়, আর user login page-এ চলে যায়।

- **কেন `response.json()` না লিখে `response.text()` তারপর `parseJson`?** কিছু সফল response-এর body একদম খালি থাকে, যেমন delete করার পর 204 No Content। খালি body-তে `response.json()` চালালে সেটা error throw করে। `response.text()` খালি body-তে শুধু `''` দেয়, আর `parseJson('')` দেয় `null`, কোনো error না। তারপর ধাপ ৪-এ `response.ok` দেখে `null` ফেরত দেওয়া হয়। `response.text()`-ও একটা Promise দেয়, তাই তার আগে `await`।
- **ধাপ 7-এর message**: অধ্যায় ১.৫-এ দেখেছ, backend-এর error-এ `message` থাকে, আর মাঝে মাঝে `details` নামে একটা list থাকে। `json.error.details?.join(', ')` মানে: `details` থাকলে তার সব লেখা কমা দিয়ে জোড়া লাগাও (`join` array-এর একটা method, যেটা সব জিনিসকে একটা string-এ জোড়ে)। `details` না থাকলে `?.` থেমে গিয়ে `undefined` দেয়, তখন `??` fallback হিসেবে `json.error.message` নেয়। যেমন `details` যদি `['email must be an email', 'password is too short']` হয়, message হবে `'email must be an email, password is too short'`।
- **ধাপ 9-এর message**: `response.statusText` হলো status code-এর সাথে আসা ছোট লেখা, যেমন `'Not Found'`। সেটা খালি হলে `||` দিয়ে `'Request failed'` নেওয়া হয়।

সহজ wrapper: `api` object

`request`-এর result থেকে প্রতিবার `data` বের করা ঝামেলা। তাই এর উপরে একটা **wrapper** বানানো হয়েছে, মানে আরেকটা layer যেটা ভেতরে `request` call করে আর ব্যবহার সহজ করে। নিচে `api` object-এর পাঁচটা method-এর দুটো দেখানো হলো, মাঝের `...` লাইনে বাকিগুলো (`getPage`, `patch`, `delete`) বাদ রাখা হয়েছে (অধ্যায় ২.১২-এর নিয়ম):

```
export const api = {
    async get<T>(url: string): Promise<T>
    {
        return dataOf(await request<T>('GET', url));
    },
    async post<T>(url: string, body: unknown): Promise<T>
    {
        return dataOf(await request<T>('POST', url, body));
    },
    ...
};
```

- **Object-এর ভেতরে method-এর ছোট লেখা।** `api` একটা object, আর `get`, `post` তার method (অধ্যায় ২.৭)। `async get(url) { ... }` অনেকটা `get: async (url) => { ... }`-এর মতো, তবে object-এর ভেতরে method হিসেবে ছোট করে লেখা। সামনে `async` থাকায় এগুলো async method।

- `dataOf` : একই file-এর আরেকটা ছোট function। Server-এর response `{ success: true, data: ... }` থেকে শুধু `data`-টা বের করে দেয়। আর response যদি `null` হয় (খালি body), তাহলে `ApiError(500, 'The server sent an empty response')` throw করে, কারণ `get` বা `post`-এর সময় data আসা উচিত ছিল।

ফলে বাকি code-এ শুধু এটুকু লিখলেই হয়। `frontend/src/features/auth/auth.api.ts` থেকে:

```
api.get<User>(ApiEndpoints.AuthMe)
```

- `ApiEndpoints.AuthMe` হলো `frontend/src/shared/constants.ts`-এ রাখা একটা নাম, যার value `'/auth/me'`। এভাবে নাম দিয়ে value-গুলো এক জায়গায় রাখার জিনিসটাকে বলে **enum** (অধ্যায় ৪)।
- `<User>` হলো একটা TypeScript type, যেটা বলে "ফেরত আসা data দেখতে `User`-এর মতো হবে", মানে তাতে `id`, `email`, `name` ইত্যাদি থাকবে (`frontend/src/features/auth/auth.types.ts`)।
- পুরো লাইনটা `GET /api/auth/me` request পাঠায়, আর একটা Promise দেয় যেটা শেষে সরাসরি user-এর object দেয়, যেমন `{ id: '1', email: 'a@b.com', name: 'Rahim', ... }`।

`<T>` বা `<User>` দেখে ভয় পেও না। এটাকে বলে **generic**। মানে "কোন ধরনের data আশা করছ, সেটা বলে দাও"। অধ্যায় ৪-এ বিস্তারিত আছে।

৩.৬ Error message এক জায়গায়: `getErrorMessage`

`try` / `catch`-এর `catch`-এ যে error আসে, সেটা **যেকোনো কিছু** হতে পারে। কারণ JavaScript-এ শুধু `Error` object না, যেকোনো value throw করা যায়: `throw 'oops'` (একটা string), এমনকি `throw 42` (একটা number)। তাই error থেকে সবসময় একটা readable error message বের করতে একটা function আছে, `frontend/src/shared/utils.ts`-এ:

```
export function getErrorMessage(error: unknown): string
{
    if (typeof error === 'string')
    {
        return error;
    }
    if (error instanceof Error)
    {
        return error.message;
    }
    return 'Something went wrong';
}
```

- `error: unknown` : `unknown` একটা TypeScript type, যার মানে "এর type জানা নেই, যেকোনো কিছু হতে পারে" (অধ্যায় ৪)। `catch` -এ আসা error-এর জন্য এটাই ঠিক।
- `: string` (প্রথম লাইনের শেষে): function সবসময় একটা string ফেরত দেবে।
- `typeof error === 'string'` : error কি একটা string? (অধ্যায় ২.৩)
- `error instanceof Error` : error কি `Error` class থেকে বানানো? `ApiError` -ও এর মধ্যে পড়ে, কারণ সে `Error` -কে extend করে (অধ্যায় ২.১৪)।

Result:

Input	Output
`'oops'`	`'oops'`
`new ApiError(0, 'Cannot reach the server')`	`'Cannot reach the server'`
`42`	`'Something went wrong'`

যে message আসে, সেটাই screen-এ user-কে দেখানো হয়।

৩.৭ `void` : "জেনেশুনে অপেক্ষা করছি না"

কখনো কখনো আমরা একটা async function call করি, কিন্তু তার Promise-এর জন্য অপেক্ষা করতে চাই না। যেমন:

```
void loadApplications();
```

`loadApplications()` (৩.৩-এর function) একটা Promise ফেরত দেয়। সামনে `void` লিখে বলা হচ্ছে: "এই Promise-টা আমি ইচ্ছা করেই ignore করছি।" কাজটা ঠিকই চলবে, শুধু আমরা এখানে তার শেষ হওয়ার অপেক্ষা করছি না।

কেন লিখতে হয়? Project-এ **ESLint** নামে একটা tool আছে, যেটা code-এর সম্ভাব্য ভুল ধরে (অধ্যায় ৫)। কোনো Promise-কে `await` ছাড়া এমনি ফেলে রাখলে ESLint সাবধান করে দেয়, কারণ ভুলে `await` বাদ পড়ে যাওয়া খুব সাধারণ bug। `void` লিখলে ESLint বোঝে এটা ভুল না, ইচ্ছাকৃত।

Project-এ এমন লাইন অনেক পাবে, যেমন `frontend/src/features/applications/ApplicationsPage.tsx`-এ:

```
void dispatch(loadApplications(filters));
```

এখানকার `loadApplications` ৩.৩-এর function না, একই নামের আলাদা একটা function: একটা Redux thunk (অধ্যায় ১১)। `dispatch` আর thunk দুটোই Redux-এর জিনিস, অধ্যায় ১০ আর ১১-এ শিখব। এখন শুধু এটুকু জানো: `dispatch(...)` একটা Promise ফেরত দেয়, আর আমরা তার অপেক্ষা করছি না, কারণ result নিজে থেকেই অন্য জায়গায় (Redux store-এ) পৌঁছে যাবে।

সারসংক্ষেপ

- Asynchronous code অপেক্ষার সময় app-কে জমিয়ে রাখে না। `setTimeout`-এর সময় millisecond-এ।
- **Promise**-এর তিন state: **pending**, **fulfilled**, **rejected**।
- `.then` chain করা যায়: একটার return value পরেরটায় যায়। `response.json()` নিজেও Promise দেয়, তাই আলাদা `.then` বা `await` লাগে।
- `async` / `await` দিয়ে async code সহজে পড়া যায়, আর `try` / `catch` দিয়ে error ধরা যায়।
- `fetch` শুধু network ব্যর্থতায় reject করে। 4xx আর 5xx নিজে চেক করতে হয়। Development-এ Vite `/api/...` request backend-এর port 3000-এ proxy করে।
- আমাদের `shared/api.ts` token, JSON আর error সামলানোর কাজ এক জায়গায় করে। `api.get` / `api.post` wrapper সরাসরি `data` দেয়।
- `getErrorMessage` যেকোনো error থেকে readable message বের করে। `void` দিয়ে Promise ইচ্ছা করে ignore করা হয়।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Promise কী? এর state-গুলো কী কী?
উত্তর: Promise হলো এমন একটা object, যেটা ভবিষ্যতে আসবে এমন একটা value-কে represent করে, যেমন server-এর response। এর তিনটা state: pending মানে কাজ চলছে, fulfilled মানে সফলভাবে value পাওয়া গেছে, rejected মানে error হয়েছে। Promise pending থেকে শুরু হয়, আর একবার fulfilled বা rejected হয়ে গেলে আর বদলায় না।

প্রশ্ন: async/await আর .then()-এর পার্থক্য কী?
উত্তর: দুটো একই কাজ করে। async/await আসলে Promise-এর উপরেই বানানো, শুধু লেখার সহজ উপায়। `.then()`-এ callback chain করে লিখতে হয়, আর error ধরতে `.catch()` লাগে। async/await-এ code পড়তে synchronous code-এর মতো লাগে, ওপর থেকে নিচে, আর error ধরা যায় সাধারণ `try` / `catch` দিয়ে। `await` শুধু `async` function-এর ভেতরে লেখা যায়।

প্রশ্ন: fetch কি 404 বা 500-এ error throw করে?
উত্তর: না। fetch শুধু network ব্যর্থ হলে reject করে, মানে server-এ একদমই পৌঁছানো না গেলে, যেমন internet নেই। Server 404 বা 500 দিলেও fetch-এর চোখে একটা response এসেছে, তাই Promise fulfilled হয়, শুধু `response.ok` হয় `false`। তাই status code বা body নিজে চেক করতে হয়। আমাদের project-এ `shared/api.ts`-এর `request` function এই চেক করে `ApiError` throw করে।

প্রশ্ন: `response.json()`-এর আগে আবার `await` কেন লাগে?
উত্তর: `fetch`-এর Promise fulfilled হয় যখন status code আর headers চলে আসে, তখনো body পুরোটা এসে না-ও পৌঁছাতে পারে। `response.json()` body পুরো আসার অপেক্ষা করে তারপর parse করে, তাই সেটাও একটা Promise দেয়, আর তার জন্য আলাদা `await` লাগে।

প্রশ্ন: একাধিক request একসাথে চালাতে চাইলে কী করবে?
উত্তর: `Promise.all` ব্যবহার করব। যেমন:

```
const [me, apps] = await Promise.all([api.get('/auth/me'), api.get('/applications')]);
```

`api.get(...)` call করার সাথে সাথেই দুটো request শুরু হয়ে যায়, একটা আরেকটার জন্য অপেক্ষা করে না। `Promise.all` সেই Promise-গুলোর array নেয়, আর সবগুলো শেষ হওয়া পর্যন্ত wait করে। তারপর একই ক্রমে result-এর একটা array দেয়, যেটা array destructuring দিয়ে `me` আর `apps`-এ রাখা হলো (অধ্যায় ২.৯)। দুটো আলাদা `await` পরপর লিখলে দ্বিতীয় request শুরু হতো প্রথমটা শেষ হওয়ার পরে, তাই বেশি সময় লাগত। কোনো একটা request ব্যর্থ হলে `Promise.all` পুরোটাই reject হয়।

প্রশ্ন: Event loop কী? (সংক্ষেপে)
উত্তর: **Thread** মানে code চালানোর একটা লাইন। JavaScript-এর একটাই thread, মানে সে একসাথে একটাই কাজ করতে পারে (single threaded)। অপেক্ষার কাজগুলো, যেমন timer বা network request, browser নিজে আলাদাভাবে সামলায়। সেগুলো শেষ হলে তাদের callback একটা **queue**-তে গিয়ে বসে। Queue মানে

লাইন, যেখানে আগে যে আসে সে আগে সুযোগ পায়। **Event loop** বারবার দেখে JavaScript-এর বর্তমান কাজ শেষ হলো কিনা। শেষ হলে queue থেকে পরের callback নিয়ে চালায়। এই কারণেই `setTimeout(..., 0)` দিলেও callback সাথে সাথে চলে না, বর্তমান code শেষ হওয়ার পরে চলে।

অধ্যায় ৪: TypeScript

আমাদের প্রজেক্টের সব ফাইল `.ts` বা `.tsx`, মানে **TypeScript**। এই অধ্যায়ে শিখব TypeScript কী, আর project-এর code পড়তে এর কোন অংশগুলো জানা দরকার।

৪.১ TypeScript কেন?

JavaScript-এ নিচের ভুলটা code চালানোর আগে কেউ ধরে না:

```
function formatSalary(info)
{
    return info.salaryMin.toLocaleString();
}

formatSalary({ salaryMn: 5000 });
```

`salaryMin`-এর বানান ভুল করে `salaryMn` লেখা হয়েছে। চালালে app crash করবে, আর সেটা হয়তো user-এর সামনেই।

TypeScript = JavaScript + Type। তুমি বলে দাও কোন variable বা parameter কোন type-এর। একই function TypeScript-এ:

```
interface SalaryInfo
{
    salaryMin: number;
}

function formatSalary(info: SalaryInfo): string
{
    return info.salaryMin.toLocaleString();
}

formatSalary({ salaryMn: 5000 });
```

এখানে `interface SalaryInfo` বলে দিচ্ছে object-টায় `salaryMin` নামে একটা `number` থাকবে (interface নিয়ে বিস্তারিত ৪.৩-এ)। এখন শেষ লাইনে VS Code সাথে সাথে লাল দাগ দেবে: `SalaryInfo`-তে `salaryMn` নামে কোনো field নেই। মানে ভুলটা **code লেখার সময়েই** ধরা পড়ল, runtime-এ (চালানোর সময়) না।

- Browser TypeScript বোঝে না। চালানোর আগে Vite এটাকে সাধারণ JavaScript-এ convert করে, আর সব type মুছে ফেলে।
- `.tsx` মানে TypeScript, যার ভেতরে JSX আছে (React-এর HTML-এর মতো লেখা, অধ্যায় ৬-এ শিখব)।
- পুরো project type-check করার command হলো `npm run typecheck` (ভেতরে এটা `tsc -b` চালায়)।

৪.২ Basic Type

```
const name: string = 'Rahim';
const page: number = 1;
const isApplied: boolean = false;
const tags: string[] = ['remote', 'senior'];
```

`: string` অংশটাকে বলে **type annotation**। `string[]` মানে string-এর array।

প্রায়ই annotation লিখতেই হয় না, কারণ TypeScript নিজেই value দেখে type বুঝে নেয়। একে বলে **type inference**:

```
const page = 1;
```

এখানে TypeScript নিজেই জানে `page`-এর type হলো `number`।

Function-এর type

```
function formatDate(value: string | null): string
{
    return value ? new Date(value).toLocaleDateString() : '—';
}
```

- `new Date(value).toLocaleDateString()` string থেকে একটা তারিখ বানিয়ে সেটাকে local format-এ string হিসেবে দেয়।
- parameter-এর type লেখা হয় নামের পরে: `(value: string | null)`। এখানে `|` মানে "অথবা": value হয় string, নয়তো null (৪.৪-এ বিস্তারিত)।
- return type লেখা হয় `)` এর পরে: `: string`।
- (project-এর আসল `formatDate` একটু বড়, এখানে সংক্ষেপে দেখানো হলো।)

কিছু return না করলে return type হয় `void`। খেয়াল করো, এই `void` একটা **type**, আর অধ্যায় ৩.৭-এর `void` ছিল Promise ignore করার একটা **operator**: একই শব্দ, দুই জায়গায় দুই কাজ।

```
function saveToken(token: string): void
{
    localStorage.setItem(TOKEN_STORAGE_KEY, token);
}
```

এখানে `TOKEN_STORAGE_KEY` হলো `shared/constants.ts`-এর একটা string constant।

Arrow function-এর type

Arrow function-এও একই নিয়ম, শুধু লেখার জায়গা একটু আলাদা:

```
const add = (a: number, b: number): number => a + b;

const sayHello = (): void => console.log('hello');
```

- `(a: number, b: number)`: parameter আর তাদের type।
- `): number =>`: `)` এর পরে, `=>`-এর আগে return type।
- কোনো parameter না থাকলে `()`, আর কিছু return না করলে `: void`। তাই লেখা হয় `(): void => ...`।

এই `(): void =>` ধরনটা পুরো বইয়ে বারবার দেখবে, বিশেষ করে button-এর click handler-এ।

আমাদের project-এ **প্রতিটা function-এর return type লেখা বাধ্যতামূলক** (এটা একটা ESLint rule, অধ্যায় ৫)। এতে code পড়লেই বোঝা যায় function কী return করে।

একটা ব্যতিক্রম আছে: যে arrow function সরাসরি অন্য একটা function-এ argument হিসেবে দেওয়া হয় (যেমন `[1, 2].map((n) => n * 2)`-এর ভেতরের `(n) => n * 2`), তার type TypeScript আশেপাশ থেকেই জেনে যায়। তাই সেখানে return type না লিখলেও ESLint কিছু বলে না। বইয়ের অনেক callback-এ (map, filter, পরে `useAppSelector`) তাই return type নেই।

৪.৩ `interface`: Object-এর shape

`frontend/src/features/auth/auth.types.ts`:

```
export interface User
{
    id: string;
    email: string;
    name: string | null;
    createdAt: string;
    updatedAt: string;
}
```

এখন কোথাও `User` লিখলে TypeScript জানে এই object-এ ঠিক এই ৫টা field আছে, আর কোনটার type কী। `user.emial` লিখলে লাল দাগ পড়বে।

Optional field: `?`

```
export interface RegisterRequest extends LoginRequest
{
    name?: string;
}
```

- `name?`: এই field থাকতেও পারে, না-ও পারে।
- `extends LoginRequest`: `LoginRequest`-এর সব field (`email`, `password`) এখানেও আছে, সাথে নতুন `name`। একে বলে **inheritance**: এক interface অন্যটার field পেয়ে যায়।

৪.৪ Union Type: "এটা অথবা ওটা"

```
name: string | null;
```

`name` হয় string, অথবা `null`।

Literal union মানে নির্দিষ্ট কয়েকটা value-এর মধ্যে একটা:

```
type HttpMethod = 'GET' | 'POST' | 'PATCH' | 'DELETE';
type ButtonVariant = 'primary' | 'secondary' | 'danger';
```

`const variant: ButtonVariant = 'primry';` লিখলে বানান ভুলটা TypeScript সাথে সাথে ধরে ফেলবে, কারণ `'primry'` ওই তিনটার কোনোটা না।

Function type (variable বা prop-এর জন্য)

একটা variable বা prop-এ function রাখলে তার type লেখা হয় এভাবে:

```
type PageChangeHandler = (page: number) => void;
```

মানে: এমন একটা function, যেটা একটা `number` নেয় আর কিছু return করে না। অধ্যায় ৬-এ props-এ এটা দেখবে।

`type` বনাম `interface`

দুটো দিয়েই type-এর নাম দেওয়া যায়।

- `interface` শুধু object-এর shape বলার জন্য।
- `type` যেকোনো কিছুর জন্য: union, function, object।

আমাদের project-এ object-এর জন্য `interface`, আর বাকি সবের জন্য `type` ব্যবহার করা হয়েছে।

৪.৫ Enum: নাম দেওয়া নির্দিষ্ট value-এর set

`frontend/src/shared/constants.ts` (সংক্ষেপে):

```
export enum AppRoutes
{
    Home = '/',
    Login = '/login',
    Applications = '/applications',
    ApplicationDetails = '/applications/:id',
    ...
}
```

এখন code-এ `'/applications'` হাতে না লিখে লেখা হয় `AppRoutes.Applications`। সুবিধা:

- বানান ভুল হলে TypeScript ধরে।
- URL বদলাতে চাইলে শুধু এক জায়গায় বদলালেই চলে।
- VS Code-এ `AppRoutes.` লিখলেই সব option-এর তালিকা দেখায়।

Project-এর আরও কিছু enum:

Enum	কোথায়	Value
`ApiEndpoints`	`shared/constants.ts`	সব API path
`RequestStatus`	`shared/types.ts`	`Idle`, `Loading`, `Succeeded`, `Failed`
`ApplyMedium`	`features/applications/applications.constants.ts`	`Link = 'LINK'`, `Mail = 'MAIL'`
`SortField`, `SortOrder`, `StatusFilter`	একই ফাইলে	sort আর filter-এর option

৪.৬ Generic: type-এর জায়গায় পরে বসবে

ধরো একটা function যেকোনো ধরনের data আনতে পারে। আগে থেকে জানি না সেটা `User` হবে নাকি `JobApplication`। তখন type-টাকেই একটা **parameter** বানানো হয়:

```
async get<T>(url: string): Promise<T>
```

`<T>` হলো একটা খালি ঘর, যেখানে পরে একটা type বসবে। call করার সময় ঘরটা ভরাট করা হয়:

```
const user = await api.get<User>(ApiEndpoints.AuthMe);
const application = await api.get<JobApplication>('/applications/42');
```

প্রথম লাইনে `user`-এর type হয় `User`, আর দ্বিতীয় লাইনে `application`-এর type হয় `JobApplication`। একই function, কিন্তু result-এর type আলাদা।

`Promise<T>` মানে: এমন একটা Promise, যেটা শেষে `T` type-এর value দেবে (Promise কী, তা অধ্যায় ৩-এ দেখেছি)।

Project-এর আরেকটা generic, `frontend/src/shared/types.ts`:

```
export interface Page<T>
{
    items: T[];
    meta: PaginationMeta;
}
```

`Page<JobApplication>` মানে application-এর একটা page: `items` হলো `JobApplication`-এর array।

৪.৭ Utility Type: পুরনো type থেকে নতুন type

TypeScript-এর কিছু built-in tool আছে, যেগুলো দিয়ে একটা type থেকে আরেকটা type বানানো যায়:

Utility	মানে	উদাহরণ
`Partial<T>`	সব field optional	`Partial<ApplicationPayload>`: শুধু কিছু field পাঠিয়ে update
`Omit<T, 'a'>`	`a` field বাদ দিয়ে বাকি সব	`Omit<JobApplication, 'id' | 'createdAt' | 'updatedAt'>`
`Record<K, V>`	`K` type-এর key, `V` type-এর value	`Record<ButtonVariant, string>`
`ReturnType<F>`	function যা return করে তার type	`ReturnType<typeof formatDate>` হলো `string`

`typeof formatDate` মানে "formatDate function-টার type"। Project-এ `ReturnType`-এর আসল ব্যবহার আছে `frontend/src/app/store.ts`-এ, `ReturnType<typeof store.getState>`; Redux chapter-এ (অধ্যায় ১০) সেটা দেখবে।

`frontend/src/features/applications/applications.types.ts` থেকে:

```
export type ApplicationPayload = Omit<JobApplication, 'id' | 'createdAt' | 'updatedAt'>;
```

Server-এ নতুন application পাঠানোর সময় `id` আর তারিখগুলো পাঠাই না, ওগুলো server নিজেই বানায়। তাই সেগুলো বাদ দিয়ে নতুন type বানানো হলো। এতে সব field দ্বিতীয়বার লিখতে হলো না।

একটু জটিল উদাহরণ

```
export type ApplicationFormErrors = Partial<Record<keyof ApplicationFormValues,
string>>;
```

ভেতর থেকে বাইরের দিকে পড়ো:

1. `keyof ApplicationFormValues`: form-এর সব field-এর নাম, মানে `'jobPost' | 'jobPostLink' | 'deadline' | ...`।
2. `Record<..., string>`: প্রতিটা field-এর নামের জন্য একটা string, মানে error message।
3. `Partial<...>`: সব optional, কারণ সব field-এ error থাকে না।

শেষে এই type দাঁড়ায় এমন:

```
{
    jobPost?: string;
    jobPostLink?: string;
    deadline?: string;
    ...
}
```

Indexed access: `Type['field']`

কোনো object type থেকে একটা নির্দিষ্ট field-এর type বের করতে:

```
type AppliedType = ApplicationFormValues['isApplied'];
```

এখানে `AppliedType` হলো `boolean`, কারণ `ApplicationFormValues`-এ `isApplied: boolean`। অধ্যায় ৭-এ এটা কাজে লাগবে।

৪.৮ Type Narrowing: TypeScript-কে প্রমাণ দেখানো

যখন কোনো value-এর type একাধিক হতে পারে, তখন `if` দিয়ে check করলে TypeScript নিজেই type ছোট করে নেয়। একে বলে **narrowing**:

```
function formatDate(value: string | null): string
{
    if (!value)
    {
        return '–';
    }
    return new Date(value).toLocaleDateString();
}
```

`if` -এর পরের লাইনে পৌঁছানো মানে `value` null না। তাই TypeScript জানে এখানে `value` নিশ্চিতভাবে `string`।

Discriminated union

আমাদের server-এর response দুই রকম হতে পারে। `frontend/src/shared/types.ts` -এ (সংক্ষেপে):

```
export interface ApiSuccessResponse<T>
{
    success: true;
    data: T;
}

export interface ApiErrorResponse
{
    success: false;
    error: { statusCode: number; message: string; };
}

export type ApiResponse<T> = ApiSuccessResponse<T> | ApiErrorResponse;
```

খেয়াল করো, `success` -এর type এখানে শুধু `boolean` না, একদম নির্দিষ্ট `true` অথবা `false`। নিচের উদাহরণে ধরো `getUserResponse()` একটা function, যেটা server থেকে `User` -এর response এনে দেয়:

```
const json: ApiResponse<User> = await getUserResponse();

if (json.success === false)
{
    console.log(json.error.message);
}
else
{
    console.log(json.data.email);
}
```

- `success === false` check করার পরে TypeScript জানে এটা `ApiErrorResponse`, তাই `json.error` লেখা নিরাপদ।
- `else`-এর ভেতরে জানে এটা `ApiSuccessResponse`, তাই `json.data` নিরাপদ।

একটা common field (`success`) দেখে কোন type সেটা বোঝার এই কৌশলের নাম **discriminated union**।

`unknown` বনাম `any`

- `any` মানে "type-check বন্ধ"। এটা বিপজ্জনক, আর আমাদের project-এ নিষিদ্ধ।
- `unknown` মানে "জানি না কী, তাই ব্যবহারের আগে check করতে হবে"। এটা নিরাপদ।

`catch`-এ কী আসবে কেউ জানে না, তাই সেখানে `unknown`:

```
catch (error: unknown)
{
    console.log(getErrorMessage(error));
}
```

`getErrorMessage` (অধ্যায় ৩.৬) ভেতরে `typeof` আর `instanceof` দিয়ে narrowing করে।

৪.৯ `import type`

```
import type { User } from '@/features/auth/auth.types';
```

`@/` মানে `src/` folder (অধ্যায় ৫.২-এ বিস্তারিত)। শুধু type import করলে `import type` লেখা হয়। Runtime-এ type-এর কোনো অস্তিত্ব থাকে না, তাই এই import পুরোপুরি মুছে ফেলা হয়। ফলে final JavaScript file-এ (যাকে বলে **bundle**) বাড়তি কিছু ঢোকে না।

আমাদের project-এ এটা বাধ্যতামূলক: শুধু type import করলে `import type` না লিখলে error আসে।

৪.১০ `as const` (পরিচিতির জন্য)

```
const SIZES = [10, 20, 50];
const FIXED_SIZES = [10, 20, 50] as const;
```

- `SIZES` -এর type হলো `number[]` : যেকোনো number, আর পরে বদলানো যায়।
- `FIXED_SIZES` -এর type হলো `readonly [10, 20, 50]` : ঠিক এই তিনটা value, আর **readonly**, মানে বদলানো যাবে না।

৪.১১ Strict mode

আমাদের TypeScript config-এ (অধ্যায় ৫.৩) `"strict": true` আর আরও কিছু কড়া নিয়ম চালু আছে:

- `strictNullChecks` (strict-এর অংশ): `null` বা `undefined` হতে পারে এমন জিনিস check না করে ব্যবহার করা যায় না।
- `noUncheckedIndexedAccess` : array বা object থেকে index দিয়ে কিছু নিলে TypeScript ধরে নেয় সেটা না-ও থাকতে পারে:

  ```
  const tags: string[] = [];
  const first = tags[0];
  ```

  এখানে `first` -এর type `string | undefined` , কারণ array খালিও হতে পারে। তাই ব্যবহারের আগে check করতে হয়।

সারসংক্ষেপ

- TypeScript ভুলগুলো **runtime-এর আগেই** ধরে ফেলে।
- Parameter আর return type লেখা হয় `(a: number): string` , আর arrow function-এ `(a: number): string => ...` ।
- `interface` object-এর shape বলে, আর `type` যেকোনো type-এর নাম দেয়।
- Union (`A | B`) মানে এটা অথবা ওটা। Enum হলো নাম দেওয়া নির্দিষ্ট value।
- Generic (`<T>`) মানে type-টা পরে বসবে।

- `Partial`, `Omit`, `Record` দিয়ে পুরনো type থেকে নতুন type বানানো যায়।
- `if` দিয়ে check করলে TypeScript type narrow করে নেয়।
- `any` নয়, `unknown` ব্যবহার করো।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: interface আর type-এর পার্থক্য কী?
উত্তর: দুটোই object-এর shape বলতে পারে। union (`'a' | 'b'`) শুধু `type` দিয়েই বানানো যায়, আর function type লেখা `type` দিয়ে সহজ। interface `extends` দিয়ে আরেকটা interface-এর field নিতে পারে। সাধারণ নিয়ম হলো object-এর জন্য interface, বাকি সবের জন্য type।

প্রশ্ন: any আর unknown-এর পার্থক্য কী?
উত্তর: `any`-তে TypeScript কোনো check করে না, যা খুশি করা যায়। `unknown`-এ কিছু করার আগে type check (narrowing) করতে হয়। তাই unknown নিরাপদ।

প্রশ্ন: Generic কেন দরকার?
উত্তর: একই function বা type যাতে আলাদা আলাদা type-এর data-র সাথে কাজ করতে পারে, অথচ type-safety না হারায়। যেমন `api.get<User>()` আর `api.get<JobApplication>()` একই function, কিন্তু result-এর type আলাদা।

প্রশ্ন: Type narrowing কী?
উত্তর: `typeof`, `instanceof`, `=== null`, বা কোনো common field (যেমন `success`) দিয়ে check করার পরে TypeScript নিজেই variable-এর type ছোট করে নেয়। যেমন `if (!value) return;` এর পরে `value` আর null থাকে না।

প্রশ্ন: Partial আর Omit কী?
উত্তর: `Partial<T>` সব field optional করে দেয়। `Omit<T, K>` `K` নামের field-গুলো বাদ দিয়ে নতুন type বানায়।

প্রশ্ন: enum কী? এর বিকল্প কী?
উত্তর: নাম দেওয়া নির্দিষ্ট value-এর set হলো enum, যেমন `AppRoutes.Login`। বিকল্প হলো string literal union (`'a' | 'b'`), অথবা `as const` দেওয়া object।

অধ্যায় ৫: Tools — যেগুলো ছাড়া project চলে না

Code লেখার আগে জানতে হবে project-এ কোন কোন tool আছে, আর প্রতিটা কী কাজ করে। এই অধ্যায়ে কোনো React নেই, শুধু tool।

৫.১ Node.js আর npm

- **Node.js** হলো browser-এর বাইরে, তোমার computer-এ JavaScript চালানোর environment। আমাদের frontend চলে browser-এ, কিন্তু তাকে **বানানোর tool-গুলো** (Vite, TypeScript, ESLint) চলে Node-এ।
- **npm** (Node Package Manager) দিয়ে অন্যদের লেখা library ডাউনলোড করা হয়। এই library-গুলোকে বলে **package**।

Command	কাজ
`npm install`	`package.json`-এ লেখা সব package ডাউনলোড করে `node_modules/` folder-এ রাখে
`npm install react-router`	নতুন একটা package যোগ করে
`npm run dev`	`package.json`-এর `"scripts"`-এ লেখা `dev` command চালায়

`package.json`

এটা project-এর পরিচয়পত্র। এতে থাকে project-এর নাম, কোন কোন package লাগবে, আর কোন কোন command আছে। `frontend/package.json` (সংক্ষেপে):

```
{
    "scripts": {
        "dev": "vite",
        "build": "tsc -b && vite build",
        "lint": "eslint .",
        "typecheck": "tsc -b",
        "format": "dprint fmt",
        "format:check": "dprint check"
    },
    "dependencies": {
        "react": "^19.2.8",
        "react-router": "^8.4.0",
        "@reduxjs/toolkit": "^2.13.0",
        ...
    },
    "devDependencies": {
        "typescript": "~6.0.2",
        "vite": "^8.3.0",
        "eslint": "^10.10.0",
        ...
    }
}
```

- `scripts` : ছোট নামে লম্বা command। `npm run typecheck` লিখলে আসলে চলে `tsc -b` ।
- `dependencies` : যা app চালাতে লাগে, যেমন React।
- `devDependencies` : যা শুধু development-এর সময় লাগে, যেমন TypeScript আর ESLint। User-এর browser-এ এগুলো কখনো যায় না।
- `^19.2.8` মানে 19.x.x-এর যেকোনো নতুন version চলবে, কিন্তু 20 না।
- `~6.0.2` আরও কড়া: শুধু 6.0.x-এর নতুন version চলবে, 6.1 না।
- `package-lock.json` প্রতিটা package-এর **হুবহু** version লিখে রাখে, যাতে সবার computer-এ একই জিনিস install হয়।

আমাদের project-এর scripts

Command	ভেতরে কী চলে	কাজ
`npm run dev`	`vite`	Development server চালু করে (৫.২)
`npm run build`	`tsc -b && vite build`	আগে type-check, তারপর production build (**production** মানে আসল user-রা যে version ব্যবহার করবে, server-এ বসানো)

Command	ভেতরে কী চলে	কাজ
`npm run typecheck`	`tsc -b`	পুরো project-এর type error খোঁজে (৫.৩)
`npm run lint`	`eslint .`	Code-এর ভুল আর খারাপ অভ্যাস খোঁজে (৫.৪)
`npm run format`	`dprint fmt`	সব file একই style-এ সাজায় (৫.৫)

`&&` মানে "আগেরটা সফল হলে তবেই পরেরটা চালাও"। তাই type error থাকলে build হবেই না।

আমাদের project-এর মূল package

Package	কাজ
`react`, `react-dom`	React নিজে, আর browser-এ দেখানোর অংশ
`react-router`	Page আর URL (অধ্যায় ৯)
`@reduxjs/toolkit`, `react-redux`	পুরো app-এর shared data রাখা (অধ্যায় ১০–১১)
`@tiptap/react`, `@tiptap/starter-kit`	Rich text editor (অধ্যায় ১৩)
`tailwindcss`	Style (৫.৬)
`typescript`	Type-check
`vite`	Dev server আর build
`eslint`	Code-এর ভুল আর খারাপ অভ্যাস ধরা
`dprint`	Code format করা

৫.২ Vite: Development server আর build tool

Browser TypeScript বা JSX বোঝে না (JSX হলো React-এ HTML-এর মতো করে লেখা, অধ্যায় ৬)। **Vite** দুটো কাজ করে:

1. `npm run dev`: একটা local server (http://localhost:5173) চালু করে। File save করলে browser-এ সাথে সাথে পরিবর্তন দেখা যায়, পুরো page reload না করেই। একে বলে **HMR** (Hot Module Replacement)।

2. `npm run build` : পুরো app-কে ছোট, দ্রুত সাধারণ JavaScript/CSS file-এ convert করে `dist/` folder-এ রাখে। আসল server-এ এটাই বসানো হয়।

Vite type-check করে না

এটা নতুনদের জন্য খুব গুরুত্বপূর্ণ। `npm run dev` চলার সময় Vite TypeScript file থেকে শুধু type-গুলো মুছে ফেলে (strip করে), কিন্তু type ঠিক আছে কিনা **check করে না**। মানে type error থাকলেও page দিব্যি চলবে।

Type error দেখার দুটো উপায়:

- VS Code-এ লাল দাগ।
- Terminal-এ `npm run typecheck` ।

`frontend/vite.config.ts`

```
export default defineConfig({
    plugins: [react(), tailwindcss()],
    resolve: {
        alias: {
            '@': fileURLToPath(new URL('./src', import.meta.url)),
        },
    },
    server: {
        port: 5173,
        proxy: {
            '/api': 'http://localhost:3000',
        },
    },
});
```

(উপরের `import` লাইনগুলো বাদ দেওয়া হয়েছে।)

অংশ	কাজ
`plugins`	Vite-কে React (JSX) আর Tailwind বোঝার ক্ষমতা দেয়
`alias '@'`	`@/shared/utils` লিখলে মানে হয় `src/shared/utils` (নিচে বিস্তারিত)
`server.port`	Dev server চলবে 5173 port-এ
`proxy`	`/api` দিয়ে শুরু হওয়া request backend-এ (3000 port-এ) পাঠিয়ে দেয়

`fileURLToPath(new URL('./src', import.meta.url))` লাইনটা শুধু `src` folder-এর পুরো path বের করে। মুখস্থ করার দরকার নেই।

Proxy কেন?

Frontend চলে 5173 port-এ, আর backend 3000 port-এ। Port আলাদা মানে browser-এর চোখে এরা আলাদা **origin** (origin = protocol + domain + port)।

Browser-এর একটা security নিয়ম আছে, যার নাম **CORS** (Cross-Origin Resource Sharing)। এক origin-এর page অন্য origin-এ request পাঠালে request ঠিকই যায়, কিন্তু server স্পষ্টভাবে অনুমতি না দিলে browser সেই **response JavaScript-কে পড়তে দেয় না**, block করে দেয়।

Proxy থাকলে frontend request পাঠায় নিজের origin-এই: `http://localhost:5173/api/...`। Vite server সেটা চুপচাপ backend-এ পাঠিয়ে দেয়, আর উত্তর ফেরত আনে। Browser দেখে সব একই origin থেকে আসছে, তাই কোনো CORS সমস্যা হয় না।

`@/` alias কেন?

Alias ছাড়া গভীর folder থেকে import করতে লিখতে হতো:

```
import { api } from '../../../shared/api';
```

Alias দিয়ে লেখা হয়:

```
import { api } from '@/shared/api';
```

File যে folder-এই থাকুক, import লাইন একই থাকে। Alias দুই জায়গায় বলতে হয়: TypeScript-কে `tsconfig.app.json`-এর `paths`-এ, আর Vite-কে `vite.config.ts`-এর `resolve.alias`-এ।

৫.৩ `tsconfig`: TypeScript-এর নিয়মবই

Frontend-এ তিনটা file আছে:

- `tsconfig.json`: নিজে কোনো নিয়ম রাখে না, শুধু বাকি দুটোকে দেখিয়ে দেয়।
- `tsconfig.app.json`: `src/` folder-এর app code-এর নিয়ম।
- `tsconfig.node.json`: `vite.config.ts`-এর নিয়ম (কারণ এটা browser-এ না, Node-এ চলে)।

`tsconfig.app.json`-এর গুরুত্বপূর্ণ setting (সংক্ষেপে)। খেয়াল করো, সব নিয়ম থাকে `"compilerOptions"`-এর ভেতরে:

```
{
    "compilerOptions": {
        "jsx": "react-jsx",
        "verbatimModuleSyntax": true,
        "noEmit": true,
        "paths": {
            "@/*": ["./src/*"]
        },
        "strict": true,
        "noUncheckedIndexedAccess": true,
        "noImplicitReturns": true,
        "noUnusedLocals": true,
        ...
    },
    "include": ["src"]
}
```

Setting	মানে
`jsx: "react-jsx"`	JSX-কে React-এর function call-এ convert করার নিয়ম
`verbatimModuleSyntax`	শুধু type import করলে `import type` লিখতে বাধ্য করে (অধ্যায় ৪.৯)
`noEmit`	TypeScript নিজে কোনো JavaScript file বানাবে না, শুধু check করবে। File বানানোর কাজ Vite-এর
`paths`	`@/` alias TypeScript-কে জানানো
`strict`	সব কড়া check চালু (অধ্যায় ৪.১১)
`noUncheckedIndexedAccess`	`array[0]` লিখলে type হয় `T | undefined`, কারণ array খালি হতে পারে
`noImplicitReturns`	Function-এর কোনো পথে `return` বাদ পড়লে error
`noUnusedLocals`	Declare করে ব্যবহার না করা variable থাকলে error
`include: ["src"]`	শুধু `src` folder-এর file check হবে

`tsc -b` কী?

`tsc` হলো TypeScript-এর নিজের command। `-b` মানে "build mode": `tsconfig.json` যে file-গুলো দেখিয়ে দেয় (app আর node দুটোই), সবগুলো একসাথে check করে। `noEmit` থাকায় কোনো file তৈরি হয় না, শুধু error থাকলে দেখায়। তোমাকে সরাসরি `tsc -b` লিখতে হবে না, `npm run typecheck` দিলেই হবে।

৫.৪ ESLint: code-এর ভুল ধরার tool

TypeScript ধরে **type-এর** ভুল। **ESLint** ধরে **খারাপ অভ্যাস আর সম্ভাব্য bug**, যেগুলো type-এর দিক থেকে ঠিক হলেও সমস্যা করতে পারে। চালানোর command: `npm run lint` (ভেতরে চলে `eslint .`, মানে পুরো folder check করো)।

`switch` হলো অনেকগুলো `if` / `else`-এর ছোট লেখা; একটা value-এর সাথে কয়েকটা case মেলায়।

প্রতিটা check-কে বলে একটা **rule**। আমাদের `frontend/eslint.config.js`-এর কিছু rule:

Rule	কী বলে
`explicit-function-return-type`	প্রতিটা function-এর return type লিখতে হবে (অধ্যায় ৪.২)
`no-explicit-any`	`any` ব্যবহার নিষেধ
`no-non-null-assertion`	`value!` নিষেধ (`!` দিয়ে "বিশ্বাস করো, এটা null না" বলে TypeScript-কে জোর করা যাবে না)
`consistent-type-imports`	শুধু type import করলে `import type`
`switch-exhaustiveness-check`	`switch`-এ enum-এর সব case ধরতে হবে
`no-floating-promises`	Promise ফেলে রাখা যাবে না, তাই ইচ্ছা করে ফেলে দিলে `void` লিখি (অধ্যায় ৩.৭)
`react-hooks/rules-of-hooks`	Hook-এর নিয়ম মানতে হবে। Hook শুরু হবে অধ্যায় ৭-এ (`useState`), নিয়ম অধ্যায় ৮-এ
`react-hooks/exhaustive-deps`	`useEffect`-এর dependency list ঠিক আছে কিনা (অধ্যায় ৮)

শেষের দুটো rule এখন পুরো বোঝার দরকার নেই, পরে এগুলোর কাছে ফিরে আসব। প্রথম পাঁচটা rule আমরা নিজেরা config file-এর `rules`-এ লিখেছি। শেষ তিনটা নিজেরা লিখিনি, এগুলো আসে `tseslint.configs.strictTypeChecked` আর `reactHooks.configs.flat.recommended` নামের তৈরি rule-set থেকে।

৫.৫ dprint: code format করা

সবাই আলাদা style-এ code লিখলে পড়তে কষ্ট হয়। **dprint** নিজে থেকেই সব file একই style-এ সাজায়। Setting থাকে `frontend/dprint.json`-এ। আমাদের style C# ভাষার মতো:

- ৪ space indentation (`"indentWidth": 4`)
- `{` নতুন লাইনে (`"bracePosition": "nextLine"`)
- String-এ single quote `'...'`, কিন্তু JSX attribute-এ double quote `"..."`
- সব statement-এর শেষে `;`
- Import-গুলো alphabetically sort করা

Command	কাজ
`npm run format`	সব file format করো
`npm run format:check`	শুধু check করো, কিছু বদলিও না

এই বইয়ের সব code snippet এই style-এ লেখা, তাই `{` নতুন লাইনে দেখলে অবাক হয়ো না।

৫.৬ Tailwind CSS: class দিয়ে style

সাধারণ CSS-এ আলাদা file-এ class বানিয়ে style লিখতে হয়। **Tailwind**-এ আগে থেকে তৈরি ছোট ছোট class সরাসরি element-এ বসিয়ে দাও।

নিচের উদাহরণটা React-এর JSX-এ লেখা। এখানে শুধু একটা জিনিস জেনে রাখো: React-এ HTML-এর `class` attribute-কে লিখতে হয় `className` (কারণ অধ্যায় ৬-এ)।

```
<button className="rounded-md bg-indigo-600 px-4 py-2 text-sm font-medium text-white
hover:bg-indigo-700">
    Log in
</button>
```

ফলাফল: indigo background color-এর, সাদা লেখার, rounded corner-ওয়ালা একটা button। Mouse উপরে নিলে রং আরেকটু গাঢ় হয়।

Class	মানে
`bg-indigo-600`	Background color indigo (600 = কতটা গাঢ়; সংখ্যা যত বড়, তত গাঢ়)

Class	মানে
`text-white`	লেখার রং সাদা
`px-4` / `py-2`	বাম-ডানে / উপর-নিচে ভেতরের ফাঁকা জায়গা (padding)
`m-4`, `mt-2`	বাইরের ফাঁকা জায়গা (margin); `t` = top
`rounded-md`	Rounded corner
`text-sm`, `text-2xl`	Font size
`font-medium`, `font-bold`	Font weight
`hover:bg-indigo-700`	Mouse উপরে গেলে background color আরও গাঢ়
`flex`, `items-center`, `justify-between`, `gap-4`	পাশাপাশি সাজানো (flexbox)
`grid`, `sm:grid-cols-3`	Grid layout; screen width ৬৪০px বা বেশি হলে ৩টা column
`w-full`, `max-w-md`, `min-h-screen`	Width আর height
`shadow`	Shadow
`space-y-4`	ভেতরের প্রতিটা element-এর মাঝে উপর-নিচে ফাঁক

Responsive

`sm:`, `md:`, `lg:` prefix মানে "screen width এই মাপের বা তার বেশি হলে তবেই এই class কাজ করবে"।

Prefix	কখন কাজ করে
(কিছু না)	সব screen-এ
`sm:`	width ≥ 640px
`md:`	width ≥ 768px
`lg:`	width ≥ 1024px

যেমন `grid sm:grid-cols-3`: mobile-এ এক column (একটার নিচে আরেকটা), আর ৬৪০px বা বড় screen-এ পাশাপাশি ৩ column। আবেদনের form-এর salary অংশ ঠিক এভাবে সাজানো।

Tailwind চালু করা

`frontend/src/index.css`-এর প্রথম লাইন:

```
@import "tailwindcss";
```

সাথে `vite.config.ts`-এর `plugins`-এ `tailwindcss()`। আর কিছু লাগে না।

সারসংক্ষেপ

- **npm** package install করে, আর **package.json**-এ থাকে package-এর তালিকা আর scripts।
- **Vite** dev server চালায় (HMR, proxy) আর production build বানায়। কিন্তু Vite **type-check করে না**।
- Type error দেখতে `npm run typecheck`, code-এর খারাপ অভ্যাস ধরতে `npm run lint`, আর format করতে `npm run format`।
- **tsconfig**-এর সব setting থাকে `compilerOptions`-এর ভেতরে।
- **Tailwind**-এ ছোট class বসিয়ে style দেওয়া হয়; `sm:` মানে ৬৪০px বা বড় screen।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: dependencies আর devDependencies-এর পার্থক্য কী?
উত্তর: dependencies হলো যা app চালাতে লাগে, যেমন react, react-dom। devDependencies হলো যা শুধু development আর build-এর সময় লাগে, যেমন typescript, eslint, vite। Production-এ user-এর browser-এ devDependencies-এর code যায় না।

প্রশ্ন: package-lock.json কেন দরকার?
উত্তর: এটা প্রতিটা package-এর, আর সেই package যেগুলোর উপর নির্ভর করে সেগুলোরও, হুবহু version লিখে রাখে। এতে সব developer-এর computer-এ আর server-এ একই code install হয়। তাই এই file git-এ commit করতে হয়। (**git** হলো code-এর সব পরিবর্তনের ইতিহাস রাখার tool, আর **commit** মানে সেই ইতিহাসে file-এর বর্তমান version save করা।)

প্রশ্ন: CORS কী? Development-এ কীভাবে সামলাও?
উত্তর: CORS হলো browser-এর একটা security নিয়ম। এক origin (protocol + domain + port) থেকে অন্য origin-এ request গেলে, server স্পষ্টভাবে অনুমতি না দিলে browser সেই response JavaScript-কে পড়তে দেয় না। Development-এ আমরা Vite proxy দিয়ে সামলাই: frontend নিজের origin-এই `/api/...`-তে request করে, আর Vite server সেটা backend-এ পাঠিয়ে উত্তর ফেরত আনে। Browser-এর চোখে তাই সব একই origin।

প্রশ্ন: HMR কী?
উত্তর: Hot Module Replacement। কোনো file বদলালে browser-এ শুধু সেই file-এর অংশটুকু বদলানো হয়, পুরো page reload হয় না। ফলে form-এ যা লিখে রেখেছিলে বা page-এর বর্তমান state সাধারণত হারায় না, আর পরিবর্তন সাথে সাথে দেখা যায়।

প্রশ্ন: Vite কি TypeScript-এর type error ধরে?
উত্তর: না। Dev server চলার সময় Vite শুধু type-গুলো মুছে দিয়ে JavaScript বানায়, type ঠিক আছে কিনা check করে না। Type error দেখতে VS Code বা `tsc` (আমাদের project-এ `npm run typecheck`) লাগে। আমাদের `build` script আগে `tsc -b` চালায়, তাই type error থাকলে build হয় না।

প্রশ্ন: TypeScript আর ESLint দুটোই কেন লাগে?
উত্তর: TypeScript দেখে type মিলছে কিনা, যেমন string-এর জায়গায় number দেওয়া হলো কিনা। ESLint দেখে code-এর অভ্যাস আর সম্ভাব্য bug, যেমন Promise ফেলে রাখা, `any` ব্যবহার, বা return type না লেখা। দুটো আলাদা ধরনের ভুল ধরে।

প্রশ্ন: Tailwind-এর সুবিধা আর অসুবিধা কী?
উত্তর:

- সুবিধা: আলাদা CSS file আর class-এর নাম ভাবতে হয় না, design সবখানে একই রকম থাকে, আর build-এ শুধু ব্যবহার করা class-গুলো থাকে, তাই CSS file ছোট হয়।
- অসুবিধা: element-এ অনেক লম্বা class-এর তালিকা হয়ে যায়, প্রথমে পড়তে কষ্ট হয়।

অধ্যায় ৬: React-এর মূল ধারণা

এখন আসল জিনিস। এই অধ্যায়ে শিখব React কী, JSX কী, component কী, আর props কীভাবে কাজ করে। শুরুতেই কয়েকটা শব্দ পরিষ্কার করে নেব, কারণ পুরো বইয়ে এগুলো বারবার আসবে।

৬.১ React কী?

React হলো user interface (UI) বানানোর একটা JavaScript library।

অধ্যায় ১-এ দেখেছি, হাতে হাতে DOM বদলানো কঠিন: কোন element খুঁজতে হবে, কোন লেখা বদলাতে হবে, কোনটা মুছতে হবে, সব নিজেকে মনে রাখতে হয়। React এই ঝামেলা নিজের কাঁধে নেয়।

React app ছোট ছোট টুকরো দিয়ে বানানো। প্রতিটা টুকরোর নাম **component**, যেমন একটা button, একটা table, বা একটা পুরো page। Component আসলে একটা JavaScript function (৬.৩-এ বিস্তারিত)।

পাঁচটা জরুরি শব্দ

শব্দ	মানে
render	React component function-টা call করে জেনে নেয় screen-এ কী দেখাতে হবে। এই কাজকে বলে render করা
state	Component-এর নিজের মনে রাখা data, যা সময়ের সাথে বদলাতে পারে। যেমন input field-এ কী লেখা আছে। পুরো আলোচনা অধ্যায় ৭-এ
mount	Component প্রথমবার screen-এ আসা (DOM-এ বসানো)
unmount	Component screen থেকে পুরোপুরি সরে যাওয়া (DOM থেকে মুছে ফেলা)
re-render	Data বদলালে React একই component আবার render করে, মানে function আবার call করে নতুন চেহারা বের করে

একটা উদাহরণ দিয়ে দেখো। Applications page খুললে table-এর প্রতিটা row screen-এ আসে, মানে **mount** হয়। একটা application "Applied" করে দিলে ওই row-এর data বদলায়, তাই row **re-render** হয়ে নতুন লেখা দেখায়। আর application delete করলে row-টা screen থেকে সরে যায়, মানে **unmount** হয়।

React-এর মূল ধারণা: UI = f(state)

> **UI = f(state)**
> Screen হলো data-র একটা function। Data (state) বদলাও, screen নিজেই বদলে যাবে।

তুমি শুধু বলো "data এমন হলে screen এমন দেখাবে"। DOM-এ ঠিক কী কী বদলাতে হবে, সেটা React নিজে বের করে। এই পদ্ধতির নাম **declarative** (কী চাই সেটা বলা)। এর উল্টো হলো **imperative** (প্রতিটা ধাপ নিজে বলা, যেমন `document.getElementById(...).textContent = ...`)।

Virtual DOM

আসল DOM বদলানো slow কাজ। তাই React আগে memory-তে DOM-এর একটা হালকা JavaScript copy বানায়, যার নাম **Virtual DOM**। Data বদলালে:

1. React re-render করে নতুন Virtual DOM বানায়।
2. পুরনোটার সাথে তুলনা করে দেখে কী কী বদলেছে। একে বলে **diffing** বা **reconciliation**।
3. আসল DOM-এ **শুধু বদলানো অংশটুকু** update করে।

ফলাফল: পুরো page নতুন করে না বানিয়ে, শুধু একটা লেখা বা একটা class বদলায়।

৬.২ JSX: JavaScript-এর ভেতরে HTML-এর মতো লেখা

Component কী ফেরত দেয়? দেয় **JSX**। তাই component-এর আগে JSX দেখে নিই। JSX লেখা হয় component-এর ভেতরে (৬.৩), আপাতত শুধু এর চেহারা আর নিয়ম দেখো।

```
return <h1 className="text-2xl">Applications</h1>;
```

দেখতে HTML, কিন্তু আসলে JavaScript। ফলাফল: screen-এ বড় font size-এ "Applications" লেখা একটা heading।

Browser JSX বোঝে না। Vite এটাকে মোটামুটি এরকম সাধারণ function call-এ convert করে (সংক্ষেপে):

```
return jsx('h1', { className: 'text-2xl', children: 'Applications' });
```

মানে JSX আসলে একটা object বানানোর সহজ লেখা, যে object React-কে বলে "এখানে একটা `h1` বানাও, এই class দিয়ে, এই লেখা ভেতরে রেখে"।

JSX-এর নিয়ম

নিয়ম	উদাহরণ
`class` নয়, `className` (কারণ `class` JavaScript-এর নিজের keyword, অধ্যায় ২.১৪)	`<div className="p-4">`
`for` নয়, `htmlFor` (একই কারণে: `for` হলো JavaScript-এর loop-এর keyword। **Loop** মানে একই code বারবার চালানো, যেমন array-র প্রতিটা item-এর জন্য একবার করে; অধ্যায় ২.৮-এর `forEach` আর `map`-ও এক ধরনের loop)	`<label htmlFor="email">`
সব tag বন্ধ করতে হবে	`<input />`, ` `
Attribute camelCase-এ	`onClick`, `onChange`, `minLength`
একটাই root element ফেরত দিতে হবে	কয়েকটা থাকলে একটা `<div>` বা `<>...</>` দিয়ে wrap করো
`{ }`-এর ভেতরে JavaScript	`<p>Hello, {name}!</p>`

`{ }`: JSX-এর ভেতরে JavaScript

```
const name = 'Rahim';
return <p>Hello, {name}!</p>;
```

ফলাফল: screen-এ "Hello, Rahim!"।

```
<p>{2 + 3}</p>
<td>{formatSalary(application)}</td>
<input placeholder={hint} />
```

- প্রথম লাইন দেখায় "5"।
- দ্বিতীয় লাইনে project-এর `formatSalary` function call হয়, আর যে string return করে (যেমন "50,000 – 80,000 BDT") সেটা table cell-এ দেখায়।
- তৃতীয় লাইনে `hint` variable-এর value attribute হিসেবে বসে।

`{ }`-এর ভেতরে যেকোনো **expression** (যা একটা value দেয়) চলে। কিন্তু `if` বা `for`-এর মতো **statement** চলে না। শর্ত দিয়ে কিছু দেখাতে কী করতে হয়, সেটা ৬.৮-এ দেখব।

Fragment: `<>...</>`

একটা component একটাই root element return করতে পারে। ধরো পাশাপাশি একটা heading আর একটা paragraph দেখাতে চাও:

```
return (
    <>
        <h1>Applications</h1>
        <p>All your job applications in one place.</p>
    </>
);
```

ফলাফল: screen-এ heading আর তার নিচে paragraph। `<>` আর `</>` হলো **Fragment**: এটা দুজনকে এক জায়গায় wrap করে, অথচ DOM-এ বাড়তি কোনো `<div>` যোগ করে না। একাধিক লাইনের JSX `( )`-এর ভেতরে রাখা হয়, যাতে পড়তে সুবিধা হয়।

৬.৩ Component: UI-এর ইট

একটা component হলো একটা **function**, যেটা JSX return করে। React এই function call করে (render করে) জানে screen-এ কী দেখাতে হবে।

Project-এর সবচেয়ে ছোট component, `frontend/src/features/applications/StatusBadge.tsx`:

```
import type { JSX } from 'react';

export function StatusBadge({ isApplied }: { isApplied: boolean; }): JSX.Element
{
    return isApplied
        ? (
            <span className="rounded-full bg-emerald-100 px-2.5 py-0.5 text-xs font-
medium text-emerald-700">
                Applied
            </span>
        )
        : (
            <span className="rounded-full bg-amber-100 px-2.5 py-0.5 text-xs font-medium
text-amber-700">
                Not applied
            </span>
        );
}
```

ব্যবহার:

```
<StatusBadge isApplied={true} />
<StatusBadge isApplied={false} />
```

প্রথমটা সবুজ "Applied" badge দেখাবে, দ্বিতীয়টা হলুদ "Not applied" badge।

Function-এর প্রথম লাইনটা ভেঙে দেখি

`function StatusBadge({ isApplied }: { isApplied: boolean; }): JSX.Element`

এখানে দুই জোড়া `{ }` আছে, আর দুটোর কাজ একদম আলাদা:

অংশ	কী	কোথায় শিখেছি
প্রথম `{ isApplied }`	Parameter-এ **destructuring**: function-কে দেওয়া object থেকে `isApplied` field বের করে নেওয়া	অধ্যায় ২.৯
`:` এর পরের `{ isApplied: boolean; }`	ওই object-এর **type**: এতে `isApplied` নামে একটা boolean আছে	অধ্যায় ৪

অংশ	কী	কোথায় শিখেছি
`: JSX.Element`	Function-এর return type। `JSX.Element` হলো একটা JSX element-এর type	অধ্যায় ৪.২

`JSX` type-টা আসে React থেকে, তাই উপরে `import type { JSX } from 'react';` (অধ্যায় ৪.৯)।

`<StatusBadge isApplied={true} />` লিখলে React আসলে function-টা এভাবে call করে: `StatusBadge({ isApplied: true })`। মানে JSX-এ যত attribute লেখো, সব একটা object-এ ভরে component-কে দেওয়া হয়। এই object-এর নাম **props** (৬.৫-এ বিস্তারিত)।

Function-এর ভেতরে আছে ternary (অধ্যায় ২.১১): `isApplied` true হলে প্রথম `<span>`, না হলে দ্বিতীয়টা।

Component-এর নিয়ম

- নাম **বড় হাতের অক্ষর দিয়ে শুরু** হতে হবে (`StatusBadge`, `statusBadge` না)। ছোট হাতের নাম দেখলে React সেটাকে HTML tag ভাবে, যেমন `<span>`।
- JSX return করতে হবে। আমাদের project-এ return type লেখা হয় `JSX.Element`।
- একই component বারবার ব্যবহার করা যায়। Table-এর প্রতিটা row-এ একই `StatusBadge` ব্যবহার হয়, শুধু props আলাদা। এজন্যই component **reusable**।

Fragment কেন লাগে: `ExternalLink`

`frontend/src/features/applications/ApplicationDetailsPage.tsx`-এ একটা ছোট component আছে:

```
function ExternalLink({ href }: { href: string | null; }): JSX.Element
{
    if (!href)
    {
        return <>–</>;
    }
    return (
        <a href={href} target="_blank" rel="noreferrer" className="break-all text-
indigo-600 hover:underline">
            {href}
        </a>
    );
}
```

Link থাকলে একটা clickable link দেখায়, না থাকলে শুধু একটা dash "—"। `target="_blank"` link-টা নতুন tab-এ খোলে, আর `rel="noreferrer"` দেওয়া হয় security-র জন্য (অধ্যায় ৯.৫)।

এখানে fragment-এ একটাই child, তাহলে fragment কেন? কারণ return type `JSX.Element`। শুধু `return '–';` লিখলে সেটা একটা string, `JSX.Element` না, তাই TypeScript error দেবে। `<>–</>` লিখলে string-টা JSX-এ wrap হয়ে যায়, আর DOM-এ বাড়তি কোনো tag-ও আসে না।

৬.৪ App শুরু হয় কোথা থেকে: `main.tsx`

`frontend/src/main.tsx` (import লাইনগুলো বাদে):

```
const rootElement = document.getElementById('root');
if (!rootElement)
{
    throw new Error('Root element #root not found in index.html');
}

void store.dispatch(restoreSession());

createRoot(rootElement).render(
    <StrictMode>
        <Provider store={store}>
            <AppRouter />
        </Provider>
    </StrictMode>,
);
```

1. **`document.getElementById('root')`**: `index.html`-এর খালি `<div id="root">` খুঁজে বের করে। এর type `HTMLElement | null`, কারণ element না-ও থাকতে পারে।
2. **`if (!rootElement) throw ...`**: element না পেলে error। এটা আসলে **narrowing** (অধ্যায় ৪.৮): `if`-এর পরে TypeScript জানে `rootElement` আর null না, তাই নিচে নিশ্চিন্তে ব্যবহার করা যায়।
3. **`store.dispatch(restoreSession())`**: আগে login করা থাকলে সেটা ফিরিয়ে আনে। এটা Redux-এর জিনিস (অধ্যায় ১০–১২), এখন শুধু জেনে রাখো যে app শুরুতেই এই কাজটা হয়।
4. **`createRoot(...).render(...)`**: React-কে বলে "এই div-এর ভেতরে পুরো app render করো"।
5. **`<StrictMode>`**: শুধু development-এ বাড়তি check চালায়। ভুল ধরার জন্য এটা প্রতিটা component **ইচ্ছা করে দুইবার render করে** (আর effect-ও দুইবার চালায়, অধ্যায় ৮)। তাই development-এ component-এর ভেতরের `console.log` দুইবার দেখলে ভয় পেয়ো না। Production-এ এর কোনো প্রভাব নেই।
6. **`<Provider store={store}>`**: Redux store পুরো app-কে দেয় (অধ্যায় ১০)।

7. `<AppRouter />`: কোন URL-এ কোন page দেখাবে, তা ঠিক করে (অধ্যায় ৯)।

এভাবে component-এর ভেতরে component বসিয়ে তৈরি হয় **component tree**। নিচের ছবিটা সংক্ষেপে; আসলে `AppRouter`-এর ভেতরে `RouterProvider` আর আরও কিছু layer আছে, সেগুলো অধ্যায় ৯-এ দেখব:

```
StrictMode
└── Provider
    └── AppRouter
        ├── AuthLayout → LoginPage / RegisterPage
        └── ProtectedRoute
            └── AppLayout
                └── ApplicationsPage
                    ├── ApplicationFilters
                    ├── ApplicationsTable
                    │   └── StatusBadge (প্রতি row-এ একটা)
                    └── Pagination
```

যে component অন্যটাকে নিজের ভেতরে বসায়, সে **parent**; আর যাকে বসানো হয়, সে **child**। যেমন `ApplicationsTable` হলো `StatusBadge`-এর parent।

৬.৫ Props: Component-কে data দেওয়া

Function যেমন parameter নেয়, component নেয় **props** (properties)। Parent component JSX attribute হিসেবে child-কে props দেয়:

```
<StatusBadge isApplied={application.isApplied} />
```

Child সেটা নেয় parameter-এ destructuring করে, যেটা ৬.৩-এ দেখেছি:

```
export function StatusBadge({ isApplied }: { isApplied: boolean; }): JSX.Element
```

Props-এর type: আলাদা `interface`

Props বেশি হলে type-টা আলাদা `interface`-এ লেখা হয়।
`frontend/src/shared/components/Pagination.tsx` (import লাইন বাদে):

```
interface PaginationProps
{
    meta: PaginationMeta;
    onPageChange: (page: number) => void;
}

export function Pagination({ meta, onPageChange }: PaginationProps): JSX.Element
{
    return (
        <div className="flex items-center justify-between">
            <Button variant="secondary" disabled={!meta.hasPrev} onClick={(): void =>
onPageChange(meta.page - 1)}>
                Previous
            </Button>
            <span className="text-sm text-slate-600">Page {meta.page}</span>
            <Button variant="secondary" disabled={!meta.hasNext} onClick={(): void =>
onPageChange(meta.page + 1)}>
                Next
            </Button>
        </div>
    );
}
```

ফলাফল: বাম পাশে "Previous" button, মাঝে "Page 1", ডান পাশে "Next" button। আগের page না থাকলে "Previous" disabled থাকে।

লাইন ধরে দেখি:

- `meta: PaginationMeta`: একটা object, যাতে আছে `page` (বর্তমান page number), `limit`, `hasPrev` আর `hasNext` (আগের/পরের page আছে কিনা, boolean)।
- `onPageChange: (page: number) => void`: এটা একটা **function-type prop** (অধ্যায় ৪.৪)। মানে parent-কে এমন একটা function দিতে হবে, যেটা একটা number নেয় আর কিছু return করে না।
- `Button` হলো আমাদের নিজের button component (৬.৭)। আপাতত সাধারণ `<button>` ভাবো।
- `disabled={!meta.hasPrev}`: আগের page না থাকলে button disabled।
- `onClick={(): void => onPageChange(meta.page - 1)}`: click করলে এই arrow function চলে। `(): void =>` মানে কোনো parameter নেই আর কিছু return করে না (অধ্যায় ৪.২)। ভেতরে parent-এর দেওয়া `onPageChange` call হয়, আগের page number দিয়ে।

ব্যবহার

```
<Pagination meta={meta} onPageChange={(page): void => console.log(page)} />
```

এখন page 1-এ থাকা অবস্থায় "Next" click করলে console-এ `2` print হবে। এখানে `page`-এর type লিখতে হয়নি, কারণ TypeScript prop-এর type (`(page: number) => void`) দেখে নিজেই বুঝে নেয় `page` একটা number।

আসল `ApplicationsPage.tsx`-এ `console.log`-এর জায়গায় Redux-এ একটা action পাঠানো হয়, যাতে নতুন page-এর data load হয়। সেটা অধ্যায় ১০-এ দেখব। কিন্তু pattern একই: Pagination শুধু খবর দেয়, কী করতে হবে সেটা parent ঠিক করে।

Function-কে prop হিসেবে দেওয়া: child থেকে parent-এ খবর পাঠানো

খেয়াল করো, Pagination নিজে জানে না page বদলালে কী করতে হবে। সে শুধু call করে `onPageChange(2)`। কী করতে হবে, সেটা ঠিক করে parent। Prop হিসেবে দেওয়া এমন function-কে বলে **callback**।

- **Data নিচে যায়** props হিসেবে (parent থেকে child)।
- **Event উপরে যায়** callback function হিসেবে (child থেকে parent)।

এটাকে বলে **one-way data flow**। React-এর সবচেয়ে গুরুত্বপূর্ণ নিয়মগুলোর একটা।

Props পড়া যায়, বদলানো যায় না

Child কখনো নিজের props বদলাবে না। Props হলো **read-only**। কিছু বদলাতে হলে child callback দিয়ে parent-কে জানাবে, আর parent নতুন props পাঠাবে।

Default value

Destructuring-এ default value (অধ্যায় ২.৯) দিয়ে optional prop-এর default ঠিক করা যায়। `frontend/src/shared/components/Feedback.tsx`-এর `Spinner` (সংক্ষেপে):

```
export function Spinner({ fullPage = false }: { fullPage?: boolean; }): JSX.Element
```

- `fullPage?` মানে prop-টা দেওয়া optional (অধ্যায় ৪.৩)।
- `<Spinner />` লিখলে `fullPage` হবে `false`, তাই ছোট একটা loading চিহ্ন দেখায়।
- `<Spinner fullPage />` লিখলে `fullPage` হবে `true`, আর পুরো screen জুড়ে মাঝখানে loading চিহ্ন। JSX-এ value ছাড়া শুধু attribute-এর নাম লিখলে সেটা `true` ধরা হয়।

৬.৬ `children`: Component-এর ভেতরের জিনিস

Component tag-এর opening আর closing অংশের মাঝে যা লেখা হয়, সেটা component পায় `children` নামের একটা বিশেষ prop হিসেবে। `frontend/src/shared/components/Field.tsx` (সংক্ষেপে):

```
import type { JSX, ReactNode } from 'react';

interface FieldProps
{
    label: string;
    htmlFor: string;
    error?: string | undefined;
    children: ReactNode;
}

export function Field({ label, htmlFor, error, children }: FieldProps): JSX.Element
{
    return (
        <div className="space-y-1">
            <label htmlFor={htmlFor} className="block text-sm font-medium text-slate-
700">
                {label}
            </label>
            {children}
            {error && <p className="text-sm text-red-600">{error}</p>}
        </div>
    );
}
```

- **`ReactNode`** হলো "React screen-এ দেখাতে পারে এমন যেকোনো কিছু"-র type: JSX, string, number, `null`, বা এদের array। `children`-এ কী আসবে আগে থেকে জানা নেই, তাই এই চওড়া type। এটাও React থেকে import করা হয়।
- **`error?: string | undefined`**: error message দেওয়া optional, আর দিলে সেটা string বা `undefined` হতে পারে। শুধু `?`-ই যথেষ্ট ছিল; `| undefined` এখানে বাড়তি, শুধু স্পষ্ট করার জন্য লেখা।
- **`{error && ...}`**: error থাকলে তবেই লাল লেখা দেখায় (৬.৮-এ বিস্তারিত)।

ব্যবহার:

```
<Field label="Email" htmlFor="email">
    <input id="email" type="email" placeholder="you@example.com" />
</Field>
```

ফলাফল: উপরে "Email" label, তার নিচে একটা input field। কোনো `error` দেওয়া হয়নি, তাই লাল লেখা নেই।

এখানে `<input ... />` হলো `children`। Field জানে না ভেতরে কী আসবে: input, select নাকি rich text editor। সে শুধু label আর error-এর জায়গা দেয়, আর মাঝে `{children}` বসিয়ে দেয়। এভাবে ছোট component জুড়ে বড় জিনিস বানানোর নাম **composition**।

(Input field-এ `value` দিয়ে React থেকে লেখা নিয়ন্ত্রণ করার নিয়ম অধ্যায় ৭-এ। এখানে শুধু `placeholder` দিয়েছি, যেটা খালি থাকলে ধূসর একটা hint দেখায়।)

৬.৭ Reusable ছোট component: আমাদের `shared/components`

Component	কাজ
`Button`, `LinkButton`	একই রকম দেখতে button, আর button-এর মতো দেখতে link
`Field`, `Input`, `Select`	Label + input + error, সব জায়গায় একই style-এ
`Spinner`, `ErrorMessage`	Loading হচ্ছে, অথবা সমস্যা হয়েছে
`Pagination`	আগের/পরের page
`RichTextEditor`	Bold, italic, list সহ লেখার editor (অধ্যায় ১৩)

উদাহরণ: `Input`, যেখানে HTML-এর সব attribute চলে

`frontend/src/shared/components/Field.tsx` থেকে:

```
const CONTROL_CLASSES =
    'w-full rounded-md border border-slate-300 bg-white px-3 py-2 text-sm text-slate-800
focus:border-indigo-500 focus:outline-none focus:ring-2 focus:ring-indigo-200';

export function Input({ className = '', ...props }: ComponentProps<'input'>):
JSX.Element
{
    return <input {...props} className={`${CONTROL_CLASSES} ${className}`} />;
}
```

- `CONTROL_CLASSES`: সব input আর select-এর common Tailwind class (পুরো width, border, rounded corner, focus করলে indigo রঙের ring)।
- `ComponentProps<'input'>`: সাধারণ `<input>`-এর যত attribute আছে (`type`, `value`, `onChange`, `required`, `placeholder`...), সবগুলোর type। এটাও React থেকে import করা একটা type।
- `{ className = '', ...props }`: `className` আলাদা করে নেওয়া হলো (না দিলে খালি string), আর বাকি সব attribute **rest** দিয়ে `props` object-এ জড়ো হলো (অধ্যায় ২.১২)।
- `<input {...props} />`: JSX-এ `{...props}` লিখলে object-এর সব field একেকটা attribute হিসেবে ছড়িয়ে বসে যায় (spread)।
- **className**: template literal (অধ্যায় ২.১০) দিয়ে আমাদের common class আর বাইরে থেকে আসা class জোড়া লাগানো হয়।

ফলে `<Input id="email" type="email" placeholder="you@example.com" />` লিখলে সাধারণ input-এর মতোই কাজ করে, শুধু দেখতে সব জায়গায় একই রকম সুন্দর।

উদাহরণ: `Button`, যেখানে variant দিয়ে রং বদলায়

`frontend/src/shared/components/Button.tsx` (শুধু `Button` অংশ):

```
type ButtonVariant = 'primary' | 'secondary' | 'danger';

const VARIANT_CLASSES: Record<ButtonVariant, string> = {
    primary: 'bg-indigo-600 text-white hover:bg-indigo-700',
    secondary: 'border border-slate-300 bg-white text-slate-700 hover:bg-slate-50',
    danger: 'bg-red-600 text-white hover:bg-red-700',
};

function buttonClasses(variant: ButtonVariant, className: string): string
{
    return `inline-flex items-center justify-center gap-2 rounded-md px-4 py-2 text-sm
font-medium transition disabled:cursor-not-allowed disabled:opacity-50 ${
        VARIANT_CLASSES[variant]
    } ${className}`;
}

interface ButtonProps extends ComponentProps<'button'>
{
    variant?: ButtonVariant;
    isLoading?: boolean;
}

export function Button({
    variant = 'primary',
    isLoading = false,
    className = '',
    type = 'button',
    disabled,
    children,
    ...props
}: ButtonProps): JSX.Element
{
    return (
        <button
            {...props}
            type={type}
            disabled={disabled === true || isLoading}
            className={buttonClasses(variant, className)}
        >
            {isLoading ? 'Please wait...' : children}
        </button>
    );
}
```

ধাপে ধাপে:

- `VARIANT_CLASSES`: প্রতিটা variant-এর জন্য রঙের class। `Record<ButtonVariant, string>` (অধ্যায় ৪.৭) নিশ্চিত করে তিনটা variant-এর একটাও বাদ পড়েনি।
- `buttonClasses`: সব button-এর common class-এর সাথে `VARIANT_CLASSES[variant]` জোড়া দেয়। `variant` যদি `'danger'` হয়, তাহলে `VARIANT_CLASSES['danger']` মানে লাল রঙের class-গুলো। এর পরে বাইরে থেকে আসা `className`। Template literal-এর `${` -এর পরে লাইন ভেঙে যাওয়াটা dprint-এর কাজ: লাইন খুব লম্বা হলে সে এভাবে ভেঙে format করে, মানে একই থাকে। আর `disabled:cursor-not-allowed` -এর মতো Tailwind-এর `disabled:` prefix মানে button disabled থাকলে তবেই class-টা কাজ করে, ঠিক `hover:` -এর মতো।
- `interface ButtonProps extends ComponentProps<'button'>`: সাধারণ `<button>` -এর সব attribute, সাথে আমাদের নিজের দুটো: `variant` আর `isLoading` (inheritance, অধ্যায় ৪.৩)।
- **Destructuring**: `variant`, `isLoading`, `className`, `type` -এর default value আছে; `disabled` আর `children` আলাদা করে নেওয়া হলো, কারণ এদের নিয়ে আলাদা কাজ আছে; বাকি সব (`onClick` ইত্যাদি) গেল `props` -এ।
- `type = 'button'`: default type `button`। কেন এটা জরুরি, অধ্যায় ৭-এ form-এর সময় দেখবে।
- `disabled={disabled === true || isLoading}`: বাইরে থেকে disabled করা হলে, অথবা loading চলাকালীন, button disabled।
- `{isLoading ? 'Please wait...' : children}`: loading হলে লেখা বদলে "Please wait..." হয়, না হলে ভেতরের আসল লেখা।

ফলাফল: `<Button variant="danger">Delete</Button>` লিখলে লাল "Delete" button আসবে। `<Button isLoading={true}>Save</Button>` লিখলে button disabled হয়ে "Please wait..." দেখাবে।

৬.৮ Conditional rendering: condition অনুযায়ী দেখানো

`{ }` -এর ভেতরে `if` চলে না, তাই শর্ত দিয়ে কিছু দেখাতে তিনটা উপায় আছে।

`&&` : শর্ত true হলে দেখাও

```
{error && <p className="text-sm text-red-600">{error}</p>}
```

`error` -এ কোনো string থাকলে লাল লেখা দেখায়। `error` undefined বা খালি string হলে কিছুই দেখায় না (অধ্যায় ২.১১-এর `&&`)।

সাবধান: বাম দিকে number থাকলে `0` screen-এ print হয়ে যায়, কারণ React `false`, `null`, `undefined` দেখায় না কিন্তু `0` দেখায়। `{items.length && <Table />}` লিখলে list খালি থাকলে screen-এ "0" দেখাবে। তাই লেখো `{items.length > 0 && <Table />}`, যেমনটা `ApplicationsPage.tsx` -এ লেখা

আছে।

Ternary: এটা অথবা ওটা

Button থেকে আগেই দেখেছ:

```
{isLoading ? 'Please wait...' : children}
```

আবেদনের form-এ apply medium "Link" হলে link-এর input field, আর "Mail" হলে mail লেখার editor দেখায়। নিচেরটা **pseudo-code** (`...` মানে অংশ বাদ, এভাবে হুবহু লিখলে চলবে না):

```
{values.applyMedium === ApplyMedium.Link
    ? <Field label="Apply link" ...> ... </Field>
    : <Field label="Application mail" ...> ... </Field>}
```

পুরো code অধ্যায় ১৩.৩-এ দেখবে।

আগেভাগে return (early return)

`frontend/src/shared/components/Feedback.tsx` :

```
export function ErrorMessage({ message }: { message: string | null; }): JSX.Element |
null
{
    if (!message)
    {
        return null;
    }
    return <p className="rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm
text-red-700">{message}</p>;
}
```

- `message` null বা খালি হলে `null` return করে। `null` return করলে React কিছুই দেখায় না।
- তাই return type `JSX.Element | null` : হয় JSX, নয়তো null।
- `<ErrorMessage message="Network error" />` লাল box-এ "Network error" দেখায়; `<ErrorMessage message={null} />` কিছুই দেখায় না।

Page-এও early return ব্যবহার হয়। নিচের snippet `ApplicationDetailsPage.tsx` থেকে (সংক্ষেপে)। এখানে `selectedStatus` , `selectedError` আর `selected` আসে Redux থেকে, আর `id` আসে URL থেকে (অধ্যায় ৯–১১)। এখন এগুলোর উৎস নিয়ে ভেবো না, শুধু early return-এর pattern দেখো:

```
if (selectedStatus === RequestStatus.Failed)
{
    return <ErrorMessage message={selectedError} />;
}
if (!selected || selected.id !== id)
{
    return <Spinner />;
}
return ( ... );
```

- Load করতে গিয়ে fail হলে: error message দেখাও, বাকি page না।
- `selected` হলো load করা application। এটা এখনো না থাকলে, অথবা থাকলেও সেটা অন্য কোনো application (যেটা আগে দেখা হয়েছিল, তাই তার `id` URL-এর `id`-র সাথে মেলে না), তাহলে ঠিক data এখনো আসেনি: spinner দেখাও।
- দুটো পরীক্ষাই পার হলে তবেই আসল page return হয় (`( ... )` অংশটা বাদ রাখা হয়েছে)।

৬.৯ List দেখানো আর `key`

Array-এর প্রতিটা item থেকে JSX বানাতে `map` (অধ্যায় ২.৮)। `ApplicationsTable.tsx` থেকে (সংক্ষেপে, class আর কিছু column বাদ):

```
<tbody>
    {applications.map((application) => (
        <tr key={application.id}>
            <td>{application.jobPost}</td>
            <td>{formatDate(application.deadline)}</td>
            ...
        </tr>
    ))}
</tbody>
```

ফলাফল: `applications` array-তে যতগুলো application, table-এ ততগুলো row। `map` প্রতিটা application-কে একটা `<tr>`-এ convert করে, আর JSX সেই array-র সব element পরপর দেখায়।

`key` কেন দরকার?

List বদলালে (নতুন item এলো, একটা মুছে গেল, ক্রম বদলাল) React-কে বুঝতে হয় কোন row কোন data-র। `key` হলো প্রতিটা row-এর **স্থায়ী পরিচয়পত্র**। এটা list-এর ভেতরে **unique** হতে হবে, আর re-render-এর মাঝে বদলানো যাবে না।

key হিসেবে	ভালো?
Database-এর `id`	সবচেয়ে ভালো: সবসময় একই, আর unique
Array index (`0, 1, 2`)	সাবধান: list-এর ক্রম বদলালে বা মাঝখান থেকে মুছলে index সরে যায়, তখন React ভুল row-এ ভুল data দেখাতে পারে
`Math.random()`	খারাপ: প্রতি render-এ নতুন key আসে, তাই React প্রতিবার সব row unmount করে নতুন করে mount করে

যে list কখনো বদলায় না, যেমন `PAGE_SIZES = [10, 20, 50]`, সেখানে value নিজেই key হতে পারে। `ApplicationFilters.tsx`-এ:

```
{PAGE_SIZES.map((size) => <option key={size} value={size}>{size}</option>)}
```

ফলাফল: dropdown-এ 10, 20, 50 তিনটা option।

`key` দিয়ে একটা component-কে "একদম নতুন করে শুরু" করানোর একটা কৌশলও আছে। সেটা বুঝতে state জানা লাগে, তাই অধ্যায় ৭-এ দেখব।

৬.১০ Event: user-এর কাজে সাড়া দেওয়া

`ApplicationsTable.tsx`-এর delete button (সংক্ষেপে, class বাদ):

```
<button type="button" onClick={(): void => onDelete(application)}>Delete</button>
```

Click করলে parent-এর দেওয়া `onDelete` callback চলে, ওই row-এর application দিয়ে।

- Event-এর নাম camelCase: `onClick`, `onChange`, `onSubmit`।
- Value হিসেবে একটা **function** দিতে হয়, function call-এর **ফলাফল** না।

তিনটা লেখা তুলনা করো:

```
onClick={handleReset}
onClick={(): void => onDelete(application)}
onClick={onDelete(application)}
```

- **প্রথমটা ঠিক**: function-টা নিজেই দেওয়া হয়েছে (call করা হয়নি, শুধু নাম)। Click হলে React এটা call করবে। `ApplicationFilters.tsx`-এর Reset button ঠিক এভাবে লেখা।

- **দ্বিতীয়টা ঠিক**: একটা নতুন arrow function দেওয়া হয়েছে (অধ্যায় ৪.২), যেটা click হলে চলবে আর ভেতরে `onDelete(application)` call করবে।
- **তৃতীয়টা ভুল**: এখানে `onDelete` **এখনই** call হয়ে যাবে, render-এর সময়েই, click-এর অপেক্ষা না করে। মানে page খুললেই সব application delete হওয়ার চেষ্টা হবে।

নিয়ম: function-কে argument দিতে হলে (এখানে `application`) arrow function-এ wrap করো।

Input field-এ লেখা, form submit করা ইত্যাদি event-এর বিস্তারিত অধ্যায় ৭-এ।

সারসংক্ষেপ

- **render** মানে React component function call করে চেহারা বের করা; **mount** মানে প্রথম screen-এ আসা, **unmount** মানে সরে যাওয়া, **re-render** মানে data বদলালে আবার render।
- React-এ **UI = f(state)**: data বদলালে screen নিজে বদলায়। React Virtual DOM দিয়ে শুধু বদলানো অংশ update করে।
- **JSX**-এ `className`, `htmlFor`, সব tag বন্ধ, একটাই root (দরকারে Fragment `<>...</>`), আর `{}` -এ JavaScript expression।
- **Component** হলো JSX return করা function, নাম বড় হাতের অক্ষরে শুরু, return type `JSX.Element`।
- **Props** parent থেকে child-এ আসে, আর read-only। Child callback দিয়ে parent-কে খবর দেয় (**one-way data flow**)।
- **children** দিয়ে component-এর ভেতরে যেকোনো কিছু বসানো যায় (**composition**)।
- Conditional rendering-এ `&&`, ternary আর early return; `null` return করলে কিছু দেখায় না।
- List-এ `map` আর স্থায়ী, unique `key`।
- Event handler-এ function দাও, function call-এর ফলাফল না।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Virtual DOM কী? কেন দরকার?
উত্তর: Virtual DOM হলো আসল DOM-এর একটা হালকা JavaScript copy, যেটা memory-তে থাকে। Data বদলালে React নতুন Virtual DOM বানিয়ে আগেরটার সাথে তুলনা করে (diffing), তারপর আসল DOM-এ শুধু পার্থক্যটুকু বসায়। আসল DOM বদলানো slow আর expensive, তাই এতে কাজ অনেক কম হয়।

প্রশ্ন: Render, mount আর re-render-এর পার্থক্য কী?
উত্তর: Render মানে React component function call করে জানা screen-এ কী দেখাতে হবে। Mount মানে component প্রথমবার DOM-এ বসানো। পরে state বা props বদলালে React আবার function call করে নতুন চেহারা বের করে, এটা re-render। আর component DOM থেকে সরে গেলে সেটা unmount।

প্রশ্ন: Props আর State-এর পার্থক্য কী?
উত্তর: Props বাইরে থেকে, মানে parent থেকে আসে, আর child সেগুলো বদলাতে পারে না, শুধু পড়ে। State হলো component-এর নিজের মনে রাখা data, যা সে নিজে বদলাতে পারে। দুটোর যেকোনোটা বদলালে component re-render হয়।

প্রশ্ন: key কেন দরকার? index কেন ভালো key নয়?
উত্তর: List বদলালে কোন element কোন data-র, সেটা React key দিয়ে চেনে। Index ব্যবহার করলে মাঝখানে কিছু যোগ বা মুছলে পরের সব item-এর index সরে যায়, তখন React ভুল element-এর সাথে ভুল data জুড়ে দিতে পারে। Database-এর id-র মতো স্থায়ী আর unique value সবচেয়ে ভালো key।

প্রশ্ন: JSX কী? Browser কি JSX বোঝে?
উত্তর: JSX হলো JavaScript-এর ভেতরে HTML-এর মতো করে UI লেখার syntax। Browser এটা বোঝে না। Build tool, আমাদের ক্ষেত্রে Vite, এটাকে সাধারণ JavaScript function call-এ convert করে, যেটা React-এর জন্য একটা object বানায়।

প্রশ্ন: One-way data flow কী?
উত্তর: Data শুধু উপর থেকে নিচে, parent থেকে child-এ props হিসেবে যায়। Child কিছু বদলাতে চাইলে নিজে বদলায় না, parent-এর দেওয়া callback function call করে। Parent তখন নিজের data বদলায় আর নতুন props পাঠায়। এতে data কোথা থেকে আসছে আর কে বদলাচ্ছে, তা সহজে বোঝা যায়।

প্রশ্ন: Fragment কী?
উত্তর: `<>...</>`। কয়েকটা element একসাথে return করতে দেয়, অথচ DOM-এ বাড়তি কোনো tag যোগ করে না। Component একটাই root element return করতে পারে বলে এটা দরকার হয়।

প্রশ্ন: StrictMode কী করে?
উত্তর: StrictMode শুধু development-এ চলে। লুকানো bug ধরার জন্য component ইচ্ছা করে দুইবার render করে, effect-ও দুইবার চালায়, আর পুরনো API ব্যবহার করলে সতর্ক করে। Production-এ এর কোনো প্রভাব নেই।

অধ্যায় ৭: State আর Event

Props বাইরে থেকে আসে। কিন্তু component-এর **নিজের** কিছু মনে রাখার দরকার হলে, যেমন input field-এ user কী লিখেছে, তখন লাগে **state** (অধ্যায় ৬.১-এ এক লাইনে দেখেছি)। এই অধ্যায়ে state, event, আর form নিয়ে পুরো আলোচনা।

৭.১ কেন সাধারণ variable দিয়ে হয় না?

ধরো login page-এ email মনে রাখতে একটা সাধারণ variable নিলাম:

```
function LoginPage(): JSX.Element
{
    let email = '';
    return <input value={email} onChange={(event): void => { email = event.target.value;
}} />;
}
```

- `onChange` চলে প্রতিবার user input field-এ কিছু টাইপ করলে। `event.target` হলো সেই input element, আর `event.target.value` হলো তাতে এখন যা লেখা আছে।

এই code কাজ করবে না, দুটো কারণে:

1. `email` বদলালে React **জানতেই পারে না** যে কিছু বদলেছে, তাই re-render হয় না, screen-ও বদলায় না।
2. কোনো কারণে re-render হলেও function আবার শুরু থেকে চলে, তাই `let email = ''` লাইনে `email` আবার খালি হয়ে যায়।

দরকার এমন একটা জিনিস, যেটা re-render-এর মাঝে value মনে রাখে, আর বদলালে React-কে re-render করতে বলে। এটাই state।

৭.২ `useState`

```
import { useState } from 'react';

const [email, setEmail] = useState('');
```

- `useState('')`: React-কে বলে "এই component-এর জন্য একটা value মনে রাখো, শুরুতে `''`"।
- এটা দুটো জিনিসের একটা array return করে, যেগুলো array destructuring (অধ্যায় ২.৯) দিয়ে নেওয়া হয়:
  - `email`: বর্তমান value।
  - `setEmail`: value বদলানোর function। একে বলে **setter**।
- `setEmail('a@b.com')` call করলে:
  1. React নতুন value মনে রাখে।
  2. Component **re-render** হয় (function আবার চলে)।
  3. এবার `useState` return করে নতুন value, তাই `email` হয় `'a@b.com'`, আর screen সেই অনুযায়ী বদলায়।

শুরুর value (`''`) শুধু প্রথম render-এ, মানে mount-এর সময় কাজে লাগে। পরের সব render-এ React নিজের মনে রাখা value দেয়। এই কথাটা ৭.১০-এ খুব কাজে লাগবে।

Hook কী?

`useState` হলো একটা **hook**। Hook মানে `use` দিয়ে শুরু হওয়া একটা React function, যেটা component-কে বাড়তি ক্ষমতা দেয় (যেমন `useState` দেয় মনে রাখার ক্ষমতা)। Hook শুধু component function-এর **top level**-এ call করা যায়: `if`, loop বা ভেতরের কোনো function-এর মধ্যে না। কেন এই নিয়ম, আর আরও hook, বিস্তারিত অধ্যায় ৮-এ।

নামকরণের নিয়ম

`[কিছু, setকিছু]`, যেমন `[email, setEmail]`, `[values, setValues]`, `[errors, setErrors]`।

Type

সাধারণত TypeScript শুরুর value দেখে type বুঝে নেয়: `useState('')` থেকে `email` হয় `string`। শুরুর value দেখে বোঝা না গেলে generic (অধ্যায় ৪.৬) দিয়ে নিজে বলে দাও। `ApplicationForm.tsx` থেকে:

```
const [values, setValues] = useState<ApplicationFormValues>(initialValues);
const [errors, setErrors] = useState<ApplicationFormErrors>({});
```

দ্বিতীয় লাইনে শুরুর value `{}`, যেটা দেখে TypeScript বুঝতে পারত না পরে এতে কী কী field আসবে। তাই `<ApplicationFormErrors>` লেখা হয়েছে (এই type অধ্যায় ৪.৭-এ দেখেছ)।

৭.৩ Render: function আবার চলা মানে কী

React component হলো একটা function। প্রতিবার state বা props বদলালে **পুরো function আবার চলে**, আর নতুন JSX return করে।

ধরো `RegisterPage`-এ বোঝার জন্য একটা `console.log` যোগ করলাম (সংক্ষেপে; আসল file-এ এই log নেই, আর আরও অনেক কিছু আছে):

```
export function RegisterPage(): JSX.Element
{
    const [name, setName] = useState('');
    console.log('render হলো, name =', name);
    return <Input value={name} onChange={(event): void => setName(event.target.value)}
/>;
}
```

Input field-এ "Rahim" টাইপ করলে console-এ দেখাবে:

```
render হলো, name =
render হলো, name = R
render হলো, name = Ra
render হলো, name = Rah
...
```

প্রতিটা অক্ষরে `setName` call হয়, আর প্রতিবার পুরো function আবার চলে।

Development-এ প্রতিটা লাইন দুইবার করে দেখতে পারো। এটা bug না, `StrictMode` ইচ্ছা করে প্রতিটা component দুইবার render করে (অধ্যায় ৬.৪)।

State হলো একটা snapshot

একটা render-এর ভেতরে state-এর value **বদলায় না**। `setName` call করলে বদলটা দেখা যায় **পরের render-এ**:

```
setName('Karim');
console.log(name);
```

এখানে এখনো পুরনো value print হবে, যেমন `'Rahim'`। কারণ এই render-এর `name` একটা সাধারণ `const`, এর value আর বদলাবে না। নতুন `'Karim'` পাওয়া যাবে পরের render-এ, যখন function আবার চলবে আর `useState` নতুন value দেবে।

Snapshot-এর ফাঁদ: পুরনো value (stale value)

একটা counter ধরো:

```
const [count, setCount] = useState(0);

function handleClick(): void
{
    setCount(count + 1);
    setCount(count + 1);
}
```

`count` 0 থাকা অবস্থায় click করলে নতুন value কত হবে? **2 না, 1।**

কারণ:

- `handleClick` function-টা তৈরি হয়েছে সেই render-এ, যখন `count` ছিল 0। একটা function যেখানে তৈরি হয় সেখানকার variable মনে রাখে, একে বলে **closure** (অধ্যায় ২.৬)। তাই এই function-এর ভেতরে `count` সবসময় 0।
- ফলে দুই লাইনই আসলে `setCount(0 + 1)`, মানে দুইবার `setCount(1)`।

এই সমস্যার নাম **stale value**: function একটা পুরনো snapshot-এর value দেখছে।

সমাধান: functional update

Setter-কে নতুন value না দিয়ে একটা **function** দাও:

```
function handleClick(): void
{
    setCount((c) => c + 1);
    setCount((c) => c + 1);
}
```

এবার click করলে value হবে **2**। React প্রতিটা function-কে সেই মুহূর্তের **সর্বশেষ** value দেয়: প্রথমটা পায় 0 আর দেয় 1, দ্বিতীয়টা পায় 1 আর দেয় 2। এখানে `count` variable-টা ব্যবহারই হয়নি, তাই পুরনো snapshot-এর প্রশ্ন নেই।

একে বলে **functional update**।

নিয়ম: নতুন state যদি পুরনো state-এর উপর নির্ভর করে, তাহলে functional update ব্যবহার করো।

৭.৪ Controlled Input: React-ই আসল মালিক

`frontend/src/features/auth/LoginPage.tsx` থেকে (সংক্ষেপে):

```
const [email, setEmail] = useState('');

<Input
    id="email"
    type="email"
    required
    value={email}
    onChange={(event): void => setEmail(event.target.value)}
/>
```

চক্রটা এরকম:

```
User টাইপ করে "a"
   ↓
onChange চলে → event.target.value = "a"
   ↓
setEmail("a") → state বদলায়
   ↓
re-render → <Input value="a" />
   ↓
screen-এ "a"
```

Input-এর value সবসময় state থেকে আসে, তাই একে বলে **controlled input**। সুবিধা:

- যেকোনো সময় `email` variable থেকে value জানা যায়।
- Value বদলানো, খালি করা বা validate করা খুব সহজ: শুধু state বদলাও।

`value` দিয়ে কিন্তু `onChange` না দিলে input "জমে" যায়, টাইপ করা যায় না, কারণ value সবসময় state থেকে আসছে আর state কখনো বদলাচ্ছে না। React তখন console-এ warning দেয়।

Checkbox

Checkbox-এ `value` নয়, `checked` ব্যবহার হয়, আর নতুন value পাওয়া যায় `event.target.checked` থেকে (true/false)। `ApplicationForm.tsx` থেকে (class বাদ):

```
<input
    type="checkbox"
    checked={values.isApplied}
    onChange={(event): void => setField('isApplied', event.target.checked)}
/>
```

Tick দিলে `isApplied` হয় `true`, tick তুলে দিলে `false`। `setField` কী, সেটা ৭.৬-এ দেখব।

৭.৫ Form submit করা

`LoginPage.tsx` থেকে (সংক্ষেপে)। Snippet-এ `dispatch(loginUser(...))` আছে, যেটা Redux-এর জিনিস (অধ্যায় ১০–১১)। এখন শুধু ধরো এই লাইন server-এ login request পাঠায়:

```
import type { JSX, SubmitEvent } from 'react';

function handleSubmit(event: SubmitEvent<HTMLFormElement>): void
{
    event.preventDefault();
    void dispatch(loginUser({ email, password }));
}

return (
    <form onSubmit={handleSubmit}>
        ...
        <Button type="submit">Log in</Button>
    </form>
);
```

- `<form>`-এর ভেতরে `type="submit"` button click করলে, অথবা কোনো input field-এ Enter চাপলে, form-এর `onSubmit` চলে।
- `event.preventDefault()` খুব জরুরি। Browser-এর পুরনো অভ্যাস হলো form submit হলে **পুরো page reload** করা। এই লাইন সেটা থামায়, কারণ আমাদের app-এ (single page app) request আমরা নিজেরাই JavaScript দিয়ে পাঠাই।
- `SubmitEvent<HTMLFormElement>` হলো এই event-এর type। `HTMLFormElement` হলো `<form>` element-এর type; মানে "একটা form থেকে আসা submit event"। `SubmitEvent` React থেকে import করা; `HTMLFormElement` browser-এর built-in type, import লাগে না।

- পুরনো tutorial-এ এর জায়গায় `FormEvent` দেখবে। আমাদের React version-এর type-এ `FormEvent` deprecated (আর ব্যবহার না করার পরামর্শ দেওয়া হয়েছে), তাই project-এ `SubmitEvent` ব্যবহার করা হয়েছে।
- `void` লেখা হয়েছে কারণ `dispatch(...)` একটা Promise return করে, আর আমরা ইচ্ছা করে তার জন্য অপেক্ষা করছি না (অধ্যায় ৩.৭)।
- `<Button type="button">` form submit করে না। আমাদের `Button`-এর default type `button` (অধ্যায় ৬.৭), যাতে form-এর ভেতরে অন্য কোনো button (যেমন "Apply by link") ভুল করে form submit না করে। Submit করতে হলে স্পষ্ট করে `type="submit"` লিখতে হয়।

HTML-এর নিজস্ব validation

`RegisterPage.tsx`-এর password field (সংক্ষেপে):

```
<Input
    id="password"
    type="password"
    required
    minLength={8}
    value={password}
    onChange={(event): void => setPassword(event.target.value)}
/>
```

আর email field-এ আছে `type="email"` আর `required`।

এই attribute-গুলো দিলে browser নিজেই submit-এর আগে check করে:

- `required`: খালি থাকলে submit হবে না।
- `type="email"`: email-এর format না মিললে submit হবে না।
- `minLength={8}`: ৮ অক্ষরের কম হলে submit হবে না।

Check fail করলে browser নিজের ছোট্ট message দেখায়, আর `onSubmit` চলেই না। Login আর Register page-এ এটুকুই যথেষ্ট, কারণ server-ও আবার validate করে।

৭.৬ অনেক field-এর form: একটা object state

আবেদনের form-এ ১২টা field। ১২টা আলাদা `useState` না লিখে একটা object রাখা হয়েছে।
`frontend/src/features/applications/ApplicationForm.tsx`:

```
const [values, setValues] = useState<ApplicationFormValues>(initialValues);

function setField<K extends keyof ApplicationFormValues>(field: K, value:
ApplicationFormValues[K]): void
{
    setValues((previous) => ({ ...previous, [field]: value }));
}
```

ব্যবহার (class আর id বাদ):

```
<Input value={values.jobPost} onChange={(event): void => setField('jobPost',
event.target.value)} />
<input type="checkbox" checked={values.isApplied} onChange={(event): void =>
setField('isApplied', event.target.checked)} />
```

প্রথম লাইনে টাইপ করলে `values.jobPost` বদলায়, বাকি ১১টা field যেমন ছিল তেমন থাকে। দ্বিতীয় লাইনে tick দিলে শুধু `values.isApplied` বদলায়।

`setField`-এর তিনটা জিনিস বুঝতে হবে।

(ক) Generic দিয়ে নিখুঁত type

`<K extends keyof ApplicationFormValues>`-কে ভাগ করে পড়ো:

- **`keyof ApplicationFormValues`**: form-এর সব field-এর নাম, মানে `'jobPost' | 'jobPostLink' | ... | 'isApplied' | ...` (অধ্যায় ৪.৭)।
- **`K`**: একটা generic type parameter (অধ্যায় ৪.৬)। Call করার সময় TypeScript নিজেই বের করে নেয় `K` কী।
- **`extends`**: এখানে `extends` মানে **constraint**, অর্থাৎ "`K`-কে অবশ্যই এর মধ্যে একটা হতে হবে"। এটা interface-এর inheritance (অধ্যায় ৪.৩) না, একই শব্দ কিন্তু আলাদা কাজ। তাই `field` হতে পারবে শুধু form-এর কোনো একটা field-এর নাম।

তারপর **`value: ApplicationFormValues[K]`** হলো indexed access (অধ্যায় ৪.৭): "`K` field-টার type"। যেমন `setField('isApplied', ...)` call করলে `K` হয় `'isApplied'`, আর `ApplicationFormValues['isApplied']` হলো `boolean`, তাই `value`-কে boolean হতে হবে।

ফলাফল:

- `setField('isApplied', true)` চলবে, কারণ `isApplied` একটা boolean।
- `setField('isApplied', 'yes')` লাল দাগ পাবে, কারণ `'yes'` boolean না।
- `setField('jobPst', 'Engineer')` লাল দাগ পাবে, কারণ `jobPst` নামে কোনো field নেই।

(খ) `[field]: value`: computed key

`{ [field]: value }` মানে object-এর key-এর নাম আসবে `field` variable-এর value থেকে। `field` যদি `'jobPost'` হয়, তাহলে এটা দাঁড়ায় `{ jobPost: value }`। আর `...previous` আগের সব field copy করে (অধ্যায় ২.১২), তারপর শুধু এই একটা field নতুন value দিয়ে বসে।

(গ) Functional update: `setValues((previous) => ...)`

এটা ৭.৩-এর functional update। নতুন object বানানো হচ্ছে পুরনো object থেকে, তাই `values` variable সরাসরি না ধরে React-এর দেওয়া সর্বশেষ `previous` ব্যবহার করা হয়েছে। এতে কখনো পুরনো snapshot-এর ফাঁদে পড়তে হয় না।

৭.৭ Immutability: State কখনো সরাসরি বদলাবে না

```
values.jobPost = 'Engineer';
setValues(values);

setValues({ ...values, jobPost: 'Engineer' });
```

প্রথম দুই লাইন ভুল: আসল object-টাই বদলে ফেলে সেই একই object আবার দেওয়া হয়েছে। শেষ লাইন ঠিক: পুরনোটা copy করে একটা নতুন object বানানো হয়েছে।

কেন? React পুরনো আর নতুন state তুলনা করে **reference** দিয়ে (`Object.is`), মানে "এটা কি memory-তে একই object?"। একই object বদলে দিলে React দেখে object একই আছে, ভাবে কিছুই বদলায়নি, তাই re-render করে না, screen-ও update হয় না।

State বদলানো যায় না, শুধু নতুন বানিয়ে বদলে দেওয়া যায়: এই নিয়মের নাম **immutability**।

Array-র বেলাতেও একই নিয়ম:

ভুল (আসলটা বদলায়)	ঠিক (নতুন বানায়)
`items.push(x)`	`[...items, x]`
`items.splice(i, 1)`	`items.filter((item) => item.id !== id)`
`items[i].done = true`	`items.map((item) => item.id === id ? { ...item, done: true } : item)`

এখানে `id` হলো যে item বদলাতে চাও তার id, আর `i` তার index।

ডান দিকের সব method (`filter`, `map`) আর spread নতুন array return করে, পুরনোটা ছোঁয় না (অধ্যায় ২.৮, ২.১২)।

(Redux Toolkit-এ এই নিয়ম লেখার ধরন একটু আলাদা, দেখব অধ্যায় ১০-এ।)

৭.৮ Derived value: State থেকে হিসাব করা যায় এমন জিনিস state-এ রাখবে না

আবেদনের form-এ "Apply by link" আর "Apply by mail" নামে দুটো button আছে। যেটা বেছে নেওয়া, সেটা indigo (`primary`), অন্যটা সাদা (`secondary`)। `ApplicationForm.tsx` থেকে (সংক্ষেপে):

```
{[ApplyMedium.Link, ApplyMedium.Mail].map((medium) => (
    <Button
        key={medium}
        variant={values.applyMedium === medium ? 'primary' : 'secondary'}
        onClick={(): void => setField('applyMedium', medium)}
    >
        {medium === ApplyMedium.Link ? 'Apply by link' : 'Apply by mail'}
    </Button>
))}
```

ফলাফল: শুরুতে "Apply by link" indigo। "Apply by mail" click করলে `values.applyMedium` বদলায়, re-render হয়, আর এবার "Apply by mail" indigo হয়ে যায়।

খেয়াল করো, "কোন button highlighted" এর জন্য আলাদা কোনো state নেই। এটা প্রতিটা render-এ `values.applyMedium` থেকে হিসাব করা হয়। একই কৌশল `applications.utils.ts`-এর `toPayload`-এও আছে:

```
const isLink = values.applyMedium === ApplyMedium.Link;
```

এখানে `isLink` একটা সাধারণ variable, প্রতিবার নতুন করে হিসাব হয়।

উল্টোটা করলে কী হতো? ধরো `const [isLinkSelected, setIsLinkSelected] = useState(true);` নামে আলাদা state রাখলাম। তখন `applyMedium` বদলালে প্রতিবার `isLinkSelected`-ও মনে করে বদলাতে হবে। একবার ভুলে গেলেই form বলবে medium "Mail", অথচ button দেখাবে "Link" selected।

State থেকে হিসাব করে বের করা এমন value-কে বলে **derived value**।

নিয়ম: যা props বা অন্য state থেকে হিসাব করা যায়, সেটা render-এর সময় সরাসরি variable-এ হিসাব করো। আলাদা state রাখলে দুটো value একে অপরের সাথে না মেলার bug তৈরি হয়।

৭.৯ Form validation: আমাদের নিজের `validateForm`

`frontend/src/features/applications/applications.utils.ts`:

```
export function validateForm(values: ApplicationFormValues): ApplicationFormErrors
{
    const errors: ApplicationFormErrors = {};

    if (!values.jobPost.trim())
    {
        errors.jobPost = 'Job post is required';
    }
    if (!isValidUrl(values.jobPostLink))
    {
        errors.jobPostLink = 'Enter a valid link starting with http:// or https://';
    }
    if (values.applyMedium === ApplyMedium.Link && !isValidUrl(values.applyLink))
    {
        errors.applyLink = 'Enter the link where you apply';
    }
    if (values.applyMedium === ApplyMedium.Mail && !htmlToText(values.applyMail))
    {
        errors.applyMail = 'Write the application mail';
    }
    if (values.cvLink && !isValidUrl(values.cvLink))
    {
        errors.cvLink = 'Enter a valid link starting with http:// or https://';
    }
    if (values.salaryMin && values.salaryMax && Number(values.salaryMin) >
Number(values.salaryMax))
    {
        errors.salaryMax = 'Maximum must be greater than minimum';
    }

    return errors;
}
```

এখানে দুটো helper function ব্যবহার হয়েছে, দুটোই `frontend/src/shared/utils.ts`-এ:

- `isValidUrl(value)` : string-টা `http://` বা `https://` দিয়ে শুরু হওয়া valid link হলে `true` , না হলে `false` । ভেতরে `new URL(value)` চেষ্টা করে; ভুল link হলে সেটা error throw করে, আর `catch` -এ `false` return হয় (অধ্যায় ২.১৩)।
- `htmlToText(html)` : Rich text editor HTML দেয়, যেমন `<p></p>` । এই function HTML tag বাদ দিয়ে শুধু লেখাটুকু বের করে। খালি string আসা মানে mail-এ আসলে কিছুই লেখা হয়নি।

ফলাফল: সব ঠিক থাকলে খালি object `{}` return হয়। ধরো বাকি সব field ঠিকঠাক পূরণ করা, শুধু job post খালি আর salary min 90000, max 50000; তাহলে return হবে `{ jobPost: 'Job post is required', salaryMax: 'Maximum must be greater than minimum' }` ।

এটা একটা সাধারণ function: values নেয়, errors object return করে। React-এর কিছুই জানে না, তাই আলাদা করে test করা সহজ।

Form-এ ব্যবহার

প্রথমে দেখো `ApplicationForm` কী কী props নেয়:

```
interface ApplicationFormProps
{
    initialValues: ApplicationFormValues;
    submitLabel: string;
    isSaving: boolean;
    onSubmit: (payload: ApplicationPayload) => void;
}
```

এখানে **`onSubmit` হলো `ApplicationForm` -এর একটা callback prop** (অধ্যায় ৬.৫), `<form>` -এর `onSubmit` attribute না। নাম এক, কিন্তু কাজ আলাদা: form নিজে জানে না data কোথায় পাঠাতে হবে, তাই validation পার হলে parent-কে data দিয়ে খবর দেয়। Parent ঠিক করে নতুন বানাবে নাকি পুরনোটা update করবে।

`ApplicationForm` -এর ভেতরে:

```
function handleSubmit(event: SubmitEvent<HTMLFormElement>): void
{
    event.preventDefault();
    const validationErrors = validateForm(values);
    setErrors(validationErrors);
    if (Object.keys(validationErrors).length === 0)
    {
        onSubmit(toPayload(values));
    }
}
```

1. Page reload থামাও।
2. `validateForm` দিয়ে সব error বের করো, আর `errors` state-এ রাখো (এতে re-render হবে আর error দেখাবে)।
3. **`Object.keys(obj)`** একটা object-এর সব key-এর নাম array হিসেবে দেয়। যেমন `Object.keys({ jobPost: '...' })` হলো `['jobPost']` । Length 0 মানে কোনো error নেই।
4. Error না থাকলে form-এর values server-এর format-এ convert করে (`toPayload` , নিচে দেখো) parent-এর `onSubmit` callback call করো।

Error থাকলে `Field` -এর নিচে লাল লেখা দেখায়, কারণ `Field` `error` prop পায় (অধ্যায় ৬.৬):

```
<Field label="Job post" htmlFor="jobPost" error={errors.jobPost}>
```

`errors.jobPost` undefined হলে কিছু দেখায় না, string হলে লাল রঙে দেখায়।

এই form-এ `<form onSubmit={handleSubmit} noValidate>` লেখা আছে। **`noValidate`** browser-এর নিজস্ব validation (৭.৫) বন্ধ রাখে, যাতে browser-এর ছোট্ট message-এর বদলে শুধু আমাদের নিজের, field-এর নিচের error message দেখায়।

Form-এর value আর server-এর data আলাদা

- Input field সবসময় **string** দেয়, যেমন salary `"50000"` , আর খালি থাকলে `""` ।
- Server চায় `50000` (number) বা `null` ।

তাই দুটো conversion function আছে, দুটোই `applications.utils.ts` -এ:

- **`toFormValues(application)`** : server-এর data থেকে form-এর values বানায় (edit-এর সময়)। যেমন salary `50000` হয়ে যায় `"50000"` , আর `null` হয়ে যায় `""` ।
- **`toPayload(values)`** : form-এর values থেকে server-এ পাঠানোর data বানায়।

`toPayload` -এর দুটো লাইন (সংক্ষেপে):

```
salaryMin: values.salaryMin ? Number(values.salaryMin) : null,
cvLink: values.cvLink.trim() || null,
```

- `"50000"` হয় `50000`, আর `""` হয় `null` (খালি string falsy, অধ্যায় ২.৪)।
- CV link-এর আগে-পিছের space কেটে খালি হলে `null`, না হলে link-টাই।

৭.১০ `key` দিয়ে component-কে নতুন করে শুরু করানো

অধ্যায় ৬.৯-এ `key` দেখেছি list-এর জন্য। কিন্তু `key` যেকোনো component-এ দেওয়া যায়, আর এর একটা জরুরি ব্যবহার আছে।

নিয়ম: কোনো component-এর `key` বদলালে React পুরনো component-টা unmount করে, আর একদম নতুন একটা mount করে। নতুন component-এর সব state শুরু হয় শূন্য থেকে।

`ApplicationFormPage.tsx` থেকে (সংক্ষেপে)। এখানে `id` আসে URL থেকে `useParams` দিয়ে (অধ্যায় ৯), আর `selected` আসে Redux থেকে (অধ্যায় ১১)। "New application" page-এ `id` থাকে না (undefined), আর "Edit" page-এ থাকে:

```
<ApplicationForm
    key={id ?? 'new'}
    initialValues={id && selected ? toFormValues(selected) : EMPTY_FORM_VALUES}
    ...
/>
```

- `id ?? 'new'` (অধ্যায় ২.১১): `id` থাকলে key হয় সেই id, না থাকলে `'new'`।
- নতুন form হলে `initialValues` খালি values, edit হলে সেই application-এর values।

Key না দিলে কী সমস্যা হতো?

৭.২-এ বলেছি: `useState(initialValues)`-এর শুরুর value **শুধু mount-এর সময়** কাজে লাগে। ধরো user প্রথমে application A edit করছিল, তারপর browser-এর Back button দিয়ে (page reload ছাড়া) "New application" page-এ ফিরে গেল। (Address bar-এ URL লিখে Enter চাপলে কিন্তু পুরো page reload হয়, সব state মুছে নতুন করে শুরু হয়, তাই সেখানে এই সমস্যা হয় না।) `frontend/src/app/AppRouter.tsx`-এ দুটো route-ই একই `<ApplicationFormPage />` element দেখায়, আর "New" page-এ `id` না থাকায় Spinner-ও আসে না। তাই একই `ApplicationForm` mount থাকে, শুধু নতুন `initialValues` prop (খালি values) পায়। কিন্তু `values` state সেটা পাত্তা দেয় না, আগের মতোই A-এর লেখা ধরে রাখে। ফলে key না থাকলে নতুন form-এ A-এর data দেখাত।

`key={id ?? 'new'}` থাকায় A-এর id থেকে `'new'`-এ গেলে key বদলায়। React তখন পুরনো form unmount করে নতুন form mount করে, আর নতুন form-এর `useState` খালি `initialValues` দিয়ে শুরু হয়।

(এক application-এর edit থেকে আরেকটার edit-এ গেলে সমস্যাটা এমনিতেই হয় না: `ApplicationFormPage` নতুন application load না হওয়া পর্যন্ত, মানে `selected?.id !== id` থাকা পর্যন্ত, form-এর জায়গায় Spinner দেখায়, তাই form তখন unmount হয়ে যায়।)

৭.১১ State কোথায় রাখবে?

State	কোথায়	উদাহরণ
শুধু এক component-এর দরকার	ওই component-এ `useState`	Form-এর values, search box-এর লেখা
পাশাপাশি কয়েকটা component-এর দরকার	তাদের সবচেয়ে কাছের common parent-এ	(**lifting state up**)
অনেক দূরের অনেক component-এর দরকার	Redux store (অধ্যায় ১০)	Login করা user, আবেদনের list আর filter

Lifting state up: দুটো sibling component (একই parent-এর দুই child) একই data চাইলে, state তাদের parent-এ নিয়ে যাও। Parent props দিয়ে দুজনকে data দেয়, আর callback দিয়ে বদলানোর সুযোগ দেয় (অধ্যায় ৬.৫-এর one-way data flow)।

আমাদের project-এ:

- Form-এর values শুধু `ApplicationForm`-এর দরকার, তাই local state।
- Login করা user লাগে navbar, `ProtectedRoute`, login page সবার, তাই Redux।
- Filter লাগে filter bar আর page দুজনের, আর details page থেকে ফিরে এলেও মনে রাখতে হয়, তাই Redux।

সারসংক্ষেপ

- `useState` component-কে re-render-এর মাঝে value মনে রাখার ক্ষমতা দেয়। Setter call করলে re-render হয়।
- **Hook** হলো `use` দিয়ে শুরু হওয়া React function, শুধু component-এর top level-এ call করা যায়।

- প্রতিটা render হলো সেই মুহূর্তের state-এর একটা **snapshot**। Setter-এর পরের লাইনে পুরনো value-ই থাকে।
- নতুন state পুরনোটার উপর নির্ভর করলে **functional update** (`setCount((c) => c + 1)`), নইলে stale value-এর ফাঁদে পড়বে।
- **Controlled input:** `value` আসে state থেকে, আর `onChange` দিয়ে state update হয়।
- `onSubmit`-এ `event.preventDefault()` দিয়ে page reload থামাতে হয়; event-এর type `SubmitEvent<HTMLFormElement>`।
- State **কখনো সরাসরি বদলাবে না**, সবসময় নতুন object বা array বানাবে (**immutability**)।
- হিসাব করা যায় এমন জিনিস state-এ রাখবে না (**derived value**)।
- `key` বদলালে component unmount হয়ে নতুন করে mount হয়, state শুরু থেকে।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: setState কি সাথে সাথে state বদলায়?
উত্তর: না। Setter call করলে React update-টা পরে করার জন্য তালিকায় রাখে (schedule করে), আর একই event-এর ভেতরের কয়েকটা update একসাথে করে একবারই re-render করে, একে বলে batching। নতুন value দেখা যায় পরের render-এ। তাই setter-এর ঠিক পরের লাইনে পুরনো value-ই পাওয়া যায়, কারণ প্রতিটা render হলো state-এর একটা snapshot।

প্রশ্ন: Functional update কখন ব্যবহার করবে?
উত্তর: নতুন state পুরনো state-এর উপর নির্ভর করলে। যেমন `setCount(count + 1)` দুইবার লিখলে দুটোই একই পুরনো snapshot-এর `count` দেখে, তাই value মাত্র 1 বাড়ে। `setCount((c) => c + 1)` দুইবার লিখলে React প্রতিবার সর্বশেষ value দেয়, তাই 2 বাড়ে। Object-এর বেলায় `setValues((previous) => ({ ...previous, jobPost: 'X' }))`।

প্রশ্ন: State কেন immutable রাখতে হয়?
উত্তর: React পুরনো আর নতুন state-এর reference তুলনা করে বোঝে state বদলেছে কিনা। আসল object বদলে দিলে reference একই থাকে, তাই React ভাবে কিছু বদলায়নি আর re-render করে না। তাছাড়া immutable data-য় কোথায় কী বদলেছে বোঝা আর bug ধরা সহজ হয়।

প্রশ্ন: Controlled আর uncontrolled component-এর পার্থক্য কী?
উত্তর: Controlled component-এ input-এর value React state-এ থাকে: `value` আসে state থেকে আর `onChange` দিয়ে state বদলায়। Uncontrolled component-এ value DOM নিজে রাখে, React শুধু দরকারের সময় পড়ে নেয়, সাধারণত `ref` দিয়ে। `ref` হলো একটা DOM element-কে সরাসরি ধরে রাখার React-এর উপায়। Controlled-এ validation, value বদলানো আর খালি করা সহজ, তাই আমাদের project-এর সব form controlled।

প্রশ্ন: Hook কী? এর নিয়ম কী?

উত্তর: Hook হলো `use` দিয়ে শুরু হওয়া React function, যেমন `useState`, যেটা component-কে state-এর মতো বাড়তি ক্ষমতা দেয়। Hook শুধু component function-এর top level-এ call করা যায়, `if`, loop বা ভেতরের function-এর মধ্যে না, যাতে প্রতিটা render-এ hook-গুলো একই ক্রমে call হয়।

প্রশ্ন: Component-এর key বদলালে কী হয়?

উত্তর: React পুরনো component unmount করে নতুন একটা mount করে, তাই সব state শুরুর value থেকে আবার শুরু হয়। Edit form থেকে New form-এ গেলে পুরনো লেখা মুছে নতুন করে শুরু করাতে এই কৌশল ব্যবহার হয়, কারণ `useState`-এর শুরুর value শুধু mount-এর সময় নেওয়া হয়।

প্রশ্ন: Lifting state up কী?

উত্তর: দুই বা তার বেশি component-এর একই data লাগলে state-কে তাদের সবচেয়ে কাছের common parent-এ নিয়ে যাওয়া। Parent props দিয়ে data দেয়, আর callback দিয়ে child-দের বদলানোর সুযোগ দেয়।

প্রশ্ন: কখন useState আর কখন global state (Redux)?

উত্তর: যে data শুধু একটা component বা ছোট একটা অংশের দরকার, সেটা useState-এ। যে data app-এর অনেক দূরের অংশের দরকার, page বদলালেও মনে রাখতে হয়, বা অনেক জায়গা থেকে বদলায়, যেমন login করা user বা server থেকে আনা list, সেটা Redux-এ।

প্রশ্ন: Re-render কখন হয়?

উত্তর: তিনভাবে: component-এর নিজের state বদলালে, parent re-render হলে (তখন child-রাও সাধারণত re-render হয়), অথবা component Redux store থেকে যে data পড়ছে সেটা বদলালে।

অধ্যায় ৮: Side Effect আর `useEffect`

Component-এর মূল কাজ হলো JSX return করা। কিন্তু মাঝে মাঝে **render-এর বাইরের** কাজ করতে হয়: server থেকে data আনা, timer চালানো। এগুলোকে বলে **side effect**।

> **আগে একটু heads-up:** এই অধ্যায়ের উদাহরণগুলো আমাদের আসল project থেকে নেওয়া, তাই সেখানে Redux-এর কিছু জিনিস আছে, যেগুলো বিস্তারিতভাবে আসবে অধ্যায় ১০–১১-এ। আপাতত শুধু এটুকু জানলেই চলবে:
>
> | কোড | এখনকার জন্য মানে |
> |---|---|
> | **store** | পুরো app-এর shared data রাখার একটা জায়গা |
> | `useAppSelector((state) => ...)` | store থেকে একটা value পড়া; value বদলালে component re-render হয় |
> | `dispatch(...)` | store-কে "এই কাজটা করো" বলা |
> | `dispatch(loadApplications(filters))` | এই `filters` দিয়ে server থেকে application-এর list আনো |
> | `dispatch(filtersChanged({ search }))` | store-এর filter-এ নতুন search লেখা বসাও |
>
> বিস্তারিত অধ্যায় ১০–১১-এ।

৮.১ Side effect কী, আর render-এর ভেতরে কেন নয়?

Render (অর্থাৎ component function চলা) হওয়া উচিত **pure**: একই props আর state দিলে সবসময় একই JSX আসবে, আর function-এর বাইরের কিছু বদলাবে না।

ধরো ভুল করে render-এর ভেতরেই server call করলাম (এটা ভুল উদাহরণ):

```
function ApplicationsPage()
{
    dispatch(loadApplications(filters));
    return ...;
}
```

তাহলে এই চক্র তৈরি হতো:

1. Component render হলো, আর render-এর ভেতরে server call গেল।
2. Data এসে store-এ বসল, তাই component আবার re-render হলো।
3. Re-render মানে function আবার চলা, তাই আবার server call গেল।
4. আবার data এল, আবার re-render... এভাবে চলতেই থাকবে।

এটাই **infinite loop**: যে loop কখনো থামে না।

তাই side effect রাখতে হয় `useEffect` -এর ভেতরে। `useEffect` একটা hook (অধ্যায় ৭), যেটা তোমার দেওয়া function চালায় **render শেষ হয়ে screen-এ দেখানোর পরে**, আর শুধু তখনই, যখন তুমি বলবে।

৮.২ `useEffect` -এর structure

মনে করিয়ে দিই (অধ্যায় ৬): component প্রথমবার screen-এ আসাকে বলে **mount**, আর screen থেকে সরে যাওয়াকে বলে **unmount**।

```
useEffect(() =>
{
    ...
    return () => { ... };
}, [dependency1, dependency2]);
```

- প্রথম argument একটা function: এর ভেতরে side effect-এর কাজ লেখা হয়।
- সেই function চাইলে আরেকটা function return করতে পারে, যাকে বলে **cleanup function**। এটা optional, ৮.৫-এ বিস্তারিত।
- দ্বিতীয় argument হলো **dependency array**: কোন কোন value বদলালে effect আবার চলবে, তার list।

Dependency array	Effect কখন চলে
`[]` (খালি)	শুধু mount-এর পরে, একবার
`[a, b]`	mount-এর পরে, তারপর যখনই `a` বা `b` বদলায়
একদমই না দিলে	**প্রতিটা** render-এর পরে (সাধারণত ভুল)

"বদলায়" মানে React আগের render-এর value আর এখনকার value `Object.is` দিয়ে তুলনা করে, যেটা প্রায় `===` -এর মতো। Object বা array হলে তুলনা হয় **reference** দিয়ে: একই object হলে "বদলায়নি", নতুন object হলে "বদলেছে", ভেতরের লেখা এক হলেও।

৮.৩ উদাহরণ ১: data আনা, আর filter বদলালে আবার আনা

`frontend/src/features/applications/ApplicationsPage.tsx` (সংক্ষেপে):

```
const dispatch = useAppDispatch();
const { filters, items, meta, listStatus, listError } = useAppSelector((state) =>
state.applications);

useEffect((): void =>
{
    void dispatch(loadApplications(filters));
}, [dispatch, filters]);
```

`useAppDispatch()` হলো `dispatch` function পাওয়ার hook (অধ্যায় ১০)। `void` লেখা হয়েছে কারণ `dispatch(loadApplications(...))` একটা Promise return করে, আর আমরা সেটার জন্য অপেক্ষা করছি না (অধ্যায় ৩)।

কী ঘটে:

1. Page mount হলো। Effect চলে, আর list load হয়।
2. User filter বদলাল। Store-এ নতুন `filters` object এল। `filters` বদলেছে, তাই effect আবার চলে, আর নতুন filter দিয়ে list আসে।
3. List load হয়ে `items` বদলাল। Component re-render হয়, কিন্তু effect চলে **না**, কারণ `filters` বদলায়নি। তাই infinite loop হয় না।

তৃতীয় ধাপটা কেন কাজ করে? Store update হলে **শুধু store-এর যে অংশ বদলায়, সেটাই নতুন object হয়**; বাকি অংশ আগের object-ই থাকে। তাই `items` বদলালেও `filters`-এর reference একই থাকে, আর React দেখে "`filters` বদলায়নি" (বিস্তারিত অধ্যায় ১০)।

`dispatch` কেন dependency-তে?

ESLint-এর `react-hooks/exhaustive-deps` rule বলে: effect-এর ভেতরে component-এর যে value ব্যবহার করবে, সব dependency array-তে দিতে হবে। `dispatch` কখনো বদলায় না (প্রতিটা render-এ একই function), তাই এটা দিলে effect বাড়তি চলে না, শুধু rule মানা হয়।

৮.৪ উদাহরণ ২: URL-এর id বদলালে নতুন data

Heads-up: `useParams` হলো React Router-এর একটা hook, যেটা URL থেকে `:id`-এর মতো অংশ পড়ে দেয় (অধ্যায় ৯)। `/applications/42` URL-এ `id` হবে `'42'`।

`ApplicationDetailsPage.tsx` (সংক্ষেপে):

```
const { id = '' } = useParams<{ id: string; }>();

useEffect((): void =>
{
    void dispatch(loadApplication(id));
}, [dispatch, id]);
```

`loadApplication(id)` মানে server থেকে একটা application আনো। ফলাফল:

- Page mount হলে effect চলে, আর ওই `id`-এর application load হয়।
- `/applications/1` থেকে `/applications/2`-তে গেলে component unmount হয় না, একই component থেকে যায়, শুধু `id` বদলায়।
- Dependency-তে `id` থাকায় effect আবার চলে, আর নতুন application load হয়।

`id` dependency-তে না থাকলে দ্বিতীয় ক্ষেত্রে নতুন application কখনো load হতো না। আমাদের page-এ `selected.id !== id` হলে Spinner দেখায় (অধ্যায় ৬.৮-এর early return), তাই screen-এ চিরকাল Spinner ঘুরত।

শুধু একবার চালানোর কাজ

`LoginPage.tsx`:

```
useEffect((): void =>
{
    dispatch(authErrorCleared());
}, [dispatch]);
```

`dispatch` কখনো বদলায় না, তাই এই effect কার্যত শুধু mount-এর পরে একবার চলে। কাজটা হলো: login page-এ আসার সময় আগের কোনো পুরনো error message মুছে ফেলা।

৮.৫ Cleanup: পরিষ্কার করে যাওয়া

Effect থেকে একটা function return করলে React সেটাকে cleanup function হিসেবে রাখে, আর চালায়:

- effect **আবার চলার ঠিক আগে**, আর
- component **unmount হওয়ার সময়**।

উদাহরণ ৩: Debounce — প্রতিটা অক্ষরে নয়, থামার পরে খোঁজো

Search box-এ "engineer" type করলে প্রতিটা অক্ষরে (৮ বার) server-এ request গেলে অপচয় হতো। তাই user **type করা থামানোর ৪০০ms পরে** একবারই খোঁজা হয়। এই কৌশলকে বলে **debounce**।

এখানে দুটো browser function লাগবে:

- `window.setTimeout(fn, 400)`: ৪০০ms পরে `fn` চালাবে, আর একটা timer id (number) return করে।
- `window.clearTimeout(timerId)`: ওই timer বাতিল করে, `fn` আর চলবে না।

`frontend/src/features/applications/ApplicationFilters.tsx` (সংক্ষেপে):

```
const filters = useAppSelector((state) => state.applications.filters);
const [search, setSearch] = useState(filters.search);

useEffect((): (() => void) | undefined =>
{
    if (search === filters.search)
    {
        return undefined;
    }
    const timer = window.setTimeout((): void =>
    {
        dispatch(filtersChanged({ search }));
    }, 400);
    return (): void => window.clearTimeout(timer);
}, [dispatch, search, filters.search]);

<Input value={search} onChange={(event): void => setSearch(event.target.value)} />
```

Return type `(() => void) | undefined` কেন?

Effect function দুই রকম জিনিস return করতে পারে:

- `() => void` type-এর একটা function, অর্থাৎ **cleanup function**, অথবা
- `undefined`, অর্থাৎ **কিছুই না**।

এই effect-এ দুটোই ঘটে: `search === filters.search` হলে `undefined` return করে (কোনো timer নেই, তাই পরিষ্কার করারও কিছু নেই), আর না হলে `clearTimeout` চালানোর cleanup function return করে। তাই type হলো এই দুইয়ের union।

সময় ধরে কী ঘটে

ধরো store-এ `filters.search` এখন `""`, আর user "eng" type করছে:

সময়	Event	Effect কী করে
0ms	"e" type → `search = "e"`	`"e" !== ""`, তাই ৪০০ms-এর timer ① বসায়
100ms	"en" type → `search = "en"`	আগে cleanup: timer ① বাতিল। তারপর নতুন timer ②
200ms	"eng" type → `search = "eng"`	cleanup: timer ② বাতিল। নতুন timer ③
200–600ms	user থামল	কিছু ঘটে না, timer ③ অপেক্ষা করছে
600ms	timer ③ চলে	`dispatch(filtersChanged({ search: "eng" }))`
তারপর	store-এ `filters.search = "eng"`	`filters.search` বদলেছে, তাই effect আবার চলে। এবার `search === filters.search` (দুটোই `"eng"`), তাই কোনো timer বসায় না

শেষ ধাপটাই `if (search === filters.search)` guard-এর কাজ। Guard না থাকলে store update হওয়ার পরে আবার timer বসত, আবার একই search dispatch হতো, যদিও কিছুই বদলায়নি।

আর store-এর `filters` বদলানোয় `ApplicationsPage`-এর effect (৮.৩) চলে, আর server-এ মাত্র **একবার** request যায়।

মূল কথা:

- **Local state (`search`)** সাথে সাথে বদলায়, তাই input-এ type করা smooth থাকে।
- **Store-এর `filters.search`** বদলায় শুধু user থামার পরে।
- **Cleanup** (`clearTimeout`) না থাকলে প্রতিটা অক্ষরের timer চলে যেত, আর ৩টা বা তার বেশি request যেত; debounce কাজই করত না।

Cleanup-এর আরো কিছু উদাহরণ

Effect-এ যা শুরু করেছ	Cleanup-এ যা করবে
`setInterval`	`clearInterval`
`window.addEventListener(...)`	`window.removeEventListener(...)`

Effect-এ যা শুরু করেছ	Cleanup-এ যা করবে
WebSocket connection খোলা	connection বন্ধ করা

৮.৬ StrictMode-এ effect দুইবার চলে কেন?

আমাদের `main.tsx` -এ পুরো app `<StrictMode>` -এর ভেতরে আছে। Development-এ StrictMode প্রতিটা component mount হওয়ার সময় ইচ্ছা করে এই কাজ করে:

1. effect চালায়,
2. cleanup চালায় (যেন component unmount হলো),
3. **আবার effect চালায়** (যেন আবার mount হলো)।

উদ্দেশ্য হলো দেখা যে তোমার cleanup ঠিকমতো লেখা আছে কিনা। Cleanup ঠিক থাকলে দুইবার চললেও ফলাফল একই হয়। তাই development-এ browser-এর Network tab-এ একই request দুইবার দেখা স্বাভাবিক। Production build-এ effect একবারই চলে।

৮.৭ Hook-এর নিয়ম (Rules of Hooks)

`use` দিয়ে শুরু হওয়া function-গুলোই hook: `useState`, `useEffect`, `useParams`, `useAppSelector`... এদের দুটো কঠোর নিয়ম:

1. **Hook শুধু component function-এর top level-এ call করবে।** অর্থাৎ সরাসরি function body-তে, `if`, loop বা nested function-এর ভেতরে না।
2. **Hook শুধু React component বা অন্য hook-এর ভেতরে call করবে।** সাধারণ JavaScript function-এ না।

কেন? React কোন `useState` কোনটা, তা নাম দিয়ে চেনে না, চেনে **call order** দিয়ে: "এই component-এর প্রথম hook, দ্বিতীয় hook..."। প্রতিটা render-এ hook একই order-এ call হতে হবে। `if` -এর ভেতরে hook থাকলে কোনো render-এ সেটা বাদ পড়তে পারে, তখন পরের সব hook-এর order এক ঘর সরে যায়, আর React ভুল state ভুল hook-কে দিয়ে দেয়।

```
if (user)
{
    const [x, setX] = useState(0);
}
```

এটা ভুল: `user` থাকলে hook call হয়, না থাকলে হয় না, তাই call order বদলে যায়।

Early return মানে function-এর শেষে পৌঁছানোর আগেই মাঝপথে `return` করা। Early return-এর পরে hook থাকলে সেটাও "কখনো call হয়, কখনো হয় না"। তাই নিয়ম হলো: সব hook আগে, early return পরে।

`LoginPage`-এ ঠিক এটাই করা হয়েছে। এখানে `Navigate`, `useLocation` আর `redirectPathFrom` হলো routing-এর জিনিস (অধ্যায় ৯); এখন শুধু hook-গুলোর জায়গা খেয়াল করো:

`LoginPage.tsx` (সংক্ষেপে):

```
export function LoginPage(): JSX.Element
{
    const dispatch = useAppDispatch();
    const location = useLocation();
    const { user, status, error } = useAppSelector((state) => state.auth);
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');

    useEffect(...);

    if (user)
    {
        return <Navigate to={redirectPathFrom(location.state)} replace />;
    }
    ...
}
```

ফলাফল: `user` থাকুক বা না থাকুক, প্রতিটা render-এ ছয়টা hook একই order-এ call হয়; তারপরে `if (user)` early return।

৮.৮ সাধারণ ভুল

ভুল	ফলাফল	সমাধান
Dependency array না দেওয়া	প্রতি render-এ effect চলে, effect-এ state বদলালে infinite loop	সঠিক dependency দাও
ব্যবহৃত variable dependency-তে না দেওয়া	Effect পুরনো value নিয়ে কাজ করে (stale closure, নিচে interview-তে)	ESLint যা বলে তা মানো
Effect-এর ভেতরে সাথে সাথে `setState`	একটা বাড়তি render, আর কোড জটিল	যা হিসাব করে বের করা যায়, তা render-এই হিসাব করো (নিচে উদাহরণ)

ভুল	ফলাফল	সমাধান
Cleanup ভুলে যাওয়া	Timer বা listener জমতে থাকে (memory leak)	Cleanup function return করো
`useEffect(async () => ...)`	Async function সবসময় Promise return করে, অথচ React চায় cleanup function অথবা কিছুই না	ভেতরে আলাদা async function বানিয়ে call করো, অথবা `void dispatch(...)`

Effect-এ `setState` না করে derived value

ভুল পদ্ধতি:

```
const [first, setFirst] = useState('Rahim');
const [last, setLast] = useState('Uddin');
const [fullName, setFullName] = useState('');

useEffect(): void =>
{
    setFullName(`${first} ${last}`);
}, [first, last]);
```

এখানে প্রথম render-এ `fullName` খালি `''` দেখায়, তারপর effect চলে `setFullName` করে, আর দ্বিতীয় render-এ `'Rahim Uddin'` দেখায়। অর্থাৎ একটা বাড়তি render, আর এক মুহূর্তের ভুল লেখা।

সঠিক পদ্ধতি (derived value, অধ্যায় ৭):

```
const [first, setFirst] = useState('Rahim');
const [last, setLast] = useState('Uddin');
const fullName = `${first} ${last}`;
```

এখানে প্রথম render থেকেই `fullName` হলো `'Rahim Uddin'`, কোনো বাড়তি state বা effect নেই।

সারসংক্ষেপ

- Render-এর বাইরের কাজ (server call, timer) হলো **side effect**, যা `useEffect`-এর ভেতরে রাখতে হয়। Render-এ রাখলে infinite loop হতে পারে।
- Dependency array ঠিক করে effect কখন চলবে: `[]` মানে শুধু mount-এর পরে, `[x]` মানে mount-এর পরে আর `x` বদলালে।

- **Cleanup function** চলে effect আবার চলার আগে আর component unmount হওয়ার সময়।
- **Debounce** তৈরি হয় `setTimeout` আর cleanup-এর `clearTimeout` দিয়ে; guard থাকলে অপ্রয়োজনীয় timer বসে না।
- Development-এ StrictMode effect → cleanup → effect চালিয়ে cleanup test করে।
- Hook শুধু component বা অন্য hook-এর top level-এ call করবে, কারণ React hook চেনে call order দিয়ে।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: useEffect-এর dependency array খালি দিলে, একদম না দিলে, আর value দিলে কী হয়?
উত্তর: খালি array দিলে effect শুধু mount-এর পরে একবার চলে; mount মানে component প্রথমবার screen-এ আসা। Array একদম না দিলে প্রতিটা render-এর পরে চলে। Value দিলে mount-এর পরে চলে, আর তারপর যখনই ওই value-গুলোর কোনোটা আগের render থেকে বদলায়, তখন আবার চলে। Object বা array-এর ক্ষেত্রে "বদলানো" মানে reference বদলানো।

প্রশ্ন: Cleanup function কখন চলে?
উত্তর: Cleanup function হলো effect-এর return করা function। এটা দুই সময় চলে: effect পরের বার চলার ঠিক আগে, আর component unmount হওয়ার সময়, অর্থাৎ screen থেকে সরে যাওয়ার সময়। Timer বাতিল, event listener সরানো, connection বন্ধ করার মতো কাজ এখানে করা হয়।

প্রশ্ন: Debounce আর Throttle-এর পার্থক্য কী?
উত্তর: Debounce event থামার নির্দিষ্ট সময় পরে কাজটা একবার চালায়; থামার আগে নতুন event এলে timer আবার শুরু হয়। যেমন search box-এ type থামার ৪০০ms পরে খোঁজা। Throttle নির্দিষ্ট সময়ে সর্বোচ্চ একবার চালায়, event চলতে থাকলেও। যেমন scroll চলার সময় প্রতি ২০০ms-এ একবার position হিসাব করা।

প্রশ্ন: Rules of Hooks কী? কেন?
উত্তর: দুটো নিয়ম। এক, hook শুধু component বা custom hook-এর top level-এ call করা যায়, `if`, loop বা nested function-এর ভেতরে না। দুই, hook শুধু React component বা অন্য hook-এর ভেতরে call করা যায়। কারণ React প্রতিটা hook-কে চেনে call order দিয়ে। কোনো render-এ একটা hook বাদ পড়লে order বদলে যায়, আর React এক hook-এর state অন্য hook-কে দিয়ে দেয়।

প্রশ্ন: Stale closure কী?
উত্তর: Closure মানে একটা function তার তৈরির সময়ের আশেপাশের variable মনে রাখে। Stale closure হলো যখন একটা function তৈরির সময়কার পুরনো state বা props মনে রাখে, আর পরে value বদলালেও পুরনোটাই দেখে। useEffect-এ ব্যবহৃত value dependency array-তে না দিলে value বদলালেও effect আবার চলে না, তাই আগের render-এর পুরনো value নিয়েই কাজ করে। সমাধান হলো সঠিক dependency দেওয়া, অথবা state update-এর জন্য functional update ব্যবহার করা।

প্রশ্ন: StrictMode-এ useEffect দুইবার চলে কেন?

উত্তর: শুধু development-এ, cleanup ঠিকমতো লেখা হয়েছে কিনা test করার জন্য। React mount-এর পরে ইচ্ছা করে একবার unmount আর আবার mount simulate করে, তাই effect চলে, cleanup চলে, আবার effect চলে। Cleanup ঠিক থাকলে ফলাফলে কোনো পার্থক্য হয় না। Production-এ effect একবারই চলে।

প্রশ্ন: Custom hook কী?

উত্তর: `use` দিয়ে নাম শুরু হওয়া একটা সাধারণ function, যেটা ভেতরে অন্য hook ব্যবহার করে। এতে state আর effect-সহ একটা logic একাধিক component-এ reuse করা যায়, আর component-এর কোড ছোট থাকে। যেমন আমাদের project-এ Tiptap (rich text editor-এর একটা library, `shared/components/RichTextEditor.tsx` -এ ব্যবহার হয়েছে; অধ্যায় ১৩-এ দেখবে)-এর `useEditor` hook একটা rich text editor বানিয়ে return করে, আর component শুধু সেটা ব্যবহার করে। আমাদের `useAppSelector` আর `useAppDispatch` -ও hook, তবে এগুলো কীভাবে বানানো হয়েছে, তা অধ্যায় ১০-এ দেখব।

অধ্যায় ৯: Routing — React Router

আমাদের app-এ কয়েকটা "page" আছে: login, register, application list, নতুন application, details, edit। কিন্তু SPA-তে আসলে HTML page একটাই। তাহলে আলাদা আলাদা page কীভাবে? উত্তর হলো **client-side routing**।

> **Heads-up:** এই অধ্যায়ের কিছু snippet-এ `useAppSelector` আর `dispatch` আছে। অধ্যায় ৮-এর মতোই: `useAppSelector` দিয়ে store (app-এর shared data) থেকে value পড়া হয়, আর `dispatch(...)` দিয়ে store-কে কাজ করতে বলা হয়। বিস্তারিত অধ্যায় ১০–১১-এ।

৯.১ Client-side routing কী?

Browser-এর একটা built-in API আছে, যার নাম **History API**। এটা দিয়ে JavaScript page reload না করেই address bar-এর URL বদলাতে পারে, আর browser-এর history-তে (Back/Forward-এর list-এ) নতুন entry যোগ করতে পারে।

Client-side routing মানে:

- URL বদলানো হয় JavaScript দিয়ে, History API ব্যবহার করে, **server থেকে নতুন page না এনে**।
- URL দেখে ঠিক করা হয় কোন component screen-এ দেখাবে।
- Browser-এর Back/Forward button ঠিকমতো কাজ করে।
- URL copy করে অন্য কাউকে দিলে সে একই page দেখে।

এই কাজের library হলো **React Router** (`react-router` package)।

৯.২ সব URL এক জায়গায়: `AppRoutes` enum

`frontend/src/shared/constants.ts`:

```
export enum AppRoutes
{
    Home = '/',
    Login = '/login',
    Register = '/register',
    Applications = '/applications',
    ApplicationCreate = '/applications/new',
    ApplicationDetails = '/applications/:id',
    ApplicationEdit = '/applications/:id/edit',
}
```

URL-কে `/` দিয়ে ভাগ করলে প্রতিটা টুকরাকে বলে **segment**। `/applications/new`-এ দুটো segment: `applications` আর `new`। দুটোই হুবহু লেখা, তাই এগুলো **static segment**।

`:id` আলাদা: এটা একটা **URL parameter**, একটা placeholder, যার জায়গায় যেকোনো লেখা বসতে পারে। `/applications/42` URL-এ `id` হবে `'42'`।

`buildPath`: placeholder-এ value বসিয়ে URL বানানো

`frontend/src/shared/utils.ts`:

```
export function buildPath(template: string, params: Record<string, string> = {}): string
{
    return template.replace(/:(\w+)/g, (_match: string, name: string): string =>
    {
        const value = params[name];
        if (value === undefined)
        {
            throw new Error(`Missing path parameter "${name}" for ${template}`);
        }
        return encodeURIComponent(value);
    });
}
```

ব্যবহার:

```
buildPath(AppRoutes.ApplicationDetails, { id: '42' });
```

ফলাফল: `'/applications/42'`।

লাইন ধরে বুঝি:

১. **Regular expression** `/:(\w+)/g` — লেখার ভেতরে একটা pattern খোঁজার নিয়ম:

অংশ	মানে
`:`	হুবহু `:` চিহ্ন
`\w+`	এক বা একাধিক অক্ষর, সংখ্যা বা `_`
`( )`	ভেতরের অংশটা আলাদা করে ধরে রাখো (**capture**)
`g`	প্রথমটায় থেমো না, **সব** match খোঁজো

তাই `'/applications/:id/edit'`-এ এটা `:id` খুঁজে পায়, আর capture করে `id`।

২. `template.replace(regex, callback)` — প্রতিটা match-এর জায়গায় callback যা return করে, তা বসায়। Callback পায়:

- প্রথম parameter `_match`: পুরো match, যেমন `':id'`। এটা আমাদের লাগে না, তাই নামের শুরুতে `_` (convention: "ব্যবহার হবে না")।
- দ্বিতীয় parameter `name`: capture করা অংশ, যেমন `'id'`।

৩. `params[name]` দিয়ে value নেওয়া হয়, যেমন `params['id']` হলো `'42'`। Value না থাকলে error throw করে, যাতে ভুল URL চুপচাপ তৈরি না হয়।

৪. `encodeURIComponent(value)` — value-তে `/`, space, `?`-এর মতো special character থাকলে সেগুলোকে URL-safe রূপে বদলায়। যেমন `'a b'` হয়ে যায় `'a%20b'`। এতে value-র ভেতরের `/` URL-এর structure ভেঙে দিতে পারে না।

একই function API-র path-এও ব্যবহার হয়, যেমন `ApiEndpoints.ApplicationById` (`'/applications/:id'`)।

৯.৩ Router তৈরি: `AppRouter.tsx`

`frontend/src/app/AppRouter.tsx` (import বাদে):

```
const router = createBrowserRouter([
    { path: AppRoutes.Home, element: <Navigate to={AppRoutes.Applications} replace /> },
    {
        element: <AuthLayout />,
        children: [
            { path: AppRoutes.Login, element: <LoginPage /> },
            { path: AppRoutes.Register, element: <RegisterPage /> },
        ],
    },
    {
        element: <ProtectedRoute />,
        children: [
            {
                element: <AppLayout />,
                children: [
                    { path: AppRoutes.Applications, element: <ApplicationsPage /> },
                    { path: AppRoutes.ApplicationCreate, element: <ApplicationFormPage
/> },
                    { path: AppRoutes.ApplicationDetails, element:
<ApplicationDetailsPage /> },
                    { path: AppRoutes.ApplicationEdit, element: <ApplicationFormPage />
},
                ],
            },
        ],
    },
    { path: '*', element: <NotFoundPage /> },
]);

export function AppRouter(): JSX.Element
{
    return <RouterProvider router={router} />;
}
```

- `createBrowserRouter([...])` route-এর list নিয়ে একটা router object বানায়, যেটা History API ব্যবহার করে।
- `<RouterProvider router={router} />` সেই router-কে React-এ চালু করে: বর্তমান URL দেখে ঠিক component render করে।

Route object

প্রতিটা `{ path, element }` মানে: "URL এই `path`-এর সাথে মিললে এই `element` দেখাও।"

Route tree (nested routes)

Route-এর ভেতরে `children` দিয়ে আরো route রাখা যায়। এই `children` ৬.৬-এর `children` prop না; এটা route object-এর একটা key, যেখানে child route-এর array থাকে। এভাবে পুরো structure একটা **route tree** হয়ে যায়। একে বলে **nested routes**।

নিচের diagram-এ প্রথম লাইনের `<Navigate>` হলো এমন একটা component, যেটা render হলেই অন্য URL-এ পাঠিয়ে দেয় (বিস্তারিত নিচে ৯.৫):

```
/                       → <Navigate>: /applications-এ পাঠিয়ে দাও
(AuthLayout)            ← মাঝখানে সাদা card
├── /login              → LoginPage
└── /register           → RegisterPage
(ProtectedRoute)        ← login না থাকলে /login-এ পাঠাও
└── (AppLayout)         ← উপরে navbar
    ├── /applications           → ApplicationsPage
    ├── /applications/new       → ApplicationFormPage (নতুন)
    ├── /applications/:id       → ApplicationDetailsPage
    └── /applications/:id/edit  → ApplicationFormPage (edit)
*                       → NotFoundPage (404)
```

যে route-এর `path` নেই, কিন্তু `element` আর `children` আছে, তাকে বলে **layout route**। এটা নিজে কোনো URL-এর সাথে মেলে না; এর কাজ হলো তার সব children-কে নিজের component দিয়ে wrap করা। যেমন `/login` খুললে screen-এ আসে `AuthLayout`, আর তার ভেতরে `LoginPage`।

`new` বনাম `:id` — কে জেতে?

`/applications/new` URL-টা `/applications/:id`-এর সাথেও মেলে (তখন `id = 'new'`)! কিন্তু React Router static segment-কে parameter-এর চেয়ে বেশি priority দেয়। তাই array-তে যে order-এই লেখা থাকুক, `/applications/new` খুললে `ApplicationCreate` route-ই জেতে।

`path: '*'`

`*` মানে "যেকোনো URL"। অন্য কোনো route-এর সাথে না মেলা সব URL এখানে আসে, আর 404 page দেখায়।

৯.৪ Layout আর `<Outlet />`

`frontend/src/app/AppLayout.tsx` (সংক্ষেপে):

```
export function AppLayout(): JSX.Element
{
    return (
        <div className="min-h-screen bg-slate-100">
            <header>... Job Tracker ... Log out ...</header>
            <main className="mx-auto max-w-6xl px-4 py-8">
                <Outlet />
            </main>
        </div>
    );
}
```

`<Outlet />` হলো React Router-এর একটা placeholder component: route tree-তে যে child route URL-এর সাথে মিলেছে, তার page ঠিক এই জায়গায় render হয়। ফলাফল:

- Navbar প্রতিটা page-এ আলাদা করে লিখতে হয় না।
- Page বদলালে শুধু `<Outlet />`-এর ভেতরটা বদলায়; navbar একই থাকে, re-mount হয় না।

`AuthLayout`-ও একই কাজ করে: মাঝখানে একটা card, আর তার ভেতরে `<Outlet />`। সেখানে login বা register page বসে।

৯.৫ Page বদলানোর তিন উপায়

(ক) `<Link>`: click করে যাওয়া

`LoginPage.tsx` আর `ApplicationsTable.tsx` থেকে (সংক্ষেপে, `className` বাদে):

```
<Link to={AppRoutes.Register}>Create one</Link>
<Link to={buildPath(AppRoutes.ApplicationEdit, { id: application.id })}>Edit</Link>
```

প্রথমটা click করলে URL হয় `/register`, দ্বিতীয়টায় যেমন `/applications/42/edit`।

সাধারণ `<a href>` ব্যবহার করলে **পুরো page reload** হতো: browser server থেকে সব নতুন করে আনত, app একদম শুরু থেকে চালু হতো, আর memory-তে থাকা সব state (component-এর `useState`, আর store, অধ্যায় ১০) মুছে যেত। `<Link>` History API দিয়ে reload ছাড়াই URL বদলায়, তাই state থেকে যায়।

বাইরের website-এর জন্য কিন্তু সাধারণ `<a>`, `ApplicationDetailsPage.tsx`-এ (সংক্ষেপে):

```
<a href={href} target="_blank" rel="noreferrer">{href}</a>
```

- `target="_blank"` : link-টা নতুন tab-এ খোলে।
- `rel="noreferrer"` : security আর privacy-র জন্য দুটো কাজ করে। এক, নতুন tab-এর page `window.opener` দিয়ে আমাদের page-কে access বা redirect করতে পারে না (যেটা `noopener` -এর কাজ, `noreferrer` সেটাও করে)। দুই, নতুন site-কে আমাদের page-এর URL (referrer) জানানো হয় না।

`LinkButton` হলো আমাদের বানানো component: ভেতরে `<Link>`, তবে দেখতে button-এর মতো। `ApplicationsPage.tsx` :

```
<LinkButton to={AppRoutes.ApplicationCreate}>+ New application</LinkButton>
```

(খ) `useNavigate` : কোড থেকে যাওয়া

কোনো কাজ শেষ হওয়ার **পরে** অন্য page-এ যেতে `useNavigate` hook লাগে। এটা একটা `navigate` function return করে; `navigate(path)` call করলে app সেই path-এ চলে যায়।

`ApplicationFormPage.tsx` (সংক্ষেপে):

```
const navigate = useNavigate();

async function handleSubmit(payload: ApplicationPayload): Promise<void>
{
    try
    {
        const saved = id
            ? await dispatch(updateApplication({ id, payload })).unwrap()
            : await dispatch(createApplication(payload)).unwrap();
        void navigate(buildPath(AppRoutes.ApplicationDetails, { id: saved.id }));
    }
    catch
    {
        return;
    }
}
```

কী ঘটে:

- `id` থাকলে (edit page) server-এ update request যায়, না থাকলে (new page) create request।
- `.unwrap()` এখানে এটুকু করে: save সফল হলে server-এর ফেরত দেওয়া application (`saved`) দেয়, আর ব্যর্থ হলে error throw করে, তখন `catch` -এ চলে যায়। বিস্তারিত অধ্যায় ১১-এ।

- Save সফল হলে তবেই নতুন application-এর details page-এ যাওয়া হয়, যেমন `/applications/42`।
- `navigate(...)` একটা Promise return করে, আর আমরা সেটার জন্য অপেক্ষা করছি না, তাই সামনে `void` (অধ্যায় ৩)।

(গ) `<Navigate>`: render-এর সময়েই সরিয়ে দেওয়া

```
if (user)
{
    return <Navigate to={AppRoutes.Applications} replace />;
}
```

কোনো component JSX হিসেবে `<Navigate>` return করলে, React Router render-এর সাথে সাথে অন্য URL-এ পাঠিয়ে দেয়। এখানে `user` থাকলে (মানে login করা আছে) `/applications`-এ চলে যায় (সরল উদাহরণ; আসল `LoginPage` কোথায় পাঠায়, তা ৯.৭-এ)।

`replace` মানে browser history-তে নতুন entry যোগ না করে বর্তমান entry-টাকেই বদলে দেওয়া। কেন দরকার? ধরো `/`-এ এসে `/applications`-এ redirect হলে। `replace` না থাকলে history-তে থাকত `/` আর `/applications`। Back চাপলে `/`-এ ফিরত, সেখান থেকে আবার `/applications`-এ redirect হতো, আর user কখনো পেছনে যেতে পারত না। `replace` থাকলে `/` history থেকেই বাদ পড়ে।

উপায়	কখন ব্যবহার করবে
`<Link>`	User click করে অন্য page-এ যাবে
`useNavigate`	কোনো কাজ (যেমন save) শেষ হওয়ার পরে কোড থেকে যাবে
`<Navigate>`	Render-এর সময়েই বোঝা গেল এই page দেখানো যাবে না

৯.৬ URL থেকে তথ্য পড়া

`useParams`: `:id`-এর value

`ApplicationDetailsPage.tsx`:

```
const { id = '' } = useParams<{ id: string; }>();
```

- `useParams` URL-এর সব parameter একটা object হিসেবে দেয়। `/applications/42`-এ সেটা `{ id: '42' }`।

- `<{ id: string; }>` হলো generic (অধ্যায় ৪): TypeScript-কে বলা হলো কোন parameter আশা করছি।
- তবুও `id`-এর type হয় `string | undefined`, কারণ TypeScript জানে না এই component আসলেই `:id` আছে এমন route-এ বসানো কিনা।
- `= ''` হলো destructuring-এর default value: `id` `undefined` হলে `''` বসবে। ফলে নিচের কোডে `id` সবসময় `string`, আর `loadApplication(id)`-এ সরাসরি দেওয়া যায়।

`ApplicationFormPage.tsx`-এ কিন্তু default নেই:

```
const { id } = useParams<{ id: string; }>();
```

এখানে `undefined` হওয়াটাই দরকারি তথ্য। এই component দুটো route-এ বসানো:

URL	id	Page
`/applications/new`	`undefined` (route-এ `:id` নেই)	নতুন application
`/applications/42/edit`	`'42'`	edit

তাই একই component দেখে নেয় কোন কাজ করবে:

```
<h1>{id ? 'Edit application' : 'New application'}</h1>
```

একই component, দুই কাজ। কম কোড, কম bug।

`useLocation`: বর্তমান location আর লুকানো state

`useLocation()` বর্তমান URL-এর তথ্য একটা **location** object হিসেবে দেয়। এতে আরো কিছু field আছে, আমাদের কাজে লাগে এই তিনটা:

Field	`/applications/42?tab=info` URL-এ
`pathname`	`'/applications/42'`
`search`	`'?tab=info'`
`state`	navigate করার সময় সাথে পাঠানো লুকানো data; কিছু না পাঠালে `null`

`state` URL-এ দেখা যায় না। এটা পাঠানো হয় navigate করার সময়, যেমন `<Navigate>`-এর `state` prop দিয়ে:

```
<Navigate to={AppRoutes.Login} replace state={{ from: location.pathname }} />
```

এতে `/login` page-এর `useLocation().state` হবে `{ from: '/applications/42' }`-এর মতো কিছু। "Login শেষে আগের page-এ ফেরা" এভাবেই কাজ করে (নিচে ৯.৭)।

৯.৭ সুরক্ষিত page: `ProtectedRoute`

`frontend/src/features/auth/ProtectedRoute.tsx` (import বাদে):

```
export function ProtectedRoute(): JSX.Element
{
    const { user, isSessionChecked } = useAppSelector((state) => state.auth);
    const location = useLocation();

    if (!isSessionChecked)
    {
        return <Spinner fullPage />;
    }
    if (!user)
    {
        return <Navigate to={AppRoutes.Login} replace state={{ from: location.pathname
}} />;
    }
    return <Outlet />;
}
```

এটা একটা **guard** layout route (পাহারা দেয়, কে ঢুকতে পারবে)। Store থেকে `user` (login করা user, না থাকলে `null`) আর `isSessionChecked` পড়ে ঠিক করে কী দেখাবে:

অবস্থা	কী দেখায়
এখনো জানা নেই login আছে কিনা (app সবে খুলল)	পুরো screen জুড়ে Spinner
Login নেই	`/login`-এ পাঠায়, সাথে `state`-এ বর্তমান path মনে রাখে
Login আছে	`<Outlet />`, মানে ভেতরের page

এটা নিজে থেকে বদলায় কীভাবে? `useAppSelector` store-এর value পড়ে, আর store-এ `user` বা `isSessionChecked` বদলালে এই component আপনাআপনি re-render হয় (অধ্যায় ১০)। তাই login হওয়ার সাথে সাথে Spinner বা redirect-এর জায়গায় `<Outlet />` চলে আসে।

`isSessionChecked` কেন দরকার? App খোলার সাথে সাথে server-কে জিজ্ঞেস করা হয় "এই token-টা কার?" (অধ্যায় ১২)। Server-এর response আসার আগে `user` থাকে `null`। তখনই যদি `user === null` দেখে login-এ পাঠিয়ে দিতাম, তাহলে login করা user-ও প্রতিবার page refresh করলে login page দেখত।

Login-এর পরে আগের page-এ ফেরা

`LoginPage.tsx` (সংক্ষেপে):

```
function redirectPathFrom(state: unknown): string
{
    if (typeof state === 'object' && state !== null && 'from' in state && typeof
state.from === 'string')
    {
        return state.from;
    }
    return AppRoutes.Applications;
}

const location = useLocation();
const { user, status, error } = useAppSelector((state) => state.auth);

if (user)
{
    return <Navigate to={redirectPathFrom(location.state)} replace />;
}
```

`'from' in state` হলো JavaScript-এর `in` operator: object-এ `from` নামের key আছে কিনা check করে। TypeScript এটা দেখে narrowing করে, তাই এরপর `state.from` লেখা যায়।

পুরো ঘটনা:

1. কেউ `/applications/42` খুলল, কিন্তু login নেই।
2. `ProtectedRoute` তাকে `/login`-এ পাঠাল, `state`-এ `{ from: '/applications/42' }` রেখে।
3. User login করল। Store-এ `user` set হলো, তাই `useAppSelector`-এর কারণে `LoginPage` re-render হলো।
4. এবার `user` আছে, তাই `<Navigate>` return হলো। `redirectPathFrom` `state.from` থেকে `'/applications/42'` বের করল।
5. User পৌঁছে গেল `/applications/42`-এ।

কেউ সরাসরি `/login` খুললে `state` থাকে `null`, তাই `redirectPathFrom` default `/applications` return করে।

React Router-এ `location.state`-এর type আসলে `any`, কারণ যেকোনো জায়গা থেকে যেকোনো কিছু পাঠানো যায়। `any`-তে TypeScript কোনো check করে না, তাই `redirectPathFrom` ইচ্ছা করে parameter-টা `unknown` হিসেবে নেয় (অধ্যায় ৪.৮), যাতে ব্যবহারের আগে ধাপে ধাপে পরীক্ষা করতেই হয়: object কিনা, `null` না, `from` key আছে কিনা, আর সেটা `string` কিনা (type narrowing, অধ্যায় ৪)।

সারসংক্ষেপ

- Client-side routing-এ History API দিয়ে page reload ছাড়াই URL আর component বদলায়।
- `createBrowserRouter` + `RouterProvider` দিয়ে route tree বানানো হয়। Static segment, `:param`-এর চেয়ে বেশি priority পায়।
- `buildPath` placeholder-এর জায়গায় value বসিয়ে URL বানায়।
- **Layout route** + `<Outlet />` দিয়ে navbar বা card একবার লিখে সব page-এ ব্যবহার করা যায়।
- `<Link>` click করে যাওয়ার জন্য, `useNavigate` কোড থেকে যাওয়ার জন্য, আর `<Navigate>` render-এর সময় সরিয়ে দেওয়ার জন্য। `replace` history-তে নতুন entry যোগ করে না।
- `useParams` `:id` পড়ে (না থাকলে `undefined`), আর `useLocation` `pathname`, `search` আর লুকানো `state` দেয়।
- `ProtectedRoute` login ছাড়া ভেতরে ঢুকতে দেয় না, আর login শেষে আগের page-এ ফেরত পাঠাতে `state` ব্যবহার করে।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: `<Link>` আর `<a href>`-এর পার্থক্য কী?
উত্তর: `<a href>` পুরো page reload করে: browser server থেকে সব নতুন করে আনে, ফলে app আর memory-তে থাকা সব state শুরু থেকে চালু হয়। `<Link>` browser-এর History API দিয়ে JavaScript থেকে URL বদলায়, reload করে না, তাই state থেকে যায় আর page বদল দ্রুত হয়। App-এর ভেতরের page-এ `<Link>`, আর বাইরের website-এ `<a>` ব্যবহার করি।

প্রশ্ন: Protected route কীভাবে বানাবে?
উত্তর: একটা layout route component বানাব, যেটা store থেকে auth state পড়ে। Login না থাকলে `<Navigate to="/login" replace />` return করে, আর থাকলে `<Outlet />`, যেখানে ভেতরের page render হয়। সুরক্ষিত সব route হবে এর children। App শুরুর সময় server থেকে session check শেষ না হওয়া পর্যন্ত spinner দেখাব, নাহলে login করা user-ও refresh-এ login page দেখবে। চাইলে redirect-এর সময় `state`-এ বর্তমান path রেখে, login শেষে সেখানে ফেরত পাঠানো যায়।

প্রশ্ন: Nested route আর Outlet কী?

উত্তর: Route-এর `children`-এ রাখা route হলো nested route। Parent route-এর component-এ `<Outlet />` রাখা হয়, আর URL-এর সাথে মিলে যাওয়া child route-এর component ঠিক সেখানে render হয়। এতে navbar বা card-এর মতো common layout একবার লিখে অনেক page-এ reuse করা যায়, আর page বদলালে layout re-mount হয় না।

প্রশ্ন: navigate-এর `replace` option কী করে?

উত্তর: Browser history-তে নতুন entry যোগ না করে বর্তমান entry-টাকে বদলে দেয়। Component-এ লেখা হয় `<Navigate to={path} replace />`, আর কোড থেকে `navigate(path, { replace: true })`। Redirect-এর ক্ষেত্রে এটা দরকার: নাহলে Back চাপলে user redirect করা page-এ ফেরে, সেখান থেকে আবার redirect হয়, আর পেছনে যেতে পারে না।

প্রশ্ন: SPA server-এ deploy করলে refresh-এ 404 আসে কেন?

উত্তর: `/applications/42`-এ refresh করলে browser server-এর কাছে ঠিক ওই path চায়, অথচ server-এ ওই নামে কোনো file নেই, কারণ SPA-তে HTML file একটাই, `index.html`। সমাধান হলো server-কে বলা, অচেনা সব path-এ `index.html` দিতে (একে fallback বলে)। তারপর app চালু হয়ে React Router নিজে URL দেখে ঠিক page দেখায়।

প্রশ্ন: URL parameter আর query string-এর পার্থক্য কী?

উত্তর: URL parameter path-এরই একটা segment, আর সাধারণত একটা নির্দিষ্ট resource বোঝায়, যেমন `/applications/42`-এর `42`; React Router-এ এটা `useParams` দিয়ে পড়া হয়। Query string হলো URL-এর `?`-এর পরের key-value অংশ, যেমন `?page=2&search=dev`; সাধারণত optional filter, sort বা pagination-এর জন্য, আর `useLocation().search`-এ পাওয়া যায়।

অধ্যায় ১০: Global State — Redux Toolkit

`useState` একটা component-এর নিজের জিনিস। কিন্তু login করা user-কে দরকার navbar-এর, `ProtectedRoute` -এর, Login page-এর, সবার। যে data অনেক দূরের component-কে একসাথে লাগে, তাকে বলে **global state**। আমাদের project-এ global state রাখার জন্য ব্যবহার করা হয়েছে **Redux Toolkit**।

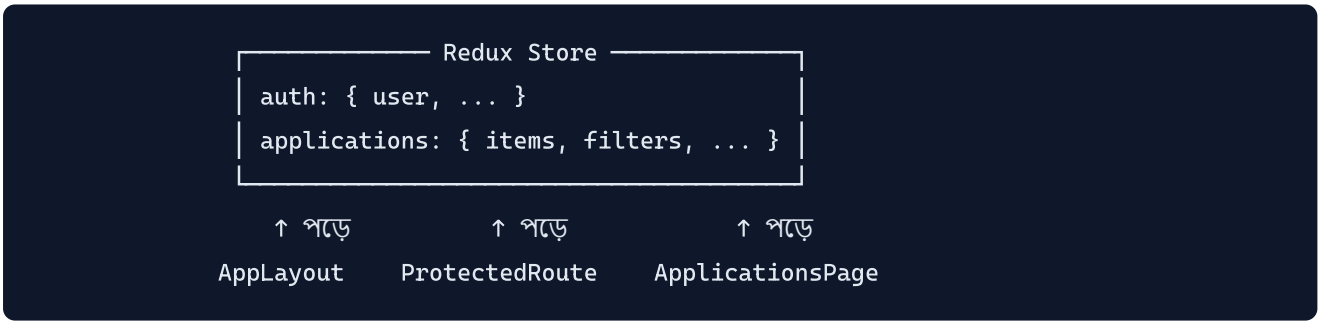
১০.১ সমস্যা: Prop drilling

ধরো আমাদের project-এ store নেই, আর `user` রাখা আছে সবচেয়ে উপরের component-এ। কিন্তু `user` লাগবে গভীরে থাকা navbar-এ। তাহলে মাঝের সব component-কে props দিয়ে হাতে হাতে `user` পৌঁছে দিতে হবে, অথচ মাঝের কেউ নিজে `user` ব্যবহারই করে না। এই কাল্পনিক chain-টা দেখতে এমন হতো:

```
App (user) → AppRouter (user) → ProtectedRoute (user) → AppLayout (user) → navbar
```

একে বলে **prop drilling**। লেখা বিরক্তিকর, আর মাঝের যেকোনো জায়গায় ভুল হওয়ার ঝুঁকি বেশি।

সমাধান: data রাখো সব component-এর বাইরে, একটা **central store**-এ। যে component-এর দরকার, সে সরাসরি store থেকে পড়বে; মাঝের কাউকে কিছু জানতে হবে না।



পুরো app-এর shared data যেহেতু এই এক জায়গাতেই থাকে, তাই store-কে বলা হয় **single source of truth**: কোনো data-র আসল value জানতে হলে একটাই জায়গায় দেখতে হয়।

১০.২ Redux-এর পাঁচটা মূল শব্দ

শব্দ	মানে	তুলনা
Store	পুরো app-এর global state রাখার একমাত্র জায়গা	ব্যাংক

শব্দ	মানে	তুলনা
State	store-এর ভেতরের data	ব্যাংকে তোমার টাকা
Action	"কী ঘটেছে" জানানো একটা ছোট object	জমা বা তোলার স্লিপ
Reducer	action দেখে নতুন state বানানোর function	ক্যাশিয়ার
Dispatch	store-এ action পাঠানো	স্লিপটা কাউন্টারে দেওয়া

Action দেখতে কেমন

```
{ type: 'applications/filtersChanged', payload: { search: 'dev' } }
```

- `type`: কী ঘটেছে, তার নাম (একটা string)।
- `payload`: সাথে পাঠানো data। এখানে মানে "search লেখা `'dev'` হয়েছে"।

Reducer-এর নিয়ম

```
নতুন state = reducer(পুরনো state, action)
```

- পুরনো state **বদলাবে না**, নতুন state return করবে (immutability, অধ্যায় ৭)।
- কোনো side effect করবে না (server call, timer; অধ্যায় ৮)।
- একই input দিলে সবসময় একই output দেবে। এমন function-কে বলে **pure function**।

One-way data flow

Redux-এ data সবসময় একই দিকে, একই পথে চলে। একে বলে **one-way data flow**:

```
UI → dispatch(action) → reducer → নতুন state (store) → useAppSelector → UI re-render
```

1. UI-তে কিছু ঘটল (যেমন user dropdown বদলাল), component একটা action dispatch করল।
2. Reducer পুরনো state আর action থেকে নতুন state বানাল।
3. Store নতুন state রাখল।
4. যে component-গুলো `useAppSelector` দিয়ে ওই state পড়ে (নিচে ১০.৭), তারা re-render হলো।

State বদলানোর অন্য কোনো পথ নেই। তাই কোনো bug হলে খুঁজতে হয় শুধু "কোন action এসেছিল, আর reducer কী করল"। এই কারণে Redux-এর state বদল **predictable**।

১০.৩ Redux Toolkit কেন?

পুরনো Redux-এ অনেক বাড়তি কোড লিখতে হতো: action type-এর আলাদা constant, action বানানোর আলাদা function, হাতে লেখা immutable update। **Redux Toolkit (RTK)** (`@reduxjs/toolkit` package) হলো Redux-এর official, আধুনিক রূপ, যেটা এই কাজগুলো automate করে:

- `configureStore`: store বানায়, আর ভালো default setting দেয়।
- `createSlice`: reducer আর action একসাথে বানায়।
- `createAsyncThunk`: server call সামলায় (অধ্যায় ১১)।
- ভেতরে থাকে **Immer** library, তাই "বদলানোর মতো" করে লিখলেও আসলে immutable update হয় (১০.৬)।

React component-কে store-এর সাথে যুক্ত করার কাজ করে আলাদা একটা package: `react-redux`। `Provider`, `useSelector` আর `useDispatch` আসে এখান থেকে।

১০.৪ Store বানানো: `app/store.ts`

`frontend/src/app/store.ts`:

```
import { applicationsReducer } from '@/features/applications/applications.slice';
import { authReducer } from '@/features/auth/auth.slice';
import { configureStore } from '@reduxjs/toolkit';

export const store = configureStore({
    reducer: {
        auth: authReducer,
        applications: applicationsReducer,
    },
});

export type RootState = ReturnType<typeof store.getState>;

export type AppDispatch = typeof store.dispatch;
```

- Store-এর state দেখতে হবে `{ auth: {...}, applications: {...} }`। প্রতিটা অংশকে বলে একটা **slice**, আর প্রতিটা slice-এর নিজের reducer আছে (`authReducer`, `applicationsReducer`)।
- `RootState` হলো পুরো state-এর type। `store.getState()` পুরো state return করে, আর `ReturnType<typeof store.getState>` (utility type, অধ্যায় ৪) সেই return-এর type বের করে আনে। নতুন slice যোগ করলে type নিজে থেকেই update হয়।

- `AppDispatch` হলো এই store-এর `dispatch` function-এর type।

Middleware

Middleware হলো dispatch আর reducer-এর মাঝের একটা ধাপ: action reducer-এ পৌঁছানোর আগে middleware সেটা দেখতে, বদলাতে, বা অন্য কিছু করতে পারে।

```
dispatch(...) → middleware → reducer
```

`configureStore` default হিসেবে কয়েকটা middleware যোগ করে দেয়, তার মধ্যে সবচেয়ে গুরুত্বপূর্ণ হলো **thunk middleware**, যেটা দিয়ে server call করা যায় (অধ্যায় ১১)। আমাদের আলাদা করে কিছু লিখতে হয়নি।

App-কে store দেওয়া: `main.tsx`

`frontend/src/main.tsx` (সংক্ষেপে):

```
<StrictMode>
    <Provider store={store}>
        <AppRouter />
    </Provider>
</StrictMode>
```

`Provider` (`react-redux` থেকে) পুরো app-কে wrap করে। এর ফলে ভেতরের যেকোনো component store ব্যবহার করতে পারে।

ভেতরে `Provider` React-এর **Context** ব্যবহার করে। Context হলো React-এর নিজস্ব ব্যবস্থা, যেটা দিয়ে props ছাড়াই উপরের একটা component থেকে নিচের যেকোনো component-এ value পৌঁছানো যায়।

১০.৫ Slice: এক বিষয়ের state + reducer + action

শুরুর filter value-গুলো রাখা আছে `frontend/src/features/applications/applications.constants.ts`-এ:

```
export const DEFAULT_FILTERS: ApplicationFilters = {
    search: '',
    status: StatusFilter.All,
    applyMedium: '',
    sortBy: SortField.CreatedAt,
    sortOrder: SortOrder.Desc,
    page: 1,
    limit: 10,
};
```

অর্থাৎ শুরুতে: কোনো search নেই, সব status, যেকোনো apply medium, নতুন যোগ করা আগে, প্রথম page, প্রতি page-এ ১০টা।

এবার slice। `frontend/src/features/applications/applications.slice.ts` (সংক্ষেপে):

```
const initialState: ApplicationsState = {
    filters: DEFAULT_FILTERS,
    items: [],
    meta: null,
    listStatus: RequestStatus.Idle,
    listError: null,
    ...
};

const applicationsSlice = createSlice({
    name: 'applications',
    initialState,
    reducers: {
        filtersChanged: (state, action: PayloadAction<Partial<ApplicationFilters>>):
void =>
        {
            state.filters = { ...state.filters, page: 1, ...action.payload };
        },
        filtersReset: (state): void =>
        {
            state.filters = DEFAULT_FILTERS;
        },
        ...
    },
    ...
});

export const { filtersChanged, filtersReset, saveErrorCleared } =
applicationsSlice.actions;

export const applicationsReducer: Reducer<ApplicationsState> =
applicationsSlice.reducer;
```

`saveErrorCleared` `saveError`-কে `null` করে।

`initialState`-এ `listStatus` আর `listError` হলো server call-এর অবস্থা (loading চলছে কিনা, error হয়েছে কিনা); `RequestStatus` নিয়ে বিস্তারিত অধ্যায় ১১-এ।

Action creator হলো এমন একটা function, যেটা action object বানিয়ে return করে। যেমন:

```
filtersChanged({ search: 'dev' });
```

এটা return করে `{ type: 'applications/filtersChanged', payload: { search: 'dev' } }`। হাতে action object লেখার বদলে আমরা action creator call করি, তাই `type`-এর বানান ভুল হওয়ার সুযোগ থাকে না।

অংশ	কাজ
`name`	action type-এর শুরু: `'applications/...'`
`initialState`	শুরুর state
`reducers`	প্রতিটা key থেকে একটা reducer **আর** একই নামের একটা action creator তৈরি হয়
`slice.actions`	সেই action creator-গুলো: `filtersChanged`, `filtersReset` ...
`slice.reducer`	পুরো slice-এর একটা reducer, যেটা store-এ বসানো হয়

`Reducer<ApplicationsState>` হলো RTK-র একটা type: "এটা এমন reducer, যার state-এর type `ApplicationsState`"।

`filtersChanged` কী করে

```
state.filters = { ...state.filters, page: 1, ...action.payload };
```

1. পুরনো সব filter রাখো (`...state.filters`)।
2. `page`-কে 1 করো। নতুন filter দিলে প্রথম page থেকে দেখানো উচিত।
3. পাঠানো নতুন value-গুলো দিয়ে overwrite করো (`...action.payload`)। Payload-এ `page` থাকলে (মানে user page বদলাচ্ছে) সেটাই জিতবে, কারণ spread-এ পরে লেখা key আগেরটাকে overwrite করে।

উদাহরণ: `filters.page` এখন 3, আর dispatch হলো `filtersChanged({ search: 'dev' })`। নতুন `filters`-এ `search` হবে `'dev'`, `page` হবে 1, বাকি সব আগের মতো।

`PayloadAction<Partial<ApplicationFilters>>` মানে এই action-এর payload হলো `ApplicationFilters`-এর কিছু field (Partial, অধ্যায় ৪)। তাই কেউ ভুল field বা ভুল type পাঠালে TypeScript error দেবে।

১০.৬ Immer: "বদলানোর মতো লেখো, নিরাপদে নতুন বানাও"

অধ্যায় ৭-এ শিখেছি state কখনো সরাসরি বদলানো যায় না। তাহলে slice-এ এটা কীভাবে চলছে?

```
state.filters = DEFAULT_FILTERS;
```

Redux Toolkit-এর reducer-এর ভেতরে `state` আসল state না; এটা Immer-এর বানানো একটা **draft**, মানে একটা temporary copy। তুমি draft "বদলাও", আর Immer সেই বদলগুলো লিখে রাখে, তারপর সেগুলো দিয়ে **একটা সম্পূর্ণ নতুন immutable state** বানিয়ে দেয়। আসল পুরনো state অক্ষত থাকে।

Immer-এর আরেকটা সুবিধা: যে অংশ বদলায়নি, নতুন state-এ সেটা পুরনো object-ই থাকে। যেমন `items` বদলালে নতুন state-এ `filters` আগের reference-ই থাকে। অধ্যায় ৮-এর `useEffect`, যার dependency array `[dispatch, filters]`, এই কারণেই অপ্রয়োজনে চলে না।

শুধু দুটো নিয়ম:

1. Draft বদলাও, **অথবা** নতুন state `return` করো। **দুটো একসাথে না।**
2. এই সুবিধা **শুধু `createSlice`-এর reducer-এর ভেতরে**। Component-এর `useState`-এ এখনো spread দিয়ে নিজে নতুন object বানাতে হবে।

নতুন state return করার উদাহরণ। ধরো slice-এর `reducers`-এ এমন একটা reducer থাকত:

```
reset: (): ApplicationsState => initialState,
```

এটা draft-এ হাত দেয় না, সরাসরি `initialState` return করে, তাই dispatch হলে পুরো slice শুরুর অবস্থায় ফিরে যায়। আমাদের আসল কোডে ঠিক এই কাজটা হয় logout-এর সময় (অধ্যায় ১১)।

১০.৭ Store থেকে পড়া আর পাঠানো: typed hook

`frontend/src/app/redux.ts` (সংক্ষেপে):

```
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();

export const useAppSelector = useSelector.withTypes<RootState>();
```

`react-redux`-এর `useDispatch` আর `useSelector` আমাদের state-এর shape জানে না। `.withTypes<>()` দিয়ে এদের **typed version** বানানো হলো: এখন `useAppSelector` জানে state-এ `auth` আর `applications` আছে, আর ভুল field লিখলে TypeScript error দেয়। **সবসময় এই দুটোই ব্যবহার করবে।**

`useAppSelector`: পড়া

```
const filters = useAppSelector((state) => state.applications.filters);
const user = useAppSelector((state) => state.auth.user);
```

- ভেতরের function-টাকে বলে **selector**: পুরো state থেকে দরকারি অংশ বেছে return করে।
- **কখন re-render হয়:** প্রতিবার store বদলালে selector আবার চলে, আর নতুন ফলাফল আগের ফলাফলের সাথে `===` দিয়ে তুলনা হয়। আলাদা হলে তবেই component re-render হয়।
- তাই যে component শুধু `state.auth.user` পড়ে, application-এর list বদলালে সে re-render হয় না।

পুরো slice select করলে কী হয়

Project-এর অনেক জায়গায় পুরো slice select করে তারপর destructure করা হয়েছে, যেমন `ApplicationsPage.tsx`:

```
const { filters, items, meta, listStatus, listError } = useAppSelector((state) =>
state.applications);
```

এখানে selector return করছে **পুরো** `state.applications` **object**। Destructuring হয় তার **পরে**, শুধু field-গুলো বের করার জন্য; re-render-এর সিদ্ধান্তে এর কোনো ভূমিকা নেই।

ফলাফল: applications slice-এর **যেকোনো** field বদলালে (যেমন `saveStatus`, যেটা এই page ব্যবহারই করে না) slice-টা নতুন object হয়, আর `ApplicationsPage` re-render হয়। ছোট app-এ এটা সমস্যা না, কিন্তু খেয়াল রাখা ভালো। শুধু একটা field দরকার হলে সরাসরি সেটাই select করা ভালো, যেমন `ApplicationFilters.tsx` করে `state.applications.filters`।

`useAppDispatch`: পাঠানো

```
const dispatch = useAppDispatch();
dispatch(filtersChanged({ status: StatusFilter.Applied }));
```

ফলাফল: store-এ `filters.status` হয়ে যায় `'applied'`, আর `page` হয় 1।

১০.৮ পুরো flow একটা উদাহরণে: Status filter বদলানো

`ApplicationFilters.tsx`-এ user "Status" dropdown থেকে "Applied" বেছে নিল।

`ApplicationFilters.tsx` (সংক্ষেপে):

```
<Select
    id="status"
    value={filters.status}
    onChange={(event): void =>
    {
        dispatch(
            filtersChanged({
                status: parseEnum(Object.values(StatusFilter), event.target.value,
StatusFilter.All),
            }),
        );
    }}
>
```

1. **Event:** `onChange` চলে। `event.target.value` হলো string `'applied'`।
2. **`parseEnum`:** string-টা আসলেই `StatusFilter`-এর কোনো value কিনা check করে, আর মিললে সেটা return করে। না মিললে `StatusFilter.All`।
3. **Action তৈরি:** action creator `filtersChanged({ status: 'applied' })` return করে `{ type: 'applications/filtersChanged', payload: { status: 'applied' } }`।
4. **Dispatch:** action store-এ যায়।
5. **Reducer:** `state.filters = { ...পুরনো, page: 1, status: 'applied' }`। Immer নতুন `filters` object বানায়, তাই applications slice-ও নতুন object হয়।
6. **Selector:** `ApplicationFilters` select করে `state.applications.filters`, যেটা নতুন object, তাই re-render হয়, আর dropdown-এ "Applied" দেখায়। `ApplicationsPage` select করে পুরো `state.applications`, যেটাও নতুন object, তাই সেও re-render হয়।
7. **Effect:** `ApplicationsPage`-এর `useEffect(..., [dispatch, filters])` দেখে `filters` বদলেছে, তাই `loadApplications(filters)` dispatch করে server থেকে নতুন list আনে (অধ্যায় ১১)।

Filter কেন Redux-এ, `useState`-এ না?

- Filter bar (`ApplicationFilters`) আর page (`ApplicationsPage`), দুটো আলাদা component-এর এটা দরকার।
- Details page দেখে list-এ ফিরে এলেও filter মনে থাকা উচিত। `useState`-এ রাখলে page ছাড়ার সাথে সাথে component unmount হয়, আর filter হারিয়ে যেত।

১০.৯ কোনটা store-এ রাখবে, কোনটা রাখবে না

Store-এ রাখো	Local `useState` -এ রাখো
Login করা user	Form-এ type করা লেখা
Server থেকে আনা list	Dropdown খোলা আছে কিনা
অনেক page-এ লাগা filter	Search box-এর তাৎক্ষণিক লেখা (debounce-এর আগে)
Server call-এর loading বা error অবস্থা	শুধু একটা component-এর UI অবস্থা

নিয়মটা সহজ: একাধিক দূরের component-এর দরকার, বা page বদলালেও মনে থাকা দরকার, এমন data store-এ। বাকি সব local।

সারসংক্ষেপ

- **Store** হলো পুরো app-এর global state রাখার একমাত্র জায়গা (single source of truth)। এতে prop drilling দূর হয়।
- **Action** `{ type, payload }` জানায় কী ঘটেছে। **Action creator** action বানায়। **Reducer** পুরনো state আর action থেকে নতুন state বানায়।
- Data চলে one-way data flow-তে: UI → dispatch → reducer → store → `useAppSelector` → UI।
- `configureStore` store বানায়, আর default হিসেবে thunk middleware যোগ করে। **Middleware** হলো dispatch আর reducer-এর মাঝের ধাপ।
- `createSlice` reducer আর action creator একসাথে বানায়।
- **Immer**-এর কারণে slice-এর ভেতরে "বদলানোর মতো" লেখা যায়, যদিও আসলে নতুন state তৈরি হয়।
- `useAppSelector` দিয়ে পড়া হয় (selector-এর ফলাফল `===` অনুযায়ী বদলালে তবেই re-render), আর `useAppDispatch` দিয়ে পাঠানো হয়। দুটোই `react-redux` -এর hook-এর typed version।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Redux কেন ব্যবহার করবে? সবসময় কি দরকার?
উত্তর: অনেক দূরের component-এর মধ্যে data share করতে, prop drilling এড়াতে, আর state কোথায় কীভাবে বদলায় তা predictable রাখতে। Redux-এ state বদলানোর একটাই পথ: action dispatch করা, আর reducer নতুন state বানানো। তাই যেকোনো বদলের কারণ খুঁজে পাওয়া সহজ। এছাড়া Redux DevTools নামের browser extension দিয়ে প্রতিটা action আর তার পরের state দেখা যায়। তবে সবসময় দরকার নেই; ছোট app-এ `useState` আর lifting state, বা React-এর Context-ই যথেষ্ট।

প্রশ্ন: Reducer কেন pure হতে হবে?
উত্তর: Pure মানে একই input-এ সবসময় একই output, আর কোনো side effect নেই। তাহলে state বদল predictable হয়, reducer test করা সহজ হয় (শুধু input দিয়ে output মেলাও), আর Redux DevTools-এ আগের action-গুলো আবার চালিয়ে আগের অবস্থায় ফেরা যায়, যাকে বলে time-travel debugging। Reducer-এ server call বা random value থাকলে এগুলো ভেঙে যায়।

প্রশ্ন: Redux Toolkit-এ state "mutate" করা যায় কীভাবে?
উত্তর: Immer library-র কারণে। `createSlice`-এর reducer-এ আসল state-এর বদলে একটা draft, মানে একটা temporary copy দেওয়া হয়। আমরা draft-এ সরাসরি assign করি, আর Immer সেই বদলগুলো থেকে একটা নতুন immutable state বানায়; আসল state অক্ষত থাকে। এটা শুধু RTK-র reducer-এর ভেতরে কাজ করে, component-এর `useState`-এ না। আর একটা reducer-এ হয় draft বদলাতে হবে, নয়তো নতুন state return করতে হবে, দুটো একসাথে না।

প্রশ্ন: useSelector কখন component re-render করে?
উত্তর: প্রতিবার store-এ action dispatch হলে selector আবার চলে, আর নতুন return value আগের value-র সাথে strict reference তুলনা, মানে `===` দিয়ে মেলানো হয়। আলাদা হলে তবেই re-render হয়। তাই selector-এ প্রতিবার নতুন object বানালে, যেমন `(state) => ({ a: state.a })`, প্রতিবার নতুন reference হয়, আর প্রতিটা action-এ re-render হয়। একইভাবে পুরো slice select করলে slice-এর যেকোনো field বদলালেই re-render হয়।

প্রশ্ন: Context API আর Redux-এর পার্থক্য কী?
উত্তর:

- Context হলো React-এর নিজস্ব ব্যবস্থা, যেটা দিয়ে props ছাড়াই উপরের component থেকে নিচের যেকোনো component-এ value পৌঁছানো যায়। Value বদলালে যারা ওই Context ব্যবহার করে তারা সবাই re-render হয়, আর state কীভাবে বদলাবে তার কোনো নিয়ম Context নিজে দেয় না।
- Redux একটা পূর্ণ state management library: selector দিয়ে শুধু দরকারি অংশ বদলালে re-render, middleware (যেমন thunk, যেটা server call সামলায়), DevTools, আর action-reducer-এর নির্দিষ্ট structure।
- কম বদলায় এমন জিনিসে (theme, ভাষা) Context ভালো। জটিল, ঘনঘন বদলানো, অনেক জায়গায় লাগা data-তে Redux ভালো।

প্রশ্ন: Prop drilling কী? সমাধান কী?
উত্তর: যে component-এর নিজের কোনো data দরকার নেই, তাকে দিয়েও props পাঠাতে হয় শুধু নিচের component-এ পৌঁছানোর জন্য। এটাই prop drilling। সমাধান: Context, Redux বা অন্য কোনো state management library, অথবা composition, মানে data যার কাছে আছে সেখানেই child component বানিয়ে `children` হিসেবে নিচে পাঠানো।

প্রশ্ন: Action creator কী?

উত্তর: Action object বানিয়ে return করা একটা function। যেমন `filtersChanged({ search: 'dev' })` return করে `{ type: 'applications/filtersChanged', payload: { search: 'dev' } }`। এতে প্রতিবার হাতে `type` লিখতে হয় না, বানান ভুলের সুযোগ থাকে না, আর payload-এর type TypeScript check করতে পারে। `createSlice` প্রতিটা reducer-এর জন্য একই নামের action creator নিজেই বানিয়ে দেয়।

অধ্যায় ১১: Server-এর data আর Redux — `createAsyncThunk`

Reducer pure হতে হয় (অধ্যায় ১০), তাই সেখানে server call চলে না। তাহলে server থেকে data এনে store-এ বসানো হবে কীভাবে? উত্তর হলো **thunk**।

১১.১ Thunk কী?

সাধারণ action হলো একটা **object**: `{ type, payload }`। **Thunk** হলো একটা **function**, যেটা action-এর মতোই dispatch করা যায়। এর ভেতরে async কাজ (server call) করা যায়, আর কাজ শেষে সাধারণ action dispatch করা যায়।

কিন্তু reducer তো function বোঝে না। তাহলে function dispatch করলে কী হয়? এখানেই অধ্যায় ১০-এর **middleware** কাজে আসে। মনে করিয়ে দিই: middleware হলো dispatch আর reducer-এর মাঝের একটা ধাপ, আর `configureStore` default হিসেবে **thunk middleware** যোগ করে দেয়। এই middleware প্রতিটা dispatch দেখে:

কী dispatch করা হলো	Thunk middleware কী করে
Object (সাধারণ action)	কিছু না, সোজা reducer-এ যেতে দেয়
Function (thunk)	Reducer-এ পাঠায় না; function-টা নিজে **চালায়**, আর তাকে `dispatch` দেয়, যাতে ভেতর থেকে সাধারণ action পাঠানো যায়

`loadApplications(filters)` আসলে কী?

অধ্যায় ৮ থেকে আমরা লিখে আসছি:

```
void dispatch(loadApplications(filters));
```

এখানে দুটো ধাপ:

1. `loadApplications(filters)` call করলে server call **হয় না**। এটা একটা **thunk action creator**: ঠিক যেমন সাধারণ action creator (অধ্যায় ১০) action object return করে, এটা return করে একটা thunk function।

2. `dispatch(...)` সেই function পায়। Thunk middleware দেখে এটা function, তাই সেটা চালায়, আর তখনই server call শুরু হয়।

`dispatch(thunk)` একটা **Promise** return করে, যেটা কাজ শেষ হলে fulfilled হয়। একটা গুরুত্বপূর্ণ ব্যাপার: `createAsyncThunk` দিয়ে বানানো thunk dispatch করলে যে Promise আসে, সেটা **কখনো reject হয় না**, server call ব্যর্থ হলেও না; ব্যর্থ হলেও এটা fulfilled হয়, শুধু ভেতরে থাকে "ব্যর্থ" অবস্থার action (১১.৬)। আমরা সাধারণত অপেক্ষা করি না, তাই সামনে `void`।

`createAsyncThunk`

Redux Toolkit-এর `createAsyncThunk` thunk action creator বানানোর কাজটা গুছিয়ে দেয়। একটা async function দাও, আর সে নিজে থেকেই তিনটা action বানায় ও ঠিক সময়ে dispatch করে:

Action	কখন	অধ্যায় ৩-এর Promise-এর কোন অবস্থা
`.../pending`	কাজ শুরু হলো	pending
`.../fulfilled`	সফল; return করা ফলাফল থাকে `action.payload`-এ	fulfilled
`.../rejected`	ব্যর্থ; `rejectWithValue` দিয়ে পাঠানো error message থাকে `action.payload`-এ। `rejectWithValue` ছাড়া error হলে `payload` হয় `undefined`, আর error-এর তথ্য থাকে `action.error`-এ	rejected

---

১১.২ আমাদের project-এর তিন layer

প্রতিটা feature-এ server-এর সাথে কাজ তিনটা আলাদা file-এ ভাগ করা। প্রতিটা file হলো একটা **layer**: নিজের কাজটুকু করে, আর শুধু তার আগের layer-কে চেনে: slice চেনে thunk-কে, thunk চেনে api-কে (slice → thunk → api)।

```
features/applications/
├── applications.api.ts      ← (১) শুধু HTTP call, আর কিছু জানে না
├── applications.thunks.ts   ← (২) api call করে, ফলাফল বা error Redux-কে দেয়
└── applications.slice.ts    ← (৩) pending/fulfilled/rejected দেখে state বদলায়
```

প্রতিটা file-এর একটাই কাজ। ফলে পড়া, বদলানো আর bug খোঁজা সহজ।

১১.৩ Layer ১: API function

`frontend/src/features/applications/applications.api.ts` (import বাদে):

```
function applicationUrl(id: string): string
{
    return buildPath(ApiEndpoints.ApplicationById, { id });
}

export function getApplications(filters: ApplicationFilters):
Promise<Page<JobApplication>>
{
    return api.getPage<JobApplication>(ApiEndpoints.Applications +
buildQueryString(filtersToQuery(filters)));
}

export function getApplication(id: string): Promise<JobApplication>
{
    return api.get<JobApplication>(applicationUrl(id));
}

export function createApplication(payload: ApplicationPayload): Promise<JobApplication>
{
    return api.post<JobApplication>(ApiEndpoints.Applications, payload);
}

export function updateApplication(id: string, payload: Partial<ApplicationPayload>):
Promise<JobApplication>
{
    return api.patch<JobApplication>(applicationUrl(id), payload);
}

export function deleteApplication(id: string): Promise<void>
{
    return api.delete(applicationUrl(id));
}
```

React বা Redux, কারো কথা এখানে নেই। শুধু "এই URL-এ এই data পাঠাও, এই type-এর response আসবে"। যেমন `getApplication('42')` `/applications/42`-এ GET request পাঠায় (`buildPath`, অধ্যায় ৯), আর একটা `JobApplication` return করে। `api` object (`shared/api.ts`) আসল `fetch` call, token যোগ করা আর error বানানোর কাজ করে।

Filter থেকে query string

`getApplications`-এ ব্যবহার হওয়া `filtersToQuery` থাকে আলাদা file-এ, `frontend/src/features/applications/applications.utils.ts`:

```
export function filtersToQuery(filters: ApplicationFilters): QueryParams
{
    return {
        page: filters.page,
        limit: filters.limit,
        sortBy: filters.sortBy,
        sortOrder: filters.sortOrder,
        search: filters.search.trim(),
        applyMedium: filters.applyMedium,
        isApplied: filters.status === StatusFilter.Applied
            ? true
            : filters.status === StatusFilter.NotApplied
            ? false
            : undefined,
        upcoming: filters.status === StatusFilter.Upcoming ? true : undefined,
        expired: filters.status === StatusFilter.Expired ? true : undefined,
    };
}
```

UI-র একটা `status` filter, server-এর কাছে কয়েকটা আলাদা query parameter হয়ে যায়। `isApplied`-এর nested ternary সহজ ভাষায়: "status Applied হলে `true`, NotApplied হলে `false`, অন্য কিছু হলে `undefined` (মানে পাঠাবই না)"।

তারপর `buildQueryString` (`shared/utils.ts`) খালি value (`''`, `undefined`, `null`) বাদ দিয়ে query string বানায়। যেমন `DEFAULT_FILTERS`-এ শুধু status বদলে Applied করলে ফলাফল:

```
?page=1&limit=10&sortBy=createdAt&sortOrder=desc&isApplied=true
```

`search` খালি আর `applyMedium` খালি, তাই বাদ; `upcoming` আর `expired` `undefined`, তাই সেগুলোও বাদ।

১১.৪ Layer ২: Thunk

`frontend/src/features/applications/applications.thunks.ts` (সংক্ষেপে):

```
import { createAppAsyncThunk } from '@/app/redux';
import * as applicationsApi from '@/features/applications/applications.api';
...

export const loadApplications = createAppAsyncThunk<Page<JobApplication>,
ApplicationFilters>(
    'applications/loadList',
    async (filters, { rejectWithValue }) =>
    {
        try
        {
            return await applicationsApi.getApplications(filters);
        }
        catch (error: unknown)
        {
            return rejectWithValue(getErrorMessage(error));
        }
    },
);
```

- `createAppAsyncThunk` : `createAsyncThunk` -এর typed version (নিচে বিস্তারিত)।
- `import * as applicationsApi` : api file-এর সব export একটা object-এ নিয়ে আসে (module, অধ্যায় ২)। তাই api-র function call হয় `applicationsApi.createApplication(...)` হিসেবে। এর সুবিধা: thunk-এর নাম `createApplication` আর api function-এর নাম `createApplication` একই হলেও কোনো name conflict হয় না।
- `<Page<JobApplication>, ApplicationFilters>` : প্রথম type হলো সফল হলে কী return করবে, দ্বিতীয়টা হলো thunk action creator-কে কী argument দিয়ে call করা হবে।
- `'applications/loadList'` : action type-এর শুরু। এখান থেকে তৈরি হয় `applications/loadList/pending` , `.../fulfilled` আর `.../rejected` ।
- **প্রথম parameter** `filters` : `loadApplications(filters)` call করার সময় যা দেওয়া হয়েছিল।
- **দ্বিতীয় parameter**: RTK একটা object দেয়, যাকে সাধারণত বলে **thunkAPI**। এতে থাকে `dispatch` , `getState` (পুরো state পড়ার function), `rejectWithValue` আর আরো কিছু। আমাদের শুধু `rejectWithValue` লাগে, তাই destructure করে সেটাই নেওয়া হয়েছে: `{ rejectWithValue }` ।
- `return await ...` : যা `return` করা হয়, সেটাই `fulfilled` action-এর `payload` হয়।
- `return rejectWithValue(message)` : ব্যর্থ হলে `rejected` action পাঠায়, আর তার `payload` -এ থাকে আমাদের **readable error message**। `getErrorMessage` (`shared/utils.ts`) যেকোনো error থেকে একটা string বের করে।

`createAppAsyncThunk` কেন?

`frontend/src/app/redux.ts`:

```
export const createAppAsyncThunk = createAsyncThunk.withTypes<{
    state: RootState;
    dispatch: AppDispatch;
    rejectValue: string;
}>();
```

অধ্যায় ১০-এর `useAppSelector`-এর মতোই এটা `createAsyncThunk`-এর typed version। সব thunk-এর জন্য একবারে বলে দেওয়া হলো: `getState` দেবে `RootState`, `dispatch`-এর type `AppDispatch`, আর `rejectWithValue`-এর payload সবসময় `string`। তাই প্রতিটা thunk-এ এগুলো আবার লিখতে হয় না।

একাধিক argument লাগলে

Thunk একটাই argument নেয়, তাই একাধিক জিনিস লাগলে একটা object-এ ভরে দিতে হয়:

`applications.thunks.ts` (সংক্ষেপে):

```
export const updateApplication = createAppAsyncThunk<
    JobApplication,
    { id: string; payload: Partial<ApplicationPayload>; }
>(
    'applications/update',
    async ({ id, payload }, { rejectWithValue }) =>
    {
        ...
    },
);
```

`ApplicationsPage.tsx`-এ ব্যবহার:

```
void dispatch(updateApplication({ id: application.id, payload: { isApplied:
!application.isApplied } }));
```

ফলাফল: ওই application-এর `isApplied` উল্টে দেওয়ার PATCH request server-এ যায়।

১১.৫ Layer ৩: Slice-এর `extraReducers`

`reducers` (অধ্যায় ১০) হলো slice-এর **নিজের** action, যেগুলোর action creator slice নিজে বানায়। Thunk-এর action তৈরি হয় slice-এর **বাইরে**, `createAsyncThunk`-এ। বাইরের action handle করা হয় `extraReducers`-এ।

`extraReducers` একটা function, যেটা একটা `builder` পায়। `builder` হলো RTK-র দেওয়া একটা object, যেটা দিয়ে "কোন action এলে কোন reducer চলবে" register করা হয়। এর দুটো method আমরা ব্যবহার করি:

Method	মানে
`addCase(actionCreator, reducer)`	ঠিক এই একটা action এলে এই reducer চালাও
`addMatcher(condition, reducer)`	`condition` যে action-এর জন্য `true` দেয়, সেগুলোর সবকটায় এই reducer চালাও

এই `condition` function-টাকে বলে **matcher**: action নিয়ে `true` বা `false` return করে। প্রতিটা method `builder`-কেই return করে, তাই একটার পর একটা chain করে লেখা যায়।

`applications.slice.ts` (সংক্ষেপে):

```
extraReducers: (builder): void =>
{
    builder
        .addCase(loadApplications.pending, (state): void =>
        {
            state.listStatus = RequestStatus.Loading;
            state.listError = null;
        })
        .addCase(loadApplications.fulfilled, (state, action): void =>
        {
            state.listStatus = RequestStatus.Succeeded;
            state.items = action.payload.items;
            state.meta = action.payload.meta;
        })
        .addCase(loadApplications.rejected, (state, action): void =>
        {
            state.listStatus = RequestStatus.Failed;
            state.listError = action.payload ?? 'Could not load applications';
        })
        ...
}
```

ফলাফল: list load শুরু হলে `listStatus` হয় Loading, সফল হলে `items` আর `meta` বসে, ব্যর্থ হলে `listError`-এ message বসে।

`action.payload`-এর type TypeScript নিজেই জানে: fulfilled-এ `Page<JobApplication>`, আর rejected-এ `string | undefined`।

Rejected-এ `payload` কেন `undefined` হতে পারে?

Thunk দুইভাবে ব্যর্থ হতে পারে:

কীভাবে ব্যর্থ	`action.payload`	`action.error`
আমরা `return rejectWithValue('...')` করলাম	আমাদের message	`{ message: 'Rejected' }`
`rejectWithValue` ছাড়াই কোনো error throw হলো (যেমন try-এর বাইরে bug)	`undefined`	আসল error-এর message ইত্যাদি

আমাদের thunk-গুলো সব error ধরে `rejectWithValue` করে, তবুও TypeScript দ্বিতীয় সম্ভাবনাটা বাদ দিতে পারে না। তাই `action.payload ?? 'Could not load applications'`: payload `undefined` হলে default message বসে।

`RequestStatus` pattern

`RequestStatus` হলো `frontend/src/shared/types.ts`-এর একটা enum:

```
export enum RequestStatus
{
    Idle = 'idle',
    Loading = 'loading',
    Succeeded = 'succeeded',
    Failed = 'failed',
}
```

প্রতিটা async কাজের জন্য slice-এ তিনটা জিনিস রাখা হয়: status, error আর data।

```
listStatus: RequestStatus;
listError: string | null;
items: JobApplication[];
```

Page এগুলো দেখে ঠিক করে কী দেখাবে। `ApplicationsPage.tsx` (সংক্ষেপে):

```
<ErrorMessage message={listError} />

{listStatus === RequestStatus.Loading && items.length === 0 && <Spinner />}

{listStatus === RequestStatus.Succeeded && items.length === 0 && (
    <p>No applications found.</p>
)}

{items.length > 0 && <ApplicationsTable applications={items} ... />}
```

ফলাফল:

- `ErrorMessage` `listError` `null` হলে কিছুই render করে না (`null` return করে); error থাকলে লাল box-এ message দেখায়।
- Spinner দেখায় শুধু loading চলছে **আর** list খালি থাকলে, অর্থাৎ শুধু প্রথমবার। পরে filter বা page বদলালে নতুন load চলার সময় পুরনো table screen-এ থেকে যায়, spinner আসে না, তাই page লাফায় না।

- Load সফল কিন্তু list খালি হলে "No applications found."।
- List-এ কিছু থাকলে table।

এক action, অনেক slice

`applications.slice.ts`:

```
.addCase(logoutUser.fulfilled, (): ApplicationsState => initialState)
```

`logoutUser` thunk থাকে auth feature-এ (`auth.thunks.ts`), কিন্তু applications slice-ও এটা handle করে। Logout-এর সময় এই reducer draft-এ হাত না দিয়ে সরাসরি `initialState` return করে (অধ্যায় ১০-এর Immer নিয়ম), তাই আগের user-এর সব application store থেকে মুছে যায়। যেকোনো slice যেকোনো action handle করতে পারে।

Server-এর response দিয়ে list update রাখা

`applications.slice.ts`:

```
.addCase(updateApplication.fulfilled, (state, action): void =>
{
    state.items = state.items.map((item) => item.id === action.payload.id ?
action.payload : item);
    if (state.selected?.id === action.payload.id)
    {
        state.selected = action.payload;
    }
})
.addCase(deleteApplication.fulfilled, (state, action): void =>
{
    state.items = state.items.filter((item) => item.id !== action.payload);
    if (state.selected?.id === action.payload)
    {
        state.selected = null;
    }
})
```

Table-এ "Not applied" badge-এ click করলে পুরো list আবার আনতে হয় না। Server update হওয়া application-টা response-এ ফেরত দেয়, আর `map` দিয়ে list-এর শুধু সেই row-টা বদলে দেওয়া হয়; বাকি row আগের মতো। Details page-এ খোলা application (`selected`) একই হলে সেটাও update হয়। Delete-এর ক্ষেত্রে `payload` হলো মুছে ফেলা `id`, আর `filter` সেই row বাদ দেয়।

`isAnyOf` : কয়েকটা action-এ একই কাজ

Create আর update দুটোরই "saving" অবস্থা একই রকম। RTK-র `isAnyOf(...)` একটা matcher বানায়, যেটা দেওয়া action-গুলোর যেকোনোটা এলে `true` দেয়।

`applications.slice.ts` :

```
.addMatcher(isAnyOf(createApplication.pending, updateApplication.pending), (state): void
=>
{
    state.saveStatus = RequestStatus.Loading;
    state.saveError = null;
})
.addMatcher(isAnyOf(createApplication.fulfilled, updateApplication.fulfilled), (state):
void =>
{
    state.saveStatus = RequestStatus.Succeeded;
})
.addMatcher(isAnyOf(createApplication.rejected, updateApplication.rejected), (state,
action): void =>
{
    state.saveStatus = RequestStatus.Failed;
    state.saveError = action.payload ?? 'Could not save the application';
});
```

ফলাফল: create বা update যেটাই হোক, শুরুতে `saveStatus` Loading, সফল হলে Succeeded, ব্যর্থ হলে Failed আর `saveError` -এ message।

দুটো নিয়ম মনে রাখো:

1. **`builder` -এ সব `addCase` আগে, তারপর `addMatcher` ।** `addMatcher` -এর পরে `addCase` লিখলে RTK error throw করে।
2. **একটা action একটা case আর একটা matcher, দুটোর সাথেই মিললে দুটো reducer-ই চলে** (আগে case, তারপর matcher)। যেমন `updateApplication.fulfilled` এলে উপরের `addCase` (আগের অংশে দেখানো) list-এর row update করে, **আর** ঠিক উপরের fulfilled matcher `saveStatus` Succeeded করে।

১১.৬ Component-এ thunk-এর ফলাফল জানা: `unwrap` আর `match`

সাধারণত component শুধু dispatch করে, আর ফলাফল আসে store থেকে (`useAppSelector`)। কিন্তু মাঝে মাঝে **সফল হলেই** পরের কাজ করতে হয়, যেমন save সফল হলে details page-এ যাওয়া। এর জন্য `dispatch(thunk)` -এর return করা Promise ব্যবহার করা হয়।

মনে করো (১১.১): এই Promise কখনো reject হয় না। সফল হলে এর ফলাফল fulfilled action, আর ব্যর্থ হলে **rejected action**। দুইভাবে বোঝা যায় কোনটা হলো।

`unwrap()`

`ApplicationFormPage.tsx` :

```
async function handleSubmit(payload: ApplicationPayload): Promise<void>
{
    try
    {
        const saved = id
            ? await dispatch(updateApplication({ id, payload })).unwrap()
            : await dispatch(createApplication(payload)).unwrap();
        void navigate(buildPath(AppRoutes.ApplicationDetails, { id: saved.id }));
    }
    catch
    {
        return;
    }
}
```

`.unwrap()` Promise-টাকে বদলে দেয়: সফল হলে সরাসরি `payload` দেয় (এখানে save হওয়া `JobApplication`), আর ব্যর্থ হলে `rejectWithValue` -এ দেওয়া value (এখানে একটা message string) **throw** করে — Error object না, তাই `error.message` লেখা যাবে না। তাই সাধারণ try/catch লেখা যায়।

ফলাফল: সফল হলে `saved.id` দিয়ে details page-এ যাওয়া হয়। ব্যর্থ হলে `catch` -এ কিছু করার নেই, কারণ rejected matcher ইতিমধ্যে `saveError` -এ message বসিয়েছে, আর page সেটা `<ErrorMessage>` -এ দেখাচ্ছে।

`.fulfilled.match()`

`ApplicationDetailsPage.tsx` :

```
const result = await dispatch(deleteApplication(selected.id));
if (deleteApplication.fulfilled.match(result))
{
    void navigate(AppRoutes.Applications);
}
```

এখানে `result` হলো শেষ action: সফল হলে fulfilled action, ব্যর্থ হলে rejected action। `deleteApplication.fulfilled.match(result)` check করে `result` fulfilled action কিনা।

`match` একটা **type guard**: যে function `true` return করলে TypeScript ওই `if`-এর ভেতরে value-র type narrow করে (অধ্যায় ৪)। তাই `if`-এর ভেতরে TypeScript জানে `result` fulfilled action, আর `result.payload` নিরাপদে ব্যবহার করা যায়।

ফলাফল: delete সফল হলে list page-এ যাওয়া হয়; ব্যর্থ হলে কিছুই হয় না, user একই page-এ থাকে।

কোনটা কখন?

	`unwrap()`	`.fulfilled.match()`
ব্যর্থ হলে	`rejectWithValue`-এ দেওয়া value (message string) throw করে; Error object না, তাই `error.message` চলে না	`false` return করে
লিখতে হয়	try/catch	সাধারণ `if`
ভালো যখন	সফল হলে payload লাগবে, যেমন `saved.id`	শুধু "সফল হলো কিনা" জানলেই চলে

১১.৭ পুরো যাত্রা: page খোলা থেকে table দেখা পর্যন্ত

```
① ApplicationsPage mount হলো
   useEffect → dispatch(loadApplications(filters))
   thunk middleware দেখে এটা function, তাই চালায়

② createAsyncThunk → dispatch({ type: 'applications/loadList/pending' })
   slice: listStatus = Loading
   page re-render → <Spinner /> (items এখনো খালি)

③ thunk-এর ভেতর: applicationsApi.getApplications(filters)
   → filtersToQuery + buildQueryString → "/applications?page=1&limit=10&..."
   → api.getPage → request('GET', ...) → fetch("/api/applications?...")
   → Vite dev server → backend (NestJS, localhost:3000) → database (PostgreSQL)

④ Server-এর response: { success: true, data: [...], meta: { page: 1, hasNext: true, ...
} }
   → api.getPage return করে { items, meta }
   → thunk সেটা return করে

⑤ createAsyncThunk → dispatch({ type: '.../fulfilled', payload: { items, meta } })
   slice: listStatus = Succeeded, items = ..., meta = ...

⑥ useAppSelector দেখে state.applications বদলেছে → ApplicationsPage re-render
   → <ApplicationsTable applications={items} /> + <Pagination meta={meta} />
```

③-এ: development-এ Vite dev server `/api` দিয়ে শুরু হওয়া request backend-এ (`localhost:3000`) পাঠিয়ে দেয় (`vite.config.ts`-এর proxy setting)।

ব্যর্থ হলে ④-এ `api` একটা `ApiError` throw করে। তখন thunk-এর `catch` থেকে `rejectWithValue('...')` হয়, slice-এ `listStatus` Failed আর `listError`-এ message বসে, আর page-এ `<ErrorMessage>` দেখায়।

সারসংক্ষেপ

- **Thunk** হলো async কাজ করার জন্য dispatch করা যায় এমন function। `configureStore`-এর thunk middleware function দেখলে সেটা চালায়, object দেখলে reducer-এ পাঠায়।
- `loadApplications(filters)` একটা **thunk action creator**: thunk function return করে। `createAsyncThunk` দিয়ে বানানো thunk dispatch করলে যে Promise আসে, সেটা কখনো reject হয় না।

- `createAsyncThunk` নিজেই **pending → fulfilled/rejected** action পাঠায়। `return` হয়ে যায় fulfilled-এর payload, আর `rejectWithValue` হয়ে যায় rejected-এর payload।
- Slice-এর `extraReducers`-এ `builder.addCase` একটা action ধরে, `builder.addMatcher` matcher দিয়ে কয়েকটা একসাথে ধরে (যেমন `isAnyOf`)। সব `addCase` আগে, `addMatcher` পরে।
- `RequestStatus` + error + data, এই তিনটা দিয়ে loading, error, empty আর data অবস্থা দেখানো হয়।
- Component-এ সফল হলেই কিছু করতে `unwrap()` (try/catch) বা `.fulfilled.match()` (type guard)।
- Layer তিনটা: **api → thunk → slice**।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Redux-এ async কাজ কোথায় করো? reducer-এ কেন নয়?
উত্তর: Thunk-এ। Thunk হলো একটা function, যেটা dispatch করা যায়; thunk middleware সেটা চালায়, আর ভেতরে server call করে শেষে সাধারণ action dispatch করা যায়। Reducer-এ করা যায় না, কারণ reducer pure আর synchronous হতে হয়: একই input-এ একই output, কোনো side effect না। Server call একটা side effect, আর তার ফলাফল পরে আসে, তাই reducer-এর নিয়ম ভাঙে।

প্রশ্ন: createAsyncThunk কোন কোন action পাঠায়?
উত্তর: তিনটা। কাজ শুরু হলে `pending`। সফল হলে `fulfilled`, যার payload হলো async function-এর return value। ব্যর্থ হলে `rejected`; `rejectWithValue` ব্যবহার করলে তার value payload-এ থাকে, আর না করলে payload `undefined` হয় এবং error-এর তথ্য থাকে `action.error`-এ।

প্রশ্ন: reducers আর extraReducers-এর পার্থক্য কী?
উত্তর: `reducers`-এর প্রতিটা key থেকে slice একটা reducer আর একই নামের action creator বানায়; এগুলো slice-এর নিজের action। `extraReducers` নতুন action creator বানায় না; এটা slice-এর বাইরে তৈরি হওয়া action handle করে, যেমন thunk-এর pending/fulfilled/rejected বা অন্য slice-এর action। এখানে `builder.addCase` দিয়ে একটা নির্দিষ্ট action, আর `builder.addMatcher` দিয়ে শর্ত মেলা কয়েকটা action ধরা হয়।

প্রশ্ন: unwrap() কী করে?
উত্তর: `createAsyncThunk` দিয়ে বানানো thunk dispatch করলে যে Promise আসে, সেটা কখনো reject হয় না; ব্যর্থ হলেও rejected action দিয়ে fulfilled হয়। `unwrap()` সেটাকে সাধারণ Promise-এর মতো বানায়: সফল হলে সরাসরি payload দেয়, ব্যর্থ হলে `rejectWithValue`-এ দেওয়া value (এখানে একটা message string) throw করে — Error object না, তাই `error.message` লেখা যাবে না। ফলে try/catch দিয়ে সফল হলে পরের কাজ, যেমন অন্য page-এ যাওয়া, আর ব্যর্থ হলে আলাদা কাজ করা যায়।

প্রশ্ন: Loading আর error state কীভাবে সামলাও?
উত্তর: প্রতিটা async কাজের জন্য slice-এ একটা `status` রাখি, যার value idle, loading, succeeded বা failed, সাথে একটা `error` আর আসল data। Thunk-এর pending-এ status loading আর error null করি, fulfilled-এ succeeded আর data বসাই, rejected-এ failed আর error message বসাই। UI status আর data দেখে ঠিক করে spinner, error message, empty message না data দেখাবে।

প্রশ্ন: Race condition কী? কীভাবে এড়াবে?
উত্তর: Race condition হলো যখন একাধিক async কাজ চলে, আর কোনটা আগে শেষ হবে তার উপর ফলাফল নির্ভর করে। যেমন user দ্রুত দুইবার filter বদলাল, দুটো request গেল, কিন্তু প্রথমটার response পরে এসে দ্বিতীয়টার ফলাফল overwrite করে দিল; screen-এ পুরনো filter-এর data দেখা গেল। এড়ানোর উপায়:

- Debounce দিয়ে request-এর সংখ্যা কমানো (আমাদের search-এ করা হয়েছে)।
- `requestId` মিলিয়ে দেখা: createAsyncThunk প্রতিটা thunk call-কে একটা unique id দেয়, যেটা `action.meta.requestId` -এ থাকে। Slice-এ সর্বশেষ request-এর id রেখে, শুধু সেই id-র fulfilled action গ্রহণ করা যায়।
- AbortController দিয়ে পুরনো request বাতিল করা: এটা browser-এর একটা API, যেটা দিয়ে চলমান fetch বাতিল করা যায়। নতুন request পাঠানোর আগে পুরনোটা বাতিল করলে তার response আর আসে না।

প্রশ্ন: RTK Query বা TanStack Query কী? কখন ব্যবহার করবে?
উত্তর: এগুলো server-এর data আনার কাজ automate করার library। এরা loading আর error state নিজে রাখে, আর data **cache** করে, মানে একবার আনা data memory-তে রেখে দেয়, যাতে একই data আবার লাগলে server-এ না গিয়েই সাথে সাথে দেখানো যায়। দরকার হলে পেছনে আবার এনে update-ও করে (refetch)। বড় app-এ এগুলো slice আর thunk-এর বাড়তি কোড অনেক কমায়। শেখার জন্য createAsyncThunk ভালো, কারণ pending, fulfilled আর rejected-এ ভেতরে কী হচ্ছে স্পষ্ট দেখা যায়।

অধ্যায় ১২: Authentication — login থেকে logout

এই অধ্যায়ে দেখব একজন user কীভাবে account খোলে, login করে, page refresh করলেও login থেকে যায়, আর logout করে। সাথে দেখব security-র কিছু শব্দ, যেগুলো interview-তে প্রায়ই আসে।

১২.১ Authentication-এর মূল ধারণা

- **Authentication** মানে "তুমি কে?" প্রমাণ করা। যেমন email আর password দিয়ে login করা।
- **Authorization** মানে "তুমি কী করতে পারবে?" ঠিক করা। যেমন আমাদের app-এ login করা user শুধু নিজের application দেখতে পারে, অন্যের application দেখতে পারে না।

HTTP নিজে আগের request-এর কথা কিছু মনে রাখে না। প্রতিটা request server-এর কাছে একদম নতুন আর আলাদা। এই স্বভাবকে বলে **stateless**। তাহলে server কীভাবে জানবে কোন request কে পাঠাচ্ছে? উত্তর হলো **token**: একটা লম্বা string, যেটা browser প্রতিটা request-এর সাথে পাঠায়, আর server সেটা দেখে user চেনে।

JWT (JSON Web Token)

আমাদের project-এ token হলো **JWT**। এর পুরো যাত্রা এরকম:

1. login সফল হলে server একটা token দেয়, যার নাম **access token**। দেখতে এরকম: `eyJhbGciOiJIUzI1NiIs...`
2. token-টা তিন ভাগে ভাগ করা, মাঝে dot (`.`) দিয়ে: `header.payload.signature`। এখানে header মানে token-এর প্রথম অংশ (কোন algorithm দিয়ে sign করা, সেই তথ্য)। এটা HTTP header না; HTTP header হলো request-এর সাথে যাওয়া বাড়তি তথ্য (অধ্যায় ১.৪)।
3. browser token রেখে দেয়, আর প্রতিটা request-এর সাথে header-এ পাঠায়:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

4. server signature verify করে, তারপর payload থেকে বোঝে request-টা কোন user-এর।

Payload-এ কী থাকে, আর কে পড়তে পারে?

Payload হলো token-এর মাঝের অংশ, যেখানে user-এর তথ্য থাকে। এটা **base64** দিয়ে লেখা। base64 হলো যেকোনো data-কে শুধু অক্ষর আর সংখ্যা দিয়ে লেখার একটা পদ্ধতি। এটা encryption না। যে কেউ token হাতে পেলে base64 decode করে payload পড়ে ফেলতে পারে।

আমাদের backend (`backend/jat/src/auth/services/auth.service.ts`) payload-এ রাখে এগুলো:

```
{
    "sub": "3f6c2a1e-8b7d-4c2f-9a10-5e2b7c9d1f04",
    "email": "rahim@example.com",
    "iat": 1760170000,
    "exp": 1760256400
}
```

Field	মানে
`sub`	"subject", মানে token-টা কার। এখানে user-এর id, যেটা একটা **UUID** string (practically unique একটা লম্বা random id)
`email`	user-এর email
`iat`	token কখন তৈরি হয়েছে (issued at), সেকেন্ডে
`exp`	token কখন **expired** হবে, সেকেন্ডে

তাহলে কেউ payload বদলে অন্যের `sub` বসিয়ে দিলে কী হবে? এখানেই **signature** কাজে আসে। server একটা **secret key** দিয়ে header আর payload-কে **sign** করে, আর ফলাফলটা token-এর শেষ অংশে বসায়। secret key শুধু server জানে। payload-এর একটা অক্ষর বদলালেও signature আর মেলে না, তাই server token-টা **invalid** বলে ফিরিয়ে দেয়।

দুটো জিনিস মনে রাখো:

- JWT পড়া যায়, কিন্তু বদলানো যায় না। তাই password বা অন্য গোপন তথ্য কখনো JWT-তে রাখা যাবে না।
- আমাদের token ১ দিন চলে। backend-এর config-এ `JWT_EXPIRES_IN=86400` (৮৬,৪০০ সেকেন্ড = ২৪ ঘণ্টা)। এরপর token expired হয়ে যায়, আর আবার login করতে হয়।

১২.২ Auth feature-এর file-গুলো

```
features/auth/
├── auth.types.ts      ← User, LoginRequest, RegisterRequest, AuthResponse, AuthState
├── auth.api.ts        ← login, register, getCurrentUser
├── auth.thunks.ts     ← restoreSession, loginUser, registerUser, logoutUser
├── auth.slice.ts      ← user, isSessionChecked, status, error
├── LoginPage.tsx
├── RegisterPage.tsx
└── ProtectedRoute.tsx
```

State

`auth.types.ts`:

```
export interface AuthState
{
    user: User | null;
    isSessionChecked: boolean;
    status: RequestStatus;
    error: string | null;
}
```

Field	মানে
`user`	login করা user, অথবা `null`
`isSessionChecked`	app খোলার পরে "আগে থেকে login আছে কিনা" check শেষ হয়েছে কিনা
`status`, `error`	login বা register চলছে, সফল, নাকি ব্যর্থ (অধ্যায় ১১-এর RequestStatus pattern)

১২.৩ Token কোথায় রাখা হয়: `localStorage`

`localStorage` হলো browser-এর একটা ছোট key-value storage। এখানে রাখা data page বন্ধ করে আবার খুললেও থেকে যায়। এ ধরনের storage-কে বলে **persistent storage**। আমাদের token-এর key হলো `'jat.accessToken'` (`shared/constants.ts`-এর `TOKEN_STORAGE_KEY`)।

`frontend/src/shared/api.ts`-এ তিনটা ছোট helper:

```
export function getToken(): string | null
{
    return localStorage.getItem(TOKEN_STORAGE_KEY);
}

export function saveToken(token: string): void
{
    localStorage.setItem(TOKEN_STORAGE_KEY, token);
}

export function clearToken(): void
{
    localStorage.removeItem(TOKEN_STORAGE_KEY);
}
```

`getToken` token পড়ে (না থাকলে `null` দেয়), `saveToken` token লেখে, আর `clearToken` token মুছে দেয়।

প্রতিটা request-এ token যোগ করে একই file-এর `request` function (সংক্ষেপে):

```
const token = getToken();
if (token)
{
    headers.Authorization = `Bearer ${token}`;
}
```

token থাকলে header-এ `Authorization: Bearer ...` বসে যায়। না থাকলে header ছাড়াই request যায়।

১২.৪ Login-এর পুরো যাত্রা

(১) Page: `LoginPage.tsx`

```
function handleSubmit(event: SubmitEvent<HTMLFormElement>): void
{
    event.preventDefault();
    void dispatch(loginUser({ email, password }));
}
```

form submit হলে browser-এর reload থামানো হয়, তারপর `loginUser` thunk dispatch হয়।

(২) Thunk: `auth.thunks.ts`

```
export const loginUser = createAppAsyncThunk<User, LoginRequest>(
    'auth/login',
    async (request, { rejectWithValue }) =>
    {
        try
        {
            const { accessToken } = await authApi.login(request);
            saveToken(accessToken);
            return await authApi.getCurrentUser();
        }
        catch (error: unknown)
        {
            clearToken();
            return rejectWithValue(getErrorMessage(error));
        }
    },
);
```

1. `POST /api/auth/login` -এ server ফেরত দেয় `{ accessToken }` ।
2. token `localStorage` -এ রাখা হয়।
3. `GET /api/auth/me` -এ (এবার token সহ) server দেয় user-এর তথ্য।
4. user return করা হয়, যেটা `fulfilled` action-এর payload হয়।
5. কোনো ধাপ ব্যর্থ হলে token মুছে error message দিয়ে `rejected` হয়।

(৩) Slice: `auth.slice.ts`

```
.addMatcher(isAnyOf(loginUser.pending, registerUser.pending), (state): void =>
{
    state.status = RequestStatus.Loading;
    state.error = null;
})
.addMatcher(isAnyOf(loginUser.fulfilled, registerUser.fulfilled), (state, action): void
=>
{
    state.status = RequestStatus.Succeeded;
    state.user = action.payload;
    state.isSessionChecked = true;
})
.addMatcher(isAnyOf(loginUser.rejected, registerUser.rejected), (state, action): void =>
{
    state.status = RequestStatus.Failed;
    state.error = action.payload ?? 'Something went wrong';
})
```

login আর register-এর action একই ভাবে সামলানো হয়, তাই `isAnyOf` দিয়ে দুটোকে একসাথে ধরা হয়েছে (অধ্যায় ১১)। fulfilled হলে `user` বসে যায়।

(৪) আবার page: সফল হলে নিজে থেকেই অন্য page-এ

```
const { user, status, error } = useAppSelector((state) => state.auth);

if (user)
{
    return <Navigate to={redirectPathFrom(location.state)} replace />;
}
```

`user` set হলে LoginPage re-render হয়, আর সাথে সাথে applications page-এ (বা user আগে যে page-এ যেতে চাইছিল সেখানে) চলে যায়। কোথাও `navigate()` call করতে হয়নি। state বদলেছে, তাই UI নিজে বদলে গেছে। এটাই **UI = f(state)**।

- Loading-এর সময়: `<Button isLoading={status === RequestStatus.Loading}>`, তাই button **disabled** থাকে (click করা যায় না) আর লেখা দেখায় "Please wait..."।
- ব্যর্থ হলে: `<ErrorMessage message={error} />` দেখায় "Invalid email or password"।

Register

একই পথ, শুধু request যায় `POST /api/auth/register`-এ। register সফল হলেই user login হয়ে যায়, কারণ server register-এর response-এই token দিয়ে দেয়।

১২.৫ Page refresh করলে login থাকে কীভাবে: `restoreSession`

Refresh করলে পুরো app আবার শুরু হয়, আর Redux store খালি হয়ে যায় (`user = null`)। কিন্তু token তো `localStorage`-এ রয়ে গেছে। সেটা দিয়েই user ফিরিয়ে আনা হয়।

`main.tsx`:

```
void store.dispatch(restoreSession());

createRoot(rootElement).render(
    <StrictMode>
        <Provider store={store}>
            <AppRouter />
        </Provider>
    </StrictMode>,
);
```

এখানে একটা জিনিস খেয়াল করো। `dispatch` শুধু কাজটা **শুরু** করে। `/auth/me` request-এর উত্তর আসতে সময় লাগে, কিন্তু app সেটার জন্য অপেক্ষা করে না। `void` লেখা হয়েছে এটা বোঝাতে যে Promise-টার জন্য ইচ্ছা করেই অপেক্ষা করা হচ্ছে না (অধ্যায় ৩)। তাই পরের লাইনের `render` সাথে সাথেই চলে।

তাহলে render-এর মুহূর্তে আমরা জানি না user login আছে কিনা। এই ফাঁকটা ঢাকতেই `isSessionChecked` আছে। check শেষ না হওয়া পর্যন্ত এটা `false` থাকে, আর `ProtectedRoute` (অধ্যায় ৯) তখন spinner দেখায়। check শেষ হলে এটা `true` হয়, তখন ProtectedRoute হয় page দেখায়, নয়তো login-এ পাঠায়।

`auth.thunks.ts`:

```
export const restoreSession = createAppAsyncThunk<User | null, undefined>
('auth/restoreSession', async () =>
{
    if (!getToken())
    {
        return null;
    }
    try
    {
        return await authApi.getCurrentUser();
    }
    catch
    {
        clearToken();
        return null;
    }
});
```

দ্বিতীয় type `undefined` মানে এই thunk কোনো argument নেয় না। তাই `main.tsx`-এ `restoreSession()` খালি call করা হয়, আর `async () =>`-এর parentheses-ও খালি (বিস্তারিত অধ্যায় ১৪.৬-এর ধাপ ৫)।

পরিস্থিতি	কী হয়
token নেই	সাথে সাথে `null`, মানে login নেই
token আছে আর valid	`/auth/me` থেকে user আসে, login ফিরে এলো
token আছে কিন্তু expired বা invalid	server দেয় 401। token থাকা অবস্থায় 401 এসেছে, তাই `request`-এর 401 handler (১২.৬) token মুছে `/login`-এ reload করে। thunk-এর `catch`-ও token মুছে `null` দেয়
server বন্ধ বা network নেই	`catch` এখানেও token মুছে দেয়, তাই user-কে আবার login করতে হয়

শেষ row-টা একটা trade-off। server সাময়িক বন্ধ থাকলেও token মুছে যায়, যদিও token-টা হয়তো ঠিকই ছিল। ভালো সমাধান হতো error-এর ধরন দেখে শুধু 401-এ token মোছা। কিন্তু code সহজ রাখতে এই project-এ যেকোনো error-এ token মোছা হয়।

Slice:

```
.addCase(restoreSession.fulfilled, (state, action): void =>
{
    state.user = action.payload;
    state.isSessionChecked = true;
})
```

thunk কখনো reject হয় না (error হলেও `null` return করে), তাই শুধু `fulfilled` সামলালেই চলে। যেকোনো ক্ষেত্রেই শেষে `isSessionChecked = true` হয়।

১২.৬ Token expired হলে: 401 সামলানো

ধরো app খোলা আছে, আর মাঝে token expired হয়ে গেল। পরের request-এ server দেয় **401 Unauthorized**। `shared/api.ts`-এর `request` function-এ (সংক্ষেপে):

```
if (response.status === 401 && token)
{
    clearToken();
    window.location.assign(AppRoutes.Login);
}
```

token মুছে login page-এ পাঠানো হয়।

- `window.location.assign` পুরো page নতুন করে load করে, তাই Redux-এর সব পুরনো state মুছে যায়। এটা সবচেয়ে সহজ আর নিশ্চিত উপায়।
- `&& token` কেন? ভুল password দিয়ে login করলেও server 401 দেয়। তখন সাধারণত token থাকে না, তাই redirect হয় না। শুধু error message দেখায়।

১২.৭ Logout

```
export const logoutUser = createAppAsyncThunk('auth/logout', (): void =>
{
    clearToken();
});
```

এখানে কোনো server call নেই, function-টা async-ও না। তবুও এটা thunk কেন, সাধারণ reducer না কেন?

1. `clearToken()` `localStorage` বদলায়। এটা একটা **side effect** (function-এর বাইরের কিছু বদলানো)। reducer pure হতে হয়, তাই reducer-এর ভেতরে এটা রাখা যায় না (অধ্যায় ১০)।
2. `createAsyncThunk` function sync হলেও `pending` আর `fulfilled` action পাঠায়। ফলে আমরা একটা `logoutUser.fulfilled` action পাই।
3. সেই একটা action দুটো slice শোনে:
    - **auth slice:** `user = null`, `status = Idle`।
    - **applications slice:** সবকিছু `initialState`-এ ফিরিয়ে দেয়, যাতে পরের user আগের জনের application না দেখে।

`user` null হলো, তাই `ProtectedRoute` নিজে থেকেই `/login`-এ পাঠিয়ে দেয়।

Navbar-এর button, `app/AppLayout.tsx`-এ (সংক্ষেপে):

```
<button
    type="button"
    onClick={(): void =>
    {
        void dispatch(logoutUser());
    }}
>
    Log out
</button>
```

click করলে token মুছে যায়, দুই slice reset হয়, আর user login page-এ পৌঁছে যায়।

১২.৮ Security নিয়ে কিছু কথা (interview-তে প্রায়ই আসে)

বিষয়	ব্যাখ্যা
localStorage-এ token	সহজ, কিন্তু page-এ কেউ নিজের JavaScript চালাতে পারলে (XSS) `localStorage` পড়ে token চুরি করতে পারে। বিকল্প হলো httpOnly cookie, যেটা JavaScript পড়তে পারে না।
XSS ঠেকানো	React `{text}`-কে সবসময় escape করে, মানে HTML হিসেবে চালায় না। ঝুঁকি শুধু `dangerouslySetInnerHTML`-এ। আমাদের server rich-text mail-টা save করার আগে `sanitize-html` দিয়ে sanitize করে (অধ্যায় ১৩.৫)।
Frontend check যথেষ্ট না	frontend-এর যেকোনো check এড়ানো যায় (কেউ সরাসরি API call করতে পারে)। আসল security server-এ: আমাদের backend প্রতিটা request-এ token verify করে, আর শুধু নিজের application দেখায়।

বিষয়	ব্যাখ্যা
ProtectedRoute	শুধু UI লুকায়, এটা security না। data-র security server-এর দায়িত্ব।

Security-র শব্দগুলো

শব্দ	মানে
Cookie	server-এর পাঠানো ছোট একটা data, যেটা browser রেখে দেয়, আর ওই server-এ প্রতিটা request-এর সাথে নিজে থেকেই পাঠায়। JavaScript-কে কিছু করতে হয় না।
httpOnly cookie	এমন cookie, যেটা JavaScript (`document.cookie`) পড়তে পারে না। browser শুধু request-এর সাথে পাঠায়। তাই XSS হলেও script token চুরি করতে পারে না।
XSS (Cross-Site Scripting)	অন্য কারো লেখা JavaScript তোমার page-এ চলে যাওয়া। যেমন কেউ নিজের নামের জায়গায় `<script>...</script>` লিখল, আর page সেটা HTML হিসেবে চালিয়ে দিল।
Escape	`<` , `>` -এর মতো বিশেষ অক্ষরকে সাধারণ লেখা হিসেবে দেখানো, যাতে browser সেগুলোকে tag ভাবে না। React এটা নিজে থেকেই করে।
Sanitize	HTML থেকে বিপজ্জনক অংশ (`<script>` , `onclick` ইত্যাদি) মুছে শুধু নিরাপদ tag রাখা।
CSRF (Cross-Site Request Forgery)	অন্য একটা website তোমার browser দিয়ে গোপনে তোমার site-এ request পাঠায়। cookie নিজে থেকেই যায়, তাই server ভাবে request-টা তুমিই পাঠিয়েছ। cookie ব্যবহার করলে এই ঝুঁকি আসে। আমাদের token header-এ হাতে পাঠানো হয়, তাই এই ঝুঁকি কম।
SameSite	cookie-র একটা setting। `SameSite=Lax` বা `Strict` দিলে অন্য site থেকে আসা request-এ browser cookie পাঠায় না। CSRF ঠেকানোর প্রধান উপায়।
Refresh token	একটা আলাদা, দীর্ঘ মেয়াদের token। access token অল্প সময় (যেমন ১৫ মিনিট) চলে। সেটা expired হলে refresh token দিয়ে চুপচাপ নতুন access token আনা হয়, user-কে আবার login করতে হয় না। সাধারণত refresh token httpOnly cookie-তে রাখা হয়। আমাদের project-এ refresh token নেই।
Server-side session	JWT-এর পুরনো বিকল্প। login হলে server নিজের database বা memory-তে "এই session id = এই user" লিখে রাখে, আর browser-কে শুধু session id দেয় (সাধারণত cookie-তে)। প্রতিটা request-এ server নিজের রেকর্ড খুঁজে user চেনে।

JWT-কে "stateless" বলে কেন? JWT-তে server কোনো session রেকর্ড রাখে না। user-এর তথ্য token-এর ভেতরেই থাকে, আর signature verify করলেই server user চিনে ফেলে। এই "stateless" আর ১২.১-এর "HTTP stateless" আলাদা জিনিস। HTTP stateless মানে protocol নিজে request-গুলোর মাঝে কিছু মনে রাখে না। JWT stateless মানে server-কেও login-এর কোনো রেকর্ড মনে রাখতে হয় না।

সারসংক্ষেপ

- Authentication মানে "তুমি কে", Authorization মানে "তুমি কী করতে পারবে"।
- login সফল হলে server **JWT access token** দেয়। payload-এ `sub` (user id), `email` আর মেয়াদ থাকে। payload পড়া যায়, কিন্তু secret key দিয়ে sign করা, তাই বদলানো যায় না। token ১ দিন চলে।
- token `localStorage`-এ থাকে, আর প্রতিটা request-এ `Authorization: Bearer ...` হিসেবে যায়।
- `loginUser` আর `registerUser` token রাখে, তারপর `/auth/me` থেকে user আনে।
- `restoreSession` app শুরুতে শুধু কাজ শুরু করে, render অপেক্ষা করে না। তাই check শেষ না হওয়া পর্যন্ত `isSessionChecked = false` থাকে আর ProtectedRoute spinner দেখায়।
- token থাকা অবস্থায় 401 এলে token মুছে login-এ reload হয়।
- `logoutUser` thunk কারণ token মোছা একটা side effect, আর এর `fulfilled` action auth আর applications দুই slice reset করে।
- frontend-এর security শুধু UI-এর জন্য, আসল security server-এ।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Authentication আর Authorization-এর পার্থক্য কী?
উত্তর: Authentication প্রমাণ করে তুমি কে। যেমন email আর password দিয়ে login। Authorization ঠিক করে login করার পরে তুমি কী করতে পারবে। যেমন আমার project-এ login করা user শুধু নিজের application দেখতে পারে। কেউ অন্যের application-এর id দিয়ে request করলে server 404 দেয়।

প্রশ্ন: JWT কী? কীভাবে কাজ করে?
উত্তর: JWT হলো তিন ভাগের একটা token: header, payload, signature। payload-এ user-এর তথ্য থাকে, যেমন আমার project-এ `sub` (user-এর UUID), `email` আর `exp` (কখন expired হবে)। server একটা secret key দিয়ে token sign করে। client প্রতিটা request-এ `Authorization: Bearer <token>` header পাঠায়, server signature verify করে user চেনে। payload base64-এ লেখা, encrypt করা না, তাই যে কেউ পড়তে পারে। কিন্তু বদলালে signature মেলে না। তাই JWT-তে কখনো password বা গোপন তথ্য রাখি না। server-কে কোনো session রেকর্ড রাখতে হয় না, এজন্য একে stateless বলে।

প্রশ্ন: Token কোথায় রাখবে: localStorage নাকি cookie?
উত্তর:

- localStorage সহজ, আমার project-এ এটাই ব্যবহার করেছি। কিন্তু XSS হলে, মানে অন্য কারো JavaScript page-এ চললে, সেটা localStorage পড়ে token চুরি করতে পারে।
- httpOnly cookie JavaScript পড়তে পারে না, তাই XSS থেকে বেশি নিরাপদ। কিন্তু cookie browser নিজে থেকে পাঠায়, তাই CSRF-এর ঝুঁকি আসে, মানে অন্য site আমার browser দিয়ে request পাঠাতে পারে। এটা ঠেকাতে cookie-তে `SameSite=Lax/Strict` দিই, অথবা আলাদা CSRF token ব্যবহার করি।

- বেশি security চাইলে সাধারণ pattern হলো: অল্প মেয়াদের access token memory-তে, আর দীর্ঘ মেয়াদের refresh token httpOnly cookie-তে।

প্রশ্ন: Refresh করলে login ধরে রাখো কীভাবে?
উত্তর: token persistent storage-এ (localStorage বা cookie) রাখি। app শুরুতে `restoreSession` thunk dispatch করি। token থাকলে সেটা `/auth/me` call করে user আনে। dispatch শুধু কাজ শুরু করে, render অপেক্ষা করে না। তাই `isSessionChecked` নামে একটা flag রাখি, check শেষ না হওয়া পর্যন্ত এটা false, আর তখন protected page-এ spinner দেখাই। না রাখলে refresh-এর মুহূর্তে user null দেখে app ভুল করে login page-এ পাঠিয়ে দিত।

প্রশ্ন: Token expired হলে কী করো?
উত্তর: server 401 দিলে, আর request-এ token থাকলে, token মুছে `window.location.assign` দিয়ে login page-এ পাঠাই। পুরো reload হয় বলে পুরনো সব state মুছে যায়। token না থাকা অবস্থায় 401 মানে সাধারণত ভুল password, তখন redirect না করে শুধু error দেখাই। আরও ভালো উপায় হলো refresh token: access token expired হলে refresh token দিয়ে চুপচাপ নতুন access token এনে request আবার পাঠানো, যাতে user-কে আবার login করতে না হয়।

প্রশ্ন: Logout-এ কোনো API call নেই, তবুও thunk কেন?
উত্তর: logout-এ `localStorage` থেকে token মুছতে হয়, এটা side effect। reducer pure হতে হয়, তাই সেখানে রাখা যায় না। thunk-এ রাখলে `createAsyncThunk` sync function-এর জন্যও `logoutUser.fulfilled` action পাঠায়। সেই একটা action-এ auth slice user null করে, আর applications slice নিজের state reset করে, যাতে পরের user আগের জনের data না দেখে।

প্রশ্ন: Frontend route guard কি security দেয়?
উত্তর: না, শুধু user experience (UX) ভালো করে। যে কেউ browser-এর DevTools দিয়ে code বদলে বা সরাসরি API call করে UI এড়াতে পারে। data-র security server-এ নিশ্চিত করতে হয়: প্রতিটা request-এ token verify করা, আর শুধু ওই user-এর data ফেরত দেওয়া।

অধ্যায় ১৩: পুরো feature একসাথে — Applications

আগের সব অধ্যায়ের জ্ঞান এখন একসাথে কাজে লাগবে। এই অধ্যায়ে "Applications" feature-এর প্রতিটা page শুরু থেকে শেষ পর্যন্ত দেখব।

১৩.১ Feature-এর file map

```
features/applications/
├── applications.types.ts      ← JobApplication, ApplicationFilters,
ApplicationFormValues, ApplicationsState
├── applications.constants.ts  ← ApplyMedium, SortField, SortOrder, StatusFilter enum;
DEFAULT_FILTERS; EMPTY_FORM_VALUES
├── applications.utils.ts      ← filtersToQuery, toFormValues, toPayload, validateForm,
formatSalary, isDeadlinePassed
├── applications.api.ts        ← HTTP call
├── applications.thunks.ts     ← loadApplications, loadApplication, createApplication,
updateApplication, deleteApplication
├── applications.slice.ts      ← state + reducers
├── ApplicationsPage.tsx       ← list page
├── ApplicationFilters.tsx     ← search, status, medium, sort, per page
├── ApplicationsTable.tsx      ← table
├── StatusBadge.tsx            ← Applied / Not applied badge
├── ApplicationFormPage.tsx    ← new/edit page
├── ApplicationForm.tsx        ← আসল form
└── ApplicationDetailsPage.tsx ← details page
```

মূল data-র shape

`applications.types.ts`:

```
export interface JobApplication
{
    id: string;
    jobPost: string;
    jobPostLink: string;
    deadline: string | null;
    applyMedium: ApplyMedium;
    applyLink: string | null;
    applyMail: string | null;
    cvLink: string | null;
    salaryMin: number | null;
    salaryMax: number | null;
    salaryCurrency: string | null;
    isApplied: boolean;
    appliedAt: string | null;
    createdAt: string;
    updatedAt: string;
}
```

তারিখগুলো `string`, কারণ JSON-এ Date type নেই। server তারিখ পাঠায় এরকম string হিসেবে: `"2026-10-11T09:21:50.103Z"`।

এটাকে বলে **ISO format**: `YYYY-MM-DDTHH:mm:ss`, মানে বছর-মাস-দিন, তারপর `T`, তারপর ঘণ্টা:মিনিট:সেকেন্ড (এখানে `.103` হলো millisecond)। শেষের `Z` মানে সময়টা **UTC**-তে, অর্থাৎ পৃথিবীর একটা common reference time-এ। বাংলাদেশের সময় UTC+6, মানে UTC-র চেয়ে ৬ ঘণ্টা এগিয়ে। তাই `09:21Z` বাংলাদেশে বিকেল ৩:২১ (15:21)।

১৩.২ List page: `ApplicationsPage`

Page-টা কয়েকটা component জুড়ে বানানো (সংক্ষেপে):

```
return (
    <div className="space-y-6">
        <div className="flex items-center justify-between">
            <h1 className="text-2xl font-bold text-slate-800">Applications</h1>
            <LinkButton to={AppRoutes.ApplicationCreate}>+ New application</LinkButton>
        </div>

        <ApplicationFilters />
        <ErrorMessage message={listError} />

        {listStatus === RequestStatus.Loading && items.length === 0 && <Spinner />}
        {listStatus === RequestStatus.Succeeded && items.length === 0 && <p>No
applications found.</p>}
        {items.length > 0 && <ApplicationsTable applications={items} onToggleApplied=
{...} onDelete={...} />}
        {meta && items.length > 0 && <Pagination meta={meta} onPageChange={...} />}
    </div>
);
```

পরিস্থিতি	কী দেখায়
প্রথমবার load হচ্ছে	Spinner
ব্যর্থ	লাল error box
সফল, কিন্তু কিছু নেই	"No applications found."
data আছে	table + pagination

Spinner-এর সাথে `items.length === 0` কেন? filter বদলালে list আবার load হয়। তখন spinner না দেখিয়ে পুরনো table দেখাতে থাকি, যাতে screen বারবার লাফিয়ে না ওঠে। শুধু প্রথমবার, যখন দেখানোর মতো কিছু নেই, তখন spinner।

Filter (অধ্যায় ৮ আর ১০-এ বিস্তারিত)

- **Search:** local state + 800ms debounce, তারপর `filtersChanged({ search })`।
- **Status, Apply via, Sort by, Per page:** সাথে সাথে `filtersChanged(...)`।
- **↑/↓ button:** sort order উল্টে দেয়।
- **Reset:** `filtersReset()`।

`<select>` থেকে আসা value সবসময় string। সেটাকে enum-এ convert করে `parseEnum`, `frontend/src/shared/utils.ts`-এ:

```
export function parseEnum<T extends string>(allowed: readonly T[], value: string,
fallback: T): T
{
    return allowed.find((item) => item === value) ?? fallback;
}
```

`allowed` হলো allowed value-গুলোর list। `value` সেই list-এ থাকলে সেটাই ফেরত আসে, না থাকলে `fallback`। `Object.values(StatusFilter)` enum-এর সব value-র array দেয়: `['all', 'applied', 'notApplied', 'upcoming', 'expired']`। যেমন `parseEnum(Object.values(StatusFilter), 'applied', StatusFilter.All)` দেয় `StatusFilter.Applied`, আর `'hello'` দিলে দেয় `StatusFilter.All`। এভাবে `as` দিয়ে জোর করে type বদলাতে হয় না।

Table: `ApplicationsTable`

Table নিজে কোনো data আনে না, কিছু বদলায়ও না। সে শুধু দেখায়, আর কোনো **event** (যেমন click) হলে callback দিয়ে parent-কে জানায়:

```
interface ApplicationsTableProps
{
    applications: JobApplication[];
    onToggleApplied: (application: JobApplication) => void;
    onDelete: (application: JobApplication) => void;
}
```

এমন component-কে বলে **presentational component**: শুধু props থেকে UI দেখায়। আর যে component data আনে ও সিদ্ধান্ত নেয় (এখানে ApplicationsPage), তাকে বলে **container component**।

Table-এর প্রতিটা row-তে থাকে:

- job post-এর নাম, যেটা click করলে details page-এ যায় (`<Link>`)
- deadline। deadline পেরিয়ে গেছে আর এখনো apply করা হয়নি, এমন হলে লাল (`isDeadlinePassed`)
- apply medium, আর salary (`formatSalary` দেয় যেমন "50,000 – 90,000 BDT")
- status badge, যেটা click করলে Applied ↔ Not applied বদলায়
- Edit link আর Delete button

Status বদলানো আর delete

```
function handleToggleApplied(application: JobApplication): void
{
    void dispatch(updateApplication({ id: application.id, payload: { isApplied:
!application.isApplied } }));
}

async function handleDelete(application: JobApplication): Promise<void>
{
    if (!window.confirm(`Delete "${application.jobPost}"?`))
    {
        return;
    }
    await dispatch(deleteApplication(application.id));
    if (items.length === 1 && filters.page > 1)
    {
        dispatch(filtersChanged({ page: filters.page - 1 }));
    }
    else
    {
        void dispatch(loadApplications(filters));
    }
}
```

- `window.confirm`: browser-এর নিজস্ব "OK / Cancel" box। user Cancel চাপলে `false` আসে, আর function সেখানেই থেমে যায়।
- `handleToggleApplied`: slice `updateApplication.fulfilled`-এ row-টা server-এর নতুন data দিয়ে বদলে দেয়, তাই badge সাথে সাথে বদলায় (অধ্যায় ১১)।
- delete সফল হলে slice নিজেই `items` থেকে row-টা সরিয়ে দেয় (অধ্যায় ১১)। তবুও list আবার load করা হয়। কেন? ধরো page-এ ১০টা item ছিল, একটা মুছলে ৯টা থাকে। আবার load করলে server পরের page-এর প্রথম item এনে page-টা আবার ১০টা দিয়ে ভরে দেয়।
- `items.length === 1` কী বোঝায়? handler-এর ভেতরের `items` হলো সেই value, যেটা এই render-এর সময় ছিল। function-টা তৈরি হওয়ার সময়ের `items` মনে রেখেছে, এটাই closure (অধ্যায় ২)। `await`-এর পরে slice বদলালেও এই `items` বদলায় না। তাই `items.length === 1` মানে "যেটা মুছলাম, সেটা এই page-এর শেষ item ছিল"।
- তখন page খালি হয়ে যেত, তাই আগের page-এ যাওয়া হয়। page বদলালে `filters` বদলায়, আর ApplicationsPage-এর `useEffect` (যার dependency `filters`) নিজেই list আবার load করে। তাই এই ক্ষেত্রে আলাদা করে `loadApplications` call করতে হয় না।

Async function কেন সরাসরি prop হিসেবে দেওয়া হয়নি?

```
onDelete={(application): void =>
{
    void handleDelete(application);
}}
```

`onDelete`-এর type `(application) => void`, কিন্তু `handleDelete` Promise return করে। সরাসরি দিলে ESLint সতর্ক করে (`no-misused-promises` rule; এটা ৫.৪-এ বলা `strictTypeChecked` rule-set-এর অংশ), কারণ কেউ সেই Promise-এর error ধরবে না। তাই একটা ছোট wrapper বানিয়ে `void` দিয়ে স্পষ্ট করা হলো যে Promise-টা ইচ্ছা করেই ছেড়ে দেওয়া হচ্ছে।

Pagination

Server মোট কতগুলো application আছে সেটা বলে না। শুধু বলে `hasNext` (পরের page আছে কিনা) আর `hasPrev` (আগের page আছে কিনা)। তাই Pagination component শুধু Previous/Next button আর "Page 2" দেখায়।

মোট সংখ্যা জানতে database-এ `COUNT(*)` চালাতে হতো। এটা একটা SQL query, যেটা table-এর সব row গুনে দেয়। বড় table-এ এটা ধীর, কারণ database-কে প্রতিটা row ছুঁয়ে দেখতে হয়।

আমাদের backend (`backend/jat/src/common/utils/pagination.util.ts`) একটা চালাকি করে। ১০টা চাইলে database-কে বলে ১১টা দাও (`take: query.limit + 1`)। যদি ১১টা আসে, তাহলে পরের page আছে, তাই `hasNext = true`, আর বাড়তি একটা বাদ দিয়ে ১০টা পাঠায়। ১১টার কম এলে `hasNext = false`। `hasPrev` আরও সহজ: page ১-এর বেশি হলেই `true`।

১৩.৩ নতুন আর edit: `ApplicationFormPage` + `ApplicationForm`

দুই component, দুই দায়িত্ব

Component	দায়িত্ব
`ApplicationFormPage`	URL থেকে id নেওয়া, edit হলে data আনা, save করা, তারপর অন্য page-এ যাওয়া
`ApplicationForm`	শুধু form: input, validation, তারপর payload বানিয়ে `onSubmit`-এ দেওয়া

`ApplicationForm` জানেই না যে server, Redux বা router আছে। তাই এটা বোঝা সহজ, আর **testable**: test-এ শুধু props দিয়ে চালিয়ে দেখা যায়।

Page-এর মূল অংশ (সংক্ষেপে)

```
const { id } = useParams<{ id: string; }>();

useEffect((): void =>
{
    dispatch(saveErrorCleared());
    if (id)
    {
        void dispatch(loadApplication(id));
    }
}, [dispatch, id]);

if (id && selected?.id !== id)
{
    return selectedStatus === RequestStatus.Failed ? <ErrorMessage message=
{selectedError} /> : <Spinner />;
}

return (
    <div className="mx-auto max-w-3xl space-y-6">
        <h1 className="text-2xl font-bold text-slate-800">{id ? 'Edit application' :
'New application'}</h1>
        <ErrorMessage message={saveError} />
        <ApplicationForm
            key={id ?? 'new'}
            initialValues={id && selected ? toFormValues(selected) : EMPTY_FORM_VALUES}
            submitLabel={id ? 'Save changes' : 'Create application'}
            isSaving={saveStatus === RequestStatus.Loading}
            onSubmit={(payload): void =>
            {
                void handleSubmit(payload);
            }}
        />
    </div>
);
```

- URL-এ id থাকলে (`/applications/:id/edit`) এটা edit, না থাকলে (`/applications/new`) নতুন। heading-ও সেই অনুযায়ী "Edit application" বা "New application" দেখায়।
- Edit হলে application-টা না আসা পর্যন্ত spinner দেখায়, কারণ form-এর **initial value** তখনো জানা নেই।

- `<ErrorMessage message={saveError} />`: save ব্যর্থ হলে form-এর উপরে লাল box-এ server-এর message দেখায়।
- `initialValues`: নতুন হলে খালি value (`EMPTY_FORM_VALUES`), edit হলে server-এর data থেকে বানানো (`toFormValues`)।
- `key`: অন্য application-এ বা নতুন form-এ গেলে form একদম নতুন করে শুরু হয় (অধ্যায় ৭-এর key reset)। `useState(initialValues)` initial value শুধু প্রথম render-এ নেয়, পরে prop বদলালেও নেয় না। তাই key বদলে নতুন component বানানোই সবচেয়ে সহজ সমাধান।

Form-এর ভেতরে conditional field (সংক্ষেপে)

```
{values.applyMedium === ApplyMedium.Link
    ? <Field label="Apply link" ...><Input ... /></Field>
    : <Field label="Application mail" ...><RichTextEditor ... /></Field>}

{values.isApplied && (
    <Field label="Applied at (leave empty for now)" htmlFor="appliedAt">
        <Input type="datetime-local" ... />
    </Field>
)}
```

- **Conditional field** মানে এমন input field, যেটা কোনো শর্ত মিললে তবেই দেখায়।
- "Apply by link" বেছে নিলে link-এর input field আসে, আর "Apply by mail" বেছে নিলে mail লেখার editor।
- "I have applied" tick দিলে তবেই apply-এর তারিখের field দেখায়।
- Label-এর "(leave empty for now)" মানে: এই field খালি রাখলে server এখনকার সময় বসিয়ে দেবে। তাই আজই apply করলে তারিখ লেখার দরকার নেই।

তারিখ: `datetime-local` input

`<input type="datetime-local">` value দেয় আর চায় এই format-এ: `"2026-10-11T15:30"`। এখানে কোনো `Z` নেই, মানে এটা user-এর computer-এর **local time**। server চায় ISO format, UTC-তে: `"2026-10-11T09:30:00.000Z"`। বাংলাদেশে (UTC+6) local 15:30 মানে UTC 09:30।

দুই দিকের **conversion function** আছে `shared/utils.ts`-এ:

```
export function fromDateTimeInputValue(value: string): string | null
{
    return value ? new Date(value).toISOString() : null;
}
```

`new Date("2026-10-11T15:30")` string-টাকে local time হিসেবে পড়ে, আর `toISOString()` সেটাকে UTC-তে লেখে। বাংলাদেশের computer-এ ফল হয় `"2026-10-11T09:30:00.000Z"`। খালি string দিলে `null` আসে।

`toDateTimeInputValue` ঠিক উল্টো কাজ করে। ISO string থেকে local বছর, মাস, দিন, ঘণ্টা, মিনিট বের করে `"2026-10-11T15:30"` বানায়, যাতে edit form-এ সঠিক সময় দেখায়।

Save করা

```
const saved = id
    ? await dispatch(updateApplication({ id, payload })).unwrap()
    : await dispatch(createApplication(payload)).unwrap();
void navigate(buildPath(AppRoutes.ApplicationDetails, { id: saved.id }));
```

সফল হলে নতুন বা update হওয়া application-এর details page-এ যাওয়া হয়। ব্যর্থ হলে `unwrap()` throw করে, তাই `navigate` আর চলে না (পুরো function একটা try/catch-এ wrap করা)। `catch`-এ শুধু `return`; error message দেখানোর কাজ slice-এর `saveError`-এর। তখন slice `saveError` set করে, আর form-এর উপরে message দেখায়। যেমন development-এ backend বন্ধ থাকলে Vite proxy `502` দেয়, তাই দেখাবে "Bad Gateway" (অধ্যায় ৩.৫-এর `request` function-এর শেষ ধাপ)। (form-এর নিজের validation, মানে `validateForm`, আগেই ভুল ধরে ফেলে, তাই server-এর validation error এখান থেকে সাধারণত আসে না।)

Business rule-গুলো কোথায়?

Business rule মানে app-এর নিজস্ব নিয়ম, যেমন "link দিয়ে apply হলে link লাগবেই"। এগুলো তিন জায়গায় আছে, আর প্রত্যেকটার কাজ আলাদা।

(ক) `validateForm`: সাথে সাথে feedback। submit-এর সময় form-এর ভেতরেই check হয়, server-এ যাওয়ার আগে। ভুল থাকলে field-এর নিচে লাল লেখা আসে, request যায় না। যেমন:

- job post খালি থাকলে "Job post is required"
- link `http://` বা `https://` দিয়ে না শুরু হলে "Enter a valid link..."
- mail medium বেছে mail খালি রাখলে "Write the application mail"
- `salaryMin` > `salaryMax` হলে "Maximum must be greater than minimum"

(খ) `toPayload`: পাঠানোর আগে data পরিষ্কার করা। এটা কিছু আটকায় না, শুধু form-এর value-কে server-এর পছন্দমতো আকারে সাজায়। যেমন:

- medium LINK হলে `applyMail: null` যায়, আর MAIL হলে `applyLink: null`। ফলে অন্য medium বেছে আগে কিছু লিখে থাকলেও সেটা save হয় না।
- "I have applied" tick না থাকলে `appliedAt: null` যায়।
- লেখার আগে-পরের ফাঁকা জায়গা `trim()` দিয়ে কাটা হয়, আর salary string থেকে number হয়।

(গ) Server: আসল validation। frontend-এর check যে কেউ এড়াতে পারে, তাই server (`backend/jat/src/application/services/application.service.ts`) সব নিয়ম আবার check করে। যেমন কেউ সরাসরি API-তে `applyMedium: "LINK"` পাঠাল কিন্তু `applyLink` দিল না, তখন server **400 Bad Request** দেয়: "applyLink is required when applyMedium is LINK"।

একটা জিনিস validation না, বরং **default**: `isApplied` true কিন্তু `appliedAt` নেই, এমন হলে server ভুল ধরে না। সে নিজেই "এখন"-এর সময় বসিয়ে দেয়। form-এর "(leave empty for now)" label এই default-এর উপরেই ভরসা করে।

১৩.৪ Rich text: Tiptap editor

চাকরির application mail-এ bold, italic, list থাকতে পারে। সাধারণ `<textarea>` এসব পারে না। তাই **Tiptap** library ব্যবহার করা হয়েছে।

`frontend/src/shared/components/RichTextEditor.tsx` (সংক্ষেপে):

```
interface ToolbarAction
{
    label: string;
    run: (editor: Editor) => boolean;
}

const TOOLBAR: ToolbarAction[] = [
    { label: 'B', run: (editor) => editor.chain().focus().toggleBold().run() },
    { label: 'I', run: (editor) => editor.chain().focus().toggleItalic().run() },
    { label: 'U', run: (editor) => editor.chain().focus().toggleUnderline().run() },
    { label: '• List', run: (editor) => editor.chain().focus().toggleBulletList().run()
},
    { label: '1. List', run: (editor) =>
editor.chain().focus().toggleOrderedList().run() },
];

export function RichTextEditor({ id, value, onChange }: RichTextEditorProps):
JSX.Element
{
    const editor = useEditor({
        extensions: [StarterKit],
        content: value,
        onUpdate: ({ editor: current }): void => onChange(current.getHTML()),
        editorProps: { attributes: { id, class: 'rich-text min-h-40 px-3 py-2 text-sm' }
},
    });

    return (
        <div className="overflow-hidden rounded-md border border-slate-300 bg-white">
            <div className="flex gap-1 border-b border-slate-200 bg-slate-50 p-1">
                {TOOLBAR.map((action) => (
                    <button
                        key={action.label}
                        type="button"
                        onClick={(): void =>
                        {
                            action.run(editor);
                        }}
                    >
                        {action.label}
                    </button>
                ))}
            </div>
```

```
            <EditorContent editor={editor} />
        </div>
    );
}
```

- `useEditor` : Tiptap-এর নিজের hook (একটা **custom hook**, মানে `use` দিয়ে শুরু হওয়া এমন function যেটা ভেতরে React-এর hook ব্যবহার করে)। এটা একটা editor বানিয়ে দেয়।
- `StarterKit` : bold, italic, underline, list-এর মতো সাধারণ feature-এর একটা package।
- `onUpdate` : editor-এ প্রতিটা বদলে লেখাটা HTML string হিসেবে parent-কে দেয়। যেমন "Hello" লিখে "HR" bold করলে parent পায় `<p>Hello <strong>HR</strong></p>` । (Tiptap bold-এর জন্য `<b>` না, `<strong>` ব্যবহার করে।)
- `({ editor: current })` মানে object-এর `editor` field বের করে `current` নামে রাখো (destructuring-এর সময় rename)। বাইরের `editor` variable-এর সাথে নাম যেন না মেলে, তাই। এখানে `: current` কোনো type না।
- `EditorContent` : editor-এর লেখার জায়গাটা screen-এ বসায়।
- `TOOLBAR` **array থেকে** `map` : পাঁচটা button আলাদা করে না লিখে data থেকে বানানো। নতুন button লাগলে array-তে একটা object যোগ করলেই হয়।

Method chaining: `editor.chain().focus().toggleBold().run()`

এখানে একটার পর একটা method dot দিয়ে জোড়া, একে বলে **method chaining**। এটা কাজ করে কারণ প্রতিটা method একটা "chain" object return করে, আর পরের method সেই object-এর উপর call হয়।

ধাপ	কী করে
`editor.chain()`	একটা খালি command-এর list শুরু করে
`.focus()`	list-এ যোগ করে: "editor-এ cursor ফেরত দাও" (button click করলে focus editor থেকে সরে যায়)
`.toggleBold()`	list-এ যোগ করে: "selected লেখা bold থাকলে বন্ধ, না থাকলে চালু"
`.run()`	এতক্ষণ জমানো সব command একসাথে চালায়

`run()` না লিখলে কিছুই ঘটে না, শুধু list তৈরি হয়ে পড়ে থাকে। ফলাফল: B button চাপলে selected লেখা bold হয়, আর cursor editor-এই থেকে যায়।

Tailwind browser-এর default list style (bullet, number) মুছে দেয়। তাই `index.css` -এ `.rich-text ul` -এ bullet আর `.rich-text ol` -এ number ফেরত আনা হয়েছে।

খালি mail চেনা: editor খালি হলেও `<p></p>` দেয়, যেটা খালি string না। তাই validation-এ HTML থেকে শুধু লেখা বের করে দেখা হয়: `htmlToText(values.applyMail)`। `<p></p>` থেকে আসে `''`, তাই error দেখায়।

১৩.৫ Details page: `ApplicationDetailsPage`

(সংক্ষেপে)

```
<dl className="divide-y divide-slate-100 rounded-xl bg-white px-6 shadow">
    <DetailRow label="Job post link">
        <ExternalLink href={selected.jobPostLink} />
    </DetailRow>
    <DetailRow label="Deadline">{formatDateTime(selected.deadline)}</DetailRow>
    <DetailRow label="Salary">{formatSalary(selected)}</DetailRow>
    ...
</dl>
```

- `DetailRow`, `ExternalLink`: একই file-এর ভেতরের ছোট component, export করা হয়নি। শুধু এই page-এ লাগে, তাই আলাদা file বানানো হয়নি।
- `<dl>`, `<dt>`, `<dd>`: "label আর তার value"-র list দেখানোর সঠিক HTML tag (description list)। `<dt>`-তে label, `<dd>`-তে value।

HTML দেখানো: `dangerouslySetInnerHTML`

React সাধারণত `{ }`-এর ভেতরের string-কে **escape** করে, মানে HTML tag-গুলোকে সাধারণ লেখা হিসেবে দেখায়। যেমন:

```
<div>{'<b>Hi</b>'}</div>
```

screen-এ দেখা যায় হুবহু `<b>Hi</b>` লেখাটা, bold "Hi" না। এটাই XSS থেকে বাঁচায় (অধ্যায় ১২): কেউ `<script>` লিখলেও সেটা লেখা হিসেবে দেখায়, চলে না।

কিন্তু mail-এর bold, list তো দেখাতে হবে। আসল HTML render করতে এই বিশেষ prop লাগে:

```
<div className="rich-text" dangerouslySetInnerHTML={{ __html: selected.applyMail }} />
```

এখানে `applyMail` যদি `<p>Hello <strong>HR</strong></p>` হয়, তাহলে screen-এ দেখায় "Hello **HR**"। নামটা ইচ্ছা করেই ভয় দেখানো, কারণ কেউ এখানে বিপজ্জনক HTML ঢোকাতে পারলে সেটা সত্যিই চলবে।

আমরা নিরাপদ কেন? backend save করার আগেই mail-কে sanitize করে (`backend/jat/src/common/utils/html.util.ts`, `sanitize-html` library)। শুধু নিরাপদ tag রাখে, যেমন `p`, `b`, `strong`, `i`, `em`, `u`, `ul`, `ol`, `li`, `a`। `<script>`, `<iframe>`, `style` সহ বাকি সব মুছে দেয়। ফলে database-এ যা আছে, সেটা দেখানো নিরাপদ।

Action button

- **Mark as applied / not applied**: `updateApplication` dispatch হয়। slice `selected` update করে, তাই page সাথে সাথে বদলায়।
- **Edit**: `LinkButton`, যেটা edit page-এ নিয়ে যায়।
- **Delete**: confirm, তারপর `deleteApplication`। সফল হলে list page-এ ফেরত যায়।

সারসংক্ষেপ

- List page: filter (Redux-এ) বদলালে effect চলে, effect thunk dispatch করে, তারপর table + pagination দেখায়। loading, error, empty প্রতিটা state আলাদা দেখানো হয়।
- Delete-এর পরে list আবার load হয়, যাতে page পরের item দিয়ে ভরে যায়। page-এর শেষ item মুছলে আগের page-এ যাওয়া হয়।
- Server `COUNT(*)` চালায় না, `limit + 1` row এনে `hasNext` বোঝে।
- Table presentational: শুধু দেখায় আর callback দিয়ে event জানায়।
- একই form page নতুন আর edit দুই কাজ করে। `key` দিয়ে form নতুন করে শুরু হয়।
- `validateForm` সাথে সাথে feedback দেয়, `toPayload` data পরিষ্কার করে, আর server আসল validation করে।
- form-এর local time আর server-এর ISO/UTC string-এর মাঝে conversion function আছে।
- Tiptap দিয়ে rich text লেখা হয়, `dangerouslySetInnerHTML` দিয়ে দেখানো হয়, আর server আগেই sanitize করে রাখে।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: Presentational আর Container component কী?
উত্তর: Presentational component শুধু props থেকে UI দেখায়, আর কোনো event হলে callback দিয়ে parent-কে জানায়। যেমন আমার ApplicationsTable: সে `applications`, `onToggleApplied`, `onDelete` props নেয়, নিজে কোনো data আনে না। Container component data আনে, state সামলায়, আর সিদ্ধান্ত নেয়। যেমন ApplicationsPage, যে Redux থেকে data পড়ে আর thunk dispatch করে। এতে UI অংশ **reusable** হয় (অন্য data দিয়ে অন্য জায়গায় ব্যবহার করা যায়) আর testable হয়।

প্রশ্ন: dangerouslySetInnerHTML কী? কখন নিরাপদ?
উত্তর: React সাধারণত string escape করে, তাই `{'<b>Hi</b>'}` screen-এ লেখা হিসেবে `<b>Hi</b>` দেখায়। আসল HTML render করতে চাইলে `dangerouslySetInnerHTML={{ __html: html }}` দিতে হয়। এতে XSS-এর ঝুঁকি আছে, কারণ HTML-এ বিপজ্জনক কিছু থাকলে সেটা চলবে। তাই শুধু বিশ্বস্ত বা sanitize করা HTML দিতে হয়। আমার project-এ server save করার আগে `sanitize-html` দিয়ে শুধু নিরাপদ tag রাখে। client-এ করতে চাইলে DOMPurify library আছে।

প্রশ্ন: Form-এ initial value বদলালে নতুন value দেখায় না কেন? সমাধান কী?
উত্তর: `useState(initialValue)` initial value শুধু প্রথম render-এ নেয়। পরে prop বদলালেও state বদলায় না। সমাধান হলো component-কে নতুন `key` দেওয়া। key বদলালে React পুরনো component unmount করে নতুনটা mount করে, তাই state নতুন initial value দিয়ে শুরু হয়। আমার project-এ `key={id ?? 'new'}` দিয়েছি। আরেকটা উপায় হলো data আসার পরে তবেই form render করা, সেটাও করেছি (data না আসা পর্যন্ত spinner)।

প্রশ্ন: Pagination কত রকম হয়?
উত্তর:

- **Offset/page-based** (`?page=3&limit=10`): সহজ, আর যেকোনো page-এ লাফ দেওয়া যায়। কিন্তু বড় data-য় ধীর, কারণ page ৩ চাইলে database-কে প্রথম ২০টা row পেরিয়ে যেতে হয়, ফেলে দেওয়ার জন্য হলেও। page যত পিছনে, তত বেশি row পেরোতে হয়।
- **Cursor-based** (`?after=<id>`): মানে "এই id-এর পরেরগুলো দাও"। database index দিয়ে সরাসরি ওই জায়গায় চলে যায়, তাই বড় data-তেও দ্রুত। মাঝে নতুন row যোগ হলেও item বাদ পড়ে না বা দুবার আসে না, তাই **stable**। কিন্তু সরাসরি page ৫-এ যাওয়া যায় না।
- মোট সংখ্যা জানতে `COUNT(*)` লাগে, যেটা বড় table-এ **expensive**। আমার project-এ server `limit + 1` row আনে: ১১টা এলে `hasNext = true`, আর ১০টা পাঠায়। তাই শুধু Previous/Next দেখাই।

প্রশ্ন: একই component দিয়ে create আর edit কেন?
উত্তর: দুটোর field আর validation একই, শুধু initial value আর save-এর API আলাদা। তাই ApplicationForm শুধু `initialValues`, `submitLabel`, `onSubmit` props নেয়, আর ApplicationFormPage ঠিক করে কোনটা দেবে। একটা component রাখলে code দ্বিগুণ হয় না, আর এক জায়গায় বদলালে দুটোই ঠিক হয়।

প্রশ্ন: Frontend validation থাকলে server validation কেন লাগে?
উত্তর: frontend validation শুধু user experience (UX)-এর জন্য, যাতে ভুল সাথে সাথে দেখা যায়। কিন্তু যে কেউ browser বাদ দিয়ে সরাসরি API-তে request পাঠাতে পারে। তাই server সব নিয়ম আবার check করে। যেমন আমার project-এ `applyMedium: "LINK"` দিয়ে `applyLink` না পাঠালে server 400 দেয়।

অধ্যায় ১৪: Project structure আর নতুন feature যোগ করার নিয়ম

Code কোথায় রাখা হয়েছে আর কেন, সেটা বুঝলে নিজে নতুন কিছু যোগ করতে পারবে। interview-তে "তোমার project-এর architecture বলো" প্রশ্নের উত্তরও দিতে পারবে। **Architecture** মানে project-এর বড় ছবি: কোন অংশ কোথায় থাকে, আর অংশগুলো একে অপরের সাথে কীভাবে কথা বলে।

১৪.১ পুরো folder structure

```
frontend/src/
├── main.tsx                ← app শুরু: store, restoreSession, Provider, Router
├── index.css               ← Tailwind + rich-text style
│
├── app/                    ← পুরো app জোড়া লাগানোর জায়গা
│   ├── store.ts            ← Redux store, RootState, AppDispatch
│   ├── redux.ts            ← useAppDispatch, useAppSelector, createAppAsyncThunk
│   ├── AppRouter.tsx       ← সব route
│   ├── AppLayout.tsx       ← navbar + Outlet (login করা page-গুলোর জন্য)
│   ├── AuthLayout.tsx      ← মাঝখানে card + Outlet (login/register)
│   └── NotFoundPage.tsx    ← 404
│
├── features/               ← প্রতিটা feature-এর নিজের folder
│   ├── auth/               ← login, register, session
│   └── applications/       ← job application-এর সবকিছু
│
└── shared/                 ← যেকোনো feature ব্যবহার করতে পারে
    ├── api.ts              ← fetch client, ApiError, token helper
    ├── constants.ts        ← ApiEndpoints, AppRoutes enum
    ├── types.ts            ← RequestStatus, Page, ApiResponse
    ├── utils.ts            ← buildPath, formatDate, isValidUrl, parseEnum...
    └── components/         ← Button, Field, Feedback, Pagination, RichTextEditor
```

এখানে **feature** মানে app-এর এমন একটা অংশ, যেটা user-এর কাছে একটা আলাদা কাজ: যেমন "login করা" বা "application সামলানো"। একে **domain**-ও বলে।

১৪.২ তিনটা layer, তিনটা নিয়ম

Layer মানে code-এর একটা ভাগ, যার নির্দিষ্ট দায়িত্ব আছে, আর যে নির্দিষ্ট কিছু ভাগকেই import করতে পারে।

Folder	কী থাকে	কাকে import করতে পারে
`shared/`	কোনো feature-এর কথা জানে না এমন জিনিস: button, তারিখ format, HTTP client	শুধু `shared/`
`features/`	একটা feature-এর সব: type, API, state, page	`shared/`, `app/redux.ts`, দরকারে অন্য feature-এর action
`app/`	সব জোড়া লাগানো: store, router, layout	সবাইকে

"দরকারে অন্য feature-এর action"-এর উদাহরণ: applications slice auth-এর `logoutUser.fulfilled` শোনে, যাতে logout হলে নিজের data মুছে ফেলে (অধ্যায় ১২)।

`shared/` কখনো `features/` import করবে না। কেউ `Button`-এর ভেতরে application-এর কথা লিখলে সেটা আর "shared" থাকে না, অন্য feature সেটা আর ব্যবহার করতে পারে না।

১৪.৩ Feature folder কেন? (type-based না সাজিয়ে)

একটা উপায় হলো file-এর ধরন অনুযায়ী সাজানো। একে বলে **type-based** structure:

```
components/  pages/  slices/  thunks/  api/  types/
```

তাহলে "applications" feature-এ একটা ছোট কাজ করতেও ছয়টা folder-এ ঘুরতে হতো।

আমরা সাজিয়েছি feature অনুযায়ী, একে বলে **feature-based** structure:

```
features/applications/   ← application-এর সব কিছু এক জায়গায়
features/auth/           ← login-এর সব কিছু এক জায়গায়
```

- একটা feature-এর কাজ এক folder-এই।
- feature মুছতে চাইলে পুরো folder মুছলেই হয়।
- বড় team-এ আলাদা মানুষ আলাদা feature-এ কাজ করে, একই file নিয়ে কারো সাথে ধাক্কা লাগে না।

১৪.৪ File-এর নামকরণ

ধরন	নাম	উদাহরণ
Component বা page	PascalCase + `.tsx`	`ApplicationsPage.tsx`, `StatusBadge.tsx`
Feature-এর logic	`<feature>.<কাজ>.ts`	`applications.api.ts`, `auth.slice.ts`
Shared logic	ছোট হাতের `.ts`	`utils.ts`, `api.ts`

File-এর নাম দেখেই বোঝা যায় ভেতরে কী আছে।

১৪.৫ এই project-এর code-এর নিয়মগুলো (code conventions)

নিয়ম	কেন
প্রতিটা function-এর return type লেখা	পড়লেই বোঝা যায় function কী দেয়, আর ভুল return করলে TypeScript ধরে
`any` নিষেধ, `unknown` + narrowing	type-safety থাকে (অধ্যায় ৪)
URL আর API path হাতে লেখা নিষেধ, enum (`AppRoutes`, `ApiEndpoints`) ব্যবহার	বানান ভুল হয় না, আর path বদলালে এক জায়গায় বদলালেই হয়
project-এর নিজের file import করতে সবসময় `@/` (package যেমন `react` এমনিই নাম দিয়ে import হয়)	যে file import করছে, সেটা অন্য folder-এ সরালেও path বদলাতে হয় না
data আনা: api → thunk → slice	প্রতিটা layer-এর একটাই কাজ
component সরাসরি `fetch` করবে না	সব network-এর কাজ api file-এ
code-এ comment নেই	নাম আর type দিয়েই code নিজেকে ব্যাখ্যা করবে
state কখনো সরাসরি বদলানো যাবে না	React আর Redux-এর immutability নিয়ম (অধ্যায় ৭, ১০)
`npm run format`, `npm run lint`, `npm run typecheck`	commit-এর আগে সব পরিষ্কার

`@/` কী? `@/` মানে `src/`। যেমন `@/shared/utils` মানে `src/shared/utils`। এটা কাজ করে কারণ `tsconfig.app.json`-এর `paths`-এ TypeScript-কে, আর `vite.config.ts`-এর `alias`-এ Vite-কে এটা বলে দেওয়া আছে (অধ্যায় ৫)। `../../shared/utils`-এর মতো relative path লিখলে file সরালে path ভেঙে যায়। `@/` সবসময় `src/` থেকে শুরু হয়, তাই ভাঙে না।

১৪.৬ নতুন feature যোগ করার checklist

ধরো একটা "Companies" feature যোগ করতে চাও: কোম্পানির নাম আর website-এর একটা list।

> **আগে জেনে রাখো:** এই project-এর backend-এ `/companies` নামে কোনো API **নেই**। নিচের ধাপগুলো একটা template, যাতে দেখা যায় frontend-এ নতুন feature কীভাবে যোগ হয়। সত্যিই চালাতে হলে আগে backend-এ কাজ লাগবে: একটা NestJS module (controller, service, repository) আর database-এর জন্য একটা Prisma model। সেই কাজ এই বই শেখায় না। backend ছাড়া চালালে page-এ শুধু একটা error message দেখাবে।

সব import-এর style `features/applications/*`-এর মতোই: type-এর জন্য `import type`, আর path সবসময় `@/` দিয়ে।

১. Endpoint আর route যোগ করো: `shared/constants.ts`

(সংক্ষেপে, `...` মানে আগের value-গুলো যেমন আছে তেমন থাকবে)

```
export enum ApiEndpoints
{
    ...
    Companies = '/companies',
}

export enum AppRoutes
{
    ...
    Companies = '/companies',
}
```

দুটোর string একই (`'/companies'`), কিন্তু জিনিস আলাদা:

- `ApiEndpoints.Companies` হলো backend-এর path। এর আগে `API_BASE_URL` (`/api`) বসে, তাই request যায় `/api/companies`-এ।
- `AppRoutes.Companies` হলো browser-এর URL। user address bar-এ দেখে `/companies`, আর React Router এটা দেখে CompaniesPage দেখায়।

একটা বদলালে অন্যটা বদলায় না। এখানে শুধু list লাগবে, তাই একটা company আনার `/companies/:id` endpoint যোগ করা হয়নি।

২. Folder বানাও: `features/companies/`

এই feature-এর সব file এই নতুন folder-এ থাকবে। এর পরের ধাপগুলোতে যে order-এ file লেখা হচ্ছে (types → api → thunks → slice → store), সেটা ইচ্ছা করেই: slice import করে thunk, আর store import করে slice। এই order-এ লিখলে কোনো ধাপে এমন কিছু import করতে হয় না যেটা এখনো লেখা হয়নি।

৩. Type: `features/companies/companies.types.ts`

```
import type { RequestStatus } from '@/shared/types';

export interface Company
{
    id: string;
    name: string;
    website: string | null;
}

export interface CompaniesState
{
    items: Company[];
    status: RequestStatus;
    error: string | null;
}
```

`Company` হলো server থেকে আসা একটা company-র shape। `CompaniesState` হলো slice-এর state: list, request-এর status, আর error (অধ্যায় ১১-এর RequestStatus pattern)।

৪. API: `features/companies/companies.api.ts`

```
import type { Company } from '@/features/companies/companies.types';
import { api } from '@/shared/api';
import { ApiEndpoints } from '@/shared/constants';

export function getCompanies(): Promise<Company[]>
{
    return api.get<Company[]>(ApiEndpoints.Companies);
}
```

`getCompanies()` call করলে `GET /api/companies` request যায়, আর একটা Promise আসে, যেটা `Company[]` দিয়ে resolve হয়।

৫. Thunk: `features/companies/companies.thunks.ts`

```
import { createAppAsyncThunk } from '@/app/redux';
import * as companiesApi from '@/features/companies/companies.api';
import type { Company } from '@/features/companies/companies.types';
import { getErrorMessage } from '@/shared/utils';

export const loadCompanies = createAppAsyncThunk<Company[], undefined>(
    'companies/load',
    async (_arg, { rejectWithValue }) =>
    {
        try
        {
            return await companiesApi.getCcompanies();
        }
        catch (error: unknown)
        {
            return rejectWithValue(getErrorMessage(error));
        }
    },
);
```

- **`import * as companiesApi`**: file-এর সব export একটা object-এ নিয়ে আসে, তাই `companiesApi.getCompanies()` লেখা যায়। নাম দেখেই বোঝা যায় function-টা api file থেকে এসেছে।
- **`<Company[], undefined>`**: প্রথমটা fulfilled-এর payload-এর type, দ্বিতীয়টা thunk-এর argument-এর type। `undefined` মানে এই thunk কোনো argument নেয় না, তাই dispatch করার সময় শুধু `loadCompanies()` লিখলেই হয়।
- **`_arg`**: thunk-এর function-এ প্রথম parameter সবসময় argument, দ্বিতীয়টা thunkAPI। আমাদের `rejectWithValue` লাগবে, যেটা দ্বিতীয় parameter-এ থাকে। তাই প্রথমটা ব্যবহার না করলেও লিখতে হয়। নামের শুরুতে `_` একটা প্রচলিত convention, মানে "এটা ইচ্ছা করেই ব্যবহার করা হয়নি"। project-এ আরেক জায়গায়ও আছে: `shared/utils.ts`-এর `buildPath`-এ `_match`।

৬. Slice: `features/companies/companies.slice.ts`

```
import { loadCompanies } from '@/features/companies/companies.thunks';
import type { CompaniesState } from '@/features/companies/companies.types';
import { RequestStatus } from '@/shared/types';
import { createSlice } from '@reduxjs/toolkit';
import type { Reducer } from '@reduxjs/toolkit';

const initialState: CompaniesState = {
    items: [],
    status: RequestStatus.Idle,
    error: null,
};

const companiesSlice = createSlice({
    name: 'companies',
    initialState,
    reducers: {},
    extraReducers: (builder): void =>
    {
        builder
            .addCase(loadCompanies.pending, (state): void =>
            {
                state.status = RequestStatus.Loading;
                state.error = null;
            })
            .addCase(loadCompanies.fulfilled, (state, action): void =>
            {
                state.status = RequestStatus.Succeeded;
                state.items = action.payload;
            })
            .addCase(loadCompanies.rejected, (state, action): void =>
            {
                state.status = RequestStatus.Failed;
                state.error = action.payload ?? 'Could not load companies';
            });
    },
});

export const companiesReducer: Reducer<CompaniesState> = companiesSlice.reducer;
```

- শুরুতে list খালি, status `Idle` ।

- pending-এ status `Loading` আর পুরনো error মোছা হয়, fulfilled-এ list বসে, rejected-এ error message বসে।
- **নিজের কোনো action না থাকলেও `reducers: {}` লিখতে হবে।** Redux Toolkit 2-এ `createSlice`-এর `reducers` field বাধ্যতামূলক, খালি object হলেও।
- শেষ লাইনে শুধু reducer export হয়েছে, কারণ store-এ এটাই লাগবে।

৭. Store-এ বসাও: `app/store.ts`

```
import { applicationsReducer } from '@/features/applications/applications.slice';
import { authReducer } from '@/features/auth/auth.slice';
import { companiesReducer } from '@/features/companies/companies.slice';
import { configureStore } from '@reduxjs/toolkit';

export const store = configureStore({
    reducer: {
        auth: authReducer,
        applications: applicationsReducer,
        companies: companiesReducer,
    },
});

export type RootState = ReturnType<typeof store.getState>;

export type AppDispatch = typeof store.dispatch;
```

এখানে নতুন শুধু দুটো লাইন: `companiesReducer`-এর import, আর `companies: companiesReducer`। নিচের `RootState` আর `AppDispatch` লাইন দুটো আগের মতোই রাখতে হবে। মুছে ফেললে পুরো app ভেঙে যাবে, কারণ `app/redux.ts` এ দুটো ব্যবহার করে। `RootState` store থেকে নিজে থেকেই তৈরি হয়, তাই নতুন slice যোগ করলে এর ভেতরে `companies` নিজে থেকেই চলে আসে। এখন `state.companies` দিয়ে companies-এর state পড়া যায়।

৮. Page: `features/companies/CompaniesPage.tsx`

`ApplicationsPage`-এর মতো করে:

```
import { useAppDispatch, useAppSelector } from '@/app/redux';
import { loadCompanies } from '@/features/companies/companies.thunks';
import { ErrorMessage, Spinner } from '@/shared/components/Feedback';
import { RequestStatus } from '@/shared/types';
import { useEffect } from 'react';
import type { JSX } from 'react';

export function CompaniesPage(): JSX.Element
{
    const dispatch = useAppDispatch();
    const { items, status, error } = useAppSelector((state) => state.companies);

    useEffect((): void =>
    {
        void dispatch(loadCompanies());
    }, [dispatch]);

    return (
        <div className="space-y-6">
            <h1 className="text-2xl font-bold text-slate-800">Companies</h1>
            <ErrorMessage message={error} />

            {status === RequestStatus.Loading && <Spinner />}

            {status === RequestStatus.Succeeded && items.length === 0 && (
                <p className="rounded-xl bg-white p-12 text-center text-slate-500
shadow">No companies yet.</p>
            )}

            {items.length > 0 && (
                <ul className="divide-y divide-slate-100 rounded-xl bg-white shadow">
                    {items.map((company) => (
                        <li key={company.id} className="px-4 py-3">
                            <p className="font-medium text-slate-800">{company.name}</p>
                            <p className="text-sm text-slate-500">{company.website ??
'—'}</p>
                        </li>
                    ))}
                </ul>
            )}
        </div>
    );
}
```

- page mount হলে `useEffect` একবার `loadCompanies()` dispatch করে (`dispatch` কখনো বদলায় না, তাই effect আবার চলে না)।
- load চলার সময় spinner, ব্যর্থ হলে লাল error box, সফল কিন্তু খালি হলে "No companies yet.", আর data থাকলে প্রতিটা company একটা `<li>`-তে। `key={company.id}` দিয়ে React প্রতিটা item চেনে (অধ্যায় ৬)।
- website না থাকলে (`null`) `??` দিয়ে "—" দেখায়।

৯. Route আর navbar link

`app/AppRouter.tsx`-এ উপরে import যোগ করো, তারপর AppLayout-এর `children`-এ নতুন route (সংক্ষেপে):

```
import { CompaniesPage } from '@/features/companies/CompaniesPage';

{
    element: <AppLayout />,
    children: [
        ...
        { path: AppRoutes.Companies, element: <CompaniesPage /> },
    ],
},
```

AppLayout-এর children-এ রাখা হয়েছে, তাই page-টা ProtectedRoute-এর ভেতরে পড়ে (login ছাড়া দেখা যায় না) আর উপরে navbar থাকে।

user যেন page-টা খুঁজে পায়, তাই `app/AppLayout.tsx`-এর navbar-এ "Job Tracker" link-এর পরে একটা link (সংক্ষেপে):

```
<Link to={AppRoutes.Companies} className="text-sm font-medium text-slate-600 hover:text-
indigo-600">
    Companies
</Link>
```

`Link` আর `AppRoutes` AppLayout-এ আগে থেকেই import করা আছে। এখন navbar-এ "Companies" click করলে page reload ছাড়াই `/companies`-এ যাওয়া যায়।

১০. Check করো

```
npm run format
npm run lint
npm run typecheck
```

তিনটাই কোনো error ছাড়া শেষ হলে browser-এ `/companies` খুলে দেখো। backend-এ endpoint না থাকলে লাল error box দেখাবে, সেটাই প্রত্যাশিত।

এই দশ ধাপ প্রতিটা নতুন feature-এর জন্য একই। project-এর সবচেয়ে বড় শক্তি এটাই: **সবকিছু একই pattern-এ**। একটা feature বুঝলে বাকিগুলোও বোঝা যায়।

১৪.৭ "Over-engineering" এড়ানো: কী কী ইচ্ছা করেই রাখা হয়নি

Over-engineering মানে এখনো যে সমস্যা নেই, তার জন্য আগেভাগে জটিল সমাধান বানানো।

যা নেই	কেন নেই	কখন লাগবে
Form library (react-hook-form)	মাত্র ৩টা form (Login, Register, ApplicationForm), `useState` + `validateForm` যথেষ্ট	অনেক বড় আর জটিল form হলে
Toast notification system	inline error message যথেষ্ট	অনেক জায়গায় popup message দরকার হলে
Custom modal	`window.confirm` যথেষ্ট	design করা dialog লাগলে
URL-এ filter রাখা	Redux-এ রাখা সহজ	filter-সহ link share করতে হলে
Data-fetching library (RTK Query)	thunk দিয়ে শেখা সহজ	অনেক endpoint আর cache লাগলে
Unit test	শেখার project	আসল product-এ অবশ্যই

মূলনীতি: যতটুকু দরকার, ততটুকুই। জটিলতা যোগ করো তখনই, যখন সমস্যাটা সত্যিই আসে।

সারসংক্ষেপ

- `app/` সব জোড়া লাগায়, `features/` প্রতিটা feature-এর code রাখে, আর `shared/` সবার common জিনিস রাখে। `shared/` কখনো `features/` import করে না।

- feature-based structure-এ একটা feature-এর সব কিছু এক folder-এ থাকে।
- `@/` মানে `src/`, তাই file সরালেও import path বদলাতে হয় না।
- নতুন feature মানে একই ১০ ধাপ:
    1. constants-এ endpoint আর route
    2. `features/<name>/` folder
    3. types
    4. api
    5. thunks
    6. slice (`reducers: {}` সহ)
    7. store-এ reducer বসানো
    8. page
    9. route আর navbar link
    10. format, lint, typecheck দিয়ে check
- এই project-এর backend-এ companies নেই। checklist-টা template, চালাতে হলে backend-এর কাজ লাগবে।
- দরকার না হলে জটিলতা যোগ করা হয় না।

ইন্টারভিউ প্রশ্ন

প্রশ্ন: তোমার React project-এর architecture বলো।
উত্তর (নমুনা): "Vite + React + TypeScript দিয়ে বানানো একটা SPA (single-page application, মানে browser একবার page load করে, তারপর JavaScript দিয়ে page বদলায়)।

- Routing: React Router-এর layout route। login লাগে এমন অংশ ProtectedRoute দিয়ে ঢাকা।
- Global state: Redux Toolkit। server-এর data আসে createAsyncThunk দিয়ে। প্রতিটা feature-এ api → thunk → slice layer: api file শুধু HTTP call করে, thunk async কাজ আর error সামলায়, slice state বদলায়।
- Code সাজানো feature অনুযায়ী: auth আর applications। common UI component আর HTTP client থাকে shared-এ।
- URL আর API path সব enum-এ, হাতে লেখা নেই।
- Styling: Tailwind।
- Type-safety: strict TypeScript আর ESLint।"

প্রশ্ন: Feature-based আর type-based folder structure-এর পার্থক্য কী?
উত্তর: Type-based-এ file-এর ধরন অনুযায়ী folder: সব component এক folder-এ, সব reducer আরেক folder-এ। ছোট project-এ চলে, কিন্তু বড় হলে একটা feature-এর একটা কাজে অনেক folder-এ ঘুরতে হয়।

Feature-based-এ একটা feature-এর সব file এক folder-এ থাকে, যেমন আমার `features/applications/` -এ সেই feature-এর type, api, thunk, slice, page সব। খোঁজা, বদলানো, মোছা সহজ, আর বড় team-এ আলাদা মানুষ আলাদা folder-এ কাজ করতে পারে।

প্রশ্ন: Component কখন আলাদা file-এ ভাঙবে?
উত্তর: যখন এটা একাধিক জায়গায় ব্যবহার হয়, যখন file এত বড় হয়ে যায় যে পড়া কঠিন, বা যখন এর দায়িত্ব স্পষ্টভাবে আলাদা। শুধু এক জায়গায় লাগা ছোট অংশ একই file-এ রাখা যায়। যেমন আমার `DetailRow` শুধু details page-এ লাগে, তাই সেটা ওই file-এর ভেতরেই আছে, export করা হয়নি। আর `Button` সব জায়গায় লাগে, তাই সেটা `shared/components` -এ।

প্রশ্ন: Separation of concerns বলতে কী বোঝো?
উত্তর: প্রতিটা অংশের একটাই দায়িত্ব থাকবে। আমার project-এ api file শুধু HTTP call করে, thunk শুধু async flow আর error সামলায়, slice শুধু state বদলায়, component শুধু দেখায়, আর utils শুধু হিসাব করে (যেমন `formatSalary`)। ফলে এক জায়গার বদল অন্য জায়গা ভাঙে না। যেমন backend-এর URL বদলালে শুধু constants আর api file বদলাতে হয়, page-এ হাত দিতে হয় না।

প্রশ্ন: নতুন feature কীভাবে যোগ করো?
উত্তর: প্রতিবার একই order মানি: constants-এ endpoint আর route, তারপর type, api function, thunk, slice, store-এ reducer, page, route আর navbar link, শেষে format, lint আর typecheck। সব feature একই pattern-এ, তাই নতুন কেউ একটা feature দেখেই বাকিগুলো বুঝে ফেলে।

অধ্যায় ১৫: Interview প্রশ্নব্যাংক

Interview-এর আগে এই অধ্যায়টা পড়ো। প্রতিটা উত্তর ছোট রাখা হয়েছে, যাতে মুখে বলা যায়। বন্ধনীতে লেখা আছে কোন অধ্যায়ে বিস্তারিত পাবে। যে বিষয় বইয়ে গভীরভাবে শেখানো হয়নি, সেখানে লেখা আছে **(বইয়ে বিস্তারিত নেই, শুধু পরিচিতি)**, আর সাথে একটা ছোট উদাহরণ, যাতে follow-up প্রশ্নেও কিছু বলতে পারো।

কৌশল: প্রতিটা উত্তরের শেষে নিজের project থেকে একটা উদাহরণ দাও। "আমার Job Tracker project-এ আমি..." বললে interviewer বোঝে তুমি শুধু মুখস্থ করোনি, নিজে কাজ করেছ।

ক. JavaScript (অধ্যায় ২–৩)

১. `let`, `const`, `var`-এর পার্থক্য কী? (অধ্যায় ২)
`const`-কে নতুন value দেওয়া যায় না, `let`-কে যায়। দুটোই **block scoped**, মানে `{ }`-এর বাইরে দেখা যায় না। `var` function scoped (পুরো function জুড়ে দেখা যায়), আর hoisting-এর কারণে অদ্ভুত আচরণ করে (নিচে #১২ক)। তাই `var` ব্যবহার করি না।

২. `==` বনাম `===`? (অধ্যায় ২)
`==` তুলনার আগে type বদলে নেয়, তাই `0 == ''` হয় `true`। `===` type-সহ তুলনা করে, তাই `0 === ''` হয় `false`। সবসময় `===` ব্যবহার করি।

৩. Truthy আর falsy কী? (অধ্যায় ২)
`if`-এর ভেতরে যে value `false`-এর মতো আচরণ করে, সেটা **falsy**: `false`, `0`, `''`, `null`, `undefined`, `NaN`। বাকি সব **truthy**, এমনকি `'0'`, `[]` আর `{}`-ও।

৪. Arrow function আর সাধারণ function-এর পার্থক্য কী? (অধ্যায় ২)
Arrow function ছোট করে লেখা যায়, আর body এক expression হলে `return` লাগে না। বড় পার্থক্য হলো arrow function-এর নিজের `this` নেই, চারপাশের `this` ব্যবহার করে (নিচে #১২খ)।

৫. `map`, `filter`, `find`, `forEach`-এর পার্থক্য কী? (অধ্যায় ২)

- `map`: প্রতিটা item থেকে একটা করে নতুন item বানিয়ে নতুন array দেয়। `[1, 2].map((n) => n * 2)` দেয় `[2, 4]`।
- `filter`: শর্ত মেলে এমন item নিয়ে নতুন array দেয়।
- `find`: শর্ত মেলে এমন প্রথম item দেয়, না পেলে `undefined`।
- `forEach`: প্রতিটার উপর কিছু চালায়, কিছু return করে না।

৬. Destructuring আর spread কী? (অধ্যায় ২)

Destructuring দিয়ে object বা array থেকে value বের করে variable-এ রাখা হয়: `const { a } = obj`। Spread (`...`) দিয়ে ভেতরের জিনিস ছড়িয়ে দিয়ে copy বা merge করা হয়: `{ ...obj, a: 1 }`। Spread **shallow copy** করে, মানে শুধু বাইরের দিকের field copy হয়, ভেতরের object আগের reference-ই থেকে যায়।

৭. `?.` আর `??` কী? (অধ্যায় ২)

`user?.name`: `user` null বা undefined হলে error না দিয়ে `undefined` দেয়। `a ?? b`: শুধু `a` null বা undefined হলে `b` দেয়। `0 ?? 5` হয় `0`, কিন্তু `0 || 5` হয় `5`। আমার project-এ `user?.name ?? user?.email` দিয়ে navbar-এ নাম, না থাকলে email দেখাই।

৮. Closure কী? (অধ্যায় ২)

একটা function যেখানে তৈরি হয়, সেখানকার variable মনে রাখে, এমনকি বাইরের function শেষ হয়ে গেলেও। উদাহরণ:

```
function makeCounter()
{
    let count = 0;
    return () =>
    {
        count = count + 1;
        return count;
    };
}

const next = makeCounter();
next();
next();
```

`makeCounter` শেষ হয়ে গেলেও ভেতরের function `count` মনে রাখে। প্রথম `next()` দেয় `1`, দ্বিতীয়টা `2`। React-এর event handler-ও closure: আমার ApplicationsPage-এর `handleDelete` render-এর সময়কার `items` মনে রাখে (অধ্যায় ১৩)।

৯. Promise কী? এর তিনটা state কী? (অধ্যায় ৩)

Promise এমন একটা value-কে represent করে, যেটা এখনো আসেনি কিন্তু ভবিষ্যতে আসবে, যেমন server-এর উত্তর। তিনটা state: **pending** (অপেক্ষা চলছে), **fulfilled** (সফল, value এসেছে), **rejected** (ব্যর্থ, error এসেছে)। একবার fulfilled বা rejected হলে আর বদলায় না।

১০. async/await কী? (অধ্যায় ৩)

Promise-কে সাধারণ, উপর থেকে নিচে চলা code-এর মতো লেখার উপায়। `await` শুধু ওই `async` function-টাকে থামায়, পুরো app-কে না। error ধরা হয় try/catch দিয়ে। আমার project-এর সব thunk

এভাবে লেখা।

১১. fetch কি 404-এ throw করে? (অধ্যায় ৩)
না। fetch শুধু network ব্যর্থ হলে (server-এ পৌঁছাতেই না পারলে) throw করে। 404 বা 500 এলেও fetch সফল ধরে নেয়, তাই `response.ok` বা `response.status` নিজে check করতে হয়। আমার `shared/api.ts`-এর `request` function এটা করে আর ভুল হলে `ApiError` throw করে।

১২. Event loop কী? (অধ্যায় ৩; call stack বইয়ে বিস্তারিত নেই, শুধু পরিচিতি)
JavaScript single threaded, মানে একসাথে একটাই কাজ চালায়। **Call stack** হলো এখন যে function-গুলো চলছে তাদের তালিকা: function call হলে উপরে যোগ হয়, শেষ হলে সরে যায়। অপেক্ষার কাজ (timer, network) browser আলাদাভাবে সামলায়, শেষ হলে তাদের callback একটা queue-তে বসে। **Event loop** বারবার দেখে call stack খালি হলো কিনা, খালি হলে queue থেকে পরের callback নিয়ে চালায়। তাই `setTimeout(fn, 0)` দিলেও `fn` চলে বর্তমান সব code শেষ হওয়ার পরে।

১২ক. Hoisting কী? (অধ্যায় ২)
JavaScript code চালানোর আগে declaration-গুলো scope-এর উপরে তুলে নেয়, একে বলে **hoisting**। `var` উপরে ওঠে `undefined` value নিয়ে, তাই declaration-এর আগে পড়লে error হয় না, `undefined` আসে:

```
console.log(a);
var a = 5;
```

এটা print করে `undefined`। কিন্তু `let` বা `const` declaration-এর আগে পড়লে `ReferenceError` আসে:

```
console.log(b);
let b = 5;
```

এটাই নিরাপদ আচরণ, কারণ ভুলটা সাথে সাথে ধরা পড়ে।

১২খ. `this` কী? (অধ্যায় ২.১৪-এ শুধু class-এর `this`; বাকিটা বইয়ে বিস্তারিত নেই, শুধু পরিচিতি)
সাধারণ function বা method-এর ভেতরে `this` হলো যে object-এর উপর function-টা call হয়েছে। arrow function-এর নিজের `this` নেই, যেখানে লেখা হয়েছে তার চারপাশের `this` ব্যবহার করে:

```
const user = {
    name: 'Rahim',
    greet()
    {
        return this.name;
    },
    greetArrow: () => this?.name,
};
```

`user.greet()` দেয় `'Rahim'`, কারণ `this` হলো `user`। `user.greetArrow()` দেয় `undefined`, কারণ arrow function `user`-কে `this` হিসেবে পায় না, বাইরের `this` নেয় (module-এর top-level-এ সেটা `undefined`)। তাই object-এর method arrow দিয়ে লিখি না। React-এর function component-এ `this` লাগেই না।

খ. TypeScript (অধ্যায় ৪)

১৩. TypeScript কেন?
type-এর ভুল code চালানোর আগেই ধরা পড়ে, editor-এ autocomplete পাওয়া যায়, আর বড় code নিরাপদে বদলানো যায়, কারণ কোথায় কী ভাঙল TypeScript দেখিয়ে দেয়। browser TypeScript বোঝে না, তাই চালানোর আগে এটা JavaScript-এ compile/transform হয়।

১৪. interface বনাম type?
দুটোই একটা type-কে নাম দেয়। `interface` শুধু object-এর আকারের জন্য, `extends` দিয়ে বাড়ানো যায়, আর **declaration merging** হয়: একই নামে দুটো interface লিখলে TypeScript দুটো জোড়া লাগিয়ে একটা বানায় (বইয়ে বিস্তারিত নেই, শুধু পরিচিতি)। `type` দিয়ে union (`'a' | 'b'`) বা যেকোনো type-এর alias-ও বানানো যায়। আমার project-এ object-এর জন্য interface, union-এর জন্য type ব্যবহার করেছি, যেমন `type ApiResponse<T> = ApiSuccessResponse<T> | ApiErrorResponse`।

১৫. any বনাম unknown?
`any` দিলে TypeScript কোনো check-ই করে না, যা খুশি করা যায়, তাই bug লুকিয়ে থাকে। `unknown` মানে "type জানা নেই", ব্যবহারের আগে narrowing করে type নিশ্চিত করতে হয়। আমার `getErrorMessage(error: unknown)` আগে check করে `error` string নাকি `Error`, তারপর message বের করে।

১৬. Generic কী?
type-কে parameter বানানো, যাতে একই function বিভিন্ন type-এর সাথে কাজ করে। যেমন `api.get<User>(...)` দেয় `Promise<User>`, আর `api.get<JobApplication>(...)` দেয় `Promise<JobApplication>`। function একটাই, ফলাফলের type আলাদা।

১৭. Union আর discriminated union কী?
Union হলো "এটা অথবা ওটা": `A | B`। Discriminated union-এ সব type-এ থাকা একটা common field থাকে (যেমন `success`), যার value প্রতিটা type-এ আলাদা (`true` বা `false`)। সেই field check করলেই TypeScript বুঝে যায় কোন type। আমার `ApiResponse`-এ `json.success === true` হলে TypeScript জানে `data` আছে, `false` হলে জানে `error` আছে।

১৮. Partial, Omit, Record কী?

- `Partial<T>`: T-এর সব field optional করে দেয়। আমার update API নেয় `Partial<ApplicationPayload>`, কারণ শুধু বদলানো field পাঠাই।
- `Omit<T, K>`: T থেকে কিছু field বাদ দেয়। `ApplicationPayload = Omit<JobApplication, 'id' | 'createdAt' | 'updatedAt'>`।
- `Record<K, V>`: যার key-গুলো K type-এর আর value V type-এর, এমন object। যেমন `Record<string, string>`।

১৯. Type narrowing কী?

`typeof`, `instanceof`, `in`, null check বা discriminant field দিয়ে check করার পরে TypeScript নিজে থেকেই type ছোট করে নেয়। যেমন `if (error instanceof Error)`-এর ভেতরে TypeScript জানে `error.message` আছে।

গ. React মূল ধারণা (অধ্যায় ৬)

২০. React কী? declarative মানে কী?

UI বানানোর একটা library। **Declarative** মানে "কী দেখাতে চাই" বলা, আর DOM কীভাবে বদলাবে সেটা React ঠিক করে। আমি state অনুযায়ী JSX লিখি, state বদলালে React screen মিলিয়ে দেয়: UI = f(state)।

২১. Virtual DOM আর reconciliation কী?

Virtual DOM হলো আসল DOM-এর একটা lightweight copy, সাধারণ JavaScript object দিয়ে বানানো। state বদলালে React নতুন copy বানিয়ে পুরনোটার সাথে তুলনা করে (এই তুলনাকে বলে **reconciliation**), তারপর আসল DOM-এ শুধু পার্থক্যটুকু বসায়। আসল DOM বদলানো ধীর, তাই এতে কাজ কম হয়।

২২. JSX কী?

JavaScript-এর ভেতরে HTML-এর মতো করে লেখা। build tool (Vite) এটাকে সাধারণ function call-এ compile/transform করে। নিয়ম: `class`-এর বদলে `className`, `for`-এর বদলে `htmlFor`, সব tag বন্ধ করতে হয়, একটা root element লাগে, আর `{ }`-এর ভেতরে JavaScript expression।

২৩. Component কী? নাম বড় হাতের কেন?

JSX return করে এমন function হলো component। নাম ছোট হাতের হলে React সেটাকে HTML tag ভাবে (`<button>`), তাই নিজের component বড় হাতের (`<Button>`)।

২৪. Props বনাম state?

Props আসে parent থেকে, আর component-এর কাছে read-only। State component-এর নিজের, আর set function দিয়ে বদলায়। দুটোর যেকোনোটা বদলালেই re-render হয়।

২৫. children prop কী?

component-এর opening আর closing tag-এর মাঝে যা লেখা হয়, সেটা `children` prop হিসেবে আসে। এভাবে component জোড়া লাগানো যায় (composition)। যেমন আমার `<Field label="Email">`-এর

ভেতরে `<Input>`, Field শুধু label আর error দেখায়, input যেকোনোটা হতে পারে।

২৬. key কেন দরকার? index কেন খারাপ?

list-এর কোন element কোন data-র, React সেটা key দিয়ে চেনে। index দিলে মাঝখানে কিছু যোগ বা মুছলে পরের সবার key বদলে যায়, তখন এক item-এর state ভুল করে অন্য item-এ জুড়ে যেতে পারে। তাই data-র **stable** id দিই, যেমন `key={application.id}`।

২৭. Conditional rendering-এর উপায় কী কী?

`&&`, ternary (`a ? b : c`), আর early return (`if (...) return ...`)। সতর্কতা: `&&`-এর বাম দিকে সংখ্যা থাকলে `{count && <List />}`-এ count `0` হলে screen-এ `0` print হয়। তাই `count > 0 &&` লিখি।

২৮. One-way data flow কী?

data যায় উপর থেকে নিচে (parent থেকে child-এ props), আর event আসে নিচ থেকে উপরে (child callback call করে parent-কে জানায়)। আমার ApplicationsTable `applications` prop পায়, আর Delete click হলে `onDelete` callback দিয়ে ApplicationsPage-কে জানায়।

২৯. Fragment কী?

`<>...</>`: বাড়তি কোনো DOM element না বানিয়েই কয়েকটা element একসাথে return করা।

৩০. StrictMode কী করে?

শুধু development-এ বাড়তি check করে। component দুইবার render করে (এটা development-এ ভুল ধরার জন্য; effect-এর দুইবার চলা অধ্যায় ৮.৬-এ), আর effect mount → cleanup → আবার mount চালায়। production-এ এর কোনো প্রভাব নেই।

ঘ. State, Event, Form (অধ্যায় ৭)

৩১. useState কীভাবে কাজ করে?

`const [value, setValue] = useState(initial)` দেয়। `setValue` call করলে React নতুন value মনে রাখে আর component আবার render করে। পরের render-এ `value` নতুনটা।

৩২. setState কি সাথে সাথে বদলায়?

না। update schedule হয়, আর একই event-এর কয়েকটা update একসাথে করা হয় (batching)। নতুন value আসে পরের render-এ। তাই `setCount(count + 1)`-এর ঠিক পরের লাইনে `count` এখনো পুরনো।

৩৩. Functional update কখন?

নতুন value পুরনো value-র উপর নির্ভর করলে: `setX((previous) => ...)`। আমার ApplicationForm-এ `setValues((previous) => ({ ...previous, [field]: value }))`, যাতে সবসময় সর্বশেষ value থেকে শুরু হয়।

৩৪. State কেন immutable?

React reference তুলনা করে বোঝে state বদলেছে কিনা। আসল object-এর ভেতরে বদলালে reference একই থাকে, তাই React ভাবে কিছু বদলায়নি, re-render হয় না। তাই সবসময় নতুন object বা array বানাই, যেমন spread দিয়ে।

৩৫. Controlled বনাম uncontrolled input?

Controlled-এ value থাকে state-এ: `value={email}` আর `onChange`-এ `setEmail`। React state-ই source of truth। Uncontrolled-এ value থাকে DOM-এর নিজের কাছে, দরকারে পড়া হয় ref দিয়ে (useRef দিয়ে, নিচে #৪৮ দেখো)। আমার project-এর সব input controlled।

৩৬. `event.preventDefault()` কেন?

form submit হলে browser নিজে থেকে পুরো page reload করে। SPA-তে সেটা চাই না, তাই submit handler-এর শুরুতে থামাই।

৩৭. Lifting state up কী?

দুই বা বেশি component একই state ব্যবহার করলে state-টা তাদের common parent-এ রাখা, আর props দিয়ে নিচে পাঠানো।

৩৮. Derived state কেন রাখবে না?

যা অন্য state থেকে হিসাব করে বের করা যায়, সেটা আলাদা state-এ রাখলে দুটো value অমিল হয়ে যাওয়ার bug হয়। render-এর সময়েই হিসাব করে নিই। যেমন আমার table-এ deadline পেরিয়েছে কিনা `isDeadlinePassed(application)` দিয়ে প্রতি render-এ হিসাব হয়, আলাদা state নেই।

৩৯. Re-render কখন হয়?

নিজের state বদলালে, parent re-render হলে, আর component যে context বা Redux selector-এর value পড়ে সেটা বদলালে।

ঙ. Effect আর Hook (অধ্যায় ৮)

৪০. useEffect কী? কখন চলে?

render শেষ হয়ে screen-এ দেখানোর পরে side effect চালায়, যেমন data আনা বা timer চালানো। dependency array ঠিক করে কখন আবার চলবে।

৪১. `[]`, `[a]`, আর কিছু না দেওয়ার পার্থক্য?

`[]` দিলে শুধু mount-এ একবার চলে। `[a]` দিলে mount-এ আর যখনই `a` বদলায়। কিছু না দিলে প্রতি render-এ চলে।

৪২. Cleanup কখন চলে?

পরের বার effect চলার আগে, আর unmount-এর সময়। timer, event listener বা subscription বন্ধ করতে লাগে।

৪৩. Debounce কীভাবে বানাবে?

effect-এ `setTimeout`, আর cleanup-এ `clearTimeout`। value বদলালে আগের timer বাতিল হয়ে নতুন timer শুরু হয়, তাই টাইপ থামার পরেই কাজ হয়। আমার search box ৪০০ms পরে filter বদলায়।

৪৪. Rules of Hooks কী?

Hook শুধু component বা custom hook-এর একদম উপরের অংশে call করা যায়, কোনো `if` বা loop-এর ভেতরে না। কারণ React hook-গুলো চেনে তাদের call order দিয়ে। কোনো render-এ একটা hook বাদ পড়লে পরের সবগুলোর মিল ভেঙে যায়।

৪৫. Stale closure কী?

callback বা effect তৈরি হওয়ার সময়ের পুরনো value দেখে, কারণ closure সেই value মনে রেখেছে (#৮)। সমাধান: dependency array ঠিক রাখা, অথবা functional update।

৪৬. Custom hook কী?

`use` দিয়ে শুরু হওয়া একটা function, যেটা ভেতরে অন্য hook ব্যবহার করে, যাতে state-সহ logic reuse করা যায়। `useAppSelector` আসলে typed `useSelector` (`withTypes` দিয়ে বানানো); custom hook-এর ভালো উদাহরণ Tiptap-এর `useEditor`।

৪৭. useMemo, useCallback, React.memo কী? (বইয়ে বিস্তারিত নেই, শুধু পরিচিতি; আমার project-এ ব্যবহার হয়নি)

- `useMemo(() => calc(a), [a])`: expensive হিসাবের ফল মনে রাখে, `a` না বদলালে আবার হিসাব করে না। হাজার item sort বা filter করার মতো ভারী কাজে।
- `useCallback(fn, [deps])`: function-এর reference একই রাখে, deps না বদলালে নতুন function বানায় না। সাধারণত `React.memo` করা child-কে function prop দেওয়ার সময়।
- `React.memo(Component)`: props না বদলালে component-কে আবার render হতে দেয় না। ভারী component, যেটার parent ঘনঘন re-render হয়।

এগুলো performance-এর জন্য। আগে মেপে দেখি সত্যিই ধীর কিনা, তারপর ব্যবহার করি, শুরুতেই না।

৪৮. useRef কী? (অধ্যায় ৭-এ এক লাইন; বইয়ে বিস্তারিত নেই, শুধু পরিচিতি; আমার project-এ ব্যবহার হয়নি)

`const ref = useRef(initial)` একটা object দেয়, যার `ref.current`-এ value রাখা যায়। render-এর মাঝে value থেকে যায়, কিন্তু বদলালে re-render হয় না। দুটো ব্যবহার: DOM element ধরা (`<input ref={inputRef} />`, তারপর `inputRef.current.focus()`), আর এমন value রাখা যেটা screen-এ দেখাতে হয় না, যেমন timer-এর id।

চ. Routing (অধ্যায় ৯)

৪৯. Client-side routing কী?
server থেকে নতুন HTML page না এনে, JavaScript দিয়েই URL আর দেখানো component বদলানো। page reload হয় না, তাই দ্রুত, আর state থেকে যায়।

৫০. Link বনাম a?
`<Link>` reload ছাড়াই page বদলায়, তাই Redux state থেকে যায়। `<a>` পুরো page reload করে। app-এর ভেতরে `<Link>`, বাইরের website-এ `<a>`।

৫১. Nested route আর Outlet কী?
route-এর ভেতরে route। parent route-এর component-এ `<Outlet />` থাকে, আর সেই জায়গায় child route-এর page বসে। এতে layout reuse হয়: আমার AppLayout-এর navbar একবারই লেখা, সব page তার Outlet-এ বসে।

৫২. useParams, useNavigate, useLocation কী করে?

- `useParams`: URL-এর `:id`-এর মতো অংশ পড়ে।
- `useNavigate`: code থেকে অন্য page-এ পাঠায়, যেমন save-এর পরে details page-এ।
- `useLocation`: বর্তমান path, আর navigation state পড়ে। navigation state হলো এমন data, যেটা navigate করার সময় সাথে পাঠানো হয় কিন্তু URL-এ দেখা যায় না।

৫৩. Protected route কীভাবে বানাবে?
একটা component বানাই, যেটা auth state দেখে:

- session check শেষ না হলে spinner।
- login না থাকলে `<Navigate to={AppRoutes.Login} replace state={{ from: location.pathname }} />`, যাতে login-এর পরে আগের page-এ ফেরা যায়।
- login থাকলে `<Outlet />`।

URL হাতে না লিখে `AppRoutes` enum ব্যবহার করি, এটা আমার project-এর নিয়ম।

৫৪. `replace` কেন?
redirect-এর entry history-তে জমা হয় না, পুরনোটার জায়গায় বসে। না দিলে Back চাপলে আবার redirect হওয়া page-এ ফিরে যায়, আর আবার redirect হয়, user আটকে যায়।

৫৫. Deploy-এর পরে refresh করলে 404 কেন?
`/applications/123`-এ refresh করলে browser server-এর কাছে সেই path চায়, কিন্তু server-এ ওই নামে কোনো file নেই, আছে শুধু `index.html`। তাই server-এ নিয়ম দিতে হয়: অচেনা সব path-এ `index.html` দাও (fallback), তারপর React Router path দেখে সঠিক page দেখায়।

ছ. Redux Toolkit (অধ্যায় ১০–১১)

৫৬. Redux কেন? কখন দরকার নেই?
অনেক দূরের component-এ একই state লাগলে, prop drilling (অনেক component-এর মধ্য দিয়ে props হাতে হাতে পাঠানো) এড়াতে, আর predictable one-way data flow-এর জন্য: state বদলায় শুধু action dispatch করে, তাই কোথায় কী বদলাল DevTools-এ দেখা যায়। ছোট app-এ `useState` বা Context যথেষ্ট।

৫৭. Store, action, reducer, dispatch, selector কী?

- Store: পুরো app-এর central (global) state যেখানে থাকে।
- Action: কী ঘটেছে তার বর্ণনা, একটা object: `{ type, payload }`।
- Reducer: (পুরনো state, action) নিয়ে নতুন state return করে এমন pure function।
- Dispatch: store-এ action পাঠানো।
- Selector: state থেকে দরকারি অংশ বেছে নেওয়ার function, যেমন `(state) => state.auth.user`।

৫৮. Reducer কেন pure?
একই input দিলে সবসময় একই output, আর বাইরের কিছু বদলায় না। ফলে আচরণ predictable, test করা সহজ, আর DevTools-এ পুরনো action আবার চালিয়ে দেখা (time-travel) যায়। side effect থাকলে এগুলো ভেঙে যায়। তাই আমার logout-এ token মোছা reducer-এ না, thunk-এ।

৫৯. createSlice কী দেয়?
একসাথে reducer আর action creator। action type নিজে থেকে তৈরি হয় slice-এর নাম আর reducer-এর নাম জুড়ে, যেমন `'applications/filtersChanged'`।

৬০. RTK-তে state সরাসরি বদলানোর মতো লেখা যায় কেন?
Immer-এর কারণে। reducer-এ আসল state না, একটা draft আসে। draft বদলালে Immer সেই বদল দেখে নতুন immutable state বানায়। এটা শুধু RTK-এর `createSlice` reducer-এর ভেতরে কাজ করে।

৬১. useSelector কখন re-render করে?
selector-এর return value আগেরটা থেকে আলাদা হলে (reference দিয়ে তুলনা)। selector প্রতিবার নতুন object বানালে, যেমন `(state) => ({ a: state.a })`, প্রতিবার আলাদা reference, তাই প্রতিটা store বদলেই re-render হয়।

৬২. Async কাজ কোথায়?
thunk-এ, কখনো reducer-এ না, কারণ reducer pure হতে হয়।

৬৩. createAsyncThunk কী করে?
একটা async function চালায়, আর তার শুরুতে `pending`, সফল হলে `fulfilled`, ব্যর্থ হলে `rejected` action পাঠায়। function যা return করে সেটা fulfilled-এর payload, আর `rejectWithValue(x)`-এর `x` হয় rejected-এর payload।

৬৪. reducers বনাম extraReducers?

`reducers` slice-এর নিজের action বানায়, যেমন `filtersChanged`। `extraReducers` বাইরে তৈরি action শোনে, যেমন thunk-এর action বা অন্য slice-এর action। আমার applications slice `extraReducers`-এ auth-এর `logoutUser.fulfilled` শোনে।

৬৫. unwrap কী?

dispatch করা thunk-এর ফল থেকে সরাসরি payload দেয়, আর ব্যর্থ হলে throw করে। তাই component-এ try/catch দিয়ে "সফল হলে navigate" লেখা সহজ। আমার form page save-এর পরে এভাবেই details page-এ যায়।

৬৬. Loading আর error কীভাবে দেখাও?

slice-এ `status` (idle, loading, succeeded, failed) আর `error` রাখি। thunk-এর তিনটা action-এ এগুলো update হয়। UI status দেখে spinner, error message, "কিছু নেই" message, অথবা data দেখায়।

৬৭. Context বনাম Redux?

Context হলো component tree-তে props ছাড়া data পাঠানোর একটা ব্যবস্থা। value বদলালে সব consumer re-render হয়, আর এর নিজের কোনো structure বা tooling নেই। Redux-এ selector দিয়ে fine-grained re-render হয় (শুধু যে অংশ পড়ছে সেটা বদলালেই), আর middleware আর DevTools আছে। Context ভালো কম বদলানো জিনিসে (theme, ভাষা), Redux জটিল আর ঘনঘন বদলানো state-এ।

৬৮. Race condition কীভাবে এড়াবে? (অধ্যায় ১১)

Race condition মানে দুটো request গেছে, কিন্তু পুরনোটার উত্তর পরে এসে নতুনটার data মুছে দিল। উপায়: debounce দিয়ে request কমানো, শুধু সর্বশেষ request-এর id-র উত্তর গ্রহণ করা, অথবা AbortController দিয়ে পুরনো request বাতিল করা।

জ. Authentication আর security (অধ্যায় ১২–১৩)

৬৯. Authentication বনাম Authorization?

Authentication: তুমি কে, সেটা প্রমাণ করা। Authorization: তুমি কী করতে পারবে, সেটা ঠিক করা। আমার project-এ email আর password দিয়ে login করা হলো authentication। login করার পরে শুধু নিজের application দেখতে পারা, অন্যের id দিলে 404 পাওয়া, এটা authorization।

৭০. JWT কীভাবে কাজ করে?

server একটা secret key দিয়ে sign করা token দেয়। payload-এ user-এর তথ্য থাকে, আমার project-এ `sub` (user-এর UUID), `email` আর `exp` (expiry, ১ দিন)। প্রতিটা request-এ `Authorization: Bearer <token>` header যায়, আর server signature verify করে user চেনে। payload base64-এ লেখা, encrypt করা না, তাই যে কেউ পড়তে পারে, কিন্তু বদলালে signature মেলে না। তাই JWT-তে গোপন কিছু রাখি না।

৭১. localStorage বনাম httpOnly cookie?

localStorage সহজ, কিন্তু XSS হলে (অন্য কারো JavaScript page-এ চললে) সেই script token পড়ে চুরি করতে পারে। httpOnly cookie এমন cookie, যেটা JavaScript পড়তে পারে না, browser শুধু request-এর সাথে নিজে থেকে পাঠায়, তাই XSS থেকে নিরাপদ। কিন্তু নিজে থেকে যায় বলে CSRF-এর ঝুঁকি আসে: অন্য website আমার browser দিয়ে গোপনে request পাঠাতে পারে, আর cookie সাথে চলে যায়। ঠেকাতে cookie-তে `SameSite=Lax` বা `Strict` দিই (অন্য site থেকে আসা request-এ cookie যায় না), অথবা আলাদা CSRF token ব্যবহার করি।

৭২. Refresh-এ login কীভাবে থাকে?

token persistent storage-এ (localStorage) রাখি। app শুরুতে `restoreSession` thunk `/auth/me` call করে user আনে। dispatch শুধু কাজ শুরু করে, render অপেক্ষা করে না, তাই check শেষ না হওয়া পর্যন্ত `isSessionChecked` false থাকে আর protected page-এ spinner দেখাই।

৭৩. 401 এলে কী করো?

request-এ token থাকা অবস্থায় 401 এলে token মুছে login page-এ reload করি। ভালো সমাধান হলো refresh token: অল্প মেয়াদের access token-এর সাথে একটা দীর্ঘ মেয়াদের refresh token, যেটা সাধারণত httpOnly cookie-তে থাকে। access token expired হলে refresh token দিয়ে চুপচাপ নতুন access token এনে request আবার পাঠাই, user-কে আবার login করতে হয় না। আমার project-এ refresh token নেই।

৭৪. XSS কী? React কীভাবে বাঁচায়?

অন্য কারো ঢোকানো script তোমার page-এ চলে যাওয়া হলো XSS। React `{ }`-এর ভেতরের string escape করে, তাই `{'<b>Hi</b>'}` screen-এ লেখা হিসেবে `<b>Hi</b>` দেখায়, HTML হিসেবে চলে না। ঝুঁকি শুধু `dangerouslySetInnerHTML`-এ, তাই সেখানে sanitize করা HTML দিতে হয় (বিপজ্জনক tag মুছে শুধু নিরাপদ tag রাখা)। আমার server mail save করার আগে `sanitize-html` দিয়ে sanitize করে।

৭৫. Frontend validation কি যথেষ্ট?

না। frontend validation শুধু user experience (UX)-এর জন্য, ভুল সাথে সাথে দেখাতে। যে কেউ সরাসরি API-তে request পাঠিয়ে এটা এড়াতে পারে। server-এ আবার validation করতেই হবে। আমার server LINK medium-এ `applyLink` না পেলে 400 দেয়।

ঝ. Tools আর project (অধ্যায় ৫, ১৪)

৭৬. Vite কী করে?

development-এ dev server চালায়: দ্রুত চালু হয়, HMR (code save করলে page reload ছাড়াই বদল দেখা যায়), আর proxy (`/api` request backend-এ পাঠানো)। production-এর জন্য ছোট, optimize করা build বানায়।

৭৭. CORS কী? development-এ কীভাবে সামলাও?
এক origin-এর (যেমন `localhost:5173`) page অন্য origin-এ (`localhost:3000`) request পাঠালে request ঠিকই যায়, কিন্তু server অনুমতি না দিলে browser সেই response JavaScript-কে পড়তে দেয় না। এই নিয়মই CORS। development-এ Vite proxy দিয়ে সামলাই: browser ভাবে request একই origin-এ যাচ্ছে, Vite পিছনে backend-এ পাঠায়। production-এ server CORS header দিয়ে অনুমতি দেয়।

৭৮. dependencies বনাম devDependencies?
`dependencies` app চালাতে লাগে (React, Redux)। `devDependencies` শুধু development, build আর check-এর সময় লাগে (TypeScript, ESLint, Vite)।

৭৯. ESLint আর TypeScript-এর পার্থক্য কী?
TypeScript type-এর ভুল ধরে, যেমন string-এর জায়গায় number। ESLint খারাপ অভ্যাস আর সম্ভাব্য bug ধরে, যেমন hook-এর নিয়ম ভাঙা বা Promise-এর error কেউ না ধরা (`no-misused-promises`)।

৮০. তোমার project-এর architecture বলো। (অধ্যায় ১৪-এর নমুনা উত্তর দেখো)

৮১. Project-এ সবচেয়ে কঠিন কী ছিল, আর কীভাবে সমাধান করেছ? (নিজের ভাষায় তৈরি রাখো)

উদাহরণ উত্তর: "Search-এ প্রতিটা অক্ষরে request যাচ্ছিল। useEffect-এ setTimeout আর cleanup-এ clearTimeout দিয়ে debounce করেছি। এখন টাইপ থামার ৪০০ms পরে একবার request যায়।" অথবা: "Page refresh-এ login হারিয়ে যাচ্ছিল। app শুরুতে restoreSession thunk দিয়ে token দিয়ে user আনি, আর ProtectedRoute-এ isSessionChecked না হওয়া পর্যন্ত spinner দেখাই, যাতে check শেষের আগেই login page-এ পাঠিয়ে না দেয়।"

শেষ কথা

তুমি এখন জানো:

- web কীভাবে কাজ করে, JavaScript, আর TypeScript
- React-এর component, props, state, effect
- React Router দিয়ে page আর protected route
- Redux Toolkit দিয়ে global state আর server-এর data
- একটা আসল project কীভাবে সাজানো হয়

পরের ধাপ: project-এ নিজে একটা ছোট feature যোগ করো। নিচেরগুলো শুধু frontend-এর কাজ, backend-এ কিছু বদলাতে হবে না:

- **"৩ দিনের মধ্যে deadline" badge:** table-এ যেসব application-এর deadline আগামী ৩ দিনের মধ্যে আর এখনো apply করা হয়নি, সেগুলোর পাশে একটা হলুদ badge। `isDeadlinePassed`-এর পাশে `applications.utils.ts`-এ একটা নতুন function লিখে শুরু করো।

- **"Clear search" button:** search box-এ কিছু লেখা থাকলে পাশে একটা ছোট × button, যেটা click করলে শুধু search খালি হয়, বাকি filter থাকে।
- **Page size মনে রাখা:** "Per page" বদলালে value-টা `localStorage`-এ রাখো, আর app খুললে সেখান থেকে পড়ে `DEFAULT_FILTERS`-এর `limit` বসাও। key-টা `shared/constants.ts`-এ রাখতে ভুলো না।
- **Navbar-এ নাম অন্যভাবে:** নাম থাকলে "Hi, Rahim", না থাকলে email-এর `@`-এর আগের অংশ দেখাও।
- **Applied count:** list page-এর উপরে লেখা "এই page-এ ১০টার মধ্যে ৪টায় apply করেছ"। `items` থেকে render-এর সময় হিসাব করো, আলাদা state না (derived state, #৩৮)।

অধ্যায় ১৪-এর "Companies" checklist বা application-এ "notes" field-এর মতো কিছু করতে চাইলে মনে রেখো, সেগুলোর জন্য আগে backend-এ কাজ লাগবে (NestJS module, Prisma model), যেটা এই বই শেখায় না।

নিজে হাতে বানালে যা শিখেছ তা পাকা হবে, আর interview-তে আত্মবিশ্বাসের সাথে বলতে পারবে। শুভকামনা!